云原生技术 · 通俗进阶指南



深入浅出 Kubernetes

从集装箱码头到云原生舰队

熊毅 著

容器 · 编排 · 云原生 · 2026

目 录

前言：写给想真正搞懂 Kubernetes 的你 17

这本书和别的书有什么不一样

全书结构

关于版本与约定

一点小建议

第1章 从集装箱码头说起：为什么世界需要 Kubernetes 19

1.1 集装箱如何改变了世界

1.2 软件世界的"货物"曾经有多混乱

1.3 容器的出现：应用有了统一的"箱子"

1.4 容器简史：从 chroot 到 Docker 的四十年

1.4.1 1979：chroot，隔离思想的萌芽

1.4.2 2000 年代：从"监狱"到"区域"

1.4.3 2006-2008：内核拼图逐渐完整

1.4.4 2013：Docker 的临门一脚

1.4.5 名字里的小心思

1.4.6 2014-2015：军阀混战与握手言和

1.5 当集装箱多了以后：谁来调度这座码头？

1.6 Kubernetes 是什么：一句大白话

1.7 Kubernetes 简史：从 Borg 到云原生霸主

1.7.1 Borg：Google 的秘密武器

1.7.2 2014：舵手启航

1.7.3 2015：一个决定性的战略——捐给中立基金会

1.7.4 2016-2017：容器编排大战

1.7.5 对照速查：Kubernetes 与竞品

1.7.6 2018 至今：云原生的绝对中心

1.8 Kubernetes 到底做了哪几件事

1.9 Kubernetes 不是什么

1.10 本章小结

第2章 云原生的地图：容器、编排与这套新玩法 29

2.1 什么是云原生：不是一个产品，而是一套方法论

2.2 传统架构 vs 云原生：一次彻底的范式转换
2.2.1 从"养宠物"到"养牛"
2.2.2 从可变基础设施到不可变基础设施
2.2.3 从命令式到声明式
2.3 微服务：为什么应用要拆开
2.3.1 单体架构：一栋什么功能都有的大楼
2.3.2 微服务：按业务拆成一栋栋小楼
2.3.3 十二要素应用：云原生应用的"宪法"
2.4 云原生技术版图：Kubernetes 周围的"星空"
2.5 上 Kubernetes 之前，先问自己几个问题
2.6 本章小结
第3章 集群、节点与控制平面：一艘航母的构造 35
3.1 集群与节点：航母与甲板
3.1.1 Node 资源对象：机器在 Kubernetes 里的"户口"
3.2 控制平面：舰桥上的五位核心成员
3.2.1 kube-apiserver：唯一的对外窗口
3.2.2 etcd：集群的记忆中枢
3.2.3 kube-scheduler：选房系统
3.2.4 kube-controller-manager：执行"期望状态"的总管家
3.2.5 cloud-controller-manager（可选）
3.3 工作节点：甲板上的四位干将
3.3.1 kubelet：节点上的工头
3.3.2 容器运行时：真正拉起容器的人
3.3.3 kube-proxy：节点上的交通警察
3.3.4 容器网络接口（CNI）插件
3.4 一张全景图：一次请求的生命周期
3.5 控制平面失联时：为什么业务还在跑
3.6 本章小结
第4章 Pod：最小的舱室，最大的学问 44
4.1 为什么是 Pod，而不是直接管理容器
4.2 Pod 的解剖：共享了什么，隔离了什么
4.2.1 网络命名空间：全 Pod 共享
4.2.2 进程、文件系统：各自隔离
4.3 编写第一个 Pod： anatomy of a YAML
4.3.1 resources：requests 与 limits，K8s 世界的"电表和水表"

4.3.2 labels：贴在资源上的"标签纸"
4.4 Pod 的生命周期：从出生到死亡
4.4.1 容器重启策略 restartPolicy
4.4.2 优雅终止：不是"咔嚓"一下就完了
4.5 探针：告诉 Kubernetes"我到底健不健康"
4.5.1 探针设计的实践智慧
4.6 多容器 Pod 与 Sidecar 模式
4.7 Init 容器：先遣队
4.8 操作 Pod 的常用命令
4.9 本章小结
第5章 用 YAML 说话：声明式 API 与资源对象 54
5.1 命令式 vs 声明式：一次深刻的转变
5.2 Kubernetes 的资源对象体系
5.2.1 metadata：名字、标签、注解与命名空间
5.2.2 命名空间：集群里的"楼层"
5.3 YAML 语法速成与避坑
5.4 kubectl：与集群对话的瑞士军刀
5.4.1 kubectl 的命令心智模型
5.4.2 get 与 describe：日常最高频的组合
5.4.3 输出过滤：jq 思维
5.5 资源清单的组织与管理
5.6 准入控制与默认值填充：提交之后发生了什么
5.7 kustomize 与"过度 Yaml 化"的隐忧
5.8 本章小结
第6章 Deployment：应用的自动驾驶 61
6.1 从手动管理 Pod 的痛苦说起
6.2 ReplicaSet：副本数量的守护者
6.3 Deployment：ReplicaSet 之上的"发布管理器"
6.4 动手玩转 Deployment
6.4.1 验证自愈：删一个试试
6.4.2 扩缩容：一条命令的事
6.5 深入控制循环：Reconcile 到底做了什么
6.6 Deployment 的状态解读
6.7 常见的反模式与最佳实践

6.8 本章小结

第7章 滚动更新与回滚：灰度发布的艺术 67

7.1 一次滚动更新的完整解剖

7.1.1 两个节奏旋钮：maxSurge 与 maxUnavailable

7.1.2 minReadySeconds 与 progressDeadlineSeconds

7.2 回滚：发布体系的"后悔药"

7.3 灰度发布的进阶玩法

7.3.1 基于 Deployment 的"穷人版金丝雀"

7.3.2 按请求特征分流

7.3.3 蓝绿部署

7.3.4 发布策略选型小结

7.4 发布卡住了怎么办：排查指南

7.5 本章小结

第8章 StatefulSet：有状态应用的专属包间 74

8.1 什么是有状态：身份、顺序与专属存储

8.2 StatefulSet 的三大承诺

8.3 一个完整的 StatefulSet 示例

8.4 有序性的细节：部署、更新、缩容、删除

8.5 StatefulSet 的行为差异与坑

8.6 有状态应用上 K8s：要想清楚的事

8.7 本章小结

第9章 DaemonSet 与 Job/CronJob：常驻守卫与定时任务 80

9.1 DaemonSet：每台机器都要有我

9.1.1 什么样的工作适合 DaemonSet

9.1.2 一个日志收集 DaemonSet 示例

9.2 Job：把一件事干完

9.2.1 一个计算圆周率的 Job

9.2.2 completionMode：索引任务模式

9.3 CronJob：定时执行 Job

9.4 工作负载家族总览

9.5 本章小结

第10章 Service：给漂泊的 Pod 一个稳定的门牌号 85

10.1 问题：Pod 的 IP 靠不住

10.2 Service 的本质：一个稳定的虚拟 IP + 一组后端
10.2.1 虚拟 IP 的"虚拟"二字：为什么没有进程监听
10.2.2 Endpoints 与 EndpointSlice：后端名单
10.3 Service 的四种类型
10.3.1 ClusterIP（默认）：仅集群内部可访问
10.3.2 NodePort：在每个节点上开一个端口
10.3.3 LoadBalancer：接入外部负载均衡器
10.3.4 ExternalName：一个 DNS 别名
10.3.5 特殊成员：headless Service
10.4 服务发现：K8s 内置的 DNS
10.5 kube-proxy 的两种模式：iptables 与 IPVS
10.6 Session 亲和与流量策略
10.7 本章小结
第11章 Ingress 与 Gateway API：集群的大门 92
11.1 七层与四层：为什么需要 Ingress
11.2 Ingress 与 Ingress Controller：规则与执行者
11.3 Ingress 实战：域名与路径路由
11.3.1 注解：控制器的"方言"
11.4 用 Ingress 做灰度：canary 注解
11.5 Gateway API：Ingress 的下一代
11.6 外部访问方案对比
11.7 本章小结
第12章 网络原理：一个数据包的旅行 98
12.1 Kubernetes 网络模型的四条铁律
12.2 微观机制：网络命名空间、veth pair 与 pause 容器
12.2.1 网络命名空间：每个 Pod 一座"孤岛"
12.2.2 veth pair：一根"两头都有插头"的虚拟网线
12.2.3 pause 容器：为 Pod 守住网络命名空间的"房主"
12.2.4 cni0 网桥：节点上的"交换机"
12.2.5 小结：三层递进的抽象
12.3 Pod IP 是怎么来的：IPAM 与 CNI 插件链
12.4 数据包的旅行：同节点与跨节点
12.4.1 同节点：veth 对与网桥
12.4.2 跨节点：三种主流方案

12.5 Service 流量在内核里的真实路径
12.6 DNS 链路：从短名到 IP 的旅程
12.7 NetworkPolicy：给扁平网络装上"门禁"
12.8 网络排障的"手术刀"
12.9 本章小结

第13章 Volume：把数据钉在地板上 107

13.1 为什么容器没有"记忆"
13.2 常用 Volume 类型分拣
13.3 emptyDir 实战
13.4 Volume 挂载的进阶技巧
13.4.1 subPath：一个卷多处挂载的不同子目录
13.4.2 把 ConfigMap 挂成配置文件
13.4.3 projected 卷：多来源合挂
13.5 容器存储的性能与最佳实践
13.6 本章小结

第14章 PV、PVC 与 StorageClass：存储的供给侧改革 111

14.1 三个角色：库房、租约申请、自动供应商
14.2 PVC：用户的申请单
14.3 StorageClass：动态供应的魔法
14.3.1 默认 StorageClass 与 immediately 的坑
14.4 回收策略：Delete 与 Retain 的生死抉择
14.5 CSI：存储的统一接口
14.6 卷快照与扩容
14.7 本章小结

第15章 ConfigMap 与 Secret：配置的魔法抽屉 117

15.1 配置与镜像分离：为什么这是铁律
15.2 ConfigMap：明文配置的容器
15.2.1 注入方式一：环境变量
15.2.2 注入方式二：挂载为文件
15.3 Secret：敏感信息的保险柜
15.3.1 一个必须戳破的幻觉："Secret 是加密的"
15.4 配置管理的工程实践
15.5 本章小结

第16章 RBAC：谁能进哪个房间 122

16.1 四个积木：Role、ClusterRole、Binding

16.2 Role 与 ClusterRole 的组合矩阵

16.3 ServiceAccount：程序的身份证

16.3.1 实战：给应用最小权限

16.4 认证与授权的分界线

16.5 检视与排障 RBAC

16.6 权限设计的进阶话题

16.7 本章小结

第17章 安全全景：从镜像到运行时的纵深防御 129

17.1 威胁模型：容器不是虚拟机

17.2 第一道防线：供应链安全（镜像）

17.3 第二道防线：Pod Security——运行时的"纪律条款"

17.3.1 Pod Security Admission：集群级的强制标准

17.4 第三道防线：网络隔离（NetworkPolicy）

17.5 第四道防线：运行时检测与密钥管理

17.6 把防线拼成体系

17.7 本章小结

第18章 调度器：Kubernetes 的选房系统 133

18.1 调度的本质：一个两轮筛选的决策过程

18.2 requests 与调度：你已经决定了调度

18.3 nodeSelector 与亲和性：指挥调度器

18.3.1 nodeSelector：最简单的节点筛选

18.3.2 nodeAffinity：更强大的节点亲和

18.3.3 podAffinity / podAntiAffinity：Pod 之间的"合租"与"分居"

18.3.4 topologySpreadConstraints：更精细的分散控制

18.4 污点与容忍：节点的"谢客牌"

18.5 优先级与抢占

18.6 多调度器与调度框架

18.7 本章小结

第19章 HPA 与自动扩缩容：让集群呼吸起来 139

19.1 HPA：副本数的自动驾驶仪

19.1.1 指标从哪来：metrics-server
19.1.2 自定义指标与缩放行为
19.2 VPA：垂直方向的"量体裁衣"
19.3 Cluster Autoscaler：节点层的伸缩
19.3.1 别让节点"缩不走"：PDB 与注解
19.4 三件套的协作全景
19.5 本章小结
第20章 日志与监控：给集群装上仪表盘 144
20.1 可观测性三支柱
20.2 指标体系：Prometheus 与其模型
20.2.1 部署与四条黄金指标
20.2.2 告警：让人该睡就睡
20.3 日志体系：从 stdout 到检索平台
20.3.1 日志规范的工程要求
20.4 追踪：分布式系统的"GPS"
20.5 排障实战：一次 5 分钟定位
20.6 本章小结
第21章 故障排查：当船漏水时 149
21.1 方法论：五个原则
21.2 排障工具箱总览
21.3 高频故障场景手册
21.3.1 Pod 起不来：Pending
21.3.2 镜像拉取失败：ImagePullBackOff
21.3.3 崩溃循环：CrashLoopBackOff
21.3.4 Running 但不就绪：0/1 Running
21.3.5 网络不通：层层剥洋葱
21.3.6 节点故障：NotReady
21.3.7 证书过期：集群"集体失联"
21.4 进阶武器：kubectl debug 与 ephemeral 容器
21.5 事故复盘：从救火到防火
21.6 本章小结
第22章 升级、备份与高可用：永不沉没的船 156
22.1 集群升级：最不该翻车的常规操作
22.1.1 升级前的准备清单

22.1.2 kubeadm 集群的标准升级流程
22.1.3 升级期间业务会中断吗
22.2 备份：唯一能救你命的东西
22.2.1 集群状态备份（etcd 与资源清单）
22.2.2 应用数据备份（PV 内容）
22.3 高可用架构：假设一切都会坏
22.4 混沌工程：主动破坏的勇气
22.5 本章小结
第23章 Helm：Kubernetes 的包管理器 160
23.1 为什么需要包管理器
23.2 五分钟上手 Helm
23.3 Chart 的解剖：模板 + 参数
23.4 多环境管理：values 的分层覆盖
23.5 Chart 仓库与分发
23.6 Helm 的边界与替代视角
23.7 本章小结
第24章 GitOps 与 Kustomize：声明式的终极形态 165
24.1 GitOps 的四原则
24.2 Argo CD：GitOps 控制器
24.3 Kustomize：无模板的配置叠加
24.4 GitOps 的目录组织与秘密管理
24.5 渐进式交付：GitOps 的下一层
24.6 本章小结
第25章 CRD 与 Operator：教 Kubernetes 新本领 169
25.1 CRD：给 API 添加新词汇
25.2 Operator：带着领域知识的控制器
25.3 生态中的 Operator：站在巨人肩上
25.4 设计 Operator 之前的三问
25.5 Reconcile 的工程纪律
25.6 本章小结
第26章 服务网格 Istio：网络里的情报系统 175
26.1 服务网格的核心思想：Sidecar 接管流量

26.2 三个核心 CRD：VirtualService、DestinationRule、Gateway
26.3 mTLS：服务间加密的"免费午餐"
26.4 可观测性的"免费升级"
26.5 Istio 的代价：清醒地看复杂度
26.6 本章小结

第27章 从零到一：部署一个微服务应用 180

27.1 场景设定
27.2 第一步：环境准备（第 3、22 章）
27.3 第二步：设计清单（第 5、15、4 章）
27.4 第三步：发布流水线（第 23、24、7 章）
27.5 第四步：上线后的守护（第 20、21、19 章）
27.6 复盘：这本书教给你的思维
27.7 继续前行的路标

第28章 多集群与混合云：舰队协同作战 187

28.1 为什么要多集群
28.2 多集群的四种架构模式
28.3 混合云与云间迁移
28.4 多集群的核心挑战
28.5 本章小结

第29章 成本优化：把钱花在刀刃上 189

29.1 看见成本：优化的前提
29.2 六个高性价比优化手段
29.3 建立成本治理机制
29.4 本章小结

第30章 平台工程：从集群到"自助平台" 191

30.1 平台工程的定位
30.2 关键技术支柱
30.3 平台工程的避坑指南
30.4 全书终章：你的云原生地图

第31章 动手实验一：从零搭建本地集群 193

31.1 实验目标

31.2 环境要求
31.3 安装 Kind 与 kubectl
31.4 创建你的第一个集群
31.5 看看集群内部组件
31.6 配置命令补全（强烈推荐）
31.7 准备两个"百宝箱"工具
31.8 部署一个"看板"：验证集群健康
31.9 实验总结与思考

第32章 动手实验二：从代码到 Pod 的完整旅程 198

32.1 实验目标
32.2 写一个最小 HTTP 应用
32.3 写一个多阶段 Dockerfile
32.4 把镜像交给 Kind 集群
32.5 运行单容器 Pod
32.6 亲手验证探针行为
32.7 多容器 Pod：Sidecar 实战
32.8 实验总结与思考

第33章 动手实验三：Deployment 发布全生命周期 205

33.1 实验目标
33.2 准备工作
33.3 创建 Deployment 并理解三层结构
33.4 验证自愈
33.5 滚动更新：一帧一帧看
33.6 制造事故 + 回滚
33.6.1 让历史可读：change-cause 注解
33.7 金丝雀发布：灰度 10%
33.8 实验总结与思考

第34章 动手实验四：Service 与网络排障实战 211

34.1 实验目标
34.2 部署后端并暴露 Service
34.3 从 Pod 内部验证服务发现与负载均衡
34.4 观察端点随就绪状态变化

34.5 三种 Service 类型实测
34.6 故障演练一：Service 没有后端
34.7 故障演练二：Pod 起了但没就绪
34.8 一套标准的网络排障口诀
34.9 实验总结与思考

第35章 动手实验五：Ingress 路由与 HTTPS 实战 215

35.1 实验目标
35.2 部署 Ingress Controller
35.3 准备两个后端服务
35.4 基于路径的路由
35.5 基于域名的路由
35.6 亲手验证"没有 Controller，Ingress 是废纸"
35.7 配置 HTTPS（自签名证书）
35.8 用注解做限流（可选进阶）
35.9 实验总结与思考

第36章 动手实验六：存储持久化实战 221

36.1 实验目标
36.2 预备：安装本地存储供应器
36.3 实验 A：容器可写层不持久
36.4 实验 B：emptyDir 的生命周期
36.5 实验 C：PVC 动态供应全链路
36.6 实验 D：StatefulSet 的稳定身份与专属存储
36.7 实验总结与思考

第37章 动手实验七：ConfigMap、Secret 与热更新 227

37.1 实验目标
37.2 创建 ConfigMap
37.3 实验 A：环境变量注入不热更新
37.4 实验 B：文件挂载可以热更新
37.5 实验 C：subPath 不热更新（著名大坑）
37.6 实验 D：看穿 Secret 的 base64
37.7 实验 E：ConfigMap 挂 Nginx 配置 + 热加载
37.8 实验总结与思考

第38章 动手实验八：RBAC 与 Pod 安全实战 233

38.1 实验目标

38.2 实验 A：RBAC 最小权限全流程

38.3 实验 B：程序如何用 token 访问 API

38.4 实验 C：被 Pod Security Admission 拦截

38.5 实验 D：一条完整的安全基线对比

38.6 实验总结与思考

第39章 动手实验九：HPA 弹性伸缩实战 239

39.1 实验目标

39.2 安装 metrics-server

39.3 部署一个"能吃 CPU"的应用

39.4 创建 HPA

39.5 压测触发扩容

39.6 停止压测，观察缩容

39.7 实验进阶：验证 requests 的影响

39.8 实验总结与思考

第40章 动手实验十：Helm 从零到发布 244

40.1 实验目标

40.2 创建一个 Chart

40.3 写模板

40.4 渲染预览（重要习惯）

40.5 部署两个环境

40.6 升级与回滚

40.7 用 values 文件（替代一堆 --set）

40.8 调试三板斧

40.9 实验总结与思考

第41章 动手实验十一：GitOps 全流程实战 249

41.1 实验目标

41.2 部署 Argo CD

41.3 准备"真相源"仓库

41.4 把应用接入 Argo CD

41.5 体验"改 Git 即改集群"

41.6 制造配置漂移，看自动纠正

41.7 回滚 = git revert

41.8 实验总结与思考

第42章 动手实验十二：故障排查实战演练 253

42.1 实验目标

42.2 故障一：ImagePullBackOff

42.3 故障二：CrashLoopBackOff

42.4 故障三：Pending（调度失败）

42.5 故障四：网络不通（逐层剥洋葱）

42.6 一套"秒级止损"清单

42.7 实验总结与思考

附录F 动手实验篇索引与完成清单

附录A 常用命令速查表 259

A.1 集群与节点

A.2 工作负载（第 4、6～9 章）

A.3 网络与服务（第 10～12 章）

A.4 存储与配置（第 13～15 章）

A.5 RBAC 与安全（第 16～17 章）

A.6 排障与调试（第 21 章）

A.7 Helm 与 GitOps（第 23～24 章）

A.8 etcd 备份（第 22 章）

附录B 关键术语对照表

附录C 学习资源推荐

C.1 官方文档（第一手资料）

C.2 动手环境

C.3 认证体系（CNCF）

C.4 进阶主题路线

C.5 保持更新

附录D 生产就绪检查清单 266

D.1 应用层（单个服务）

D.2 流量与网络

D.3 数据层

D.4 发布与回滚
D.5 可观测性
D.6 集群与平台层
D.7 流程与人
附录E 高频面试题精选与解析
E.1 基础概念类
E.2 原理机制类
E.3 网络与存储类
E.4 实战排障类
E.5 设计与架构类
附录G 常用 YAML 模板库 **272**
G.1 无状态服务（Deployment 全套）
G.2 有状态服务（StatefulSet）
G.3 定时任务（CronJob）
G.4 守护进程（DaemonSet）
G.5 配置与密钥
G.6 网络入口（Ingress）
G.7 存储（PVC）
G.8 RBAC（最小权限）
G.9 NetworkPolicy（默认拒绝 + 白名单）
附录H 参考文献与延伸阅读 **282**
H.1 官方文档与规范
H.2 经典书籍
H.3 核心组件与开源项目文档
H.4 认证与社区资源

前言：写给想真正搞懂 Kubernetes 的你

你大概率已经听说过 Kubernetes 了。在今天的招聘启事里，它和"熟悉 Linux""会写 SQL"一样，几乎成了后端工程师的默认技能；在技术分享会上，总有人张口闭口"我们上云了""我们迁移到了 K8s"。但如果你追问一句——"Kubernetes 到底解决了什么问题？Pod 为什么不能直接对外服务？控制器、调度器、etcd 这些东西到底在背后干什么？"——能讲清楚的人，往往没有想象中那么多。

这本书，就是想帮你把这些问题彻底讲清楚。

这本书和别的书有什么不一样

市面上关于 Kubernetes 的资料已经很多了：官方文档严谨但门槛高，博客文章零散且良莠不齐，视频教程生动却往往流于操作演示。本书想做的是介于"入门教程"和"官方文档"之间的那一层——**用通俗的语言和形象的类比，把 Kubernetes 的核心机制真正讲透**，让你不仅会"照着抄命令"，更理解"为什么要这样做"。

我有三条贯穿全书的写作原则：

第一，**先讲"为什么"，再讲"是什么"**。任何一个复杂的技术概念，都不是凭空冒出来的，它背后一定对应着一个具体要解决的问题。比如，Pod 之所以存在，是因为容器之间需要共享网络和存储；Service 之所以存在，是因为 Pod 的生命周期太短、IP 会变。理解了问题，概念就顺理成章。

第二，**善用类比，但绝不停留在类比**。我会把集群比作"码头"、把 Pod 比作"集装箱"、把调度器比作"选房系统"。类比能帮你快速建立直觉，但直觉不能替代严谨。所以每个类比之后，我都会回到技术本身，讲清楚真实的数据结构和运行机制。

第三，**面向有基础的读者**。我默认你了解 Linux 的基本操作、知道容器和镜像是什么、会写简单的 YAML。如果你连 `docker run` 都没跑过，建议先补一下 Docker 的基础，再回来看这本书，会轻松很多。

全书结构

本书分为十二个部分、四十二章，外加八个附录，你可以按顺序读，也可以按需跳读：

- **第一部分「启航」**（第 1～2 章）：从集装箱革命讲到云原生，回答"世界为什么需要 Kubernetes"。
- **第二部分「概念地图」**（第 3～5 章）：建立集群、节点、Pod、声明式 API 的核心世界观。
- **第三部分「工作负载」**（第 6～9 章）：Deployment、StatefulSet、DaemonSet、Job 等控制器，让应用"跑起来、跑得稳"。

- **第四部分「网络」**（第 10～12 章）：Service、Ingress 以及背后的网络原理。
- **第五部分「存储」**（第 13～14 章）：Volume、PV/PVC、StorageClass，让数据不再随风飘散。
- **第六部分「配置与安全」**（第 15～17 章）：ConfigMap、Secret、RBAC 与安全全景。
- **第七部分「调度与弹性」**（第 18～19 章）：调度器原理与自动扩缩容。
- **第八部分「可观测性与运维」**（第 20～22 章）：日志监控、故障排查、升级备份与高可用。
- **第九部分「生态与进阶」**（第 23～26 章）：Helm、GitOps、Operator、服务网格。
- **第十部分「实战」**（第 27 章）：从零部署一个微服务应用，把前面所学串起来。
- **第十一部分「延伸与展望」**（第 28～30 章）：多集群与混合云、成本优化、平台工程。
- **第十二部分「动手实验」**（第 31～42 章）：十二个可逐条照做的实操实验，从搭集群、容器化、发布回滚、网络存储，到安全、弹性、Helm、GitOps 与故障排查，每个实验都含完整 YAML、命令与验证步骤。

关于版本与约定

本书以 Kubernetes 1.28～1.30 版本为主要参照，书中的 YAML 示例和命令在这些版本上均已验证可运行。由于 Kubernetes 迭代较快，个别 API 的版本号（如 `apps/v1`、`networking.k8s.io/v1`）可能在更新的版本中有所变化，请以你实际使用的版本为准。

书中的代码块、命令和 YAML 都经过整理，你可以直接复制到本地练习。所有示例都遵循"最小可用"原则——去掉不必要的字段，突出核心概念。

一点小建议

如果你时间充裕，我建议你**边读边动手**：每学完一个概念，就在本地（用 Minikube、Kind 或 Docker Desktop 自带的 Kubernetes）跑一遍对应的命令和 YAML。Kubernetes 是典型的"实践学科"，很多东西看十遍不如亲手敲一遍。

如果你时间紧张，可以先通读一遍建立整体框架，把感兴趣的章节标记出来，日后需要时再回头精读。

好了，让我们登船，一起驶向云原生的世界。

> **提示：** 本书所有术语首次出现时会给出中文解释；一些关键英文术语会保留原文，因为它们是你查阅资料、搜索问题时的"通行证"。建议你熟悉这些英文原词，而非只记中文译名。

熊毅

二〇二六年十月

第1章　从集装箱码头说起：为什么世界需要 Kubernetes

在正式进入 Kubernetes 的技术细节之前，我们需要先回答一个更根本的问题：**它到底解决了一个什么难题？** 如果不理解这个问题，后面学的每一个概念都会像是无源之水——你只知道"有这个东西"，却不知道为什么非要有它。

而要回答这个问题，最好的切入点，是半个多世纪前的一场物流革命。

1.1 集装箱如何改变了世界

1956 年，美国卡车司机马尔科姆·麦克莱恩做了一件当时看起来有点疯狂的事：他把 58 个卡车货箱直接装上了一艘改造过的油轮，从新泽西运往休斯敦。这个看似平淡的操作，被认为是现代"集装箱运输"的开端。

在集装箱出现之前，港口卸货是一件极其混乱的事：货物五花八门——木箱、麻袋、木桶、托盘，形状大小不一，装卸全靠码头工人一件件搬。每一件货都要单独处理，不同的船、不同的码头、不同的运输工具之间，接口全都不一样。结果就是：运输成本高得吓人，效率低得可怜。

集装箱做了什么？它做了一件**极其简单却又极其重要的事：标准化**。

集装箱有一个统一的尺寸标准（20 英尺、40 英尺等），有统一的接口（吊装角件），码头上的起重机、卡车、火车、轮船，全都围绕这个标准来设计。从此以后，码头工人不需要关心箱子里装的是电视机还是土豆——**只要是标准集装箱，就能用同一套流程装卸、堆叠、运输**。货物在船、卡车、火车之间转运时，箱子本身不用打开，直接整体搬运。

这场标准化带来的连锁反应是惊人的：装卸效率提升了数十倍，运输成本下降了一个数量级，全球贸易的格局从此被彻底改写。今天我们能在超市买到来自世界各地的商品，很大程度上要归功于这个"铁盒子"。

> **比喻：** 集装箱的本质，是把"五花八门的货物"包装成"统一的、可标准搬运的单元"。这个思想，正是理解容器和 Kubernetes 的一把钥匙。

1.2 软件世界的"货物"曾经有多混乱

让我们把镜头从码头拉回到软件开发。

在"容器"这个概念流行起来之前，软件交付的世界，和集装箱出现之前的码头几乎一模一样混乱：

一个典型的场景是这样的：开发者在自己的电脑上写好了代码，本地跑得好好的，一部署到服务器上就出问题——"我本地明明是好的啊"。为什么？因为本地和服务器的环境不一样：操作系统版本不一样，依赖库版本不一样，环境变量不一样，甚至时区、语言编码都不一样。

于是就有了那句著名的甩锅话："**在我机器上能跑啊**。"

更麻烦的是，一台服务器上往往要同时跑多个应用，这些应用之间的依赖还会互相打架：A 应用需要 Python 2.7，B 应用需要 Python 3.8，C 应用需要某个特定版本的 glibc。你装了这个就得罪那个，运维人员天天在"依赖地狱"里挣扎。

部署本身也是一场噩梦。早期的手工部署流程大概是：登录服务器 → 用 FTP 或 scp 传文件 → 手动改配置 → 重启服务 → 祈祷别出错。每次发布都像走钢丝，一不留神就线上事故。

虚拟机的出现缓解了一部分问题——你可以在同一台物理机上跑多个互相隔离的操作系统。但虚拟机太重了：每个虚拟机都要跑一整套完整的操作系统内核，启动慢、占用资源多，一台机器上跑不了几个。

软件世界迫切需要自己的"集装箱"——一个能把应用和它的运行环境打包在一起、可以随处运行的标准单元。这个"集装箱"，就是容器。

1.3 容器的出现：应用有了统一的"箱子"

容器的核心思想，和集装箱如出一辙：**把应用和它运行所需的一切（代码、运行时、系统库、配置、环境变量）打包成一个标准化的、可移植的单元**。

2013 年，Docker 的出现让容器技术从实验室走进了大众视野。它做了一件关键的事：把复杂的底层技术（Linux 的 namespace、cgroup 等）封装成一个极其友好的工具，让普通开发者用一条命令就能创建、运行、分发容器。

有了容器之后，很多老问题迎刃而解：

- **"我本地能跑"变成了"容器里能跑，所以哪里都能跑"**。开发、测试、生产环境跑的是同一个镜像，环境一致性有了保证。
- **依赖冲突消失了**。每个容器有自己的隔离环境，A 应用和 B 应用的依赖互不干扰。
- **交付物统一了**。交付的不再是"一堆代码 + 一页部署文档"，而是一个标准的镜像。

容器很美好，但很快，一个更棘手的问题浮出了水面。

1.4 容器简史：从 chroot 到 Docker 的四十年

很多人以为容器是 2013 年 Docker 发明的。其实，容器背后的技术积累，跨越了整整四十年。了解这段历史，能帮你理解"容器为什么是现在这个样子"。

1.4.1 1979：chroot，隔离思想的萌芽

1979 年，Unix 第 7 版引入了一个不起眼的系统调用——**chroot**（change root）。它的作用很简单：把一个进程的"根目录"圈到文件系统的某个子目录里，进程从此看不到外面的世界。

当时的动机其实和"隔离"无关，只是想为系统安装提供一个干净的构建环境。但 chroot 种下了一个至关重要的思想：**进程的运行环境，是可以被"圈定"和隔离的**。

1.4.2 2000 年代：从"监狱"到"区域"

2000 年，FreeBSD 推出了 **Jail**（直译就是"监狱"）——它不仅隔离文件系统，还隔离进程、网络、用户，把一个 FreeBSD 系统切分成多个互不相通的"牢房"。这个名字起得实在形象：进程被关进去，就别想越狱了。

此后，隔离技术进入百花齐放的阶段：2001 年 Linux 社区有了 VServer，2004 年 Sun 的 Solaris 推出 Zones。这些方案各有千秋，但都面临同一个问题：**太复杂、太底层，普通人玩不转**。

1.4.3 2006-2008：内核拼图逐渐完整

容器需要两块内核基石：**namespace**（隔离"能看到什么"）和 **cgroup**（限制"能用多少"）。

namespace 从 2002 年起陆续进入 Linux 内核；cgroup 则始于 2006 年 Google 工程师的贡献（当时叫 process containers）。2008 年，整合了这两块基石的 **LXC（Linux Containers）** 进入内核主线——Linux 上第一个真正完整的容器方案诞生了。

但 LXC 依然是"高手工具"：配置繁琐、文档晦涩、镜像无从分发。容器离"走进千家万户"，还差临门一脚。

1.4.4 2013：Docker 的临门一脚

2013 年 3 月，一家叫 dotCloud 的小公司，把内部工具 **Docker** 开源了。创始人 Solomon Hykes 在 PyCon 大会上做了一场五分钟的闪电演讲，演示了"一条命令启动一个应用"。

Docker 的成功秘诀，**不是发明了新技术**（底层仍然是 namespace、cgroup、LXC），而是做对了一件事——**把复杂的技术封装成傻瓜式的体验**：

- **Dockerfile**：用几行声明就能描述"我的应用需要什么环境"；
- **分层镜像**：像搭积木一样复用基础镜像，秒级构建、秒级启动；
- **Docker Hub**：第一个"镜像商店"，构建一次、随处分发。

这就像当年的图形界面之于命令行：技术还是那个技术，但使用门槛从"系统管理员"降到了"每个开发者"。

> **比喻：** 如果说 LXC 是"零件和图纸"，那 Docker 就是"乐高套装"——它把散落的零件标准化、打包、配上说明书，让普通人都能拼出自己想要的东西。

1.4.5 名字里的小心思

Docker 这个名字本身就是个绝妙的隐喻——它的本意是**"码头工人"**。码头工人搬运的正是集装箱；而 Docker 搬运的，是装了应用的"软件集装箱"。连 logo 都是一只驮着集装箱的鲸鱼（昵称 Moby Dock），把"航运 + 容器"的意象焊死在一起。

1.4.6 2014-2015：军阀混战与握手言和

Docker 火了之后，行业并没有立刻统一。2014 年，CoreOS 推出竞品 **Rocket（rkt）**，公开批评 Docker 的"大而全"设计，容器江湖一度"军阀混战"——各家标准不一，镜像互不兼容，用户无所适从。

2015 年，行业终于意识到：继续打下去，对谁都没好处。于是 Docker 牵头成立了 **OCI（Open Container Initiative，开放容器倡议）**，把自己核心的运行时（后来演化成 runc）和镜像规范捐了出来，形成统一标准。

> **深入：** 这一决策影响深远——"容器"从此有了行业标准，任何厂商都可以实现兼容的运行时。这也正是几年后 Kubernetes 能够"去 Docker 化"（改用 containerd 等标准运行时）的前提。

1.5 当集装箱多了以后：谁来调度这座码头？

想象一下，你现在有几十个、几百个甚至几千个容器需要运行，而且这些容器还有各种各样的要求：

- 有的容器是 Web 服务，需要对外提供 HTTP 访问；
- 有的容器是后台任务，需要每天凌晨定时跑一次；
- 有的容器是有状态的数据库，数据不能丢，重启后还要能找到原来的数据；
- 有的容器是另一个容器的"伴侣"，必须和它部署在同一台机器上；
- 有的容器挂了要自动重启，有的容器流量大了要自动扩容；
- 你要做灰度发布，先让 10% 的流量走新版本，观察没问题再全量；
- 某台服务器宕机了，上面的容器要自动迁移到健康的机器上。

如果靠人工来管理，这几乎是不可能完成的任务。你需要在几十台机器之间手动决定"哪个容器放到哪台机器"，手动配置网络让它们互相连通，手动监控它们的健康状态，手动处理故障……

容器解决了"单个应用怎么打包"的问题，但没有解决"成百上千个容器怎么协同工作"的问题。 后者需要的，是一套自动化的"码头操作系统"——它能像一座现代化码头那样，自动完成货物的装卸、堆叠、调度、监控、故障处理。

而这，正是 Kubernetes 的使命。

1.6 Kubernetes 是什么：一句大白话

Kubernetes（常缩写为 K8s，因为 K 和 s 之间有 8 个字母）是一个**开源的容器编排平台**。如果用一句话概括，它是：

> **一个帮你自动部署、调度、管理和扩展容器化应用的"码头操作系统"。**

你不再需要手动决定"这个容器放在哪台机器上""怎么让它们互相访问""挂了怎么重启"，你只需要用声明的方式告诉 Kubernetes"我想要什么"——"我想要 3 个 Web 服务副本，每个分配 512MB 内存，对外暴露 80 端口"——剩下的，Kubernetes 会自动去实现，并持续地维护这个状态。

Kubernetes 的名字来源于希腊语，意思是"舵手"或"领航员"——这很贴切，因为它正是为整个容器舰队掌舵的角色。它诞生于 Google 内部，凝聚了 Google 十多年大规模容器管理（Borg 系统）的经验，于 2014 年开源，2015 年捐赠给 CNCF（云原生计算基金会），此后迅速成为容器编排领域事实上的标准，如今已是全球最活跃的开源项目之一。

1.7 Kubernetes 简史：从 Borg 到云原生霸主

如果说容器的历史是一部"技术平民化"的历史，那么 Kubernetes 的历史，则是一部"内部秘密武器如何变成全球标准"的历史。这段故事同样精彩。

1.7.1 Borg：Google 的秘密武器

故事的起点在 Google 内部，时间比 Docker 早得多——2003 年，Google 就启动了一个叫 **Borg** 的大规模集群管理系统。

Borg 有多强？它管理着 Google 全球数据中心里数以万计的机器、上百万个任务，自动完成调度、监控、故障处理、资源回收。可以说，**没有 Borg，就没有 Google 搜索、Gmail、YouTube 的稳定运行**。

名字的由来很"极客"：Borg 来自科幻经典《星际迷航》里的**博格人（Borg）**——一个靠集体意识同化一切的外星种族，寓意"把所有机器整合成一个有机整体"。Google 内部还流传着一句玩笑话：**"Borg 就是我们的秘密武器。"**

Borg 的继任者是 **Omega**，进一步改进了调度架构。这两套系统积累的经验，后来全都注入了 Kubernetes。

1.7.2 2014：舵手启航

Google 深知一件事：**内部系统做得再好，如果只有自己用，也成不了行业标准**。

2014 年 6 月，Google 联合几位 Borg 老兵，把 Borg 的核心思想重新实现，并开源——这就是 **Kubernetes**。

名字同样讲究：Kubernetes 源自希腊语 **κυβερνήτης（kybernetes）**，意思是**"舵手"或"领航员"**。为庞大的容器舰队掌舵，再贴切不过。它和"控制论"（cybernetics）同根同源——都源于"掌舵"这个词根。常缩写为 **K8s**，因为 K 和 s 之间隔着 8 个字母。

logo 是一枚**七边形的船舵**——七根辐条，正是向 Borg 致敬。这枚船舵，如今已是云原生世界最醒目的图腾。

1.7.3 2015：一个决定性的战略——捐给中立基金会

2015 年 7 月，Kubernetes 1.0 发布。几乎同一时间，Google 联合 Linux 基金会成立了 **CNCF（云原生计算基金会）**，并把 Kubernetes 捐给了这个中立的基金会。

这一招看似"把到手的宝贝拱手送人"，实则极其高明：**中立托管打消了竞争对手"这是 Google 的产品"的顾虑**。Red Hat、IBM、华为、微软……各家巨头才敢放心投入、共同建设。可以说，没有 CNCF 的中立治理，就没有后来 Kubernetes 的一统江湖。

1.7.4 2016-2017：容器编排大战

接下来的几年，是一场被称为"容器编排大战"的激烈竞争，三足鼎立：

- **Docker Swarm**：Docker 亲儿子，上手简单，但功能偏弱；
- **Apache Mesos**：老牌通用调度器，强在"什么都能调"，但复杂；
- **Kubernetes**：站在 Google 十年 Borg 经验之上，架构最成熟。

最终 Kubernetes 胜出。原因众说纷纭，但公认有三点：**成熟的架构**（Borg 十年血泪换来的）、**先进的声明式 API**、以及**中立的基金会治理**。

2017 年，标志性事件发生：**Docker 官方宣布在自家产品中原生支持 Kubernetes**——连最大的对手都"投诚"了，大战正式落幕。

1.7.5 对照速查：Kubernetes 与竞品

历史讲到这里，你可能想问：当年和 Kubernetes 同台竞技的对手们，现在怎么样了？它们各自有什么特点？这张对照表能帮你快速建立认知：

维度	Kubernetes	Docker Swarm	Apache Mesos	Nomad
出身	Google（Borg 经验）	Docker 官方	Apache 基金会	HashiCorp
上手难度	中（概念较多）	低（一条命令）	高（两层调度）	低（单二进制）
调度能力	强（过滤+打分，可扩展）	较弱	极强（通用调度）	中等
自愈与伸缩	完善	基础	强大	基础
可调度负载	容器为主	仅容器	容器+大数据+任意框架	容器+裸进程+Java
生态	极其丰富（CNCF）	弱	碎片化	与 HashiCorp 家族集成
现状	事实标准	基本停滞	边缘化	小众但稳定

下面逐个说说它们的"性格"。

Docker Swarm——最像"亲儿子"的对手。 Swarm 的最大优点是简单：`docker swarm init` 一条命令就能搭起集群，语法和 Docker Compose 几乎一样，老 Docker 用户十分钟上手。但它成也简单、败也简单——调度策略、自愈、伸缩、网络都只能算"够用"，面对复杂的大规模场景力不从心。2019 年后，Docker 基本停止了对 Swarm 的投入，把宝押回了 Kubernetes。

Apache Mesos——错位竞争的"大力士"。 Mesos 的定位其实和 K8s 不完全重叠：它是一个**通用资源调度器**，不仅能调度容器，还能调度 Spark、Hadoop 等大数据框架，甚至任意进程。Twitter、Airbnb 都曾大规模使用。但它的代价是复杂度——Mesos 是"两层调度"（Mesos 管资源 + Marathon 等框架管应用），心智负担重、生态碎片化。当容器成为主流、K8s 把"容器调度"做到极致后，Mesos 的通用性优势反而成了包袱。

Nomad——小而美的"独行侠"。 HashiCorp 出品的 Nomad 走的是极简路线：单个二进制文件、没有外部依赖、上手十分钟。它还能调度非容器负载（裸进程、Java 应用），与 Consul（服务发现）、Vault（密钥）无缝集成。对于不需要 K8s 全家桶的简单场景（尤其是"我还不想容器化"的传统应用），Nomad 依然是一个优雅的选择。

> **提示：** 竞品的"失败"往往不是技术不行，而是**生态和时机**。Swarm 输在功能天花板太低，Mesos 输在复杂度与定位错位，Nomad 则退守到"简单场景"这个细分市场——这恰恰印证了一个朴素的道理：在平台竞争中，**生态的厚度，往往比单一功能的强弱更决定生死**。

与 PaaS、Serverless 的关系：不是竞品，是上下游

有两个概念常被拿来和 Kubernetes 比较，其实它们不在同一个层次：

传统 PaaS（Heroku、Cloud Foundry） 是更高一层的"应用平台"：你 `git push` 代码，它帮你构建、部署、托管，全程无需关心服务器。它的定位是"把复杂度彻底藏起来"。而 Kubernetes 把控制权交给你、也把责任交给你——它更像"**搭建 PaaS 的积木**"，而不是 PaaS 本身。事实上，今天的许多 PaaS（包括云厂商的托管服务）底层就是 Kubernetes。

Serverless（函数计算，如 AWS Lambda） 是另一种更高层的抽象：你只写函数，连"服务常驻"都不管了，按调用次数计费。它与 Kubernetes **互补而非竞争**：常驻的在线服务交给 K8s，事件驱动、波峰波谷剧烈的短任务交给 Serverless。很多团队的实际架构是"K8s + Serverless"混用。

把这几者的层次关系画出来，是这样的：

```
Serverless（函数）      ← 最省心，只管代码
   PaaS（应用平台）     ← git push 即部署
Kubernetes（编排）      ← 本书的主角，管容器
   容器（Docker）       ← 打包应用
  虚拟机 / 物理机       ← 最底层
```

一句话总结这段对照：**Kubernetes 赢在了"生态厚度 + 时机 + 架构成熟度"的组合，而 PaaS 和 Serverless 不是它的对手，是站在它肩膀上、面向不同需求的更高层抽象**。

1.7.6 2018 至今：云原生的绝对中心

此后 Kubernetes 一路狂奔：2018 年后成为云厂商的标配服务（各家纷纷推出托管版），2021 年前后开始"去 Docker 化"（用 containerd 等标准运行时替代 Docker 守护进程，但镜像标准不变），如今已是全球最活跃的开源项目之一，牢牢占据云原生版图的中心。

下图把容器与 Kubernetes 两段历史串在一起，一图纵览这场持续四十年的进化：

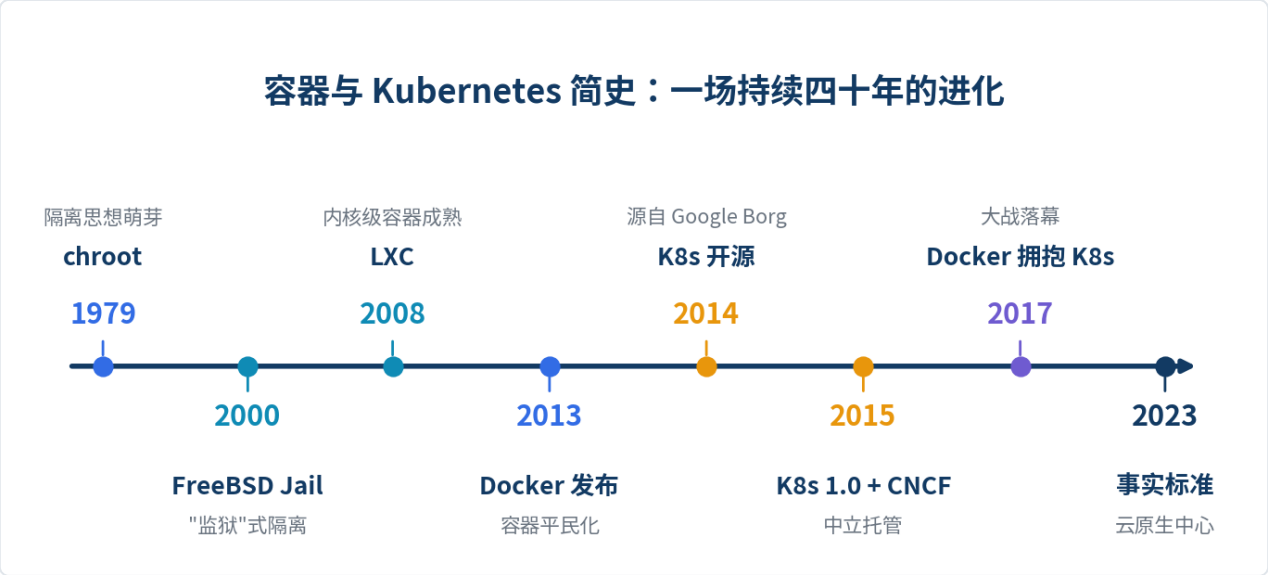


图1-1 容器与 Kubernetes 简史：一场持续四十年的进化

1.8 Kubernetes 到底做了哪几件事

为了让你对全书要讲的内容有个整体印象，我们先提纲挈领地列出 Kubernetes 的核心能力。这些能力，本书后面的章节会逐一展开：

1. **服务发现与负载均衡**：Kubernetes 可以为每个容器分配一个稳定的"身份"（DNS 名），并自动在多个副本之间做流量分发。
2. **自动装箱（调度）**：根据容器的资源需求和约束，自动决定把它放到哪个节点上运行，就像调度系统决定货物放进哪个货位。
3. **自愈能力**：容器挂了自动重启；节点挂了自动把上面的容器迁移走；不满足健康检查的容器会被替换。
4. **水平伸缩**：一条命令或一条规则，就能让应用副本数从 3 个变成 30 个，再根据 CPU 负载自动伸缩。
5. **存储编排**：自动挂载本地磁盘、云盘、网络存储等各种存储系统，让有状态应用也能从容运行。
6. **自动发布与回滚**：支持滚动更新、灰度发布，出问题可以一键回滚到旧版本。
7. **密钥与配置管理**：把配置和敏感信息（密码、证书）与镜像解耦，不用重新打包镜像就能更新配置。
8. **批处理执行**：管理定时任务和一次性任务，替换失败的任务、追踪任务进度。

你会发现，这八件事几乎覆盖了运维一个应用的全部生命周期。这也是为什么 Kubernetes 被称为"云原生的操作系统"——它之于容器，就像 Linux 之于进程。

1.9 Kubernetes 不是什么

为了避免期望错位，我们还需要澄清几个常见的误解：

Kubernetes 不是虚拟机，也不是 IaaS（基础设施即服务）。 它不提供虚拟化的操作系统，它管理的是容器这个更轻量的单元。你仍然需要底层有真实的或虚拟的机器来运行它。

Kubernetes 不是 PaaS（平台即服务）的完全替代品。 传统的 PaaS 如 Heroku，提供的是"把代码推上去就能跑"的极致体验；而 Kubernetes 把更多的控制权交给你，也把更多的责任交给你。它更接近"一个搭建 PaaS 的积木"，而不是"开箱即用的 PaaS"。

Kubernetes 不限制应用的类型。 无状态服务、有状态服务、批处理任务、甚至数据库，都可以在它上面运行——前提是应用被容器化。

Kubernetes 不替你写代码，也不重新发明应用。 它不改变你的业务逻辑，只是改变了应用的"部署和运行方式"。如果你的应用本身写得不好，Kubernetes 也无能为力。

> **提示：** 理解"Kubernetes 是什么、不是什么"，能帮你建立正确的预期。很多对 Kubernetes 的失望，其实源于把它当成了它本来不是的东西。

1.10 本章小结

这一章，我们用一个跨越半个世纪的类比，回答了"为什么需要 Kubernetes"：

- **集装箱**通过标准化，解决了货物运输的混乱；**容器**通过打包应用和其环境，解决了软件交付的混乱。
- 当容器数量从几个增长到成百上千，就需要一个**自动化的调度和管理系统**，这就是 **Kubernetes**——一个容器编排平台。
- Kubernetes 提供八大核心能力：服务发现、自动装箱、自愈、伸缩、存储编排、发布回滚、配置管理、批处理。
- Kubernetes 不是虚拟机，不是传统 PaaS 的替代品，它把控制权交给你，也把运维的责任交给你。

带着这个整体认知，我们进入下一章，看看"云原生"这艘大船，到底要驶向何方。

第2章 云原生的地图：容器、编排与这套新玩法

上一章我们回答了"为什么需要 Kubernetes"，这一章我们要绘制一张更大的地图——**云原生（Cloud Native）**。Kubernetes 是这张地图的中心，但绝不是全部。理解了云原生的整体图景，你才能明白 Kubernetes 在技术版图中的位置，以及为什么整个行业都在往这个方向迁移。

2.1 什么是云原生：不是一个产品，而是一套方法论

"云原生"这个词被用得太多了，以至于它快成了一个"什么都能装的筐"。要理解它，最权威的参照是 CNCF（云原生计算基金会）的定义。这个定义虽然绕口，但值得细读：

> 云原生技术有利于各组织在公有云、私有云和混合云等新型动态环境中，构建和运行可弹性扩展的应用。云原生的代表技术包括容器、服务网格、微服务、不可变基础设施和声明式 API。

我们把这句话拆开看，云原生其实包含几个关键词：**弹性扩展、容器化、微服务、不可变基础设施、声明式 API**。这些不是五个孤立的技术，而是相互支撑的一套体系。

> **比喻：** 如果把构建现代软件系统比作建造一座城市，那么容器就是标准化的预制构件，微服务是功能各异的建筑，Kubernetes 是市政管理系统，服务网格是城市的交通与通信网络，而声明式 API 就是城市规划图——你画好图纸，市政系统负责把城市建设到位。

理解云原生最快的方式，是把它和它的对立面——"传统架构"做对比。

2.2 传统架构 vs 云原生：一次彻底的范式转换

2.2.1 从"养宠物"到"养牛"

业界有一个经典的比喻来说明这种转变：**宠物 vs 牛羊**。

传统运维方式下，服务器是"宠物"：每台机器都有名字（比如 web-prod-01），都有独特的配置，运维人员对每台机器了如指掌。机器生病了，运维会像照顾宠物一样，登录上去"医治"它——排查问题、修复依赖、打补丁。机器坏了，大家都很难受，因为没人说得清这台机器上到底跑了什么，重新搭一遍要几天。

云原生方式下，服务器是"牛羊"：每台机器只有编号（node-3f8a、node-9c21），配置完全一致，没有任何一台是特别的。机器"病"了？直接杀掉，换一头新的上来——因为所有状态都在外部系统里（镜像仓库、持久化存储、配置中心），机器本身是"可抛弃"的（disposable）。

这个转变看似冷酷，实则高效：**当你不再心疼任何一台机器时，你才真正拥有了自动化的可能**。Kubernetes 的整个设计哲学，都建立在"节点和容器是可抛弃的"这个前提之上。

2.2.2 从可变基础设施到不可变基础设施

与"养宠物"相伴的，是"可变基础设施"（Mutable Infrastructure）：服务器买回来装好系统后，会随着时间推移被不断修改——装新软件、改配置、打安全补丁。日子久了，没有任何两台机器是完全一样的，文档永远赶不上变化，出问题时"这台机器到底经历了什么"成了未解之谜。

云原生推崇的是**不可变基础设施**（Immutable Infrastructure）：基础设施一旦部署就不再修改。要更新？不是在旧机器上打补丁，而是构建一个新镜像、部署一批新容器，然后把旧的替换掉。任何一次变更都有清晰的版本记录，任何一台出问题的机器都可以被原样重建。

你可能会问：那数据库的数据怎么办？数据总不能"不可变"吧？没错——这正是 Kubernetes 把"计算"与"状态"分离的原因：容器本身无状态、可抛弃，数据通过 Volume 和持久化存储外挂到容器之外。这也是为什么"有状态应用上 K8s"是全书要专门花笔墨讲的难题（详见第 8 章和第 14 章）。

2.2.3 从命令式到声明式

传统运维是**命令式**（Imperative）的：你写一个脚本，第一步做什么、第二步做什么，脚本按顺序执行。如果脚本执行到一半失败了，再跑一遍可能会出各种奇怪的问题（比如目录已经建了、包已经装了）。

云原生推崇**声明式**（Declarative）：你不描述"怎么做"，只描述"要什么结果"。系统持续对比"当前实际状态"和"你期望的状态"，只要发现不一致，就自动采取措施把实际状态"掰"回期望状态。

> **比喻：** 命令式像是你指挥司机："直行 500 米，左转，再右转，靠边停车"；声明式像是你直接说导航目的地："我要去机场"。路上的拥堵、绕行、红绿灯，导航系统自己会处理。

Kubernetes 是声明式理念的集大成者：你提交一个 YAML，声明"我要 3 个副本"，Kubernetes 里一个叫"控制器"的组件会永远盯着集群，确保副本数始终是 3——少了一个就补一个，多了一个就杀掉一个。哪怕有人手滑删掉了一个 Pod，它也会立刻被重建。这套"控制循环"（Control Loop）机制，是理解 Kubernetes 的一把总钥匙，第 5、6 章会深入展开。

2.3 微服务：为什么应用要拆开

云原生的另一个支柱是**微服务架构**（Microservices）。在聊它之前，先聊聊它的前身——单体架构（Monolith）。

2.3.1 单体架构：一栋什么功能都有的大楼

早期应用大多是单体架构：所有功能模块——用户管理、订单、支付、报表——都写在一个代码库里，编译成一个程序，部署在一台（或几台一样的）服务器上。

单体架构并非一无是处。它简单直接：开发调试容易，一个进程内函数调用性能极高，部署也只要拷一个包。对于小团队、小项目，单体依然是合理的甚至更优的选择。

但当业务规模增长后，单体的痛点会逐渐显现：

- **发布风险大**：改一行报表的代码，也要整个应用重新发布，任何一处 bug 都可能拖垮整个系统。
- **团队协作拥堵**：几十个工程师在一个代码库里工作，合并冲突、互相踩脚是家常便饭。
- **技术栈被锁死**：想用新语言、新框架？对不起，整个应用得一起换。
- **伸缩不灵活**：只有订单模块是热点，却不得不把整个应用复制多份，资源浪费严重。

2.3.2 微服务：按业务拆成一栋栋小楼

微服务架构把应用按业务边界拆成一组独立的小服务：用户服务、订单服务、支付服务、库存服务……每个服务独立开发、独立部署、独立伸缩，服务之间通过网络接口（通常是 HTTP/gRPC）通信。

微服务与容器、Kubernetes 是"天作之合"：每个微服务打包成一个容器镜像，天然隔离、独立发布；而管理几十上百个微服务容器，恰恰是 Kubernetes 的看家本领。

但要清醒地认识到：**微服务不是免费的午餐**。服务拆细之后，分布式系统的一系列难题随之而来——网络会失败、请求会超时、数据要保证一致性、一次用户请求可能跨越十几个服务导致排查问题困难重重。服务拆分的粒度要依团队规模和业务复杂度而定，盲目微服务化往往"拿起分布式系统的全部麻烦，却没换来它任何好处"。

2.3.3 十二要素应用：云原生应用的"宪法"

2011 年，Heroku 的工程师们总结了一套开发云原生应用的方法论，称为"十二要素应用"（The Twelve-Factor App）。它是微服务和云原生架构的理论基石之一，到今天依然非常值得参考。我们逐条过一遍，并标注它与 Kubernetes 的对应关系：

#	要素	含义	在 Kubernetes 中的体现
1	基准代码	一份代码库，多次部署	一份代码构建一个镜像，可部署到任意环境
2	依赖	显式声明依赖	镜像内打包全部依赖，依赖清单入库
3	配置	配置与代码分离，存于环境	ConfigMap / Secret（第 15 章）
4	后端服务	数据库、缓存等视为附加资源	Service + Volume，通过环境变量/DNS 访问
5	构建、发布、运行	严格分离三个阶段	CI 构建镜像 → 推送仓库 → K8s 拉取运行
6	进程	应用是无状态进程	容器即进程，状态外置（第 13、14 章）
7	端口绑定	应用自带 HTTP 服务器，不依赖外部容器	容器监听端口，Service 暴露（第 10 章）
8	并发	通过进程水平扩展	副本数、HPA 自动扩缩容（第 19 章）
9	易处理	进程可快速启动、优雅退出	探针、优雅终止、preStop 钩子（第 4 章）
10	开发环境与线上环境等价	缩小环境差异	同一镜像跑遍所有环境
11	日志	日志作为事件流输出到 stdout	容器日志收集（第 20 章）
12	管理进程	一次性管理任务与常驻进程同构	Job / CronJob（第 9 章）

> **提示：** 如果你正在开发新的服务，把这份表格当作自查清单，会让你的应用天然适配 Kubernetes，省去大量改造工作。其中第 3、6、9、11 条是最容易踩坑的，值得特别留意。

2.4 云原生技术版图：Kubernetes 周围的"星空"

Kubernetes 是云原生版图的核心，但围绕它还有一整圈配套技术。CNCF 维护的"云原生全景图"（Landscape）上有上千个项目，初学者很容易迷失。我们不妨按功能把它们归入几个大类，这样地图就清晰了：

运行时层——容器怎么跑

- **containerd**：Docker 捐给 CNCF 的容器运行时，如今是 Kubernetes 最主流的运行时。
- **CRI-O**：Red Hat 主导的轻量运行时，专为 Kubernetes 设计。

> **深入：** 你可能听说过 Kubernetes 弃用 Docker 的新闻。准确地说，弃用的是 dockershim（K8s 内嵌的 Docker 适配层），而不是"不能再用 Docker 了"。Docker 构建的镜像完全兼容，只是节点上不再直接运行 Docker 守护进程，改由 containerd 等符合 CRI 标准的运行时接手。第 3 章会展开讲。

网络与存储层——容器怎么通信、数据放哪里

- **CNI 插件**：Calico、Cilium、Flannel 等，负责 Pod 网络的连通与策略（第 12 章）。
- **CSI 驱动**：对接各类存储系统，如 Ceph（Rook）、各类云盘（第 14 章）。

镜像与交付层——应用怎么分发

- **Harbor**：企业级镜像仓库，支持镜像扫描、签名。
- **Helm**：Kubernetes 的"包管理器"（第 23 章）。
- **ArgoCD / Flux**：GitOps 持续交付工具（第 24 章）。

可观测性层——系统状态看得见

- **Prometheus**：指标监控的事实标准。
- **Grafana**：可视化仪表盘。
- **Loki / EFK**：日志收集与分析（第 20 章）。

安全层

- **cert-manager**：证书自动签发与轮换。
- **OPA（Open Policy Agent）**：统一策略引擎。
- **Falco**：运行时异常行为检测（第 17 章）。

服务网格层

- **Istio、Linkerd**：在应用之外接管服务间流量管理、加密与可观测性（第 26 章）。

不必被这个清单吓到——**Kubernetes 本体才是地基**，其他工具都是在它的地基上按需加盖的楼层。本书会带你把地基打牢，再逐层介绍这些"楼层"。

2.5 上 Kubernetes 之前，先问自己几个问题

作为一本务实的指南，我们有必要在兴奋地拥抱新技术之前，泼一点理性的冷水。Kubernetes 功能强大，但它也带来了实实在在的复杂度。以下问题值得每个团队在"上 K8s"之前认真回答：

1. **应用容器化了吗？** Kubernetes 编排的是容器。如果你的应用还是传统部署方式、依赖复杂且难以容器化，先解决容器化的问题。
2. **团队规模和业务复杂度配得上这套复杂度吗？** 如果只有两三个服务、一台服务器就够用，Docker Compose 可能是更经济的选择。Kubernetes 的价值随服务数量、团队规模、变更频率的增长而增长。
3. **运维能力跟得上吗？** 引入 Kubernetes 意味着要维护一个分布式系统——etcd 要备份、证书要轮换、版本要升级、网络要排障。用云厂商的托管服务（EKS、ACK、TKE、GKE）可以大幅降低门槛。
4. **成本账算过吗？** 除了机器成本，还有学习成本、迁移成本、机会成本。

> **注意：** "别人都在用所以我也要用"是最糟糕的上云理由。Kubernetes 解决的是规模化问题，规模不到，它的收益体现不出来，复杂度倒是先到了。反过来，当你的服务数超过十个、发布频率超过每天一次、或者经常需要跨机器调度资源时，Kubernetes 的价值就会迅速凸显。

2.6 本章小结

这一章我们绘制了云原生的整体地图：

- **云原生**是一套方法论：容器化、微服务、不可变基础设施、声明式 API，共同支撑"弹性扩展"这一终极目标。
- 从"养宠物"到"养牛羊"、从可变到不可变、从命令式到声明式，是三个最重要的范式转换，Kubernetes 的设计哲学全部建立在这三个转换之上。
- **微服务**拆分了应用，容器打包了微服务，Kubernetes 编排了容器，三者是相互成就的"铁三角"。
- 十二要素应用是云原生应用的"宪法"，其中配置外置、无状态进程、日志输出 stdout、优雅退出这几条与 Kubernetes 关系最密切。
- CNCF 生态浩如烟海，但地基只有一块——**Kubernetes 本体**。其余工具按需取用即可。

地图已经铺开，下一章，我们登上 Kubernetes 这艘航母，看看它的内部构造：控制平面、工作节点、以及那个存放一切"图纸"的 etcd。

第3章 集群、节点与控制平面：一艘航母的构造

从这一章开始，我们正式进入 Kubernetes 的内部。本章的目标，是让你建立起 Kubernetes 集群的整体架构图景：一个集群由哪几部分组成，每个部分负责什么，它们之间如何协作。搞懂了这一章，后面所有的概念都有了安放的位置。

> **比喻：** 我们不妨把一个 Kubernetes 集群想象成一艘航空母舰。控制平面是"舰桥"，里面坐着舰长、航海长、作战指挥——他们负责决策，不亲自搬运货物；工作节点是"甲板"，是集装箱真正堆放和装卸的地方；etcd 是"航海日志库"，记录着舰队的一切状态；而 kubelet，则是每个甲板区段的"工头"，负责把舰桥的命令落到实处。

3.1 集群与节点：航母与甲板

一个 **Kubernetes 集群（Cluster）** 由一组机器组成，这些机器在 Kubernetes 的术语里叫**节点（Node）**。节点分为两类角色：

- **控制平面节点（Control Plane Node）**：运行 Kubernetes 的"大脑"组件，负责决策和管理。
- **工作节点（Worker Node）**：运行用户真正的工作负载——也就是一个个 Pod。

在小型集群中，控制平面和工作节点可以混部在同样的机器上；在生产环境中，控制平面通常至少部署三台做高可用，与工作节点分开。

当你执行 `kubectl get nodes` 时，看到的就是集群中的所有节点：

```
$ kubectl get nodes
NAME           STATUS   ROLES           AGE   VERSION
k8s-master-1   Ready    control-plane   30d   v1.29.4
k8s-worker-1   Ready    worker          30d   v1.29.4
k8s-worker-2   Ready    worker          30d   v1.29.4
k8s-worker-3   Ready    worker          12d   v1.29.4
```

注意 `VERSION` 这一列——节点上运行的是 kubelet 的版本，而控制平面的版本可能与之略有差异（Kubernetes 允许 kubelet 版本略落后于控制平面，但最多落后两个小版本）。

3.1.1 Node 资源对象：机器在 Kubernetes 里的"户口"

有一个容易混淆的点：**Node 既是真实机器，又是 Kubernetes 里的一种资源对象**。kubelet 会把机器的信息——CPU、内存、操作系统、运行时版本、调度约束——注册成集群里的一个 Node 对象。这个对象就是机器在集群里的"户口"。

查看一个节点的详细信息，你会看到机器的"体检报告"：

```
$ kubectl describe node k8s-worker-1
```

输出中（节选）有几段特别值得看：

```
Capacity:           # 机器的全部资源
  cpu:              8
  memory:           16384628Ki
  pods:             110
Allocatable:        # 扣除系统预留后，可供 Pod 使用的资源
  cpu:              7820m
  memory:           15274948Ki
  pods:             110
Conditions:         # 健康状况
  MemoryPressure    False
  DiskPressure      False
  PIDPressure       False
  Ready             True
```

- `Capacity` 与 `Allocatable` 之差，是 kubelet、系统进程等预留的部分。每个节点默认最多能跑 110 个 Pod（这是可配置的，但默认值背后的考量——IP 地址分配、性能上限——值得了解）。
- `Conditions` 里的 `MemoryPressure`、`DiskPressure` 等是节点自我汇报的健康信号，调度器会参考它们避开"亚健康"的节点。

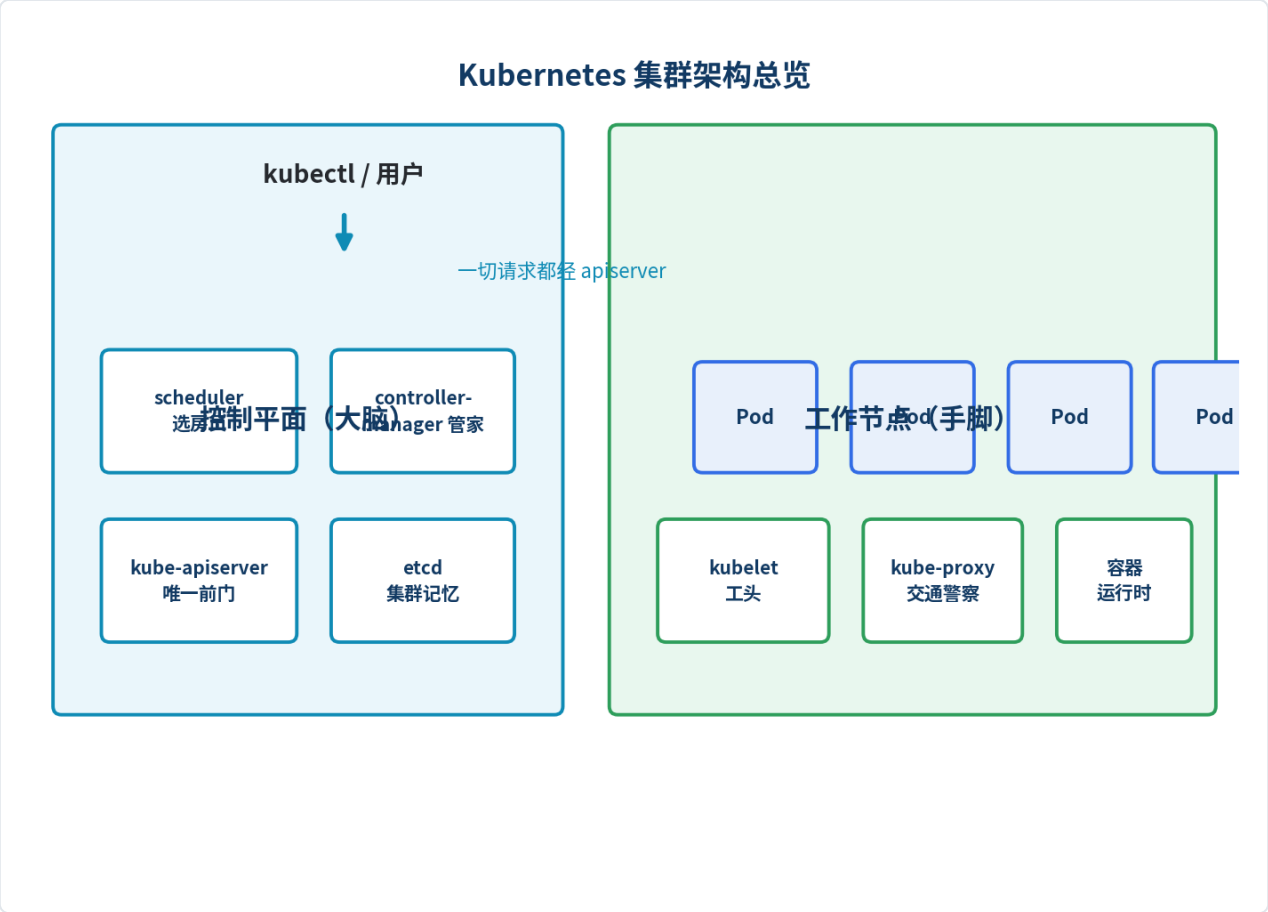


图3-1 Kubernetes 集群架构总览：控制平面与工作节点的组件分工

3.2 控制平面：舰桥上的五位核心成员

控制平面由一组组件构成，它们共同决定了"集群应该是什么状态"以及"如何达到这个状态"。我们逐个认识。

3.2.1 kube-apiserver：唯一的对外窗口

kube-apiserver 是控制平面的核心，也是整个集群唯一的"前门"。无论是你执行 `kubectl` 命令、仪表盘操作，还是集群内部组件之间的通信，**一切都要经过 apiserver**。

apiserver 的三大职责：

1. **提供 REST API**：集群里所有资源（Pod、Service、Node……）都表现为 RESTful 资源，通过标准的 HTTP 方法增删改查。
2. **认证与授权**：每个请求都要"验明正身"，并检查是否有权限执行该操作（第 16 章的 RBAC 就发生在这一层）。
3. **数据持久化**：apiserver 把所有资源的最终状态写入 etcd。**注意：apiserver 是唯一能读写 etcd 的组件**。

> **深入：** apiserver 本身是无状态的，可以水平扩展多副本。集群的高可用，很大程度上就是 apiserver 和 etcd 的高可用。生产集群 apiserver 出问题，等于舰桥失联——所有 `kubectl` 命令都会失败，但已运行的 Pod 通常不受影响（这是一个重要的"减震"特性，稍后详述）。

3.2.2 etcd：集群的记忆中枢

etcd 是一个分布式键值存储（基于 Raft 一致性协议），它保存着集群的**全部状态**：所有 Pod、Service、ConfigMap 的定义，所有 Secret 的内容，所有节点的注册信息……可以毫不夸张地说：

> **etcd 里的数据，就是集群本身。丢了 etcd，就丢了整个集群。**

这带来两个重要的运维启示：

第一，**备份 etcd 是最高优先级的运维任务**。etcd 数据丢了，恢复的唯一手段是备份（详见第 22 章）。

第二，**不要让 etcd 过载**。etcd 的性能对集群整体表现影响巨大。频繁的 List 全量资源、超大对象的频繁更新，都可能把 etcd 压垮。这就是为什么大集群要谨慎使用 `kubectl get pods --all-namespaces` 这类全量操作，也是为什么社区持续优化"Watch 缓存"机制。

3.2.3 kube-scheduler：选房系统

kube-scheduler 负责一件事：**为新创建的 Pod 挑选一个合适的节点**。

注意，调度器只"决策"，不"执行"——它决定"这个 Pod 应该去 node-2"，然后把决策结果（通过绑定 Pod 与 Node 的关系）写回 apiserver，真正把 Pod 拉起来的是节点上的 kubelet。

调度的过程像一个两轮筛选的选房流程（第 18 章会深入展开）：

1. **过滤（Filtering）**：淘汰所有不满足条件的节点——资源不够的、端口冲突的、污点不容忍的、亲和性不匹配的。
2. **打分（Scoring）**：对剩下的节点打分排序——资源更均衡的、镜像已存在的、亲和更近的优先。

最终得分最高的节点胜出。

3.2.4 kube-controller-manager：执行"期望状态"的总管家

kube-controller-manager 是一组控制器的集合，每个控制器都是一个独立的"控制循环"，负责让集群的某一方面趋向期望状态：

- **Node 控制器**：监测节点健康，节点失联超时后驱逐其上的 Pod。
- **Deployment 控制器**：确保 Deployment 声明的副本数始终成立。

- **ReplicaSet 控制器**：确保 ReplicaSet 声明的 Pod 数量成立。
- **Job 控制器**：确保批处理任务完成。
- ……

> **比喻：** 如果说 apiserver 是"前台登记处"，scheduler 是"选房员"，那么 controller-manager 就是一位不知疲倦的"总管家"：他每隔一段时间就巡视一遍（Reconcile），发现"现实和期望不符"就立即纠正。家里少了个花瓶，他补上；多了个杂物，他收走。第 5 章讲声明式 API 时，我们会看到这套机制的精妙之处。

3.2.5 cloud-controller-manager（可选）

如果集群跑在公有云上，还有 **cloud-controller-manager** 负责对接云厂商的 API：创建 LoadBalancer 类型的 Service 时自动开通云负载均衡器、节点失联时更新云上的路由等等。自建集群则没有这个组件。

3.3 工作节点：甲板上的四位干将

每个工作节点上运行着一组组件，它们才是"亲手干活"的人。

3.3.1 kubelet：节点上的工头

kubelet 是每个节点上的核心代理，负责：

- **监视分配到本节点的 Pod**：kubelet 通过 apiserver 的 Watch 机制，实时获知"哪些 Pod 应该在本节点运行"。
- **管理容器生命周期**：调用容器运行时创建、启动、停止容器；执行健康检查探针（第 4 章）；挂载存储卷（第 13 章）。
- **汇报节点状态**：定期上报节点的资源使用、健康状态。

kubelet 是"声明式"理念在节点上的落点：它不关心 Pod 是谁创建的、为什么创建，它只负责"让本节点上的 Pod 呈现 PodSpec 里声明的状态"。

3.3.2 容器运行时：真正拉起容器的人

kubelet 自己不会创建容器，它把工作委托给**容器运行时（Container Runtime）**，两者之间通过 **CRI（Container Runtime Interface）** gRPC 接口通信。

主流的运行时是 **containerd**——它原本是 Docker 内部的核心组件，后来独立出来成为 CNCF 项目。你执行 `crictl ps` 可以直接查看节点上的容器：

```
$ crictl ps
CONTAINER      IMAGE                          STATE    NAME
a1b2c3d4e5f6   nginx:1.25                     Running  web-7d9c8b6f5-xk2p9
...
```

> **深入：** Kubernetes 1.24 版本正式移除了 dockershim。这不是"Docker 不能用了"——Docker 构建的镜像依然完全兼容（因为镜像遵循 OCI 标准），只是节点上的运行时从 Docker 换成了 containerd。对绝大多数用户而言，迁移是无感的；只有那些直接依赖 Docker 特有功能（如在 Pod 里执行 `docker build`）的做法才需要调整。

3.3.3 kube-proxy：节点上的交通警察

kube-proxy 在每个节点上运行，负责实现 **Service 的虚拟 IP 与负载均衡**。当 Pod A 访问 Service 的 ClusterIP 时，是 kube-proxy 把流量"劫持"并转发到某个真实的后端 Pod。它的实现方式（iptables、IPVS 模式）与性能差异，是第 10 章的重点内容。

3.3.4 容器网络接口（CNI）插件

CNI 插件（如 Calico、Cilium、Flannel）负责给 Pod 分配 IP 地址、打通节点间的 Pod 网络。Kubernetes 有个著名的网络模型约束："**所有 Pod 之间无需 NAT 即可直接互通**"。如何实现这个约束，是 CNI 插件的职责，第 12 章会深入拆解。

3.4 一张全景图：一次请求的生命周期

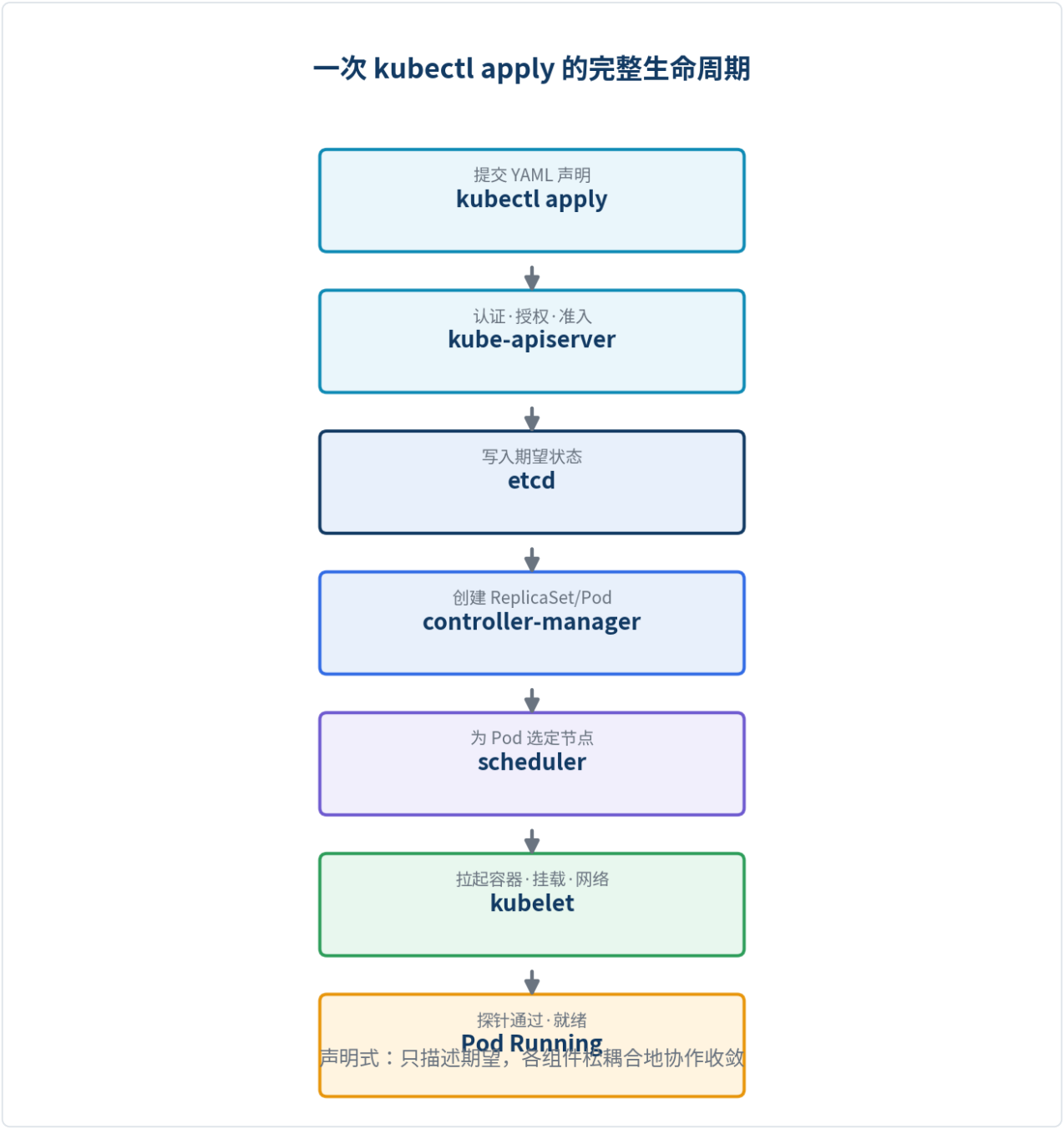


图3-2 一次 kubectl apply 的完整生命周期：各组件松耦合地协作推进状态

把所有组件拼起来，我们走一遍完整的流程——**"你执行 `kubectl apply -f deployment.yaml` 之后，集群里发生了什么"**：

1. **kubectl** 将 YAML 发送给 **kube-apiserver**（经过认证、授权、准入控制）。
2. **apiserver** 校验请求，将 Deployment 对象写入 **etcd**。此刻，"期望状态"已登记，但还没有任何 Pod 被创建。

3. **Deployment 控制器**（在 controller-manager 里）Watch 到新 Deployment，创建对应的 **ReplicaSet** 对象。
4. **ReplicaSet 控制器**发现"期望 3 个副本，实际 0 个"，创建 3 个 **Pod 对象**——注意，此时 Pod 还只是 etcd 里的记录，`kubectl get pods` 会看到它们状态为 `Pending`。
5. **kube-scheduler** 监视到未被调度的 Pod，经过过滤和打分，为每个 Pod 选定节点，把调度结果（`nodeName`）写回 apiserver。
6. 对应节点上的 **kubelet** Watch 到"有 Pod 分配到了本节点"，调用 **容器运行时（containerd）** 拉取镜像、创建容器，并挂载存储、配置网络。
7. kubelet 执行**健康检查探针**，Pod 状态从 `Pending` → `ContainerCreating` → `Running`。
8. **kube-proxy** 与 CNI 插件配合，更新网络规则，使 Service 能把流量导向新 Pod。

整个过程，没有任何组件"命令"另一个组件"你去创建 Pod X"。每个组件只 Watch 自己关心的资源，各司其职地推进状态。这种**松耦合、基于消息（资源变更事件）驱动**的架构，正是 Kubernetes 能容纳如此庞杂生态的原因——任何人都可以写一个控制器，Watch 某种资源，实现自己的逻辑，而不需要改动 Kubernetes 的任何核心代码。第 25 章讲 Operator 时，你会看到这个特性如何被发挥到极致。

3.5 控制平面失联时：为什么业务还在跑

一个值得单独强调的架构特性：**控制平面组件和工作节点是松耦合的**。

kubelet、kube-proxy、容器运行时都运行在每个节点上，Pod 的运行不依赖 apiserver 实时在线。所以当 apiserver 或 etcd 故障时：

- **已运行的 Pod 继续运行**，业务流量照常进出（kube-proxy 的转发规则已在内核里生效）；
- 但**一切变更都会停摆**：无法发布新版本、无法扩缩容、挂掉的 Pod 不会自动重建、调度停止。

这就像舰桥暂时失灵，但甲板上的水手仍在各司其职。这个特性给了我们宝贵的缓冲时间——apiserver 故障不等于业务灾难，可以冷静修复。但它也提醒我们：**控制平面的可用性，决定了集群"自愈能力"的可用性**。自愈本身也需要舰桥在线才能运转。

3.6 本章小结

- **集群** = 控制平面（大脑）+ 工作节点（手脚）。
- 控制平面五大组件：**apiserver**（唯一前门）、**etcd**（唯一的事实来源，务必备份）、**scheduler**（选房员）、**controller-manager**（总管家，控制循环的集合）、**cloud-controller-manager**（云端适配，可选）。
- 节点四大组件：**kubelet**（工头）、**容器运行时**（拉起容器）、**kube-proxy**（Service 流量转发）、**CNI 插件**（Pod 网络）。
- 一切变更的路径都是：提交期望状态 → 控制器发现差异 → 调度器分配节点 → kubelet 落地执行。

· 控制平面与节点松耦合：apiserver 故障时业务仍在运行，但变更和自愈全部停摆。

理解了这艘航母的构造，下一章我们钻进最小的"舱室"——Pod，看看这个 Kubernetes 世界里最核心、也最容易误解的对象，到底是怎么工作的。

第4章 Pod：最小的舱室，最大的学问

如果说 Kubernetes 世界里只能选一个概念来学，那一定是 **Pod**。它是 Kubernetes 调度和管理的最小单元，是所有工作负载的载体。但 Pod 也是初学者最容易产生误解的概念——很多人以为"Pod 就是一个容器"，或者"Pod 就是一个虚拟机"，这些理解都会在后续学习中埋下隐患。

这一章，我们把 Pod 从里到外彻底拆解一遍。

4.1 为什么是 Pod，而不是直接管理容器

首先要回答一个"为什么"层面的问题：Kubernetes 为什么要发明 Pod 这个概念，而不是直接调度一个个容器？

原因有三个，层层递进。

第一，有些进程天生就是"绑在一起"的。 典型例子是一个 Web 应用和它的日志收集器：Web 进程把日志写到共享目录，日志收集进程实时读取并上传。它们必须共享存储，其中一个挂了最好一起重启。如果 Kubernetes 以容器为最小调度单元，这两个容器就可能在调度器"自由意志"下被拆到两台机器上——协作就无从谈起了。Pod 把一组"必须同生共死"的容器绑定为一个调度单元，彻底解决了这个问题。

第二，一组进程需要共享运行环境。 同一 Pod 内的容器共享同一个网络命名空间——它们有相同的 IP 地址、相同的端口空间，可以通过 `localhost` 直接互访，就像同一台机器上的两个进程。它们也可以共享存储卷。这种"共享环境、隔离进程"的模式，非常符合 Unix 系统的传统设计哲学。

第三，为未来留出扩展空间。 一个 Pod 里可以运行"主容器 + 辅助容器"的组合模式（sidecar 模式），比如：主容器跑业务，辅助容器负责采集监控指标、同步配置、做流量代理。这种模式让业务镜像保持纯粹，把通用能力下沉到辅助容器。

> **比喻：** 容器是一个个"船员"，Pod 是给船员分配的"舱室"。有的舱室只住一个人（单容器 Pod），有的舱室住几个人（多容器 Pod）——住一起的船员共享同一部内线电话（IP 地址）和同一个储物柜（存储卷），因为他们的工作必须紧密协作。调度器调度的最小单位是"舱室"，而不是"船员"。

4.2 Pod 的解剖：共享了什么，隔离了什么



图4-1 Pod 内多容器共享网络命名空间、各自隔离文件系统

理解 Pod 的关键是搞清楚：**Pod 内的容器共享什么、又各自隔离什么**。

4.2.1 网络命名空间：全 Pod 共享

同一 Pod 内的所有容器：

- 共享**同一个 IP 地址**（Pod IP）；
- 共享**同一套端口空间**——容器 A 占了 80 端口，容器 B 就不能再用 80；
- 可以通过 `localhost:端口` **互相访问**——这是多容器协作的最直接方式；
- 共享同一套网络设备与路由。

Pod IP 是整个 Pod 对外的"门牌号"，Pod 内的容器对外不可见——外界只能看到 Pod 这个整体。

4.2.2 进程、文件系统：各自隔离

- 每个**容器有自己的 PID 命名空间**：容器 A 看不到容器 B 的进程。
- 每个容器有自己的**文件系统根**（基于各自的镜像），除非显式挂载共享的 Volume。
- 资源限制（CPU、内存）以容器为单位设置，但 Pod 的资源需求是所有容器之和。

> **深入：** 从 Linux 底层视角看，Pod 并不是一个"实体"，而是一组共享部分 namespace（网络、IPC）的 cgroup 和进程的集合。Kubernetes 通过 pause 容器（也叫 sandbox 容器）来"持

> 有"这些共享的 namespace：pause 容器最先启动、永远睡眠，只要它活着，Pod 的网络命名空间就不会消失，其他容器加入/退出都不影响共享环境。你在 `crictl ps` 里看到的大量 pause 镜像容器，就是它们。

4.3 编写第一个 Pod： anatomy of a YAML

现在，让我们亲手定义一个 Pod。下面这个 YAML 是 Kubernetes 世界里的"Hello World"：

```
apiVersion: v1          # API 版本：Pod 属于核心组，所以是 v1
kind: Pod               # 资源类型
metadata:
  name: web-pod         # Pod 的名字
  labels:               # 标签：键值对，用于筛选和组织
    app: web
    env: dev
spec:                   # 期望状态（PodSpec）
  containers:
  - name: nginx         # 容器名
    image: nginx:1.25   # 镜像（强烈建议带 tag，不要用 latest）
    ports:
    - containerPort: 80  # 声明容器监听的端口（仅为文档性信息）
    resources:           # 资源请求与限制（重要！）
      requests:
        cpu: 100m        # 100 毫核 = 0.1 核
        memory: 128Mi
      limits:
        cpu: 500m
        memory: 256Mi
```

把它保存为 `pod.yaml`，创建并验证：

```
$ kubectl apply -f pod.yaml
pod/web-pod created

$ kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
web-pod   1/1     Running   0          10s
```

`READY` 列的 `1/1` 表示"1 个容器就绪 / 共 1 个容器"。我们来逐段解读这个 YAML 中最重要的部分。

4.3.1 resources：requests 与 limits，K8s 世界的"电表和水表"

`resources` 是初学者最容易忽略、生产事故却最常见的配置。它分两层：

- **requests（请求值）**：声明"这个容器**至少需要**多少资源"。调度器根据 requests 来选节点——节点上"已分配"的资源总量不能超过其容量。requests 也是容器运行时的最低保障。
- **limits（上限）**：声明"这个容器**最多能**用多少资源"。超出 CPU 上限会被限流（throttle），超出内存上限会被 **OOMKill**（内核强制杀掉进程）。

`100m` 中的 m 表示 millicore（毫核），`500m` 即 0.5 个 CPU 核。内存单位 `Mi` 表示 mebibyte（1Mi = 1024×1024 字节），`Gi` 表示 gibibyte。

> **警示：** 不设置 resources 的 Pod 会以"BestEffort"（尽力而为）等级运行，节点资源紧张时第一批被驱逐；只设置 limits 不设置 requests，requests 会被自动设为与 limits 相同。生产环境请务必为每个容器设置合理的 requests/limits——具体设置方法与"QoS 等级"的关系，第 18 章会展开。

4.3.2 labels：贴在资源上的"标签纸"

`labels` 是键值对，本身没有任何功能，但它让整个 Kubernetes 系统得以"组织和检索"。你可以用标签选择器（selector）筛选一组资源：

```
$ kubectl get pods -l app=web          # 筛选 app=web 的 Pod
$ kubectl get pods -l 'env in (dev,test)'  # 更复杂的筛选
```

第 6 章你会看到，Deployment 正是通过 label selector 来认领自己管理的 Pod；第 10 章你会看到，Service 也是通过 label selector 找到后端 Pod 的。**标签是 Kubernetes 松耦合架构的"粘合剂"**——各组件不直接引用对方的名字，而是通过标签建立多对多的关联。

4.4 Pod 的生命周期：从出生到死亡

Pod 是"易逝的"（ephemeral）——这是理解 Pod 行为的关键前提。Pod 不会"自愈"：节点宕机，Pod 就没了；Pod 被删除，就是真没了。自愈是上层控制器（如 Deployment）的职责：它们发现副本少了，就创建新的 Pod——注意，是**新的** Pod，有新的名字、新的 IP，而不是把旧的"救活"。

一个 Pod 从创建到消失，会经历几个阶段，我们对照下面的命令输出理解：

```
$ kubectl get pod web-pod --watch
NAME      READY   STATUS              RESTARTS   AGE
web-pod   0/1     Pending             0          0s
web-pod   0/1     ContainerCreating   0          1s
web-pod   1/1     Running             0          5s
```

- **Pending**：Pod 已被 Kubernetes 接受，但尚未在节点上运行起来。可能是正在调度，也可能调度失败（比如资源不足、镜像拉不下来）。用 `kubectl describe pod` 查看事件可以找到原因。
- **ContainerCreating**：镜像在拉取中、容器在创建中。
- **Running**：至少一个容器在运行中（注意：Running 不等于"业务正常"，所以有了下面的探针）。
- **Succeeded / Failed**：所有容器终止且不会重启（多见于 Job 类 Pod）。
- **Unknown**：通常是与节点失联。

4.4.1 容器重启策略 restartPolicy

Pod 内的容器退出后，kubelet 依据 `restartPolicy` 决定是否重启容器：

- `Always`（默认）：容器无论如何退出都重启——适合长期运行的服务；
- `OnFailure`：仅在非零退出码时重启——适合批处理任务；
- `Never`：从不重启——适合一次性任务。

重启采用**指数退避**延迟（10s、20s、40s……最长 5 分钟），避免故障容器疯狂重启雪崩。注意 `RESTARTS` 计数：容器重启不会改变 Pod 的身份，但累计次数会显示出来。

4.4.2 优雅终止：不是"咔嚓"一下就完了

删除一个 Pod 时，背后有一套严谨的流程，理解它能帮你避免大量"发布时请求报错"的事故：

1. Pod 被标记为 `Terminating`，并从 Service 的后端端点中移除（其他 Pod 不再往它发新流量）；
2. kubelet 向容器发送 **SIGTERM** 信号，同时开始倒计时 `terminationGracePeriodSeconds`（默认 30 秒）；
3. 应用应捕获 SIGTERM，停止接收新请求、处理完存量请求后退出（"优雅退出"）；
4. 倒计时结束应用还没退出？kubelet 发送 **SIGKILL** 强杀；
5. 若 Pod 定义了 `preStop` 钩子，它会在 SIGTERM 之前执行。

一个经典的坑是：**端点移除与容器退出之间存在竞态**——Service 的规则更新需要一点时间传播，旧 Pod 可能在"已从端点移除"之后仍然收到零星请求。工程上的标准解法是在 `preStop` 中加一个短暂的 sleep（比如 5 秒），给规则传播留出时间窗：

```
spec:
  terminationGracePeriodSeconds: 45
  containers:
  - name: app
    lifecycle:
      preStop:
        exec:
          command: ["sh", "-c", "sleep 5"]   # 等端点移除传播完成
```

> **注意：** 如果你的应用不处理 SIGTERM（很多框架默认不处理），优雅终止就是一句空话，kubelet 等满 30 秒后照样强杀——在途请求全部失败。请务必确认你的应用（或其框架）正确处理了 SIGTERM。

4.5 探针：告诉 Kubernetes"我到底健不健康"

`Running` 只说明容器的主进程活着，不代表业务可用。设想一种情况：应用陷入死锁，进程还在，但所有请求都挂起——Kubernetes 默认对此毫无感知。**探针（Probe）** 就是解决这个问题的机制，它让应用主动报告健康状态。

三种探针，各司其职：

- **livenessProbe（存活探针）**：失败 → 重启容器。用于检测"死锁、僵死"这类无法自愈的故障。
- **readinessProbe（就绪探针）**：失败 → 从 Service 后端摘除。用于检测"暂时不能服务"（比如还在加载缓存、依赖的数据库暂时不可用）。**容器不会被重启**，恢复后自动回到端点。
- **startupProbe（启动探针）**：专为"启动慢"的应用设计。在它成功之前，liveness/readiness 探针不会生效——避免应用还没启动完就被存活探针误杀。

一个典型的配置：

```
spec:
  containers:
  - name: app
    image: myapp:2.1
    startupProbe:
      httpGet:
        path: /healthz
        port: 8080
      failureThreshold: 30      # 允许失败 30 次
      periodSeconds: 10         # 每 10 秒探测一次 → 最多等 5 分钟启动
    livenessProbe:
      httpGet:
        path: /healthz
        port: 8080
      periodSeconds: 10
      timeoutSeconds: 2
      failureThreshold: 3       # 连续失败 3 次才重启，避免误杀
    readinessProbe:
      httpGet:
        path: /ready            # 就绪检查通常比存活检查更严格
        port: 8080
      periodSeconds: 5
```

探针支持三种实现方式：`httpGet`（HTTP 状态码 2xx/3xx 为成功）、`tcpSocket`（端口能建立 TCP 连接为成功）、`exec`（命令退出码 0 为成功）。

4.5.1 探针设计的实践智慧

探针配置不当，轻则误杀，重则雪崩。几条来自生产实践的忠告：

- **liveness 探针要"宽松"**：只检查"进程是否还能响应"，千万别在 liveness 里检查下游依赖（比如数据库连不上就返回失败）——下游抖动会触发大面积重启，把局部故障放大成全局事故。
- **readiness 探针要"严格"**：它决定"我能不能接客"，检查依赖、检查缓存加载状态都合理，因为"不接客"比"接了做错"更安全。
- **检查接口要便宜**：探针每几秒执行一次，别让 `/healthz` 去查数据库做重操作。
- **区分存活与就绪的语义**：一个常用的分工是 `/healthz` 回答"我还活着吗"（几乎总是 true），`/ready` 回答"我现在能服务吗"。

> **提示：** 排查探针问题时，`kubectl describe pod` 的 Events 会记录探针失败详情，`kubectl get pod` 中 Pod 的重启计数（RESTARTS）突然增长，通常就是 liveness 探针在杀容器。

4.6 多容器 Pod 与 Sidecar 模式

单容器 Pod 是主流，但多容器 Pod 是 Kubernetes 的精髓之一。经典模式有：

- **Sidecar（边车）**：辅助容器增强主容器的能力，如日志收集器（filebeat/fluent-bit 读取共享卷里的日志）、本地缓存同步器。
- **Ambassador（大使）**：辅助容器代理外部服务的访问，主容器只管连 localhost。
- **Adapter（适配器）**：辅助容器把主容器的输出改造成统一格式（如监控指标格式转换）。

一个日志收集的 Sidecar 示例：

```
spec:
  containers:
  - name: app                    # 主容器：写日志到共享卷
    image: myapp:2.1
    volumeMounts:
    - name: log-volume
      mountPath: /var/log/app
  - name: log-collector          # 边车：读共享卷，转发到日志系统
    image: fluentbit:2.2
    volumeMounts:
    - name: log-volume
      mountPath: /var/log/app
      readOnly: true
  volumes:
  - name: log-volume
    emptyDir: {}
```

两个容器通过 `log-volume` 共享目录协作，这正是 Pod 存在意义的缩影。

Kubernetes 1.28+ 引入了**原生 Sidecar 支持**（`initContainers` 中设置 `restartPolicy: Always` 的容器）：这类容器在主容器启动前启动、Pod 存活期间始终运行、且其重启不影响主容器。这解决了老式 sidecar "Pod 终止时退出顺序不可控"的老大难问题。

4.7 Init 容器：先遣队

Init 容器在主容器启动**之前**按顺序运行，每个都必须成功结束，才会启动下一个；任何一个失败，按 restartPolicy 决定重试。它们适合"准备工作"：等待依赖服务就绪、生成配置文件、下载资源、做数据库 Schema 检查等。

```
spec:
  initContainers:
  - name: wait-for-db
    image: busybox:1.36
    command: ['sh', '-c', 'until nc -z db-service 5432; do echo waiting for db; sleep 2; done']
  containers:
  - name: app
    image: myapp:2.1
```

> **提示：** Init 容器与 Sidecar 的区别：Init 容器"跑完就退"，适合一次性的前置工作；Sidecar"全程陪伴"，适合持续性的辅助工作。二者可以组合使用。

4.8 操作 Pod 的常用命令

```
kubectl apply -f pod.yaml          # 创建/更新
kubectl get pods                   # 列出（-o wide 显示 IP 和节点）
kubectl describe pod web-pod       # 详细信息 + 事件（排障第一入口）
kubectl logs web-pod               # 看日志（-f 跟随，-c 指定容器）
kubectl logs web-pod -p            # 上一次崩溃容器的日志
kubectl exec -it web-pod -- sh     # 进入容器（多容器需 -c 指定）
kubectl port-forward web-pod 8080:80   # 本地端口转发，临时调试利器
kubectl delete pod web-pod         # 删除
kubectl top pod                    # 资源使用（需 metrics-server）
```

其中 `kubectl describe pod` 是排障的瑞士军刀：镜像拉取失败、探针失败、调度失败、挂载失败……几乎所有问题的第一手线索都在它的 Events 里。

4.9 本章小结

- **Pod 是调度的最小单元**，存在的意义是把"必须协作的容器"绑定在一起：共享网络命名空间（同 IP、localhost 互通）、可共享存储卷。
- Pod 是**易逝的**：节点故障即消失，自愈由上层控制器负责，新 Pod 是"新的"而非"复活的"。
- **requests/limits** 决定资源保障与上限，忘配是生产事故高发原因。
- **优雅终止**依赖应用正确处理 SIGTERM；preStop + 宽限期配置可以规避端点移除竞态。
- 三种探针的分工：**liveness 管生死、readiness 管接客、startup 管慢启动**；liveness 要宽松、readiness 要严格。
- **Init 容器**做前置准备，**Sidecar** 做全程辅助；两者组合构成了"关注点分离"的容器设计范式。

Pod 是最灵活的积木，但你不会希望手动管理成百上千个 Pod——下一章，我们先花一点时间掌握 Kubernetes 的"通用语言"（YAML 与声明式 API），然后就可以请出第一件"自动化神器"：Deployment。

第5章 用 YAML 说话：声明式 API 与资源对象

在使用 Kubernetes 之前，我们需要先掌握它的"通用语言"。这门语言有两个层面：**语法层面**是 YAML 的书写规则与资源对象的组织方式；**哲学层面**是声明式 API 与控制循环。本章把两个层面一次讲透——这是理解后续所有章节的地基。

5.1 命令式 vs 声明式：一次深刻的转变

先做一组对比实验。同一个目标——"跑一个 nginx，暴露 80 端口，3 个副本"——两种风格的命令如下：

命令式风格（类似 Docker Swarm 时代的做法）：

```
kubectl run web --image=nginx:1.25 --port=80 --replicas=3
```

声明式风格：

```
kubectl apply -f deployment.yaml
```

表面看只差了"是否写文件"，本质差异却非常大：

命令式命令描述的是"动作"——"运行这个镜像、开这个端口、给三个副本"。动作一旦执行完，系统与命令之间的关系就结束了。你无法回答"现在集群里的 web 是什么配置"这个问题，除非去翻操作记录。

声明式清单描述的是"状态"——"web 应该有 3 个副本，用 nginx:1.25 镜像"。这个文件提交后，始终是集群状态的"真实来源"（source of truth）。它躺在 Git 里，可以 diff、可以 review、可以回滚，任何人任何时候都能回答"现在应该是什么样"。

> **比喻：** 命令式像是给装修工人下达一条条口头指令："刷白墙、装吊灯、铺木地板"；声明式像是交给工人一份装修图纸。口头指令说一遍就散了，事后无人知晓；图纸始终存在，验收时对照图纸，发现问题随时改图重做。

声明式的威力在"偏离"时刻显现：假设有人手动删掉了一个 Pod，或者某天凌晨一个 Pod 因为 OOM 挂掉——**只要你提交过"3 个副本"的声明，控制器就会自动把副本数补回 3**。你睡觉的时候，Kubernetes 在替你值守。

这就是 Kubernetes 的核心循环：

```
期望状态(你声明)  ←——对比——  实际状态(集群现状)
        |                          |
        └——发现差异——→ 控制器执行动作，消除差异
```

这个循环在 Kubernetes 里被称为**调谐循环（Reconcile Loop）**，它永不停歇地运行着。

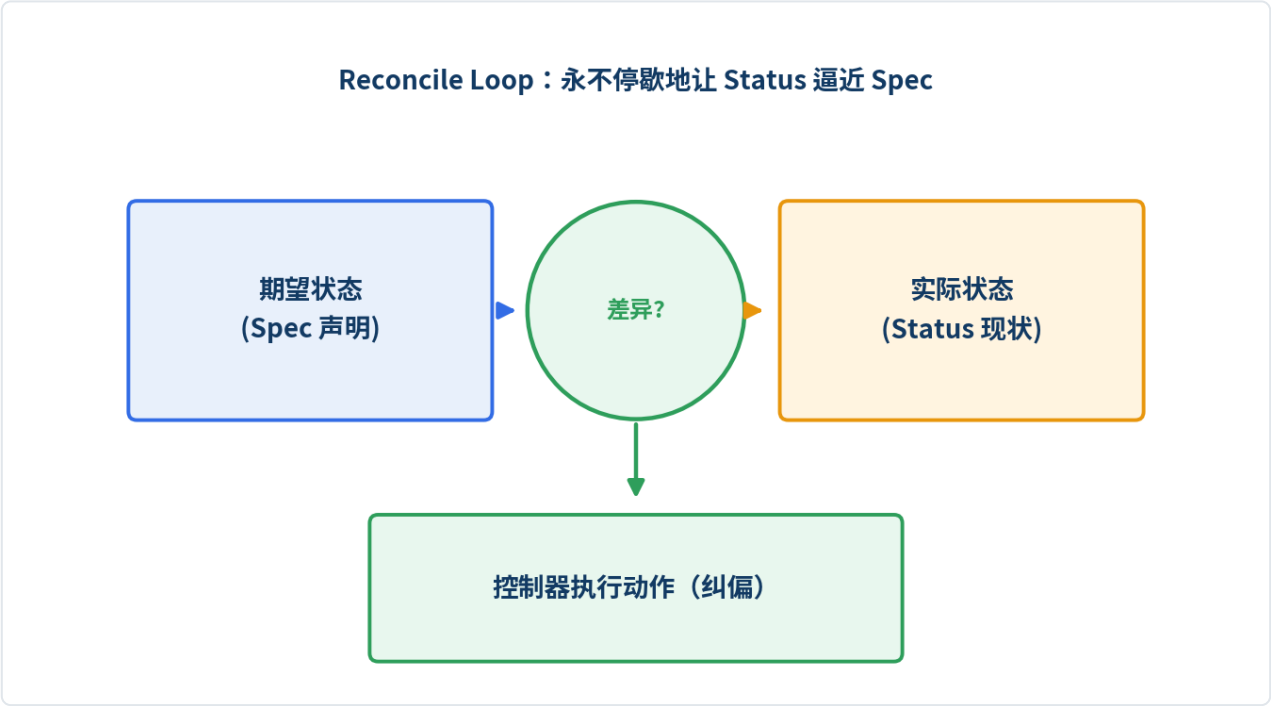


图5-1 调谐循环：控制器持续对比期望状态与实际状态并纠偏

5.2 Kubernetes 的资源对象体系

Kubernetes 里的一切都是**资源对象（Resource Object）**：Pod、Service、ConfigMap、Node……每个对象都由三部分组成：

```
apiVersion: apps/v1     # ① API 组与版本：对象定义在哪个 API 里
kind: Deployment        # ② 类型：这是什么对象
metadata:               # ③ 元数据：名字、标签、注解、所属命名空间
  name: web
spec:                   # ④ 期望状态：你希望它是什么样
  replicas: 3
  ...
status:                 # ⑤ 实际状态：系统观测到的现状（通常只读）
  readyReplicas: 3
```

- **apiVersion**：`v1` 表示核心组（Pod、Service、ConfigMap 等），其他对象分属不同 API 组，如 `apps/v1`（Deployment）、`networking.k8s.io/v1`（Ingress）。API 组机制让 Kubernetes 可以无侵入地扩展新资源（第 25 章的 CRD 就利用了这一点）。

- **spec（期望）与 status（实际）的分离**是声明式设计的精髓：你只写 spec，status 由系统填写。控制器的全部工作，就是让 status 无限逼近 spec。

5.2.1 metadata：名字、标签、注解与命名空间

name（名字）：对象在同一命名空间、同一类型下唯一。命名规范：小写字母、数字、中划线，不超过 253 字符。

labels（标签）：第 4 章已经介绍——用于**筛选与关联**。它是控制器、Service 认领资源的依据。

annotations（注解）：也是键值对，但不用于筛选，用于**携带非标识性的元信息**——构建信息、负责人、工具的私有配置（比如 Ingress 控制器的行为开关几乎都通过注解传递）。经验法则：**要用来选东西的，用 label；只是"记录点信息"的，用 annotation**。

namespace（命名空间）：把集群切分成多个虚拟空间，实现资源隔离。详见下文。

5.2.2 命名空间：集群里的"楼层"

Namespace 适合用来隔离**不同项目、不同团队、不同环境**：

```
$ kubectl create namespace team-a
$ kubectl get pods -n team-a          # 查看指定命名空间的 Pod
$ kubectl get pods -A                 # 查看所有命名空间（-A = --all-namespaces）
```

关于 namespace，有几个必须澄清的认知：

1. **namespace 不提供网络隔离**。默认情况下，不同 namespace 的 Pod 可以直接互通（通过 Service 或 Pod IP）。要限制访问，需要 NetworkPolicy（第 17 章）。
2. **namespace 不是安全边界**。跨 namespace 的 RBAC 控制能限制"谁能操作哪个空间的资源"，但它不是硬隔离，namespace 内的特权 Pod 可以威胁整个集群。
3. 有些资源是"集群级"的，**不属于任何 namespace**：Node、Namespace 本身、PersistentVolume、StorageClass、ClusterRole 等。`kubectl api-resources` 可以查看每个资源是否带命名空间（NAMESPACED 列）。
4. 配额与限额（ResourceQuota、LimitRange）按 namespace 生效——这是 namespace 在多团队集群里最重要的价值：**防止一个团队吃光整个集群**。

5.3 YAML 语法速成与避坑

Kubernetes 的 YAML 大部分是简单的键值与列表，但有些细节坑点值得单独强调。

缩进即结构：YAML 用两个空格（约定俗成）表示层级，**绝不能使用 Tab**。

列表用 `-`，映射用 `key: value`：

```
containers:              # 列表
- name: web
  image: nginx:1.25
  env:                    # 嵌套列表
  - name: LOG_LEVEL
    value: info
```

引号规则：包含特殊字符（冒号+空格、`#`、换行）的字符串需要加引号。多行文本用 `|`（保留换行）或 `>`（折叠换行）：

```
command:
- sh
- -c
- |
  echo "第一行"
  echo "第二行"
```

数值陷阱：`version: 1.0` 会被解析为浮点数，`version: 1.30` 会变成 `1.3`——对版本号、端口号等"形似数字的字符串"，请加引号 `version: "1.30"`。

锚点与合并（减少重复的高级技巧）：

```
resources: &default-res
  requests: {cpu: 100m, memory: 128Mi}
  limits:   {cpu: 500m, memory: 256Mi}
...
  resources:
    <<: *default-res        # 引用上面的定义
```

> **提示：** 在编辑器中安装 YAML 校验与 Kubernetes Schema 插件（如 YAML Language Server + kubernetes schema），可以在写错字段名时立刻标红——Kubernetes 的 API 字段数以千计，人肉记忆不现实，工具校验能省下大量"apply 报错 → 改 → 再 apply"的循环。

5.4 kubectl：与集群对话的瑞士军刀

kubectl 是官方命令行工具，几乎所有日常操作都通过它完成。掌握它的心智模型比记住所有命令更重要。

5.4.1 kubectl 的命令心智模型

kubectl 的命令可以按"对资源做什么"分为几类：

类别	典型命令	说明
声明式管理	`apply`、`delete`	提交清单文件，首选方式
命令式操作	`create`、`run`、`expose`	直接生成资源，适合临时实验
查询观测	`get`、`describe`、`logs`、`top`、`events`	看状态、找线索
调试交互	`exec`、`port-forward`、`cp`、`debug`	钻进容器里排查
修改	`edit`、`scale`、`set`、`patch`、`label`、`annotate`	就地修改（慎用于生产）
高级	`diff`、`apply --prune`、`wait`、`rollout`	精细控制

其中 `rollout` 系列（第 7 章详述）和 `patch` 值得特别关注：

```
kubectl patch deployment web -p '{"spec":{"replicas":5}}'   # 精准修改某个字段
```

`patch` 用 JSON Merge Patch 语法只改指定字段，比 `edit` 全文件修改更适合脚本化操作。

5.4.2 get 与 describe：日常最高频的组合

```
kubectl get pods -o wide          # 更多列：IP、所在节点
kubectl get pods -o yaml          # 输出完整 YAML（看 status 字段非常有用）
kubectl get deploy,svc,pvc        # 一次查多种资源
kubectl get pods --show-labels    # 显示标签
kubectl describe pod web-pod      # 事件与详情
```

一个高频排查套路：**`get` 看状态异常 → `describe` 看事件 → `logs` 看日志 → `exec` 进容器**。90% 的问题沿这条路径可以定位。

5.4.3 输出过滤：jq 思维

```
kubectl get pods -o json | jq '.items[] | {name: .metadata.name, phase: .status.phase}'
kubectl get pods --field-selector=status.phase!=Running   # 字段过滤
kubectl get pods -s custom-columns='NAME:.metadata.name,NODE:.spec.nodeName'
```

这些技巧在写运维脚本、做批量检查时威力巨大。

5.5 资源清单的组织与管理

单个 YAML 文件只能定义一个对象，但实际项目动辄十几个对象（Deployment + Service + ConfigMap + Ingress + HPA……）。有三种主流的组织方式：

方式一：多对象一个文件（用 `---` 分隔）：

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: web-config
data:
  APP_ENV: "prod"
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
...
```

`kubectl apply -f` 会按顺序创建文件中的所有对象。适合小型项目。

方式二：目录整体提交：

```
kubectl apply -f manifests/    # 目录下所有 YAML 递归应用
kubectl diff -f manifests/     # 预览变更（强烈推荐的护栏习惯）
```

方式三：模板化工具（Helm / Kustomize）：当配置需要按环境变化（dev/staging/prod 参数不同）时，纯 YAML 的复制粘贴会失控，第 23、24 章专门讲这两个工具。

> **提示：** 无论用哪种方式，请把清单文件放进 Git 仓库管理。这不仅是备份，更是审计与回滚能力的基础——"谁在什么时候改了什么配置"这个问题，只有版本控制能回答。

5.6 准入控制与默认值填充：提交之后发生了什么

`kubectl apply` 提交后、对象进入 etcd 前，apiserver 还有两道处理工序，了解它们能解释很多"咦，我的 YAML 怎么变了"的现象。

第一道：Schema 校验与默认值填充。apiserver 会校验字段合法性，并为省略的字段填上默认值。比如你没写 `restartPolicy`，提交后 `get -o yaml` 会看到它被补成了 `Always`。所以：**应用后用 `kubectl get -o yaml` 检查实际生效的完整配置，是验证假设的好习惯**。

第二道：准入控制（Admission Control）。一串插件对请求做更深入的审查甚至改写：ResourceQuota 检查命名空间配额、LimitRange 填充默认资源限制、Pod Security 检查安全上下文（第 17 章）、以及各种 Webhook（比如Istio 的 sidecar 自动注入就是通过 MutatingWebhook 实现的）。

5.7 kustomize 与"过度 Yaml 化"的隐忧

最后聊一个务实的话题。当团队把所有配置都写成 YAML 后，常见病症是：同一个 Deployment 要为 dev/staging/prod 复制三份，只改镜像 tag 和副本数；一个字段的修改要在 N 个文件里重复。这被称为"YAML 地狱"。

两条应对路线：

- **Kustomize**（kubectl 内置）：以"基础清单 + 补丁覆盖"的方式组织配置，无模板语言，纯声明式叠加。详见第 24 章。
- **Helm**：以 Go 模板的方式参数化生成 YAML，生态里有大量现成的 Chart 包。详见第 23 章。

在项目早期，不必急于引入这些工具；但当你发现"为了改一个镜像版本要改三个文件"时，就是时候了。

5.8 本章小结

- **声明式 API 是 Kubernetes 的灵魂**：你声明期望状态（spec），控制循环负责让实际状态（status）持续逼近它。清单文件是状态的"真实来源"，应纳入版本控制。
- 资源对象由 apiVersion、kind、metadata（name/labels/annotations/namespace）、spec、status 组成；label 用于筛选关联，annotation 用于携带信息。
- **Namespace 是逻辑分区**，不是安全边界、不隔离网络；集群级资源不属于任何 namespace。
- YAML 的常见坑：Tab 缩进、数字型字符串、多行文本的 `|` 与 `>`；编辑器 Schema 校验是刚需。
- kubectl 的核心心智：`apply/diff` 管变更，`get/describe/logs/exec` 管排障，`patch/scale/rollout` 管精细操作。
- apply 之后，apiserver 会做校验、填默认值、过准入控制——用 `get -o yaml` 验证"实际生效的配置"。

语言关已过。接下来，我们请出 Kubernetes 自动化能力的第一个主角——Deployment，看它如何让你的应用获得"自动驾驶"能力。

第6章 Deployment：应用的自动驾驶

从这一章起，我们进入 Kubernetes 的"工作负载"世界。第一个登场的是使用频率最高的对象——**Deployment**。绝大多数无状态服务的部署，都是从它开始的。

6.1 从手动管理 Pod 的痛苦说起

假设你要上线一个 Web 服务，需要 3 个实例来保证可用性和吞吐。如果直接管理 3 个 Pod（第 4 章的做法），你会立刻遇到这些麻烦：

- 要手动 `kubectl apply` 三份几乎相同的 YAML，只有名字不同（web-1、web-2、web-3）；
- 某个 Pod 挂了，你得自己发现、自己重建——半夜挂了怎么办？
- 要发新版本，得逐个删旧 Pod、建新 Pod，中间必然有服务空窗；
- 误删了一个 Pod？没有任何东西会帮你补回来。

你需要的，其实是一个**替你盯着 Pod 的"管理者"**：声明"我要 3 个健康的 Pod"，它负责让这句话永远成立。这个角色，在 Kubernetes 里由一组互相配合的控制器承担——**Deployment 与 ReplicaSet**。

6.2 ReplicaSet：副本数量的守护者

ReplicaSet 的职责简单而纯粹：**确保"指定标签的 Pod"始终有 N 个在运行**。

它的控制循环逻辑：

1. 通过 label selector 找到所有匹配的 Pod，数一数有几个健康的；
2. 实际数量 < 期望数量 → 创建 Pod；
3. 实际数量 > 期望数量 → 删除多余 Pod（优先删最"不健康"的）；
4. 循环往复，永不停止。

所以无论你手动删掉一个 Pod、还是某个 Pod 因 OOM 挂掉，ReplicaSet 都会在几秒内补上新的。这就是 Kubernetes "自愈"的真正来源——不是 Pod 自己自愈，而是**ReplicaSet 无休止地纠偏**。

> **比喻：** ReplicaSet 像一位严格的航班值机主管：柜台前必须始终有 3 位柜员。少一个，立刻从休息室叫一位补上——注意是叫"新人"上岗，而不是把晕倒的那位扶起来。旧 Pod 一旦死了就死了，补上来的是全新的 Pod（新名字、新 IP）。

那你可能会问：既然 ReplicaSet 已经这么强了，为什么我们日常用的都是 Deployment？答案藏在"发布"这个动作里。

6.3 Deployment：ReplicaSet 之上的"发布管理器"

ReplicaSet 只会"凑数量"，但**更新应用版本**是它的盲区。设想 nginx 从 1.25 升级到 1.26：如果让 ReplicaSet 直接把 3 个 Pod 的镜像都改掉，它只会"重建 3 个 Pod"——同一瞬间旧版本全部下线，服务中断。

Deployment 就是为解决这个问题而生的。它的设计是：不直接管理 Pod，而是管理 ReplicaSet；每次更新镜像，就创建一个"新一代" ReplicaSet，新旧 ReplicaSet 协作完成滚动替换。

三者的关系是一条清晰的链：

```
Deployment（发布策略）
    └── 管理 ReplicaSet（新版）与 旧版 ReplicaSet（保留用于回滚）
        └── 管理 Pod（真正的进程）
```

> **比喻：** Deployment 是"项目经理"，ReplicaSet 是"车间主任"，Pod 是"工人"。项目要换新版工艺时，项目经理开一条新产线（新 ReplicaSet），让新旧产线并存、逐渐把工人从旧产线调到新产线，全部调完后再保留旧产线的编制以防万一——需要回滚时，让工人全部回到旧产线就行。

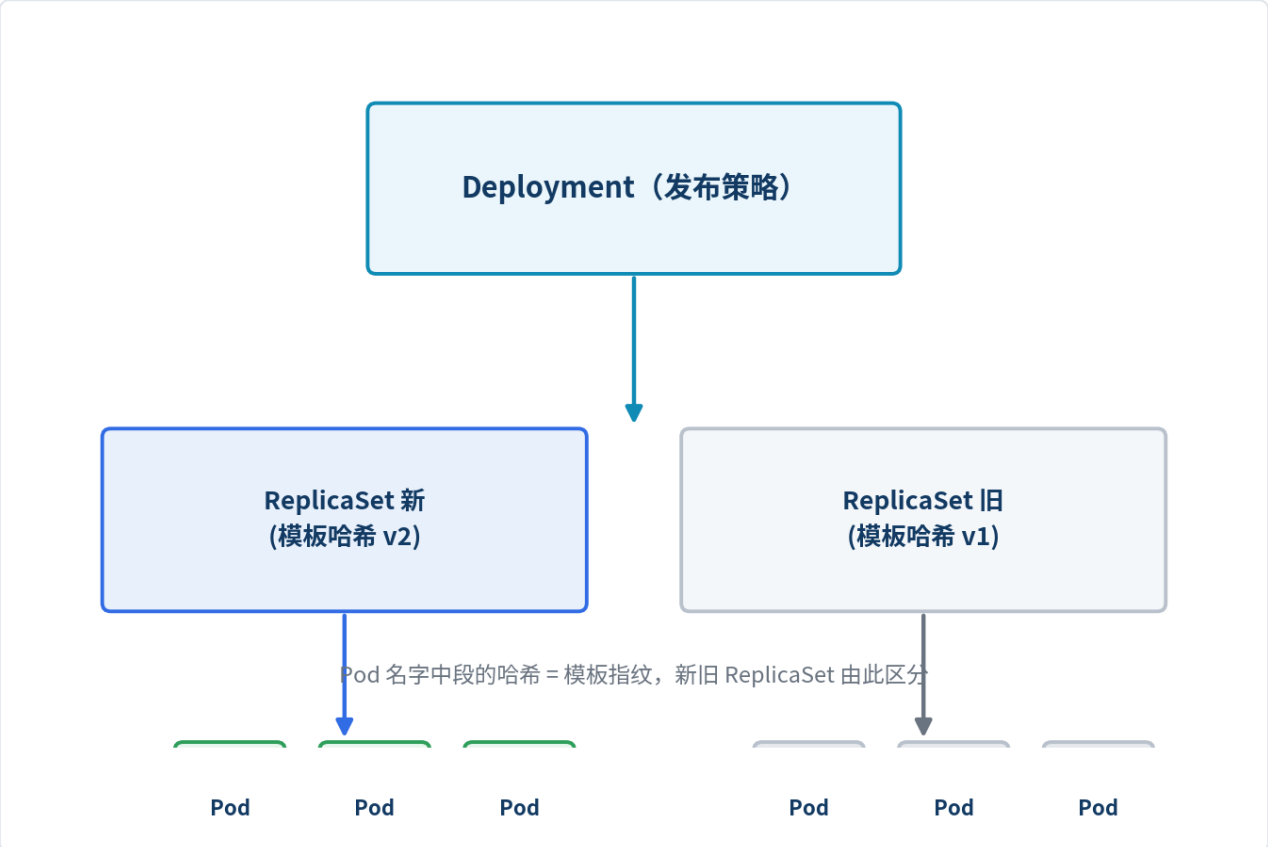


图6-1 Deployment → ReplicaSet → Pod 的三层管理关系

一个完整的 Deployment 清单：

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  labels:
    app: web
spec:
  replicas: 3                     # 期望副本数
  selector:                       # 【关键】如何认领 Pod
    matchLabels:
      app: web
  strategy:                       # 发布策略
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1                 # 滚动时最多允许多出 1 个 Pod
      maxUnavailable: 0           # 滚动时不允许少于期望数
  template:                       # Pod 模板：新 Pod 都从这里"复印"
    metadata:
      labels:
        app: web                  # 【关键】必须与 selector 匹配
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
        - containerPort: 80
        resources:
          requests: {cpu: 100m, memory: 128Mi}
          limits:   {cpu: 500m, memory: 256Mi}
        readinessProbe:
          httpGet: {path: /, port: 80}
```

这个 YAML 里有三处"关键"，值得逐个强调：

第一，selector 与 template 的绑定关系。 Deployment 通过 `selector.matchLabels` 认领 Pod，所以 Pod 模板的 labels 必须能被 selector 匹配。如果两者不匹配，Deployment 创建后会立刻报错（apiserver 会校验），所有 Pod 永远凑不齐。

第二，selector 一旦设定就不可更改。 这是 Kubernetes 的设计：selector 是 Pod 与控制器之间的"契约"，改契约等于让控制器"失忆"，后果是产生孤儿 Pod。

第三，template 的任何改动都会触发"新版本"。 镜像、环境变量、探针、资源限制……只要 Pod 模板变了，Deployment 就会生成新 ReplicaSet 开始滚动。理解了这一点，你就理解了第 7 章"滚动更新"的全部机制。

6.4 动手玩转 Deployment

```
# 创建
$ kubectl apply -f deployment.yaml
deployment.apps/web created

# 查看：注意有个 ReplicaSet 也是它的"子女"
$ kubectl get deployment,rs,pod
NAME                 READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/web  3/3     3            3           30s

NAME                           DESIRED   CURRENT   READY   AGE
replicaset.apps/web-7d9c8b6f5   3         3         3       30s

NAME                           READY   STATUS    RESTARTS   AGE
pod/web-7d9c8b6f5-xk2p9        1/1     Running   0          30s
pod/web-7d9c8b6f5-ql4mz        1/1     Running   0          30s
pod/web-7d9c8b6f5-pt7vd        1/1     Running   0          30s
```

注意到 Pod 名字的规律了吗？`web-7d9c8b6f5-xk2p9` = Deployment 名字 + ReplicaSet 哈希 + 随机后缀。**中间那段哈希就是 Pod 模板的"指纹"**——模板一变，哈希就变，新旧 ReplicaSet 由此区分。

6.4.1 验证自愈：删一个试试

```
$ kubectl delete pod web-7d9c8b6f5-xk2p9
pod "web-7d9c8b6f5-xk2p9" deleted

$ kubectl get pods -l app=web
NAME                   READY   STATUS    RESTARTS   AGE
web-7d9c8b6f5-ql4mz    1/1     Running   0          2m
web-7d9c8b6f5-pt7vd    1/1     Running   0          2m
web-7d9c8b6f5-hr8sn    1/1     Running   0          3s   # 新生补位
```

3 秒内，新 Pod 已经补上。这就是把 Pod 交给 Deployment 的意义。

6.4.2 扩缩容：一条命令的事

```
$ kubectl scale deployment web --replicas=10
$ kubectl get pods -l app=web | wc -l
```

但请注意：`scale` 是"命令式"的——它改了集群里的副本数，却没改你的 YAML 文件。下次别人 `apply` 旧文件，副本数又会被改回去。**正确的做法是把 YAML 里的 replicas 改了再 apply**，保持"文件即真相"。第 19 章会讲如何让副本数随负载自动伸缩（HPA），届时这条纪律还有更多细节。

6.5 深入控制循环：Reconcile 到底做了什么

第 3 章我们提过控制循环，现在以 Deployment 为例把它讲透。控制器的通用范式是：

> 读取期望状态（spec）→ 读取实际状态（status）→ 计算差异 → 执行动作消除差异 → 暂歇 → 再来一轮

Deployment 控制器每一轮做的事：

1. 对比"期望的 Pod 模板哈希"与现有 ReplicaSet 的哈希——有新哈希 → 创建新 ReplicaSet；
2. 按滚动策略计算"这一轮应该把多少 Pod 从旧 ReplicaSet 挪到新 ReplicaSet"——缩旧扩新；
3. 持续观察新 Pod 的就绪情况——只有新 Pod 就绪了才继续下一批（第 7 章的 minReadySeconds）；
4. 达到滚动完成条件后，把旧 ReplicaSet 缩到 0（但保留对象，供回滚）。

而 ReplicaSet 控制器每一轮做的事更简单：数 Pod、补 Pod、删 Pod。

这套设计的精妙之处在于**层级解耦**：Deployment 只和 ReplicaSet 对话，ReplicaSet 只和 Pod 对话，Pod 只被 kubelet 关心。每一层都只做简单的事，组合起来却实现了复杂的发布编排。这也是 Kubernetes 生态的扩展范式——第 25 章 Operator 的本质，就是把某个"领域知识"写成一个专用控制器。

> **深入：** 控制循环的触发是**事件驱动 + 定期兜底**：Watch 到相关资源变化立即触发；同时每隔一段（Resync 周期）强制跑一轮，防止错过事件或应对环境漂移。每轮循环是**幂等**的——跑一百遍和跑一遍效果相同。幂等性是声明式系统能自我修复的理论根基。

6.6 Deployment 的状态解读

`kubectl get deployment` 输出中有几列数字，生产排障时天天打交道：

```
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
web    2/3     3            2           10m
```

- **READY** `2/3`：3 个期望副本中 2 个就绪（模板里定义的 readinessProbe 通过才算就绪）。
- **UP-TO-DATE** `3`：已更新到最新模板的副本数。若小于期望值，说明滚动更新进行中或卡住了。
- **AVAILABLE** `2`：可用副本数（就绪且满足 minReadySeconds）。

READY 一直上不去？大概率是这几种原因：**探针失败**（应用没真正就绪）、**资源不足**（调度不出去，Pod 停在 Pending）、**镜像拉取失败**（ImagePullBackOff）。用 `kubectl describe deployment web` 和 `kubectl describe pod <卡住的pod>` 顺着事件链查即可。

> **注意：** Deployment 并不保证"任何时刻都恰好 N 个 Pod 存活"——它保证的是"向 N 个收敛"。节点故障时，替换 Pod 需要时间（调度 + 拉镜像 + 启动 + 就绪）。如果业务对可用副本数有硬性要求（比如 quorum 类应用），可以配合 PodDisruptionBudget（第 22 章）约束"自愿驱逐"时至少保留多少副本。

6.7 常见的反模式与最佳实践

反模式一：Pod 模板里不写探针。 没有 readinessProbe 的 Pod 一启动就被算作"可用"，而此时应用可能还在加载——流量打过去就是 502。请把"就绪的定义"明确告诉 Kubernetes。

反模式二：用 Deployment 跑有状态服务。 Deployment 生成的 Pod 名字随机、身份平等，重启后不保留任何身份。MySQL、Kafka、ZooKeeper 这类需要稳定身份和稳定存储的服务，请用 StatefulSet（第 8 章）。

反模式三：手动改 Pod。 `kubectl edit pod` 改了个环境变量？Pod 会立即被 ReplicaSet 杀掉重建——因为你的修改让 Pod 与模板"不一致"了。要改配置，请改 Deployment 的模板。

反模式四：所有服务共用一个 Deployment。 服务的发布节奏、资源画像、故障域各不相同，混在一个 Deployment 里（或用一个 Pod 装多个服务）会让升级和伸缩都变得别扭。按"一个可独立发布的服务 = 一个 Deployment"划分。

最佳实践清单：镜像 tag 明确（不用 latest）；设置 requests/limits；配置三类探针；合理设置 replicas（至少 2，关键服务跨节点分布）；清单文件进 Git；变更走 PR review。

6.8 本章小结

- **ReplicaSet** 负责"凑数量"：通过 label selector 认领 Pod，数量不对就增删，自愈能力由此而来。
- **Deployment** 负责"管发布"：管理 ReplicaSet 的世代更替，每次模板变更生成新 ReplicaSet，新旧并行为滚动更新铺路（第 7 章展开）。
- 三层结构 Deployment → ReplicaSet → Pod，每层只做简单的事，组合出强大的自动化；Pod 名字中间的哈希就是模板指纹。
- 控制循环的核心是**幂等的 Reconcile**：期望状态与实际状态有差异就消除，没有差异就不动。
- 日常使用要避开四大反模式：不写探针、用 Deployment 跑有状态服务、手改 Pod、服务边界混乱。

你已经能让应用"跑起来、挂了自动补"。但发新版本时如何做到"用户无感知"？如何出问题一键回滚？这正是下一章的主题——滚动更新与回滚的艺术。

第7章 滚动更新与回滚：灰度发布的艺术

上一章埋下的伏笔现在揭晓：Deployment 管理多个 ReplicaSet 的真正目的，就是为了让**版本更新**可以平滑进行、且可以随时反悔。这一章我们把发布这件事彻底讲透——从内置的滚动更新机制，到金丝雀发布、蓝绿部署，再到回滚与卡住排查。

发布能力是衡量一个团队工程成熟度的核心指标：**能在工作时间从容发布新版本的团队，才配得上"持续交付"这四个字**。

7.1 一次滚动更新的完整解剖

把第 6 章的 Deployment 镜像从 `nginx:1.25` 改为 `nginx:1.26`：

```
$ kubectl set image deployment/web nginx=nginx:1.26
# 或者（推荐）：修改 YAML 后
$ kubectl apply -f deployment.yaml
```

观察整个过程（另开一个终端）：

```
$ kubectl get pods -l app=web -w
web-7d9c8b6f5-xk2p9   1/1   Running   0     # 旧 ReplicaSet 的 3 个 Pod
web-66f8c94d21-wq9rt  0/1   Pending   0     # 新 RS 创建第 1 个新 Pod
web-66f8c94d21-wq9rt  1/1   Running   0     # 新 Pod 就绪
web-7d9c8b6f5-xk2p9   1/1   Terminating     # 就绪了才杀旧 Pod
web-66f8c94d21-tr4lp  0/1   Pending   0     # 继续第 2 个
...（循环往复）
web-66f8c94d21-tr4lp  1/1   Running   0     # 全部就绪，更新完成
```

再看 ReplicaSet 的世代痕迹：

```
$ kubectl get rs
NAME               DESIRED   CURRENT   READY   AGE
web-66f8c94d21     3         3         3       5m    # 新版
web-7d9c8b6f5      0         0         0       2d    # 旧版，缩到 0 但保留
```

整个过程的编排逻辑可以概括为一句话：**新 Pod 就绪一个，旧 Pod 才下线一个**。



图7-1 滚动更新：新副本就绪一个，旧副本才下线一个

新 Pod 什么时候算"就绪"？由 readinessProbe 决定（这再次说明探针配置的重要性——没有就绪探针的滚动更新，等于蒙眼开车）。

7.1.1 两个节奏旋钮：maxSurge 与 maxUnavailable

滚动快慢与平稳程度，由两个参数控制（都是相对值或绝对值）：

- **maxSurge**：滚动过程中，允许**超出**期望副本数的最大值（"先多生几个"）；
- **maxUnavailable**：滚动过程中，允许**低于**期望副本数的最大值（"可以暂时少几个"）。

三种典型配置的风格对比：

配置	行为	适用场景
`maxSurge=1, maxUnavailable=0`	只增不减，容量始终够	资源充足、要求零容量缺口
`maxSurge=0, maxUnavailable=1`	只减不增，省资源	测试环境、资源紧张
`maxSurge=25%, maxUnavailable=25%`（默认）	平衡	大多数场景

> **提示：** `maxUnavailable=0` 搭配就绪探针，是"无损发布"的经典组合：新 Pod 完全就绪并加入流量后才替换旧的，全程容量不缩水。代价是滚动期间需要额外资源余量（maxSurge 个 Pod 的资源）。

7.1.2 minReadySeconds 与 progressDeadlineSeconds

两个容易被忽略但极有价值的参数：

```
spec:
  minReadySeconds: 15          # 新 Pod 需保持就绪 15 秒，才计入"可用"
  progressDeadlineSeconds: 600 # 600 秒内没有进展，标记发布失败
```

- **minReadySeconds** 给每个新 Pod 留了"观察期"——就绪探针刚通过就立刻杀旧 Pod 是危险的，有些问题（如启动后的内存峰值、连接池初始化失败）要跑一会儿才暴露。15～60 秒的观察期能拦下大部分"闪亮登场然后秒挂"的新版本。
- **progressDeadlineSeconds** 定义了"多久没有进展算失败"。超时后 Deployment 状态变为 `ProgressDeadlineExceeded`，配合 CI 系统可以自动触发告警甚至自动回滚。

7.2 回滚：发布体系的"后悔药"

新版本上线后发现严重 bug？别慌，这正是 Deployment 保留旧 ReplicaSet 的意义——回滚只是"让旧世代复活"：

```
# 查看发布历史
$ kubectl rollout history deployment/web
deployment.apps/web
REVISION  CHANGE-CAUSE
1         nginx:1.24
2         nginx:1.25
3         nginx:1.26       # 当前

# 回滚到上一版
$ kubectl rollout undo deployment/web

# 或明确回滚到指定版本
$ kubectl rollout undo deployment/web --to-revision=1
```

回滚本身也是一个滚动更新过程——旧的镜像号通过新的滚动流程重新铺开，同样是平滑的。

两个让回滚更好用的细节：

记录 CHANGE-CAUSE。默认发布历史只有哈希没有说明，回滚时两眼一抹黑。给 Deployment 模板加注解，并让 CI 在发布时更新它：

```
spec:
  template:
    metadata:
      annotations:
        kubernetes.io/change-cause: "upgrade to nginx:1.26, fix CVE-2026-1234"
```

revisionHistoryLimit。默认保留 10 个历史 ReplicaSet。历史太多会堆积无用对象，设为 3～5 足够；但**至少保留 1**，否则无法回滚。

> **警示：** 回滚只能回滚"模板变更"。如果你的故障源于**配置中心的数据、数据库的 Schema 变更**、或**下游服务的契约变化**——这些在 Kubernetes 视野外的东西——回滚 Deployment 无济于事。数据库迁移脚本必须设计成"向后兼容"（新旧代码都能跑），才能让代码回滚变得安全。

7.3 灰度发布的进阶玩法

滚动更新是"全员逐步替换"，但很多时候你想要更精细的流量控制：先让 5% 的用户尝鲜，观察一小时，没问题再全量。这就是**金丝雀发布（Canary Release）**——名字源于矿井里预警毒气的金丝雀。

7.3.1 基于 Deployment 的"穷人版金丝雀"

不引入额外工具时，可以利用"Service 按 label 分发流量"的机制手工实现：

1. 主版本 Deployment 用标签 `app=web, version=stable`；
2. 额外创建一个 canary Deployment（同一 Service 的 selector 只匹配 `app=web`），镜像为新版本，副本数为 1，而主版本有 19 个；
3. Service 的 selector 匹配 `app=web`，流量按副本数比例分给 stable:canary = 19:1；
4. 验证无误后，更新主 Deployment 的镜像，滚动铺开，删除 canary。

这个方案的粒度受限于副本数比例（想精确 5% 需要更多副本），但胜在零依赖、立即可用。对大多数中小团队，这就够了。

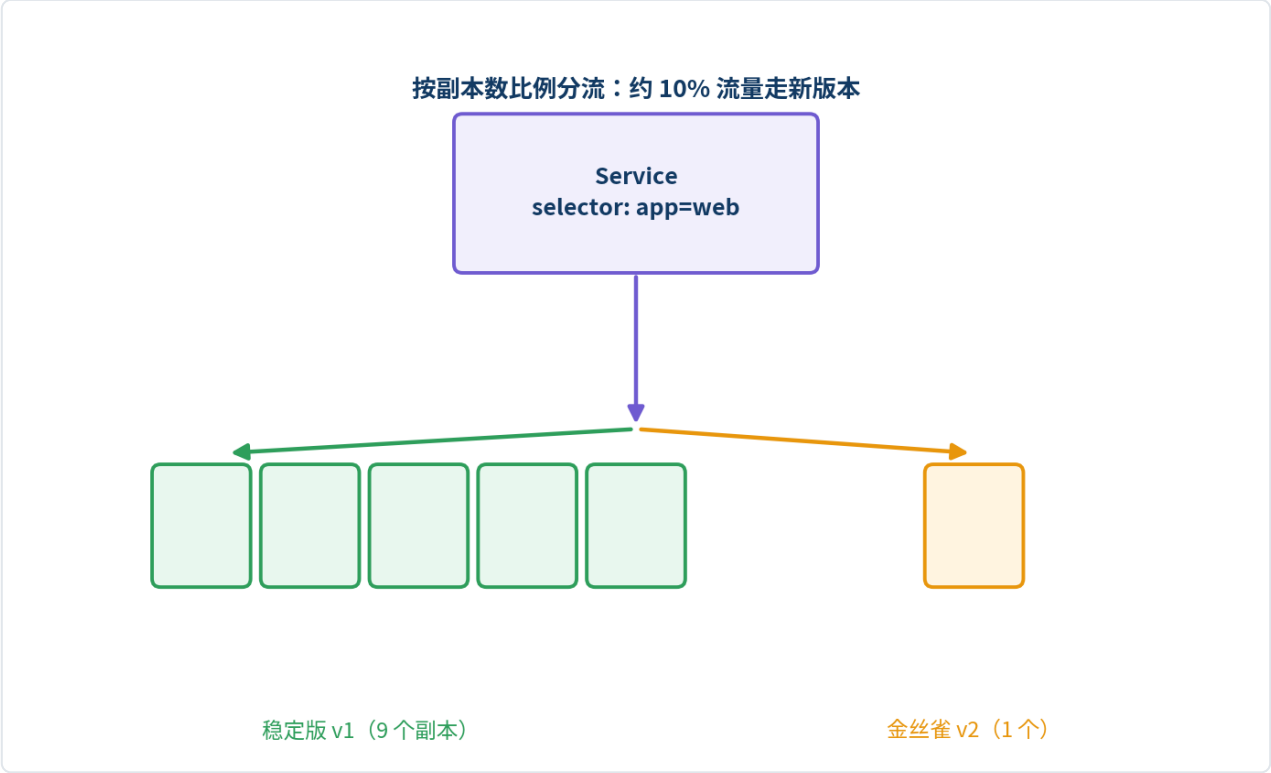


图7-2 金丝雀发布：Service 按副本数比例把少量流量引向新版

7.3.2 按请求特征分流

需要"内部员工先体验新版本""特定 header 的请求走新版本"这类能力时，副本数比例无能为力，需要 Ingress 层的流量切分能力（基于 header/cookie/权重），第 11 章会讲到 Ingress 注解的实现方式；更体系化的方案是服务网格（第 26 章 Istio 的 VirtualService 可以做到按百分比、按 header 的任意分流）。

7.3.3 蓝绿部署

蓝绿部署（Blue-Green） 是另一种发布哲学：新旧两套环境（蓝=旧、绿=新）同时完整存在，验证绿环境没问题后，**把流量一次性从蓝切到绿**。

在 Kubernetes 中，蓝绿就是两个 Deployment（web-blue、web-green）+ 一个可切换的 Service。切换方式有三种，从土到洋：

1. 改 Service 的 selector（如 `version: blue` → `version: green`）；
2. 更新主 Deployment 的镜像，走内置滚动更新（那其实又回到滚动更新了）；
3. 使用流量切换工具或网关能力做瞬时切换。

蓝绿的优势是切换快、回滚只需切回 selector（秒级）；劣势是**双倍资源**占用，且有状态服务（数据库 Schema）无法用这种方式简单切换。资源敏感的团队通常选择滚动更新 + 金丝雀，而非蓝绿。

7.3.4 发布策略选型小结

策略	资源开销	流量控制粒度	复杂度	适用
滚动更新	低（surge 部分）	无（按 Pod 比例）	零依赖	日常迭代
金丝雀	低～中	副本比例或请求特征	中	有风险的新版本
蓝绿	高（双倍）	一次性切换	中	需要快速整体回滚
网格分流	中（网格开销）	任意百分比/header	高（引入 Istio 等）	大规模微服务

7.4 发布卡住了怎么办：排查指南

滚动更新进行中，Pod 一直起不来，整个发布僵在那里。这是高频事故场景，排查有固定套路：

第一步：找到卡住的 Pod。

```
$ kubectl get pods -l app=web
web-66f8c94d21-wq9rt   0/1   Running           0    5m   # 起来了但没就绪?
web-66f8c94d21-tr4lp   0/1   ImagePullBackOff  0    4m   # 镜像拉取失败
```

第二步：describe 看事件。

```
$ kubectl describe pod web-66f8c94d21-tr4lp
Events:
  Warning  Failed     Failed to pull image "myapp:v2.1": not found
```

常见病因速查表：

现象	典型原因	处理
ImagePullBackOff	镜像 tag 不存在 / 仓库认证失败 / 节点网络	核对镜像名；配置 imagePullSecrets
CrashLoopBackOff	应用启动即崩溃（配置错误、依赖不可达）	`kubectl logs --previous` 看崩溃前日志
Pending	资源不足 / 调度约束冲突 / PVC 未绑定	describe 看 FailedScheduling 原因
Running 但 0/1 就绪	readinessProbe 失败（依赖未就绪、探针路径写错）	describe 看探针失败详情
OOMKilled	内存 limits 太小	调大 limits 或排查内存泄漏

第三步：止损优先于排查。 生产发布卡住时，先回滚（`kubectl rollout undo`），保住线上稳定，再慢慢定位新版本的问题。永远不要在"线上正卡着"的状态下从容 debug。

> **提示：** `kubectl rollout status deployment/web --timeout=120s` 可以在 CI 流水线里等待发布完成并超时失败——这是自动判断"发布是否成功"的标准姿势，配合自动回滚构成发布流水线的安全网。

7.5 本章小结

- 滚动更新的本质是**新旧 ReplicaSet 并存、逐个替换**：新 Pod 就绪才下线旧 Pod；maxSurge/maxUnavailable 控制节奏，minReadySeconds 提供观察期，progressDeadlineSeconds 定义失败。
- **回滚 = 旧 ReplicaSet 复活**，同样是平滑滚动；配合 change-cause 注解与 rollout history 可精确回滚任意历史版本。但记住：回滚救不了数据库和配置的变更。
- 金丝雀发布用"独立 canary Deployment + 共享 Service"实现流量比例灰度；蓝绿用双环境瞬时切换，资源翻倍。按风险等级与资源预算选策略。
- 发布卡住的排查三板斧：找卡住的 Pod → describe 看事件 → 对照病因速查表；**止损（回滚）永远优先于排查**。

至此，无状态服务的发布闭环已经完整。但有一大类应用——数据库、消息队列、注册中心——需要的不只是"凑数量"，而是**稳定的身份和专属的存储**。下一章，我们认识为它们量身定做的控制器：StatefulSet。

第8章 StatefulSet：有状态应用的专属包间

到目前为止，我们讨论的所有 Pod 都是"匿名的"：Deployment 里的 Pod 叫 `web-7d9c8b6f5-xk2p9`，重启后换名字，杀掉后重建，彼此之间没有亲疏。这对无状态服务来说恰到好处——但对于 MySQL 主从、Kafka 集群、ZooKeeper、Elasticsearch 这类**有状态应用**，"匿名且随时可换"恰恰是致命的。

这一章，我们认识 Kubernetes 为有状态应用准备的专属机制——**StatefulSet**，并借此机会把"有状态"这个概念本身讲清楚。

8.1 什么是有状态：身份、顺序与专属存储

先建立一个判断标准。一个应用是"有状态的"，通常意味着它至少依赖以下三样东西之一：

稳定的网络身份。 Kafka 集群里，`kafka-0` 通常是特定分区的 Leader；ZooKeeper 的成员列表写死了 `zk-0`、`zk-1`、`zk-2` 这些名字。如果每次重启 Pod 名字都随机变化，集群成员就无法互相辨认。

稳定的专属存储。 数据库实例 A 的数据必须始终跟着实例 A——Pod 重建后要能"找回原来的数据"，而不是领一块全新的空盘。

有序的部署与伸缩。 很多分布式系统对启动顺序敏感：ZooKeeper 要等前一个成员就绪才能加入；主从数据库要先起主库再起从库；滚动更新时也常常要求"从高序号往低序号"逐个更新，保证 quorum 不丢。

无状态服务（Web 前端、API 网关）对以上三项全都无所谓：谁是谁不重要，数据不落在本地，同时起同时停都行。

> **比喻：** 无状态 Pod 像电影院的散座观众——坐哪儿都行，走了一个补一个，无所谓；有状态应用像包间里的固定会员——每个人都有专属包间（稳定存储），有登记在册的编号（稳定身份），入场要按顺序（有序部署），退场也要按顺序（有序终止）。

8.2 StatefulSet 的三大承诺

StatefulSet（缩写 sts）为上述需求提供了三个核心保证：

一、稳定的 Pod 身份。 Pod 名字形如 `kafka-0`、`kafka-1`、`kafka-2`——序号固定、重建不变。无论 Pod 被调度到哪个节点，`kafka-0` 就永远是 `kafka-0`。

二、稳定的网络标识。 StatefulSet 要求配合一个 **headless Service**（无头 Service，即 `clusterIP: None` 的 Service），为每个 Pod 生成专属 DNS 记录：

```
kafka-0.kafka-hs.default.svc.cluster.local
kafka-1.kafka-hs.default.svc.cluster.local
kafka-2.kafka-hs.default.svc.cluster.local
```

集群成员之间通过这些 DNS 名字互相找到对方。这是普通 Deployment 做不到的——它的 Pod 没有个体 DNS 记录，只能靠 Service 负载均衡"雨露均沾"。headless Service 的原理第 10 章详述。

三、稳定的专属存储。 通过 **volumeClaimTemplates（存储卷申请模板）**，StatefulSet 为每个 Pod 自动创建专属的 PVC：`kafka-0` 对应 `data-kafka-0`，`kafka-1` 对应 `data-kafka-1`，一一对应、终生绑定。Pod 被删除重建后，重新挂载的还是原来那块盘。存储机制（PV/PVC）的细节第 14 章展开，本章先记住"一个 Pod 一块专属盘"这个结果。

8.3 一个完整的 StatefulSet 示例

以一个三节点的 Redis 集群为例（示意），看 StatefulSet 的典型长相：

```yaml
apiVersion: v1
kind: Service
metadata:
  name: redis-hs            # headless Service，供成员互访
spec:
  clusterIP: None           # 关键：无头
  selector:
    app: redis
  ports:
  - port: 6379
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis
spec:
  serviceName: redis-hs     # 【关键】关联 headless Service
  replicas: 3
  selector:
    matchLabels:
      app: redis
  podManagementPolicy: OrderedReady   # 默认：有序管理
  updateStrategy:
    type: RollingUpdate     # 默认从序号最大的开始倒序更新
  volumeClaimTemplates:     # 【关键】每个 Pod 一块专属盘
  - metadata:
      name: data
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
          storage: 10Gi
      storageClassName: ssd
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
      - name: redis
        image: redis:7.2
        ports:
        - containerPort: 6379
        volumeMounts:
        - name: data
          mountPath: /data
        resources:
          requests: {cpu: 500m, memory: 1Gi}
          limits:   {cpu: "2", memory: 4Gi}
```

三个标注【关键】的字段，就是 StatefulSet 区别于 Deployment 的全部精髓：

- `serviceName: redis-hs`：告诉控制器"用这个 headless Service 给 Pod 发门牌号"；
- `volumeClaimTemplates`：告诉控制器"每个 Pod 出生时，按这个模板申请一块专属盘"；
- 有序管理（podManagementPolicy 与 updateStrategy 的默认行为）：保证部署、扩容、更新、缩容全部按序号进行。

创建后观察 Pod 的生成节奏：

```
$ kubectl get pods -l app=redis -w
redis-0   0/1   Pending   0    # 先创建序号 0
redis-0   1/1   Running   0
redis-1   0/1   Pending   0    # redis-0 就绪后才创建 redis-1
redis-1   1/1   Running   0
redis-2   0/1   Pending   0    # 再等 redis-1 就绪
redis-2   1/1   Running   0
```

同时自动创建了三块专属存储：

```
$ kubectl get pvc
NAME          STATUS   VOLUME                                     CAPACITY
data-redis-0  Bound    pvc-a1b2c3d4-...                           10Gi
data-redis-1  Bound    pvc-e5f6a7b8-...                           10Gi
data-redis-2  Bound    pvc-9c0d1e2f-...                           10Gi
```

8.4 有序性的细节：部署、更新、缩容、删除

StatefulSet 的"有序"贯穿四个操作，行为各不相同，值得逐一了解：

部署与扩容：默认 `OrderedReady` 策略——按 0,1,2 顺序创建，前一个就绪才创建后一个。如果把 `podManagementPolicy` 改为 `Parallel`，则全部并发创建（适合成员间无顺序依赖的场景，如某些无 Leader 的集群）。

更新：`updateStrategy: RollingUpdate` 默认**从最大序号开始倒序**更新（2 → 1 → 0）。这个方向对多数分布式系统更安全——比如 3 节点 quorum 系统，更新高序号节点时低序号节点还保持着多数派。还可以设置 `partition` 参数实现"只更新序号 >= N 的 Pod"，这本身就是内置的金丝雀发布：先把 partition 设为 N-1（只更新最后一个），观察正常后再逐步减小 partition。

缩容：从最大序号开始删除（先删 `redis-2`），且**默认不删除对应的 PVC**——数据保住了，将来扩容回来还能重新挂上。这正是"专属包间"的体现：人可以走，包间留着。

删除 StatefulSet：Pod 会全部删除，但 PVC 同样默认保留。

> **注意：** 缩容有状态集群前，请务必确认剩余成员仍满足 quorum/多数派要求（比如 5 副本最多缩到 3）。盲目缩容可能导致集群"脑裂"或只读。

8.5 StatefulSet 的行为差异与坑

使用 StatefulSet 时，有几点与 Deployment 的差异需要特别注意：

删除 Pod 不会自动重建？ 不对——StatefulSet 也会维持副本数，删掉 `redis-1` 后它会重建一个**同名**的 `redis-1`（这是与 Deployment 的本质区别：StatefulSet 的 Pod 名字是身份，必须原样恢复；Deployment 的 Pod 名字是随机的，重建即可）。

存储绑定节点。 `ReadWriteOnce` 的卷通常只能挂在一个节点上。如果 `redis-1` 所在节点宕机，而卷还被"僵尸"节点占用着，新 Pod 可能卡在挂载阶段。云环境的解决方案是强制 detach（`kubectl delete pod --force --grace-period=0` 触发驱逐流程），第 21 章的故障排查会再回到这个场景。

headless Service 必须先行。 StatefulSet 依赖 serviceName 指向的 Service 存在，如果先建 StatefulSet 后建 Service，Pod 依然能跑，但 DNS 记录缺失，集群成员互访会失败。惯例是 Service 和 StatefulSet 写在同一个清单文件里（如本章示例）。

不要用 StatefulSet 跑无状态服务。 反过来也不成立：StatefulSet 的有序性会拖慢无状态服务的发布速度，还占用序号身份的"精神负担"。各归其位。

8.6 有状态应用上 K8s：要想清楚的事

这是一个值得冷静讨论的话题。把数据库搬进 Kubernetes，社区争论了很多年，直到今天仍有分歧。我们不站队，但给出决策框架。

支持上 K8s 的理由：

- 统一运维界面：所有应用（含数据库）用同一套工具部署、监控、伸缩；
- 弹性：测试环境快速拉起/销毁数据库实例；
- Operator 生态成熟：主流数据库（MySQL、PostgreSQL、MongoDB、Kafka、ES）都有成熟的 Operator（第 25 章），封装了运维知识——主从切换、备份恢复、版本升级。

反对（或谨慎）的理由：

- 数据库是"最贵"的工作负载：故障的代价最大，而 Kubernetes 的 Pod 驱逐、节点维护等机制都可能波及数据库；
- 存储性能：网络存储的 IOPS/延迟通常不如本地盘，数据库这种 IO 密集应用可能掉速；
- 运维复杂度叠加：数据库专家 + K8s 专家，两个稀缺角色都得有。

务实的建议：

1. **生产核心数据库**，若团队没有足够的 K8s 运维经验，托管数据库服务（云 RDS）往往是更稳的选择；
2. **测试/开发环境**的数据库、缓存、消息队列，放心用 StatefulSet（或 Operator），弹性收益明显；
3. 若决定生产上，**用成熟 Operator，不要手搓 StatefulSet YAML**——主从切换、备份、监控这些运维知识，Operator 已经踩过所有坑；
4. 无论如何，**备份策略先行**，且备份必须定期演练恢复（第 22 章）。

> **深入：** Operator 模式可以说是 StatefulSet 的"精神续作"：StatefulSet 只懂"身份、存储、顺序"这些通用有状态语义，而每个数据库的领域知识（谁是主、怎么选主、怎么备份）需要专用控制器。学会了本章，第 25 章的 Operator 会显得顺理成章。

8.7 本章小结

- 有状态应用需要三样东西：**稳定身份、稳定存储、有序管理**——StatefulSet 的三大承诺与之精确对应。
- StatefulSet 通过 **headless Service** 给每个 Pod 发专属 DNS，通过 **volumeClaimTemplates** 给每个 Pod 发专属 PVC，通过 **序号命名与有序策略**保证成员有序。
- Pod 重建恢复**同名同盘**（与 Deployment 的随机名字形成鲜明对比）；缩容默认保留 PVC。
- 更新默认倒序进行，partition 参数提供内置金丝雀能力；`Parallel` 策略适合无顺序依赖的场景。
- 有状态应用上 K8s 是收益与风险的权衡：核心生产数据库优先考虑托管服务或成熟 Operator；备份与恢复演练永远先行。

我们已经能让"有身份"的应用在 K8s 里安家。但无论无状态还是有状态，应用之间总要互相通信——下一章进入 Kubernetes 网络的世界，先从最基础的问题开始：Service 如何给漂泊不定的 Pod 一个稳定的门牌号。

第9章 DaemonSet 与 Job/CronJob：常驻守卫与定时任务

工作负载家族还有两位重要成员，它们的形态与 Deployment/StatefulSet 截然不同：**DaemonSet** 不关心"跑几个副本"，只关心"每台机器都要有"；**Job/CronJob** 不追求"永续运行"，只为"把一件事干完"。本章把这两位讲透，工作负载的全家族就集齐了。

9.1 DaemonSet：每台机器都要有我

DaemonSet（守护进程集） 保证：**在每个（匹配的）节点上，有且只有一个该 Pod 的副本**。节点加入集群，Pod 自动创建；节点移除，Pod 随之回收。

> **比喻：** 如果说 Deployment 是"招聘 N 名外卖骑手，人多就多派"，那 DaemonSet 就是"每个配送站点必须配一名驻站安全员"——站点开一个，安全员派一个，绝不多派，绝不缺岗。

9.1.1 什么样的工作适合 DaemonSet

凡是"按节点部署"才有意义的工作负载，都是 DaemonSet 的主场：

- **日志收集**：fluent-bit / filebeat 挂载所有容器的日志目录，采集本节点全部 Pod 的日志（第 20 章的主角）；
- **监控采集器**：node-exporter 采集节点的 CPU、内存、磁盘指标；
- **网络插件**：Calico、Cilium 的代理组件本身就是 DaemonSet——每台节点都得有，否则这台节点的 Pod 没法组网；
- **存储与设备插件**：CSI 节点驱动、GPU 设备插件；
- **安全与合规代理**：主机入侵检测、漏洞扫描 agent。

这些组件的共同点：**它们服务于"节点"这个物理/虚拟实体，而不是服务于"应用"这个逻辑实体**。

9.1.2 一个日志收集 DaemonSet 示例

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: log-collector
  namespace: logging
spec:
  selector:
    matchLabels:
      app: log-collector
  updateStrategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1        # 每次只更新一个节点上的采集器
  template:
    metadata:
      labels:
        app: log-collector
    spec:
      hostNetwork: true        # 使用主机网络（采集 agent 常见做法）
      tolerations:             # 【关键】容忍控制平面节点的污点
      - key: node-role.kubernetes.io/control-plane
        operator: Exists
        effect: NoSchedule
      volumes:
      - name: varlog
        hostPath:
          path: /var/log       # 直接读宿主机日志目录
      containers:
      - name: fluent-bit
        image: fluent/fluent-bit:2.2
        volumeMounts:
        - name: varlog
          mountPath: /var/log
          readOnly: true
```

三个值得注意的配置：

hostPath 卷：DaemonSet 经常需要直接访问宿主机文件系统（日志目录、/proc、设备文件）。这是 DaemonSet 的"特权"，也意味着要格外小心安全（第 17 章）。

容忍污点（tolerations）：控制平面节点默认有"NoSchedule"污点，普通 Pod 不会调度上去。如果你希望控制平面节点上也跑日志/监控 agent，必须显式容忍污点。污点机制第 18 章详讲。

滚动更新：DaemonSet 也支持 RollingUpdate，`maxUnavailable=1` 表示逐台节点更新，避免所有节点的 agent 同时中断。

9.2 Job：把一件事干完

Job（任务） 与前面所有工作负载的本质区别在于目标：Deployment 追求"永远运行"，Job 追求"**成功完成并退出**"。Pod 跑完退出了？任务成功，收工；Pod 失败了？按策略重试，直到成功或放弃。

9.2.1 一个计算圆周率的 Job

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: pi
spec:
  completions: 1              # 总共要成功完成几次
  parallelism: 1              # 同时最多跑几个 Pod
  backoffLimit: 4             # 失败最多重试 4 次
  activeDeadlineSeconds: 600  # 整个 Job 的总超时
  template:
    spec:
      restartPolicy: Never    # 【必填】Job 里 Pod 的重启策略
      containers:
      - name: pi
        image: perl:5.34
        command: ["perl", "-Mbignum=bpi", "-wle", "print bpi(2000)"]
```

> **注意：** Job 的 Pod 模板里 `restartPolicy` 必须显式设置为 `Never` 或 `OnFailure`（不允许 `Always`）——语义上很自然："跑完就退出"的容器若设为永远重启，任务就永远"完成"不了。

几个参数的组合语义：

- `completions=10, parallelism=3`：总任务 10 份，同时最多 3 个 worker 在跑——典型的批量并行处理模型；
- `backoffLimit`：失败重试上限，超过后 Job 标记为 Failed；
- `activeDeadlineSeconds`：无论进展如何，超时即终止——防止失控任务占着资源；
- `ttlSecondsAfterFinished: 3600`：完成/失败 1 小时后自动清理 Job 与 Pod（推荐设置，否则历史 Job 会越积越多）。

9.2.2 completionMode：索引任务模式

`completionMode: Indexed` 是一个强大但常被忽视的模式：每个 Pod 会获得索引编号（0 到 completions-1），通过环境变量 `JOB_COMPLETION_INDEX` 可知自己是几号。适合"把任务切片分给固定数量的 worker"的场景——比如批量迁移：预先把数据分成 100 片，建 completions=100 的 Indexed Job，每个 worker 按自己的索引号处理对应的分片，天然负载均衡且可断点续跑。

9.3 CronJob：定时执行 Job

CronJob 就是"带调度器的 Job"——按 cron 表达式定期创建 Job：

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: db-backup
spec:
  schedule: "30 2 * * *"        # 每天凌晨 2:30
  timeZone: "Asia/Shanghai"     # 1.27+ 支持指定时区
  concurrencyPolicy: Forbid     # 上一次还没跑完，本次跳过
  successfulJobsHistoryLimit: 3 # 保留最近 3 次成功的记录
  failedJobsHistoryLimit: 1
  startingDeadlineSeconds: 300  # 错过调度点超过 5 分钟则不再补跑
  jobTemplate:
    spec:
      ttlSecondsAfterFinished: 86400
      template:
        spec:
          restartPolicy: OnFailure
          containers:
          - name: backup
            image: postgres:16
            command: ["sh", "-c", "pg_dump ... > /backup/db-$(date +%F).sql"]
```

cron 表达式五段式：`分 时 日 月 周`，`*` 表示任意，`*/5` 表示每 5 个单位，`30 2 * * *` 即每天 02:30。

concurrencyPolicy 三个选项必须理解清楚，选错会出事故：

- `Allow`（默认）：并行跑，上一次没结束也开新的——备份任务用这个可能造成备份文件互相覆盖；
- `Forbid`：跳过本次——适合"宁可漏跑不可重跑"的任务（如计费）；
- `Replace`：杀掉正在跑的，换新的——慎用。

> **提示：** 定时任务的三大经典事故：任务没跑完新任务又开始了（用 Forbid）；时区不对（用 timeZone 显式指定）；错过调度窗口后疯狂补跑（用 startingDeadlineSeconds 限定补跑窗口）。这三个参数请在生产 CronJob 里都显式配置。

9.4 工作负载家族总览

至此，四大工作负载全部登场。用一张表总结它们的定位差异：

控制器	目标形态	副本语义	典型场景
Deployment	永续运行的无状态服务	N 个等价匿名副本	Web 服务、API、网关
StatefulSet	永续运行的有状态服务	N 个有身份的个体	数据库、消息队列、注册中心
DaemonSet	每节点一个	按节点数	日志、监控、网络、设备插件
Job/CronJob	跑完即止	completions 次成功	批处理、迁移、备份、巡检

> **深入：** 这四种控制器覆盖了绝大多数场景，但并非全部。ReplicationController 是 ReplicaSet 的前身（已淘汰）；若四种都不满足——比如"主从异构部署"（1 主 2 从，主和从的容器配置不同）——就需要自定义控制器（Operator，第 25 章），或者拆成多个 Deployment 组合。

9.5 本章小结

- **DaemonSet** 的语义是"每节点一份"：日志采集、监控、网络插件等按节点部署的组件是它的主场；hostPath、容忍污点是其常见搭配。
- **Job** 的语义是"成功完成 N 次"：completions/parallelism 控制批量并行，backoffLimit 控制重试，activeDeadlineSeconds 控制总超时，ttlSecondsAfterFinished 自动清理；Indexed 模式支持分片并行。
- **CronJob** 是定时版 Job：三大生产参数——concurrencyPolicy 防并发踩踏、timeZone 防时区错乱、startingDeadlineSeconds 防疯狂补跑。
- 四大工作负载各司其职：**按服务状态选 Deployment/StatefulSet，按部署粒度选 DaemonSet，按生命周期选 Job**。

工作负载家族集齐。接下来该解决一个悬而未决的大问题：这些 Pod 之间、Pod 与外部之间，到底怎么通信？下一章进入网络部分——从 Service 这个"稳定门牌号"讲起。

第10章 Service：给漂泊的 Pod 一个稳定的门牌号

现在进入本书最重要的部分之一：Kubernetes 网络。我们先从一个最朴素的问题开始——**Pod 之间怎么互相访问?**

10.1 问题：Pod 的 IP 靠不住

Kubernetes 给每个 Pod 分配了集群内唯一的 IP 地址，Pod 之间可以直接用 IP 通信——这很简单。但这个 IP 有个致命弱点：**它是不稳定的**。

回顾一下我们已经学过的知识：

- Pod 会因为节点故障被销毁重建——新 Pod 有**新 IP**；
- 发布新版本时，Deployment 滚动替换——所有 Pod 都换成**新 IP**；
- 扩容时新加副本——又一批**新 IP**；
- 缩容时——一批 IP 消失。

设想你的订单服务要调用用户服务：如果调用方写死了用户服务的 Pod IP，那么用户服务每次滚动更新，订单服务就得跟着改配置。Pod 越多、变更越频繁，这套"IP 电话簿"就越难维护。

> **比喻：** Pod IP 像员工的**手机号**——人会离职、入职、换号；而你需要的是公司的**总机号码**：不管里面的人怎么换班，总机号码永远不变，打总机，接线员自动把你转给当前在岗的员工。

这个"总机"，就是 **Service**。

10.2 Service 的本质：一个稳定的虚拟 IP + 一组后端

Service 的作用可以概括为：**提供一组稳定的访问入口（VIP + DNS 名），并把流量分发到一组动态变化的 Pod 上**。

定义一个 Service：

```
apiVersion: v1
kind: Service
metadata:
  name: user-service
spec:
  type: ClusterIP          # 默认类型
  selector:
    app: user              # 【关键】通过标签找到后端 Pod
  ports:
  - port: 80               # Service 自己监听的端口
    targetPort: 8080       # 转发到 Pod 的哪个端口
```

创建后，这个 Service 获得一个**集群内稳定的虚拟 IP**（ClusterIP）：

```
$ kubectl get svc user-service
NAME           TYPE        CLUSTER-IP     PORT(S)   AGE
user-service   ClusterIP   10.96.34.120   80/TCP    5s
```

无论后端 Pod 怎么生生死死，`10.96.34.120:80` 这个地址永远有效。流量打到它，就被转发到当前健康的一组 `app=user` 的 Pod 上。

10.2.1 虚拟 IP 的"虚拟"二字：为什么没有进程监听

ClusterIP 最反直觉的一点是：**没有任何进程在监听这个 IP**。10.96.34.120 既不属于任何 Pod，也不属于任何节点，它是一个纯粹的"规则产物"。

它的实现依赖 Linux 内核的流量劫持能力（第 3 章介绍的 kube-proxy 负责）：当 Pod A 访问 `10.96.34.120:80` 时，数据包在离开 Pod 网络栈时被 iptables/IPVS 规则拦截，**目的地址被改写**为某个真实后端 Pod 的 IP:Port（比如 `172.20.3.15:8080`），然后正常路由过去。回程包再对称地改写回来，通信双方毫无察觉。

这就是 **DNAT（目标地址转换）**。理解了这一点，很多现象就顺理成章了：

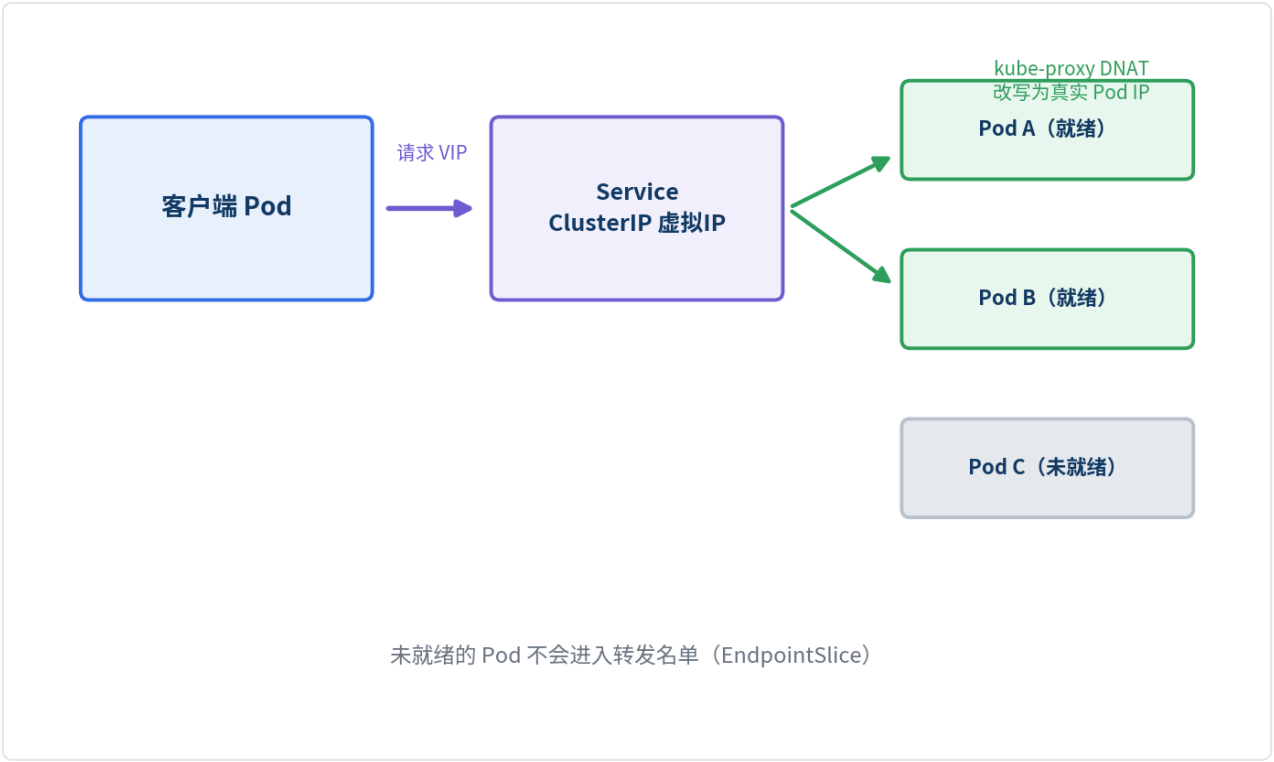


图10-1 Service 虚拟 IP 经 kube-proxy 做 DNAT，转发到就绪的后端 Pod

- 为什么 ping Service 的 ClusterIP 不通？——因为 ICMP ping 不会命中 DNAT 规则，规则只对 TCP/UDP 生效；
- 为什么 Service 没有日志？——因为它不是进程，没有"自我"；
- 为什么 Service 要配健康检查？——因为转发规则由 kube-proxy 维护，它需要知道后端哪些 Pod 活着。

10.2.2 Endpoints 与 EndpointSlice：后端名单

Service 的 selector 匹配到哪些"就绪的" Pod？这个动态名单由 **EndpointSlice** 对象维护（旧版本叫 Endpoints）：

```
$ kubectl get endpointslices -l kubernetes.io/service-name=user-service
NAME             ADDRESSES        PORTS
user-service-x7k  172.20.3.15,...  8080
```

就绪探针失败或正在终止的 Pod 会从这个名单里摘除——这正是"Service 自动避开不健康后端"的机制。注意：**Service 只认"就绪的"Pod**，所以第 4 章强调的 readinessProbe 在这里再次发挥着关键作用。

10.3 Service 的四种类型

`type` 字段决定 Service 的"暴露范围"，从小到大依次是：

10.3.1 ClusterIP（默认）：仅集群内部可访问

虚拟 IP 只在集群内有效。集群外的客户端无法访问。适合内部服务间调用——这应该也是你绝大多数 Service 的类型。

10.3.2 NodePort：在每个节点上开一个端口

```
spec:
  type: NodePort
  ports:
  - port: 80
    targetPort: 8080
    nodePort: 30080      # 可省略，自动分配（默认 30000-32767）
```

访问任意节点的 IP:30080，即可到达服务。原理：kube-proxy 在每个节点的这个端口上做了转发（依然走 DNAT，先转 ClusterIP 再转 Pod，或直接转 Pod）。

NodePort 的问题也很明显：端口号难记（30000+ 高位端口）、每个服务占一个端口、缺乏统一的 80/443 入口与 TLS 终止。**它适合临时调试或简单暴露，不适合作为生产入口**。

10.3.3 LoadBalancer：接入外部负载均衡器

```
spec:
  type: LoadBalancer
```

在支持的环境里（公有云、或装了 MetalLB 的自建集群），创建 Service 会自动开通一个外部负载均衡器，获得一个外部 IP。请求路径：外部 LB → 节点 → Pod。

注意两个现实问题：一是**每个 LoadBalancer 类型的 Service 都会开通一个 LB**，成本随服务数量线性增长——这就是为什么大家转而用 Ingress 统一入口（第 11 章）；二是自建环境默认没有 LB 实现，Service 会一直处于 `<pending>` 状态，需要部署 MetalLB 这类组件。

10.3.4 ExternalName：一个 DNS 别名

```
spec:
  type: ExternalName
  externalName: db.example.com
```

不转发任何流量，只是返回一条 CNAME DNS 记录。用途：给外部服务起个"集群内名字"，让代码统一用 `user-db` 这样的名字访问，将来后端从外部服务换成集群内服务时，只改 DNS 不改代码。

10.3.5 特殊成员：headless Service

第 8 章埋的伏笔在这里揭晓。`clusterIP: None` 的 Service 什么都不转发，它只做一件事：**为每个匹配的 Pod 生成一条 DNS 记录**。

```
# 普通 Service 的 DNS 解析：解析到虚拟 IP
$ nslookup user-service
10.96.34.120

# headless Service 的 DNS 解析：直接列出所有 Pod IP
$ nslookup redis-hs
172.20.5.11
172.20.5.12
172.20.5.13
```

客户端可以拿到全部后端地址自己做负载均衡，或者精确访问某个成员——StatefulSet 的成员互访正依赖于此。

10.4 服务发现：K8s 内置的 DNS

上一节反复出现 DNS。Kubernetes 集群内置了 DNS 服务（CoreDNS），自动为每个 Service 创建记录，格式：

```
<service-name>.<namespace>.svc.cluster.local
```

同命名空间内可以直接用短名：访问 `user-service` 自动补全为 `user-service.default.svc.cluster.local`。跨命名空间访问要带上命名空间：`user-service.team-b`。

Pod 的 DNS 解析由其 `/etc/resolv.conf` 配置完成（kubelet 自动注入），你可以 `kubectl exec` 进 Pod 验证。这套机制意味着：**服务发现是"零配置"的**——新服务上线即有域名，无需注册中心、无需配置更新。相比在应用里集成 Eureka/Consul 等注册中心，这是 Kubernetes 的一大红利。

> **深入：** CoreDNS 的配置可通过 `kube-system/coredns` 的 ConfigMap 定制（如添加自定义域、对接外部 DNS）。排障时 `nslookup` / `dig` 从 Pod 内测试是标准动作——"Pod 内域名解析失败"是高频问题，常见原因包括 NetworkPolicy 误伤 DNS 端口（53）、CoreDNS 异常、resolv.conf 的 ndots 问题等。

10.5 kube-proxy 的两种模式：iptables 与 IPVS

kube-proxy 把 Service 的转发规则写进内核，写法有两种主流模式，性能特征差异明显。

iptables 模式（默认）：为每个 Service 生成一组 iptables 规则，随机/轮询概率地选一个后端 DNAT 过去。规则是线性匹配的——Service 和后端越多，规则链越长，每个包的处理开销越大。几百个 Service 时没问题；上万 Service 时，规则更新慢（全量刷新）、转发延迟上升。

IPVS 模式：基于内核的 LVS/IPVS 模块，用**哈希表**存储转发规则，查找 O(1)，且原生支持多种负载均衡算法（rr 轮询、lc 最少连接、sh 源地址哈希）。大规模集群（数千 Service）应启用 IPVS。

判断当前模式：`kubectl get cm kube-proxy -n kube-system -o yaml | grep mode`（空字符串即 iptables 模式）。

> **提示：** 如果集群规模不大，默认 iptables 模式完全够用，不必为了"听起来高级"切换 IPVS——切换有操作风险。规模上去（Service 数千）或需要丰富 LB 算法时再切换。

还有一个更前沿的方向：**用 eBPF 直接实现 Service 转发**（Cilium 等项目），绕过 iptables 的性能天花板。eBPF 相关内容第 12 章会再提及。

10.6 Session 亲和与流量策略

两个常用的行为微调：

会话亲和（sessionAffinity）：默认行为是"每次请求随机选后端"。如果应用把状态存在本地内存、要求同一客户端连续请求打到同一 Pod（反模式，但现实存在），可以设置：

```
spec:
  sessionAffinity: ClientIP    # 同一来源 IP 固定打到同一 Pod
```

代价是破坏负载均衡的均匀性，某 Pod 重启时其"绑定"用户会短暂报错。**正确的方向是把应用改造成无状态**，而非依赖亲和。

trafficPolicy: Local：默认 `Cluster` 策略下，NodePort/LoadBalancer 的外部流量可能被转发到**其他节点**上的 Pod（多一跳网络）。`Local` 策略强制只转发到"收到流量的那个节点"上的 Pod——减少一跳延迟，且天然保留客户端源 IP（很多七层应用需要真实源 IP）。代价：本节点没有健康 Pod 时流量被丢弃。

10.7 本章小结

- **Pod IP 不可靠，Service 提供稳定入口**：ClusterIP + DNS 名字恒定，后端 Pod 动态变化。这是"总机 vs 手机号"的抽象。
- ClusterIP 是**虚拟 IP**：没有进程监听，靠 kube-proxy 在内核做 DNAT 转发；EndpointSlice 维护就绪后端名单；ping 不通是正常的。

- 四种类型按暴露范围递进：**ClusterIP（内部）→ NodePort（节点端口）→ LoadBalancer（外部 LB）**；ExternalName 是 DNS 别名；headless Service 返回全部 Pod IP，是有状态集群互访的基础。
- 服务发现靠内置 **CoreDNS**：`svc.ns.svc.cluster.local` 域名体系，新服务零配置即可被找到。
- kube-proxy 有 iptables（默认）与 IPVS（大规模）两种模式；`sessionAffinity` 与 `trafficPolicy: Local` 是两个常用微调。
- Service 只把流量给**就绪的** Pod——再次印证：readinessProbe 是流量安全的基石。

集群内部的服务间通信已经打通。但外部用户怎么进来？怎么统一管理 HTTPS 证书和路由规则？下一章请出集群的"大门"——Ingress 与 Gateway API。

第11章 Ingress 与 Gateway API：集群的大门

上一章解决了"集群内部互相访问"。但还有一个方向的问题没有解决：**外部用户如何访问集群里的服务？** 总不能给每个服务都开一个 NodePort，或者买一个 LoadBalancer 吧？

这一章的主角是 Ingress——把 HTTP(S) 流量统一收拢到"一个大门"，再按域名和路径分发到各个内部服务。

11.1 七层与四层：为什么需要 Ingress

先厘清一个容易混淆的概念。Service（NodePort/LoadBalancer）工作在**四层**（TCP/UDP）——它只认识 IP 和端口，看不懂 HTTP 的内容。而真实世界的入口需求是**七层**（HTTP）的：

- 根据域名分发：`shop.example.com` 到订单服务，`api.example.com` 到 API 服务；
- 根据路径分发：`/api/orders` 到订单服务，`/api/users` 到用户服务；
- 统一终止 TLS（HTTPS 证书集中管理）；
- 统一的重定向、限流、鉴权、灰度。

这些能力都需要"看懂 HTTP"。四层的 LoadBalancer 做不了，于是需要七层负载均衡器——也就是 Nginx 这类反向代理干的事。**Ingress 就是"用 Kubernetes 声明式方式管理七层代理规则"的机制**。

> **比喻：** 如果把集群比作一栋大楼，LoadBalancer 是给每个部门开一扇临街大门（又贵又乱），而 Ingress 是把所有访客统一引到**前台总台**，由前台根据"访客找哪个部门、办什么事"（域名、路径）把人带到对应楼层。

11.2 Ingress 与 Ingress Controller：规则与执行者

一个极其重要的认知：**Ingress 资源对象本身只是一份"规则"，它不会自己干活**。真正干活的，是集群里运行的 **Ingress Controller**（入口控制器）——一个七层代理程序，它 Watch 所有 Ingress 资源，把规则翻译成自己的配置并热加载。

这就像"红绿灯规则"（Ingress 资源）和"红绿灯设施"（Ingress Controller）的关系——只装了灯泡没有控制器，路口照样乱套。

常见的 Ingress Controller：

- **Ingress NGINX**：基于 Nginx，社区使用最广，功能全、文档多；
- **Traefik**：原生云代理，配置简洁，自带 Dashboard；

· **HAProxy Ingress**、**Envoy Gateway**、**Kong Ingress** 等。

> **警示：** 安装了 Ingress Controller 才有 Ingress 的效果。裸集群里创建 Ingress 资源，会显示创建成功但完全无效——这是新手最常见的"我创建了 Ingress 为什么访问不了"的原因。用 `kubectl get ingressclass` 确认控制器已安装并设为默认。

11.3 Ingress 实战：域名与路径路由

一个典型的 Ingress：

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: main-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx          # 指定用哪个控制器
  tls:
  - hosts:
    - shop.example.com
    secretName: shop-tls           # TLS 证书存在 Secret 里
  rules:
  - host: shop.example.com
    http:
      paths:
      - path: /api/orders
        pathType: Prefix
        backend:
          service:
            name: order-service
            port:
              number: 80
      - path: /api/users
        pathType: Prefix
        backend:
          service:
            name: user-service
            port:
              number: 80
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend
            port:
              number: 80
```

解读几个关键点：

pathType：`Prefix`（前缀匹配，`/api` 匹配 `/api/orders`）、`Exact`（精确匹配）、`ImplementationSpecific`（交给控制器解释）。1.19+ 的 API 要求显式指定，不再默认 Prefix。

tls 段：控制器读取 `shop-tls` 这个 Secret（类型 `kubernetes.io/tls`，包含 tls.crt/tls.key），为该域名终止 TLS。配合 cert-manager（自动从 Let's Encrypt 等签发并续期证书），可实现全自动 HTTPS。

traffic 流向：用户 → Ingress Controller 的 Pod（通过 LoadBalancer/NodePort 暴露）→ 按 host/path 匹配 → 转发到对应 Service → 最终落到某个 Pod。注意 Ingress 转发的对象是 Service，不是 Pod 直接对接。

11.3.1 注解：控制器的"方言"

细心的你注意到了 `annotations` 里的 `nginx.ingress.kubernetes.io/rewrite-target`。**Ingress 的标准 API 只定义了路由，而限流、重写、超时、跨域等行为都通过注解传递**，且注解前缀因控制器而异（`nginx.ingress.kubernetes.io/...` vs `traefik.ingress.kubernetes.io/...`）。

这意味着 Ingress 清单在不同控制器之间**不可移植**。这是 Ingress API 的历史包袱，也是 Gateway API 诞生的原因之一（见 11.5 节）。

常用 NGINX 注解速查：

```
nginx.ingress.kubernetes.io/limit-rps: "100"             # 每秒限流 100
nginx.ingress.kubernetes.io/proxy-body-size: "10m"       # 上传体积限制
nginx.ingress.kubernetes.io/proxy-read-timeout: "60"     # 读超时
nginx.ingress.kubernetes.io/enable-cors: "true"          # 跨域
nginx.ingress.kubernetes.io/canary: "true"               # 金丝雀 Ingress（见下）
```

11.4 用 Ingress 做灰度：canary 注解

第 7 章提过的"按 header/权重分流"，在 Ingress NGINX 里通过金丝雀注解实现。为同一个后端建第二个 Ingress，打上金丝雀标记：

```
metadata:
  name: main-ingress-canary
  annotations:
    nginx.ingress.kubernetes.io/canary: "true"
    nginx.ingress.kubernetes.io/canary-weight: "10"        # 10% 流量走这里
    # 或者按 header 精确分流：
    # nginx.ingress.kubernetes.io/canary-by-header: "X-Canary"
    # nginx.ingress.kubernetes.io/canary-by-header-value: "true"
spec:
  rules:
  - host: shop.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend-canary        # 注意：后端是新版服务
            port:
              number: 80
```

配合主 Ingress 使用：带 `X-Canary: true` 的请求（或 10% 随机流量）进入新版服务，其余走旧版。验证无误后，把主 Ingress 的后端切到新版，删除金丝雀 Ingress。这是中小团队实现灰度的轻量方案。

11.5 Gateway API：Ingress 的下一代

Ingress API 的设计局限越来越明显：

1. **注解方言化**：高级能力全靠注解，不可移植、难校验；
2. **角色不清**：基础设施团队和应用团队都编辑同一个 Ingress，权限交织；
3. **只覆盖 HTTP**：TCP/UDP/gRPC 的路由表达不了。

社区因此推出了 **Gateway API**（2023 年发布 1.0，正式 GA），它用一组更精细的资源重新定义入口：

- **GatewayClass**：定义"用哪类网关实现"（类似 IngressClass）——基础设施团队管；
- **Gateway**：声明"开一个网关实例，监听这些端口、挂这些证书"——平台团队管；
- **HTTPRoute / TCPRoute / GRPCRoute**：声明"这些流量去哪里"——应用团队管。

角色分离的设计让"平台管网关、业务管路由"成为可能；路由规则原生支持权重分流、header 匹配，不再需要注解方言。主流控制器（NGINX、Envoy Gateway、Traefik、Istio ambient 等）都已支持。

> **提示：** 新项目可以直接评估 Gateway API（尤其已有 Envoy/Istio 技术栈时）；存量 Ingress 没有必要紧急迁移——社区会长期支持 Ingress API。理解了 Ingress 的概念模型，Gateway API 上手非常快。

11.6 外部访问方案对比

把集群对外暴露的所有方案放在一张表里：

方案	层级	入口数量	成本	适用
NodePort	四层	每服务一个高位端口	零	临时调试
LoadBalancer	四层	每服务一个外部 IP	LB 费用 ×N	少量服务/直连需求
Ingress	七层	统一域名入口	1 个 LB + 控制器	HTTP 服务的标准方案
Gateway API	四/七层	统一，角色分离	1 个 LB + 控制器	新一代标准方案

生产环境的典型组合：**一个 LoadBalancer 把流量引到 Ingress Controller → Ingress 按域名/路径分发 → 后端 Service**。整个集群只需要一个（或一对主备）LB。

11.7 本章小结

- 外部访问收拢到**七层入口**是标准做法：统一域名、统一 TLS、按 host/path 分发，比每服务一个 LB 便宜且好管理。
- **Ingress 是规则，Ingress Controller 才是执行者**——没装控制器，Ingress 就是废纸；IngressClass 指定用哪个控制器。
- 注解是 Ingress 的"方言"：限流、重写、金丝雀都靠它，但不同控制器注解互不兼容。
- **Gateway API** 用 GatewayClass/Gateway/Route 三层资源实现角色分离与原生高级路由，是 Ingress 的继任者。
- 生产组合拳：LoadBalancer → Ingress Controller → Service → Pod，全集群一个入口。

至此，"流量怎么进来、怎么分发"已经清楚。但你可能还有更深一层的疑问：这些转发到底是怎么在内核里发生的？Pod 的 IP 是谁分配的？不同节点上的 Pod 为什么能直接互通？下一章，我们潜到网络的水面之下，看数据包的完整旅行。

第12章 网络原理：一个数据包的旅行

前两章我们讲了 Service 和 Ingress 这两张"地图"，这一章潜入水面之下，看看地图之下的真实地形：**Pod 的 IP 从哪来？不同节点的 Pod 凭什么直接互通？一个数据包从 Pod A 到 Pod B，到底经历了什么？**

这一章的内容偏底层，但值得啃下来——90% 的"网络玄学问题"（Pod 间不通、DNS 偶发超时、NodePort 时通时不通），只有理解了底层原理才能定位。

12.1 Kubernetes 网络模型的四条铁律

Kubernetes 对网络做了一个大胆而简单的约定，任何 CNI 插件只要满足这个模型就合格。核心是四条规则：

1. **Pod 之间**（无论在不在同一节点）：直接用 Pod IP 通信，**无需 NAT**；
2. **节点与 Pod 之间**：直接通信，无需 NAT；
3. **Pod 看到的自身 IP**：就是别人看到的它的 IP（地址不做二次翻译）；
4. **每个 Pod 有独立 IP**：Pod 内容器共享之（第 4 章讲过）。

注意这个模型的激进之处：**扁平网络**。整个集群里所有 Pod 共处一个"扁平"的地址空间，任意两点直连——这与传统虚拟机/物理机"网段划分、边界防火墙"的思路完全不同。好处是应用视角极简（Pod 间就是"局域网直连"），坏处是默认没有任何隔离（所以需要 NetworkPolicy，第 17 章）。

至于**如何实现**这个扁平网络，Kubernetes 把决定权完全交给 **CNI 插件**——这就是"接口与实现分离"的设计。

12.2 微观机制：网络命名空间、veth pair 与 pause 容器

在讲 Pod IP 从哪来之前，我们先潜到最底层，看清三个 Linux 内核机制——理解它们，Pod 网络的"魔法"就透明了。

12.2.1 网络命名空间：每个 Pod 一座"孤岛"

网络命名空间（Network Namespace，简称 netns） 是 Linux 内核提供的一种隔离机制，它让每个命名空间拥有**独立的一套网络资源**：自己的网卡（网络接口）、自己的 IP 地址、自己的路由表、自己的端口空间、自己的 iptables 规则。

这意味着：

· 不同 netns 里的进程，可以监听**同一个端口**而互不冲突——因为端口是"每个 netns 一份"的；

- 不同 netns 里的网络设备、路由互不可见，天然隔离；
- 想通信？必须通过某种方式"架桥"——这就是 veth pair 出场的地方。

Pod 的"共享网络"的本质，就是**把 Pod 里的所有容器放进同一个 netns**。这也是第 4 章说"同 Pod 内容器共享 IP、共享端口空间、localhost 互通"的底层原因：它们根本就在同一个网络命名空间里，localhost 就是它们自己。

12.2.2 veth pair：一根"两头都有插头"的虚拟网线

veth pair 是 Linux 的虚拟以太网设备，它**成对出现**，像一根网线，但两头都是虚拟网卡：

- 一端放进 Pod 的 netns，被命名为 `eth0`——Pod 里的进程看到的"网卡"就是它；
- 另一端留在宿主机（节点）的网络命名空间，通常插到网桥上。

数据从 Pod 的 eth0 发出，会立刻出现在宿主机那一端；反之亦然。**veth pair 是"把 netns 和外部世界连起来"的标准工具**。

12.2.3 pause 容器：为 Pod 守住网络命名空间的"房主"

一个容易困惑的问题是：Pod 里的容器是各自启动的，它们怎么"共用"同一个 netns？答案藏在每个 Pod 里那个永远睡眠的 **pause 容器**（也叫 sandbox 容器）身上：

1. 容器运行时**先创建 pause 容器**，并为它建立 Pod 的网络命名空间（创建 netns、创建 veth pair、把 eth0 放进 netns、配置 IP 和路由）；
2. 之后启动的**业务容器**，只需"加入"pause 容器已经建好的这个 netns，就能共享同一套网络；
3. pause 容器本身不做任何事（就是 `pause()` 一个死循环），但**只要它活着，netns 就活着**——业务容器随便重启、退出，都不会影响 Pod 的网络环境。

> **比喻：** pause 容器像租房的"主承租人"：它签下房子（netns）的租约、装好水电（veth、IP、路由）；其他容器是"合租室友"，直接搬进来共享这套设施。主承租人只要不搬走，房子就一直都在，室友来来去去都不影响。

12.2.4 cni0 网桥：节点上的"交换机"

节点上通常有一个**网桥（bridge）**（常见名字是 `cni0`，Flannel/Calico 等都用它或类似机制）。所有 Pod 的 veth 对端都"插"在这个网桥上。网桥工作在二层（链路层），负责把数据帧在插在它上面的各个接口之间转发——本质上就是**一台虚拟交换机**。

于是同节点通信的最简路径就出来了：

```
Pod A (eth0) → veth 对 → cni0 网桥 → veth 对 → Pod B (eth0)
```

网桥根据 MAC 地址表转发：帧从 Pod A 的 veth 进来，网桥查到 Pod B 的 MAC 地址在另一个 veth 接口上，就把帧从那个口转发出去。整个过程发生在节点内核里，**根本不经过物理网卡**，所以同节点 Pod 通信又快又稳。

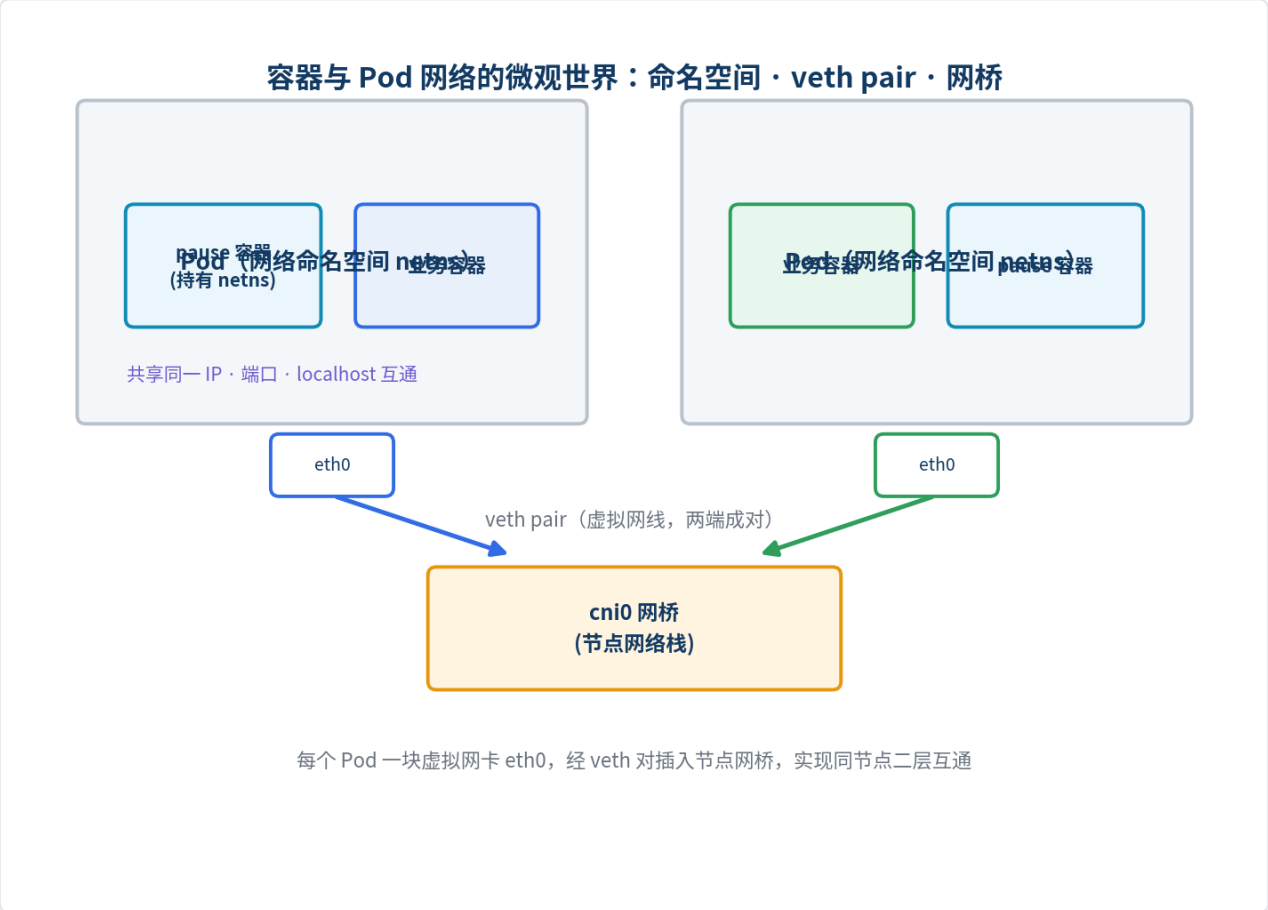


图12-1 容器与 Pod 网络的微观结构：pause 容器持有 netns，veth pair 接入 cni0 网桥

12.2.5 小结：三层递进的抽象

把上面的内容压缩成一句话：**Pod 的网络 = 一个 netns（由 pause 容器持有），通过 veth pair 接入节点的 cni0 网桥，进而连入整个集群网络。** 理解了这一层，后面讲"Pod IP 从哪来"和"跨节点怎么通"就都水到渠成了。

12.3 Pod IP 是怎么来的：IPAM 与 CNI 插件链

CNI（Container Network Interface）是一套标准接口：kubelet 创建 Pod 的网络命名空间时，按配置依次调用一组 CNI 插件的二进制程序。执行链条大致是：

1. kubelet 调用容器运行时创建 Pod 的 sandbox（pause 容器）；
2. 运行时调用 **IPAM 插件**（如 host-local、各 CNI 自带的地址管理器）从地址池里**分配一个 Pod IP**；

3. 运行时调用**主 CNI 插件**（calico/cilium/flannel），把网络接口（veth 对）接入 Pod 命名空间、配置路由；
4. 节点上（通常还有）**portmap** 等辅助插件，把 HostPort/NodePort 的映射规则准备好。

Pod IP 的地址池按**节点**切分：每个节点分到一段（如 172.20.1.0/24），本节点的 Pod 从本段取地址。这样路由表可以按网段聚合——"去 172.20.1.0/24 的包发给 node-1"——无需维护每 Pod 一条路由。

12.4 数据包的旅行：同节点与跨节点

12.4.1 同节点：veth 对与网桥

同一节点上两个 Pod 通信，是最简单的场景。每个 Pod 的网络命名空间里有一块虚拟网卡（eth0），它的另一端（veth pair 的对端）"插"在节点的网络栈里（连接到 cni0 网桥或直接由插件管理）。数据包路径：

```
Pod A (eth0) → veth → 节点网桥 → veth → Pod B (eth0)
```

这就是一台"虚拟交换机"——同一节点上的 Pod 处于同一个二层广播域，包走二层转发，几乎没有开销。

12.4.2 跨节点：三种主流方案

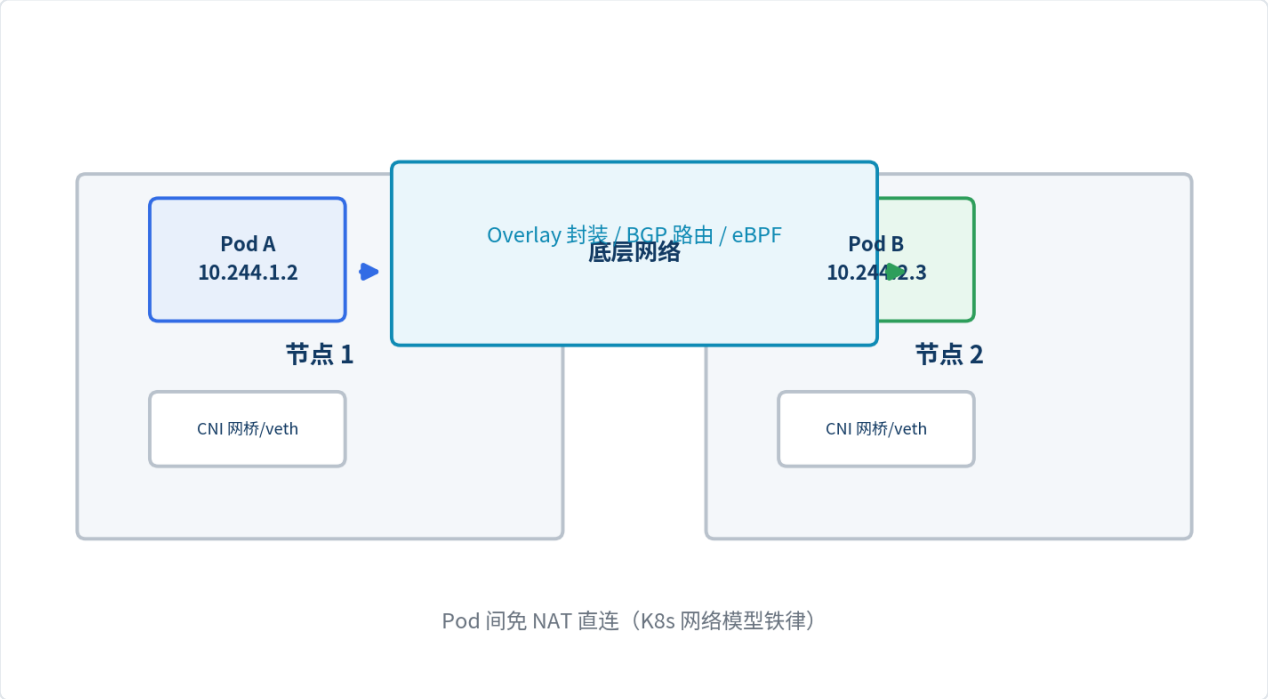


图12-2 跨节点 Pod 通信：Overlay 封装、BGP 路由或 eBPF 直通

真正有技术含量的是跨节点通信：Pod A 在 node-1（IP 10.0.0.1），Pod B 在 node-2（IP 10.0.0.2），A 的包怎么到 B？三大流派：

流派一：Overlay 隧道（封装）。代表：Flannel VXLAN、Calico IPIP 模式。把 Pod 的原始包封装进新的"外层包"（外层源目地址是节点 IP），像把信装进信封，通过普通网络寄到对端节点再拆封。

> [外层：node1 → node2 | 内层：PodA → PodB] → 到达 node2 后拆封 → 交给 PodB

优点：对底层网络要求极低（只要节点间三层可达即可），兼容一切环境；缺点：封装有开销（MTU 变小、CPU 加解封）、抓包排障困难（要抓两层）。

流派二：路由（BGP）。代表：Calico 默认模式。不封装！让每个节点充当一台路由器，通过 BGP 协议向集群广播路由："172.20.1.0/24 在我这儿"。包从 node-1 直接三层路由到 node-2，**全程不封装不拆封**。

优点：性能好、排障直观（包就是原始包）；缺点：要求底层网络支持——路由信息要么节点间直连交换（同二层网络可直通），要么要把 BGP 路由"外泄"给机房的交换机（ToR 对接），跨子网/云环境部署较麻烦。

流派三：eBPF 原生加速。代表：Cilium。用 eBPF 程序直接在内核的数据路径上做转发决策，绕过 kube-proxy 的 iptables，甚至可以绕过 TCP 栈的部分环节。性能最优，还能提供 L7 感知的网络策略与可观测性。缺点：内核版本要求高（4.19+，建议 5.10+），复杂度也最高。

选型上没有绝对赢家，经验法则是：**环境复杂、跨云跨网段 → overlay 最省心；追求性能、网络可控 → Calico 路由模式；追求极致性能与可观测、内核新 → Cilium**。

12.5 Service 流量在内核里的真实路径

有了 Pod 网络的基础，现在可以完整回答第 10 章留下的问题：**访问 ClusterIP 的包，到底经历了什么？**

以 Pod A 访问 Service（ClusterIP 10.96.34.120）为例，iptables 模式下：

1. 包从 Pod A 出发：目的 IP = 10.96.34.120，目的端口 80；
2. 包到达节点网络栈，经过内核的 **nat 表 PREROUTING → OUTPUT 链**，命中 kube-proxy 写入的规则；
3. 规则做两件事：**DNAT**——把目的地址改成随机（概率加权）选中的后端 Pod IP（172.20.3.15）；同时做 **conntrack（连接跟踪）记录**，保证回程包能正确改回；
4. 包按新的目的地址正常路由——如果后端 Pod 在本节点，走网桥直送；在其他节点，走 12.4 的跨节点方案送过去；
5. 后端 Pod 收到包，看到的目的是自己的 IP:8080（targetPort），毫不知情地"以为"对方直接找的它；
6. 回程包按 conntrack 记录对称改写，原路返回。

几个由此衍生的实用认知：

为什么 Service 的 IP ping 不通？ ping 是 ICMP，nat 表的规则只匹配 TCP/UDP，ICMP 包没有 DNAT 目标，直接被丢弃。

为什么连接"粘"在一个 Pod 上？ conntrack 以"连接"为粒度（五元组），一条已建立的连接会被持续 NAT 到同一个后端。所以长连接（如 gRPC）会固定在一个后端上——如果后端 Pod 挂了，这条连接就断了（不会自动切走）。这就是 gRPC 在 K8s 里需要客户端负载均衡（如 headless Service）的根因。

为什么 terminating 的 Pod 还能收到请求？ conntrack 里的存量连接会继续转发，直到连接结束。这正是第 4 章 preStop sleep 方案要对抗的窗口。

12.6 DNS 链路：从短名到 IP 的旅程

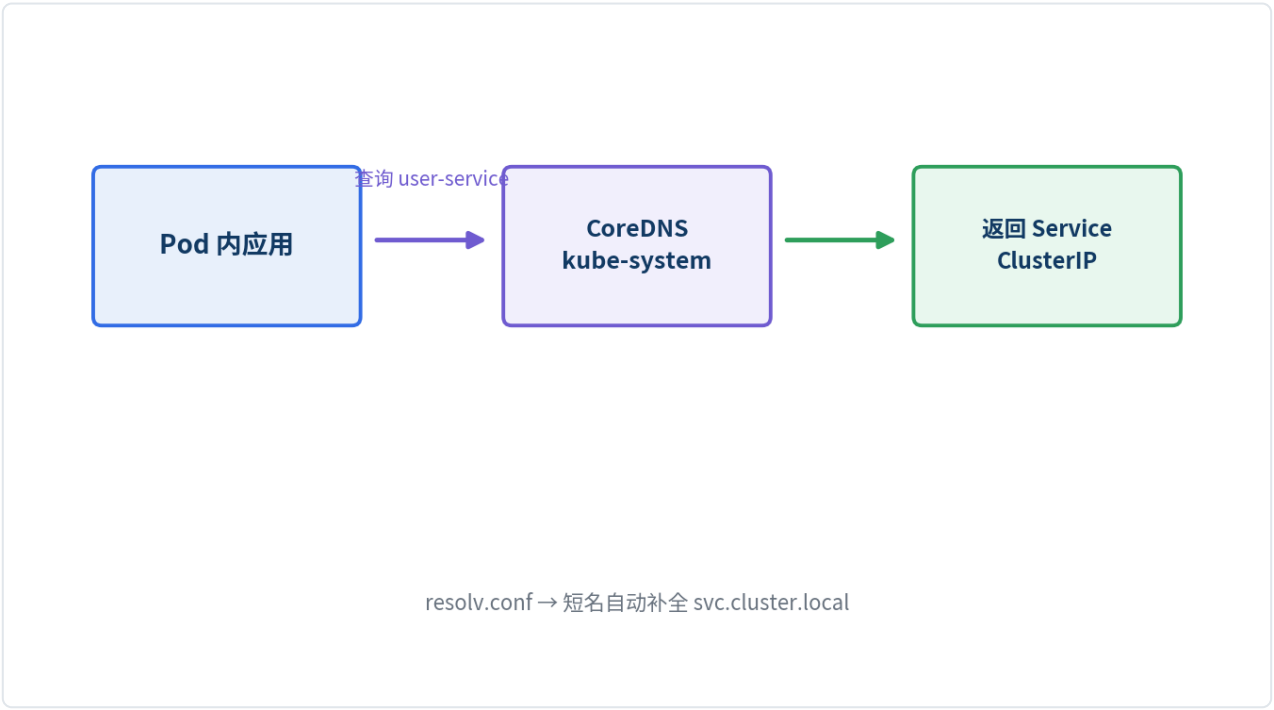


图12-3 服务发现的 DNS 链路：Pod 查询 CoreDNS 后获得 Service 地址

Pod 里 `curl user-service` 的域名解析链路同样值得走一遍：

1. 容器内的 `/etc/resolv.conf` 指向 CoreDNS 的 Service（通常是 `kube-dns.kube-system`，ClusterIP 10.96.0.10）；
2. Pod 发起 DNS 查询（UDP 53），同样经过上面的 NAT 机制到达 CoreDNS；
3. CoreDNS 按 `<ns>.svc.cluster.local` 的规则查内部记录，返回 Service 的 ClusterIP（或 headless 的 Pod IP 列表）；
4. 拿到 IP 后，按 12.5 的路径发包。

这里有个著名的 **ndots 坑**：Kubernetes 默认给 Pod 的 resolv.conf 配置 `ndots:5`——意思是"域名中的点少于 5 个时，先把它当作短名，依次追加搜索域尝试解析"。于是你访问一个外网域名 `api.github.com`（2 个点），解析顺序会变成：

```
api.github.com.default.svc.cluster.local   → 失败
api.github.com.svc.cluster.local           → 失败
api.github.com.cluster.local               → 失败
api.github.com                             → 成功
```

每次外部域名访问都要先白跑 3 次集群内查询。在高并发场景下，这会给 CoreDNS 制造可观的无效负载，DNS 偶发超时（5 秒卡顿）多源于此。缓解办法：外部调用直接用绝对域名（末尾加点 `api.github.com.`）、在 Pod spec 里自定义 dnsConfig 降低 ndots、或用 NodeLocal DNSCache 就近缓存。

12.7 NetworkPolicy：给扁平网络装上"门禁"

扁平网络默认"处处直通"——任何 Pod 可以访问任何 Pod 的任何端口。这在多租户、合规场景下不可接受。**NetworkPolicy** 是标准的网络隔离机制：

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-orders-only
  namespace: app
spec:
  podSelector:
    matchLabels:
      app: user            # 规则作用于谁：app=user 的 Pod
  policyTypes: ["Ingress"]
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: orders      # 只允许 app=orders 的 Pod 访问
    ports:
    - port: 8080
```

语义要点：**policyTypes 里的方向，一旦有策略选中某组 Pod，未明确允许的流量全部拒绝**（默认拒绝）；本例中 `app=user` 的 Pod 只接受来自 `app=orders` Pod 的 8080 端口流量，其余一律丢弃。

> **警示：** NetworkPolicy 必须由 CNI 插件**支持并实现**才生效。Flannel 就不支持（策略写了不报错但等于没写）；Calico、Cilium 支持。一个常见的"假安全"：团队写了 NetworkPolicy，用

> 的 CNI 却不实现它——流量照样畅通。务必验证：`kubectl` 创建一个测试 Pod，尝试被禁止方向的访问，确认真的不通。

Cilium 还提供 L7 层策略（允许"GET /api/健康检查"但拒绝"DELETE"），以及更易读的 Hubble 可观测面板，适合安全要求高的场景。

12.8 网络排障的"手术刀"

最后奉上一套实用的网络排障工具箱：

```
# 1. 验证 DNS：进入任意 Pod
kubectl exec -it debug-pod -- nslookup user-service

# 2. 验证 Service 连通：curl ClusterIP 与域名
kubectl exec -it debug-pod -- curl -sv http://user-service:80

# 3. 查看 Service 的后端是否正常
kubectl get endpointslices -l kubernetes.io/service-name=user-service

# 4. 观察节点上的转发规则（iptables 模式）
iptables-save | grep user-service

# 5. 抓包（需要 ephemeral 容器或特权调试 Pod）
kubectl debug node/node-1 -it --image=nicolaka/netshoot
tcpdump -i any host 172.20.3.15 -w /tmp/capture.pcap

# 6. 查看连接跟踪
conntrack -L | grep 10.96.34.120
```

> **提示：** 建议常备一个 `netshoot`（含 tcpdump、dig、curl、mtr 等全套网络工具）的调试 Pod 模板，出问题时临时部署到目标命名空间——它是 K8s 网络排障的"瑞士军刀套装"。

12.9 本章小结

- K8s 网络模型的灵魂是**扁平、免 NAT 的 Pod 直连网络**；实现完全交给 CNI 插件，Pod IP 由节点级的 IPAM 按网段分配。
- 跨节点通信三大流派：**Overlay 封装**（兼容性最好）、**BGP 路由**（性能好但要求网络可控）、**eBPF 原生加速**（性能与可观测最佳，内核要求高）。
- ClusterIP 流量靠内核 **DNAT + conntrack**：ping 不通、长连接粘滞、terminating 收包，都能从这套机制找到解释。

- DNS 链路经过 CoreDNS，**ndots 默认 5** 会拖慢外部域名解析——高并发场景要主动优化。
- **NetworkPolicy 是"默认拒绝"式的门禁，但前提是 CNI 支持**；部署后必须实测验证。
- 网络排障靠一套固定组合拳：DNS 验证 → 连通性验证 → 端点检查 → 内核规则 → 抓包。

网络部分完满收官。接下来的问题是：应用产生的**数据**怎么办？Pod 会死，盘不能亡。下一章进入存储部分，从最简单的 Volume 讲到完整的持久化体系。

第13章 Volume：把数据钉在地板上

容器有个让无数新人栽跟头的特性：**容器里的文件系统是临时的**。容器一删除，里面的一切灰飞烟灭——代码、日志、上传的文件，全没了。这一章开始解决数据的问题，先从最基础的 Volume（存储卷）讲起。

13.1 为什么容器没有"记忆"

回忆一下第 4 章的内容：容器的文件系统来自**镜像的分层叠加**。镜像的各层是只读的，容器启动时在最上面挂一层**可写层**，进程的所有写入都落在这层上。

问题就出在这个可写层：它依附于容器本身的生命周期。容器删除，可写层即被回收。而且它还有第二个缺点：**性能差**——多层叠加的文件系统（overlayfs）做随机写、小文件操作比原生文件系统慢不少，数据库这类 IO 密集应用绝不能把数据写在这里。

解决方案就是 **Volume**：把一块"外挂的存储"挂载到容器的某个路径上，让数据写到卷里而不是可写层里。卷的生命周期与容器解耦——容器死了，卷可以继续存在，被下一个容器接手。

> **比喻：** 容器像酒店房间——退房（删除）后一切归零；Volume 是你自带的**保险柜**，房间换了，保险柜里的东西还在，跟着你搬去下一个房间。

13.2 常用 Volume 类型分拣

Volume 类型很多，按用途可以分成三类，记住分类比记住全部类型更重要。

第一类：临时卷——生命周期与 Pod 相同

- **emptyDir**：一个空目录，Pod 创建时创建、Pod 删除时销毁。多容器共享（第 4 章的日志 Sidecar 就用它）、临时缓存（如/tmp）。注意：**Pod 被驱逐或节点故障，数据就没了**。
- **downwardAPI**：把 Pod 的元数据（标签、注解、资源限制）以文件形式暴露给容器。

第二类：宿主机卷——直通节点的文件系统

- **hostPath**：把节点的某个目录/文件挂进容器。DaemonSet 的最爱（读节点日志、/proc、设备文件），但**危险**：Pod 重建后若被调度到另一个节点，"原来的数据"就找不到了（数据绑在节点上，不跟着 Pod 走），而且挂载敏感路径（如根目录、/var/run/docker.sock）会带来严重安全风险（第 17 章的提权风险案例）。

第三类：持久卷——与 Pod、节点都解耦

- **PVC**（PersistentVolumeClaim 的引用）：这才是"正经持久化"的正确姿势——数据存在集群外的存储系统里（云盘、Ceph、NFS……），与 Pod 的生死、所在节点完全无关。细节下一章展开。

> **提示：** 判断标准一句话：**数据跟着 Pod 生死的用 emptyDir，数据跟节点绑定的用 hostPath（仅限系统级组件），数据要超越两者生命的用 PVC。**

13.3 emptyDir 实战

```
apiVersion: v1
kind: Pod
metadata:
  name: cache-pod
spec:
  containers:
  - name: app
    image: myapp:2.1
    volumeMounts:
    - name: tmp-cache
      mountPath: /tmp/cache
  - name: sidecar
    image: busybox:1.36
    command: ["sh", "-c", "tail -f /shared/log/app.log"]
    volumeMounts:
    - name: tmp-cache
      mountPath: /shared
  volumes:
  - name: tmp-cache
    emptyDir:
      medium: Memory     # 用内存当临时盘（tmpfs），速度快但占内存
      sizeLimit: 256Mi   # 必须限制，否则内存型卷可吃光节点内存
```

`medium: Memory` 的 emptyDir 就是 tmpfs——读写飞快，但数据在内存里，且**不计入容器的内存 limits，计入节点的消耗**，所以必须配 `sizeLimit`，否则可能把节点内存吃穿引发驱逐连锁反应。

13.4 Volume 挂载的进阶技巧

13.4.1 subPath：一个卷多处挂载的不同子目录

同一个卷的不同 Pod（比如多个 Job 共享一块盘）希望各用各的子目录，用 `subPath`：

```
volumeMounts:
- name: shared-data
  mountPath: /data
  subPath: worker-1     # 只挂载卷里的 worker-1 子目录
```

13.4.2 把 ConfigMap 挂成配置文件

Volume 的妙用之一是把配置文件"注入"容器：

```
volumes:
- name: nginx-conf
  configMap:
    name: nginx-config     # 引用第 15 章讲的 ConfigMap
containers:
- name: nginx
  volumeMounts:
  - name: nginx-conf
    mountPath: /etc/nginx/conf.d
    readOnly: true
```

ConfigMap 更新后，挂载的文件会**自动更新**（有分钟级延迟），Nginx reload 后即可生效——这是"配置热更新"的标准姿势，第 15 章详述。

13.4.3 projected 卷：多来源合挂

一个 Pod 需要 ConfigMap + Secret + downwardAPI 三种信息？可以用 projected（投影）卷合并成一个挂载点，减少 mount 次数，也更清晰：

```
volumes:
- name: all-in-one
  projected:
    sources:
    - configMap: {name: app-config}
    - secret: {name: app-tls}
    - downwardAPI:
        items:
        - path: labels
          fieldRef: {fieldPath: metadata.labels}
```

13.5 容器存储的性能与最佳实践

一、能不落盘就不落盘。 无状态设计（十二要素第 6 条）永远是首选：状态外置到数据库、对象存储、消息队列，容器里只跑逻辑。Volume 是兜底手段，不是设计借口。

二、emptyDir 的三个高频用途。 共享文件（Pod 内多容器协作）、临时大文件缓存（避免写爆可写层）、以及 checkpoint 交换。凡是"丢了也不心疼"的数据都可以用它，且**比写可写层快**。

三、警惕 hostPath 的两个坑。 坑一：数据绑节点，Pod 漂移即失联；坑二：安全风险——挂载 `/` 或 Docker socket 等于把节点钥匙交给容器（第 17 章 Pod Security 会拦截高危 hostPath）。业务负载原则上禁用 hostPath，仅基础设施组件（DaemonSet）按需使用。

四、监控磁盘使用。 emptyDir、容器可写层、节点根盘是共享的。日志失控、缓存膨胀都会写满节点磁盘，触发 DiskPressure 驱逐，殃及整节点 Pod。给日志加轮转（logrotate/容器日志驱动）、给缓存设上限、给 emptyDir 设 sizeLimit，是三条基本卫生习惯。

13.6 本章小结

- 容器可写层**随容器销毁**且性能一般，持久化必须走 Volume 外挂。
- Volume 三分类记心间：**临时卷**（emptyDir，随 Pod 生死）、**宿主机卷**（hostPath，随节点，慎用）、**持久卷**（PVC，跨生死跨节点，生产标配）。
- subPath 精挂子目录、ConfigMap 挂成文件、projected 合并挂载，是三个高频进阶技巧。
- 内存型 emptyDir（tmpfs）快但耗内存，务必设 sizeLimit。

emptyDir 解决不了"数据要活得比 Pod 长"的问题。真正的持久化——PV、PVC、StorageClass 这套体系，正是下一章的主角：存储的"供给侧改革"。

第14章 PV、PVC 与 StorageClass：存储的供给侧改革

上一章的 Volume 解决了"挂个盘"的问题，但生产级的持久化存储有一连串更深的问题：盘从哪里来？谁负责开通？用户申请和运维供给怎么对接？不同存储后端（云盘、Ceph、NFS）怎么统一管理？

这一章的 PV/PVC/StorageClass 三件套，就是 Kubernetes 给出的标准答案。这套设计思路值得细品——它本质上是把"存储的供给"与"存储的消费"解耦，做了一次**供给侧改革**。

14.1 三个角色：库房、租约申请、自动供应商

先建立整体图景，三个角色各司其职：

- **PersistentVolume（PV，持久卷）**：一块**已经实际存在**的存储资源，是集群级的"库房库存"。它对应一块真实的云盘、一个 Ceph 卷、一个 NFS 导出目录。PV 由管理员（或 StorageClass 自动）创建。
- **PersistentVolumeClaim（PVC，持久卷声明）**：用户对存储的**申请单**——"我要 100GB、可读写单节点、SSD"。用户创建 PVC，系统把它和合适的 PV 绑定。**PVC 是 Pod 里实际引用的对象**。
- **StorageClass（存储类）**：存储的"供货目录"+"自动供应商"——定义有哪些档位的存储（快速 SSD、大容量 HDD、共享 NFS），以及**自动开通**（动态供应）的能力。

> **比喻：** 这套设计像现代公寓的租赁体系：StorageClass 是"房源供应商目录"（有精装公寓、普通住宅、合租房等类型）；PVC 是租客提交的**租房申请**（我要 100 平、带电梯、月付）；PV 是**实际到手的房源**。租客不用关心房源从哪里来（管理员手动提供，还是供应商自动开通），只要申请被批准，钥匙就到手。

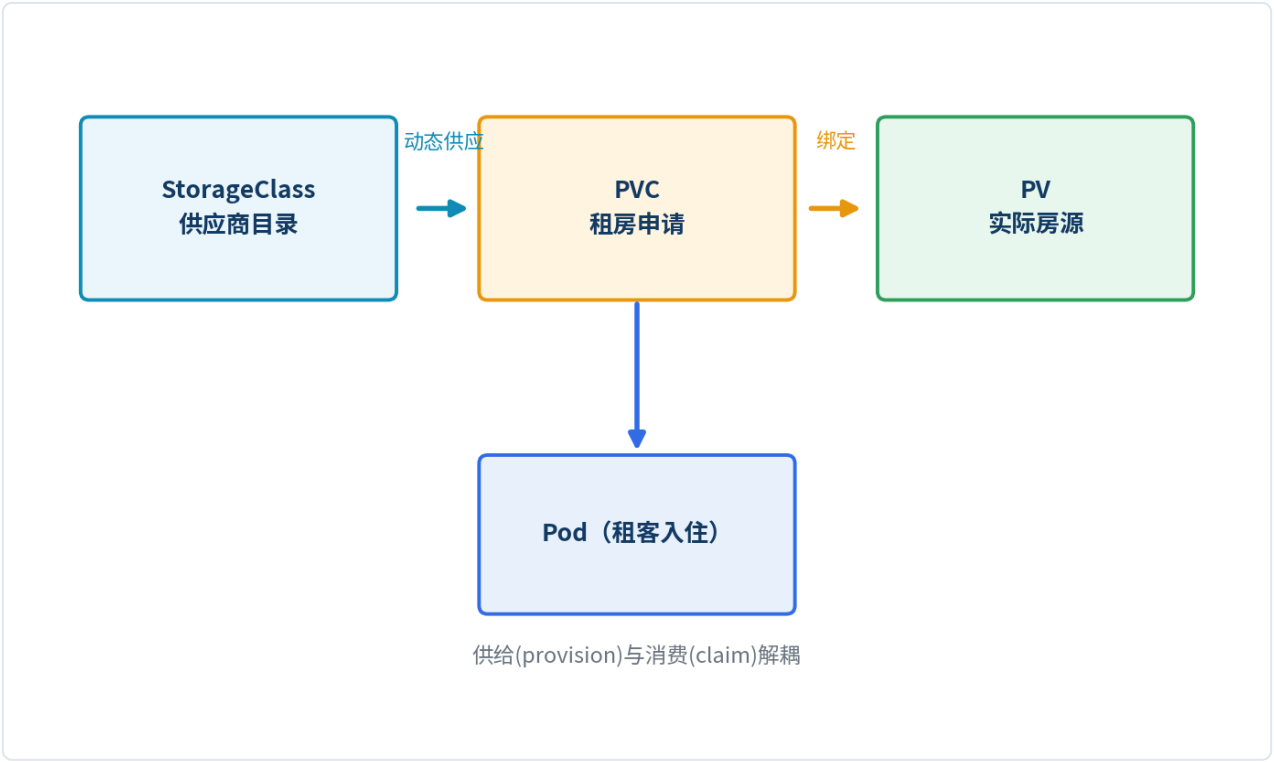


图14-1 PV/PVC/StorageClass 三者关系：供给与消费解耦

为什么要这么绕？直接让 Pod 挂载云盘不行吗？不行——那样存储的使用细节（云盘 ID、挂载参数）就硬编码进了应用清单，换环境、换存储后端都要改应用配置。PVC 这层抽象让**应用声明"我要多少什么样的存储"，而无需知道存储的具体实现**——应用可移植性的关键一环。

14.2 PVC：用户的申请单

一个 PVC 的定义：

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-claim
spec:
  accessModes:
  - ReadWriteOnce          # 访问模式
  resources:
    requests:
      storage: 50Gi        # 容量
  storageClassName: fast-ssd   # 指定存储类
```

accessModes（访问模式） 三种取值的真实含义，新手最容易误解：

- `ReadWriteOnce`（RWO）：**一个节点**读写（不是"一个 Pod"！）。同一节点上的多个 Pod 可以共享它；

- `ReadOnlyMany`（ROX）：多个节点只读挂载；
- `ReadWriteMany`（RWX）：多个节点读写——**只有支持共享文件系统的存储**（NFS、CephFS、云厂商的文件存储）才能做到；块设备云盘（EBS 类）做不到。

> **提示：** 想让多个 Pod（在不同节点上）同时读写同一份数据？RWO 的块存储做不到，必须选 RWX 的共享文件存储，或者改架构（各 Pod 一块独立盘 + 上层同步）。这是"多副本服务共享盘"需求的常见误区。

容量：PVC 申请 50Gi。动态供应时，新卷的实际大小取决于 StorageClass 的 `allowVolumeExpansion`（扩容）与供应逻辑。

14.3 StorageClass：动态供应的魔法

StorageClass 包含三件事：**供应者（provisioner）、参数（parameters）、回收策略（reclaimPolicy）**。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: ebs.csi.aws.com        # 由谁供货（云盘 CSI 驱动）
reclaimPolicy: Delete               # 卷删除时：删盘 or 留盘
allowVolumeExpansion: true          # 允许在线扩容
parameters:                         # 传给供应者的参数
  type: gp3
  encrypted: "true"
```

动态供应（Dynamic Provisioning）的完整流程——这是理解整个体系的钥匙：

1. 用户创建 PVC（申请 50Gi、storageClass `fast-ssd`）；
2. 集群里的 **CSI 供应控制器**发现这块"无人认领"的 PVC；
3. 它调用云厂商 API（或 Ceph 等），**真实地创建一块 50Gi 的盘**，并生成对应的 PV 对象；
4. PV 与 PVC **绑定**（Bound）；
5. Pod 引用这个 PVC，kubelet 调用 CSI 驱动把盘**挂载**到 Pod 所在节点、再挂进容器。

也就是说：**用户只提交了一张申请单，一块真实的盘凭空出现、自动挂载**。对比静态供应（管理员手动买盘、手动建 PV），这就是"供给侧改革"——供给由系统按需自动完成。

14.3.1 默认 StorageClass 与 immediately 的坑

一个高频坑：**PVC 一直 Pending**。原因排查顺序：

1. 命名的 StorageClass 不存在？`kubectl get sc` 确认；
2. 没写 storageClassName 且集群没有**默认 StorageClass**（`(default)` 标记）？PVC 无人供应，永远 Pending；
3. StorageClass 的 provisioner 没部署（CSI 驱动没装）？
4. 配额不足、区域无货等云侧问题——看 PVC 的 Events。

另一个经典坑是**可用区不匹配**：云盘与 Pod 不在同一可用区，挂载失败。多可用区集群需要 StorageClass 配置 `topology`（拓扑感知）或使用延迟绑定（`volumeBindingMode: WaitForFirstConsumer`——先等 Pod 调度定节点，再在对应可用区开盘）。**强烈建议多 AZ 环境用 WaitForFirstConsumer 模式**。

14.4 回收策略：Delete 与 Retain 的生死抉择

当 PVC 被删除时，它绑定的 PV 怎么办？由 `reclaimPolicy` 决定：

- **Delete**：PV 连同真实存储一起删除——**数据永久消失**。动态供应默认 Delete，方便但危险；
- **Retain**：PV 进入 Released 状态保留，盘还在，数据可救。需要管理员手动处理。

生产建议：**重要数据的 StorageClass 用 Retain**（宁可多花盘钱，不可丢数据）。或者用 Delete 但配合存储侧的快照/回收站机制。

再强调一次 StatefulSet 的相关行为（第 8 章）：缩容 StatefulSet、删除 Pod 都**不会删 PVC**，数据安全；但 `kubectl delete statefulset` 也只删工作负载。真正要小心的是**删除 PVC** 和**删除 namespace**（会连带删除其中所有 PVC！）。删 namespace 前先检查里面的 PVC——这是删库跑路的经典入口。

14.5 CSI：存储的统一接口

StorageClass 里的 `provisioner: ebs.csi.aws.com` 引出了一个关键词——**CSI（Container Storage Interface）**，它是存储插件的行业标准接口（与网络的 CNI、运行时的 CRI 并列）。

CSI 驱动实现了三类操作：**创建/删除卷**（对应动态供应）、**Attach/Detach**（把云盘"插"到节点）、**Mount/Unmount**（把卷挂进容器）。kubelet 与控制器通过 gRPC 调用驱动完成这一切。

这套接口让存储生态大爆发——你现在可以像装应用一样 `kubectl apply` 部署一个存储后端（如 Rook 部署 Ceph），然后它就自动成为集群的 StorageClass 供货源。常见选择：

场景	代表方案
云上块存储	各云 CSI 驱动（EBS/PD/云盘）
自建分布式块/文件	Rook + Ceph
共享文件	NFS Server Provisioner、云文件存储（EFS/NAS）
本地高性能盘	local-path-provisioner、OpenEBS LocalPV

> **提示：** 本地开发/测试环境可以用 `local-path-provisioner`（一行 Helm 命令装好），让 PVC 动态供应在裸环境里也能工作，避免"测试环境没法持久化"的尴尬。

14.6 卷快照与扩容

两个生产刚需能力：

在线扩容（StorageClass 需 `allowVolumeExpansion: true`）：

```
kubectl patch pvc data-claim -p '{"spec":{"resources":{"requests":{"storage":"100Gi"}}}}'
```

只能扩大不能缩小；文件系统层由 CSI 驱动自动扩展（ext4/xfs 支持）。

卷快照（VolumeSnapshot）：

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: data-snap-20260320
spec:
  volumeSnapshotClassName: csi-snapclass
  source:
    persistentVolumeClaimName: data-claim
```

快照基于存储后端的 COW 能力，秒级完成，可用于备份与回滚（第 22 章的备份策略会引用）。注意快照 ≠ 备份：快照通常与源卷同域，**跨可用区/跨区域的备份仍需复制机制**。

14.7 本章小结

- **PV 是库存、PVC 是申请单、StorageClass 是供应商目录**——三层抽象实现了存储的"按需自动供应"与"应用与存储解耦"。

- accessModes 的真实语义是**按节点计**：RWO 单节点读写、ROX 多节点只读、RWX 需共享文件系统；"多 Pod 共享盘"先想清楚选型。
- PVC Pending 的四大排查方向：SC 不存在/无默认、provisioner 未装、可用区不匹配（用 WaitForFirstConsumer）、配额。
- **回收策略关乎生死**：生产重要数据用 Retain；删 namespace 会连坐其中所有 PVC，删前必查。
- CSI 是存储的统一插件接口，驱动以"应用方式"部署；快照与在线扩容是两个高频生产能力。

数据的大门和地板都修好了。接下来补齐应用的"随身物品"——配置和密码。下一章讲 ConfigMap 与 Secret：配置的魔法抽屉。

第15章 ConfigMap 与 Secret：配置的魔法抽屉

应用上容器化之后，一个老问题以新的形式回归：**配置放哪里？** 打进镜像里？改一个环境参数就要重新构建发布，不可接受。写在 Pod 的 YAML 里？数据库密码裸奔在代码库里，更不可接受。

Kubernetes 的答案是两个专门的资源对象：**ConfigMap**（明文配置）与 **Secret**（敏感信息）。本章讲透它们的用法、注入方式与热更新行为，以及那些容易踩的坑。

15.1 配置与镜像分离：为什么这是铁律

回忆十二要素的第 3 条：**配置存储于环境，而非代码**。同一段代码，在开发环境连测试库、在生产环境连生产库、日志级别不同、开关不同——这些差异就是"配置"。

把配置打进镜像的代价清单：

- 改一个日志级别 = 一次完整的构建、推送、发布流程；
- 一个镜像要复制 N 份（每环境一份），镜像仓库里充斥着 `myapp:2.1-dev`、`myapp:2.1-staging`、`myapp:2.1-prod`；
- 密码进了镜像层，**永远可以从镜像历史里挖出来**——安全事故的经典来源。

正确的姿势：**一个镜像走天下，配置在部署时注入**。Kubernetes 用 ConfigMap 与 Secret 承接"注入"这个动作。

15.2 ConfigMap：明文配置的容器

ConfigMap 本质是一组键值对，值可以是短字符串，也可以是整个文件内容：

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: "production"              # 短配置：环境变量
  LOG_LEVEL: "info"
  nginx.conf: |                      # 文件型配置：多行文本
    server {
      listen 80;
      location / {
        proxy_pass http://app:8080;
      }
    }
binaryData:
  favicon.ico: AAABAAEAEBA...        # 二进制内容用 base64
```

15.2.1 注入方式一：环境变量

```
containers:
- name: app
  image: myapp:2.1
  envFrom:
  - configMapRef:
      name: app-config        # 整个 ConfigMap 逐键注入环境变量
  env:
  - name: LOG_LEVEL           # 或只注入个别键（可改名）
    valueFrom:
      configMapKeyRef:
        name: app-config
        key: LOG_LEVEL
```

重要认知：环境变量注入的配置不会热更新。 ConfigMap 变了，Pod 里的环境变量纹丝不动，必须重启 Pod（重建 Deployment）才能生效。原因：环境变量是进程启动时一次性读取的，这是 Linux 进程模型的常识，Kubernetes 无法"隔空"改写已运行进程的环境。

15.2.2 注入方式二：挂载为文件

第 13 章已经预演过：把 ConfigMap 挂载成文件，进程按需读取：

```
containers:
- name: nginx
  image: nginx:1.25
  volumeMounts:
  - name: conf
    mountPath: /etc/nginx/conf.d
    readOnly: true
volumes:
- name: conf
  configMap:
    name: app-config
    items:                          # 可选择性挂载并改名
    - key: nginx.conf
      path: default.conf
```

文件挂载的配置可以热更新（kubelet 会周期性同步，约一分钟内生效，可通过环境调节更快）。但两个关键细节：

1. **符号链接机制**：ConfigMap 挂载目录实际是符号链接，kubelet 更新时通过原子替换目录实现。所以**以"直接读文件句柄长驻"方式打开的文件（如直接 open 后持续 read）不会自动更新**，需重新打开文件或支持 SIGHUP 重载；
2. **subPath 挂载不热更新**：`subPath` 方式挂载的单个文件会失去自动更新能力——这是官方文档明确的行为，也是高频踩坑点。要热更新就整目录挂载。

> **提示：** 热更新能不能"生效"还取决于应用是否支持：Nginx 支持 `nginx -s reload`（配合 reloader 工具或 sidecar 监听文件变化），Spring Cloud Kubernetes、很多云原生框架原生支持 ConfigMap 热刷新。**配置"能更新到文件"与"应用能感知更新"是两回事**。

15.3 Secret：敏感信息的保险柜

Secret 与 ConfigMap 结构几乎一样，区别在于：**值以 base64 编码存储**、访问需要专门权限（RBAC 可分别控制）、etcd 中可配置静态加密。

```
apiVersion: v1
kind: Secret
metadata:
  name: db-credential
type: Opaque
stringData:            # 明文直接写（创建时自动转 base64）
  username: admin
  password: S3cure!Pass
---
# 或从命令行创建（避免密码出现在文件里）
# kubectl create secret generic db-credential \
#   --from-literal=username=admin \
#   --from-literal=password='S3cure!Pass'
```

在 Pod 里使用 Secret 的三种方式：环境变量注入（`secretKeyRef` / `envFrom`）、文件挂载（作为 volume）、以及**镜像拉取凭证**：

```
# 拉私有仓库镜像的专用 Secret
kubectl create secret docker-registry regcred \
  --docker-server=registry.example.com \
  --docker-username=ci-bot \
  --docker-password=<token>
```

```
spec:
  imagePullSecrets:
  - name: regcred
```

15.3.1 一个必须戳破的幻觉："Secret 是加密的"

很多团队以为用了 Secret 就安全了。**真相是：Secret 只是 base64 编码，不是加密**——`echo 'YWRtaW4=' | base64 -d` 一秒钟还原。真正的安全取决于配套措施：

1. **etcd 静态加密**（EncryptionConfiguration，apiserver 层配置）——防 etcd 数据泄露；
2. **RBAC 收紧**：Secret 的读权限要精确到命名空间与资源，默认的 cluster-admin 或宽泛的 `get secrets` 都是隐患（第 16 章）；
3. **传输安全**：集群内 API 通信默认 TLS，这一点 K8s 已兜底；
4. **不要把 Secret 提交进 Git**——CI/CD 的真正解法是 Sealed Secrets、SOPS+KMS、External Secrets Operator（对接 Vault/云密钥管理）这类工具；
5. 高安全场景直接上 **Vault** 这类专用密钥管理系统，Secret 只做引用。

> **警示：** `kubectl get secret xxx -o yaml` 任何有权限的人都能拿到明文。Secret 的安全边界是**权限模型 + 加密配置**，而不是那个 base64。安全审计时第一条就是检查"谁有集群级 secrets 读权限"——答案往往是"所有人"，这就出事了。

15.4 配置管理的工程实践

实践一：按环境分 ConfigMap，而不是分镜像。 同一个 Deployment 模板，通过覆盖 ConfigMap 的名字或 namespace 区分环境。Helm/Kustomize（第 23、24 章）正是做这件事的利器。

实践二：配置变更也是"发布"，要走审阅。 ConfigMap 放进 Git，改配置也要走 PR。因为配置错误和代码错误一样会引发生产事故，而且配置变更往往没有测试兜底，更需要 review。

实践三：让应用兼容两种注入方式。 环境变量简单但不热更；文件挂载热更但应用要支持读文件。成熟的云原生应用通常：启动配置用环境变量（如连接串），运行时可变配置用文件（如日志级别）。

实践四：警惕"配置漂移"。 有人用 `kubectl edit configmap` 改了线上的配置，Git 里的文件却没改——下次 GitOps 同步时改动被悄悄覆盖。解法是把"手动改线上配置"列入禁止事项，一切变更走代码库（第 24 章 GitOps 的核心价值就在于此）。

实践五：大配置拆分与瘦身。 单个 ConfigMap 有约 1MB 的上限（受 etcd 单对象限制），不要把整个静态网站塞进 ConfigMap；静态资源请走镜像或对象存储。

15.5 本章小结

- **配置与镜像分离是铁律**：一个镜像走天下，环境差异全部交给 ConfigMap/Secret 注入。
- ConfigMap/Secret 的两种注入方式：**环境变量**（简单，但不热更新）、**文件挂载**（可热更新，但 subPath 除外，且应用要支持感知变化）。
- **Secret 的 base64 不是加密**。真正的安全 = etcd 静态加密 + RBAC 精确授权 + 不进 Git + 密钥管理工具。
- 配置管理五实践：按环境分配置、配置变更走审阅、双注入方式兼容、防配置漂移、配置瘦身。

配置抽屉整理完毕。但还有一个隐秘的角落没有打扫：**谁有权打开这些抽屉？** 下一章进入安全的地基——RBAC：谁能进哪个房间。

第16章 RBAC：谁能进哪个房间

任何一个多用户系统，绕不开的问题都是权限：谁能做什么？Kubernetes 的答案是 **RBAC（Role-Based Access Control，基于角色的访问控制）**。这一章把它的模型、写法与工程实践讲清楚。

RBAC 的重要性怎么强调都不过分：它直接决定了"生产事故的人为边界"——一个错误的 RoleBinding，可以让实习生删除生产数据库；一次过宽的授权，可以让一个被攻破的 Pod 拿走整个集群的 Secret。

16.1 四个积木：Role、ClusterRole、Binding

RBAC 的模型出奇地简洁，一共四个对象：

- **Role**：一组权限规则，**作用在某个命名空间内**。规则 = 对哪些资源（resources）做哪些操作（verbs）；
- **ClusterRole**：同上，但**作用在整个集群**（可管集群级资源如 Node、PV，也可跨命名空间统一管理）；
- **RoleBinding**：把角色**授予**某些主体，作用在命名空间内；
- **ClusterRoleBinding**：同上，作用在整个集群。

> **比喻：** Role 是"房间钥匙串"（定义哪些门能开）；RoleBinding 是"把钥匙串交给某个人"的登记簿。ClusterRole 则是"大楼万能钥匙串"或"每层楼标准钥匙串的模板"。

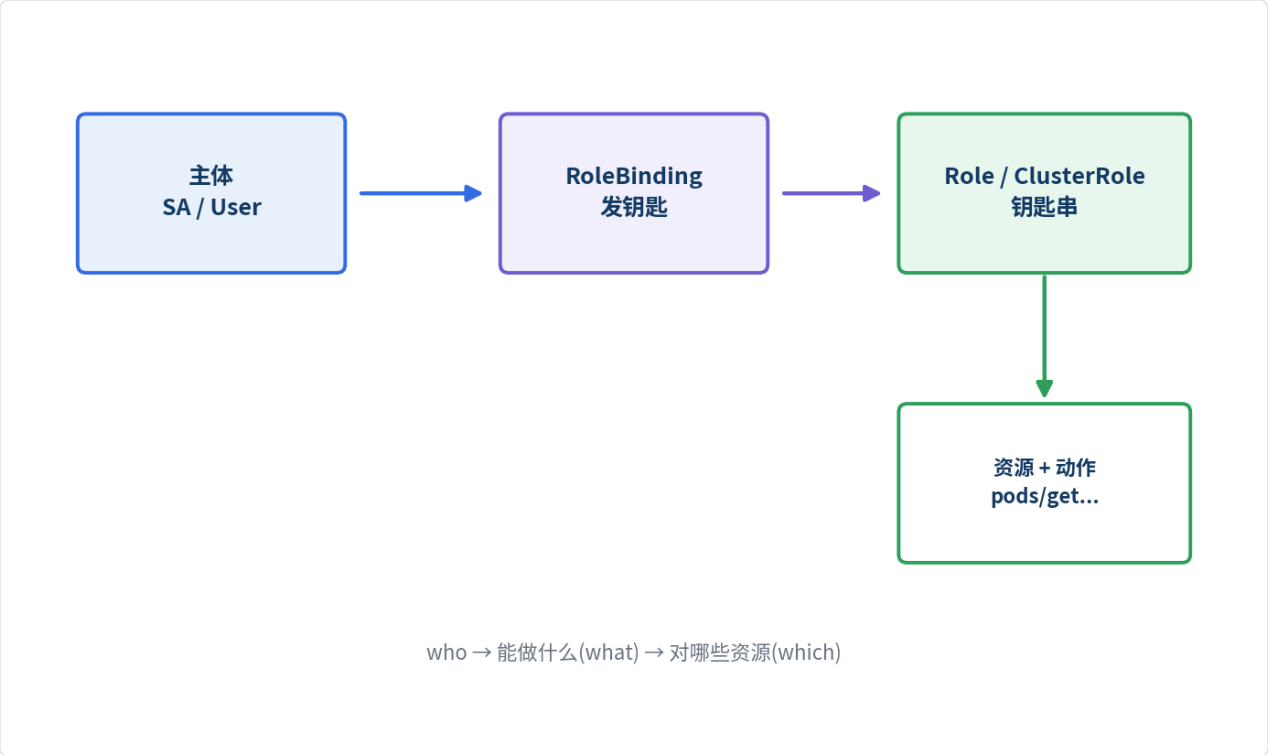


图16-1 RBAC 的关系模型：主体经绑定获得角色定义的权限

主体（Subject） 有三种：**User**（外部用户，K8s 自己不存用户，靠证书/OIDC 等方式认证）、**Group**（用户组）、**ServiceAccount**（为集群内程序准备的"机器身份"）。

一个最小的例子：

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: team-a
rules:
- apiGroups: [""]              # "" 表示核心组（Pod、Service 等）
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: team-a
subjects:
- kind: ServiceAccount
  name: ci-bot
  namespace: team-a
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

语义：在 `team-a` 命名空间里，`ci-bot` 这个 ServiceAccount 可以查看 Pod（get/list/watch），但不能创建、删除，不能看其他命名空间，也不能看 ConfigMap。

verbs 是操作的动词表：`get`（单个）、`list`（列表）、`watch`（监听变更）、`create`、`update`、`patch`、`delete`、`deletecollection`，以及特殊的 `*`（全部）。**resources** 可以细分到子资源，如 `pods/log`、`pods/exec`、`deployments/scale`。

16.2 Role 与 ClusterRole 的组合矩阵

四个对象可以交叉组合出灵活的模式，常见的三种用法值得记住：

模式一：Role + RoleBinding——命名空间内授权。最常见：给 team-a 的 CI 账号 team-a 内的部署权限。

模式二：ClusterRole + RoleBinding——"每层楼发同一串钥匙"。ClusterRole 定义一次权限，RoleBinding 在每个命名空间里引用它，效果是"该命名空间内的这些权限"。好处：权限定义集中管理，各命名空间按需引用。这是多团队集群的推荐姿势。

模式三：ClusterRole + ClusterRoleBinding——全局授权。该主体在全集群所有命名空间拥有这些权限。慎用：范围太大，只应授予平台运维类账号。

> **提示：** 权限最小化原则在 K8s 里的落地口诀：**能用 RoleBinding 就不用 ClusterRoleBinding；能授 list 就不授 `*`；能限定 apiGroups 就不写 `["*"]`**。授权时多花十分钟，审计时少熬十个通宵。

16.3 ServiceAccount：程序的身份证

人要权限，**程序也要权限**。集群里运行的每一个 Pod，默认都挂着一个自动创建的 ServiceAccount（`default`），它拥有什么权限，直接决定"Pod 被攻破后攻击者能走多远"。

关键机制 **TokenRequest**：Pod 启动时，kubelet 为其 ServiceAccount 向 apiserver 申请一个**有时限的 JWT token**，挂载到容器的文件系统（`/var/run/secrets/kubernetes.io/serviceaccount/token`）。Pod 里的任何进程都可以拿这个 token 调用 apiserver。

这就引出一个安全基线：**业务 Pod 如果不需要访问 K8s API（大多数不需要），应该禁用默认 SA 的自动挂载**：

```
spec:
  automountServiceAccountToken: false    # 不给 Pod 发 token
  containers:
  - name: app
    ...
```

或者更进一步：为需要 API 访问的组件创建**专用 SA**（如 Prometheus 的采集 SA 只授 watch pods/metrics 的权限），而不是共用 default。

16.3.1 实战：给应用最小权限

以一个需要读取自己 ConfigMap 的应用为例，完整的"SA + Role + 绑定 + Pod"组合：

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa
---
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: app-role
rules:
- apiGroups: [""]
  resources: ["configmaps"]
  resourceNames: ["app-config"]     # 精确到具体对象名!
  verbs: ["get", "watch"]
---
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: app-binding
subjects:
- kind: ServiceAccount
  name: app-sa
roleRef:
  kind: Role
  name: app-role
---
# Pod 模板里指定
spec:
  serviceAccountName: app-sa
```

`resourceNames` 是经常被忽略的"精确打击"手段——连对象名都限定，把权限面压到最小。

16.4 认证与授权的分界线

理解 RBAC 的位置，需要先看清一个请求的完整安检流水线（apiserver 内部）：

```
请求 → ① 认证(Authentication): 你是谁?
     → ② 授权(Authorization): 你能做吗?  (RBAC 在这一关)
     → ③ 准入控制(Admission): 特殊规则检查/改写 (第 5 章提过)
     → 处理请求
```

认证解决"你是谁"：客户端证书（kubectl 的 kubeconfig 里配的）、Bearer Token（SA 的 JWT）、OIDC（对接企业 SSO，生产主流）等。Kubernetes **不存储用户**——用户身份都来自外部系统。

授权解决"你能不能"：默认 Authorizer 有多种，**RBAC 是标准选择**（还有 AlwaysAllow 已淘汰、Node 授权器服务节点自举、Webhook 授权器对接外部系统如 OPA）。

准入控制解决"即使你有权限，这条请求是否合规"：配额检查、Pod 安全准入（第 17 章）、Webhook 等。注意一个容易混淆的点：**准入控制在授权之后，哪怕 RBAC 放行，准入也可能拦截**——比如 namespace 配额满了，你有 create 权限也会失败。

16.5 检视与排障 RBAC

日常管理 RBAC 的实用命令：

```
# 检查"我"是否有权限（kubectl auth 用法）
kubectl auth can-i create deployments -n prod
kubectl auth can-i delete pods --as=system:serviceaccount:team-a:ci-bot   # 模拟他人

# 谁能用某个角色？（审计利器）
kubectl get rolebinding -A -o yaml | grep -A5 ci-bot

# 403 了怎么办：看 apiserver 日志里的 RBAC 拒绝详情
kubectl logs -n kube-system kube-apiserver-master-1 | grep RBAC
```

`kubectl auth can-i --as=...` 是排障神器：不需要真的切换身份，就能回答"这个 SA 能不能做这件事"。配合 `--list` 可以列出某主体的全部权限。

> **提示：** CI/CD 系统是 RBAC 事故高发区——流水线账号往往被授予 cluster-admin 图省事。正确姿势：为每个环境建专用 SA，授最小权限；生产环境的发布 SA 只允许操作指定 namespace 的 Deployment/Service 等少数资源。

16.6 权限设计的进阶话题

多租户隔离的现实。RBAC 能做到"team-a 的用户看不到 team-b 的资源"（配合 namespace 隔离），但**不能防止**：node 层面的资源争抢（要配 ResourceQuota/LimitRange）、网络层面的互访（要配 NetworkPolicy）、以及特权 Pod 的逃逸（要配 Pod Security）。完整的"软多租户"方案 = RBAC + 配额 + 网络策略 + Pod 安全四件套（第 17 章汇总）。

用户身份对接企业 SSO。生产集群推荐用 **OIDC 认证**对接公司统一登录（如企业微信/飞书/Keycloak），把"人"的身份交给 IdP 管理，RBAC 只管"组 → 权限"的映射。这样离职员工禁用 SSO 账号即失效，不需要在 K8s 里逐个清理证书。

审计权限蔓延。权限只加不减是常态。建议定期（季度）用 `kubectl-who-can`、`rbac-lookup` 这类工具做权限盘点，回收僵尸绑定。

16.7 本章小结

- RBAC 四积木：**Role（钥匙串）/ClusterRole（全局钥匙串）+ RoleBinding（发放登记）/ ClusterRoleBinding（全楼发放）**；主体为 User/Group/ServiceAccount。
- 推荐姿势：**ClusterRole 定义权限 + 各 namespace RoleBinding 引用**，集中定义、按需发放。
- **ServiceAccount 是程序的身份证**：业务 Pod 默认不挂 token（`automountServiceAccountToken: false`），需要 API 访问的组件用专用 SA + 最小权限（可用 `resourceNames` 精确到对象）。
- 请求安检流水线：**认证（你是谁）→ 授权（RBAC，你能不能）→ 准入（合不合规）**；403 先查授权，还有准 入这一关。
- `kubectl auth can-i --as=` 是模拟排障神器；CI/CD 账号权限最小化是重中之重。

权限的地基打好了。但 RBAC 只是安全的"门禁卡"，下一章我们把整个安全大厦看全：Pod 的安全上下文、网络隔离、镜像安全与运行时防护——安全全景。

第17章 安全全景：从镜像到运行时的纵深防御

RBAC 解决了"谁有权操作集群"，但 Kubernetes 的安全远不止于此。一个容器从**构建镜像**到**被调度运行**再到**对外服务**，每个环节都有攻击面。这一章我们把安全版图看全，建立"纵深防御"的整体思维。

17.1 威胁模型：容器不是虚拟机

一切讨论的前提，是理解容器与虚拟机的根本差异：**容器共享宿主机内核**。虚拟机里跑的是完整操作系统，与宿主之间有硬件级隔离；容器只是宿主机上的"一组被隔离约束的进程"，靠 namespace 隔离视图、cgroup 限制资源——**隔离强度远弱于虚拟机**。

由此推出的铁律：

1. **内核漏洞是容器逃逸的最大威胁**。内核一旦被攻破（提权漏洞），容器边界形同虚设。所以节点系统补丁与内核版本管理是容器安全的"地基工程"；
2. **"以 root 运行容器" 是高危行为**。容器内 root 默认具备大量内核能力（capabilities），即使有部分限制，提权风险也远高于非 root；
3. **共享宿主机资源的组件（hostPath、宿主网络、Docker socket）等于把钥匙交出去**。挂载 `/var/run/docker.sock` 的容器可以直接控制宿主上的所有容器——著名的"逃逸捷径"。

安全体系就是围绕"如何在共享内核的前提下尽量收紧每一层"展开的。

17.2 第一道防线：供应链安全（镜像）

镜像就是应用的"出厂环境"，镜像不干净，后面全白搭。 供应链安全有三个抓手：

抓手一：小镜像，少攻击面。 同样的应用，`ubuntu:22.04` 基础镜像几百 MB、装着几十个系统工具；换成 `alpine` 或发行版专属精简镜像（如 `gcr.io/distroless`）只有几十 MB、工具寥寥——**能被利用的软件组件越少，攻击面越小**。distroless 甚至没有 shell，攻击者即使进来也"无事可干"。

抓手二：镜像扫描纳入流水线。 用 Trivy、Grype 等工具在 CI 中扫描镜像漏洞（CVE），高危漏洞阻断构建；Harbor 等企业仓库自带扫描门禁。配合定期重扫 + 重建（修复基础镜像 CVE），保持"在产镜像"持续可防御。

抓手三：镜像签名与来源验证。 用 cosign 对镜像签名，集群侧策略（Kyverno/Gatekeeper）校验"只允许运行签名过的、来自可信仓库的镜像"，阻断未经审核的镜像进入生产。

17.3 第二道防线：Pod Security——运行时的"纪律条款"

SecurityContext（安全上下文） 是给容器（或整个 Pod）设定的"行为纪律"：

```
spec:
  securityContext:            # Pod 级
    runAsNonRoot: true        # 禁止 root 运行
    runAsUser: 10001
    fsGroup: 10001
  containers:
  - name: app
    securityContext:
      allowPrivilegeEscalation: false   # 禁止提权（setuid 等）
      readOnlyRootFilesystem: true      # 根文件系统只读（写路径须挂卷）
      capabilities:
        drop: ["ALL"]                   # 丢弃全部内核能力
        # add: ["NET_BIND_SERVICE"]     # 需要绑定低端口时单独加回
      seccompProfile:
        type: RuntimeDefault            # 默认 seccomp 规则集
```

逐条解释这份"安全清单"的用意：

- **runAsNonRoot**：非 root 运行。需要镜像里存在非 root 用户；
- **readOnlyRootFilesystem**：根目录只读，攻击者无法落地写文件（持久化恶意程序），需要写的路径（/tmp、日志）单独挂卷；
- **capabilities: drop ALL**：Linux 把 root 的特权拆成几十项能力（CAP_NET_ADMIN 改网络、CAP_SYS_ADMIN 几乎等于 root……）。全部丢弃后按需加回，是最小权限的直接体现；
- **allowPrivilegeEscalation: false**：封堵 setuid 二进制的提权路径；
- **seccompProfile**：限制可用的系统调用集合，"攻击面再收窄一圈"。

17.3.1 Pod Security Admission：集群级的强制标准

给每个 Pod 手写 SecurityContext 靠自觉？不现实。Kubernetes 内置了 **Pod Security Admission（PSA，1.25+ 正式）**，按 namespace 强制执行三档安全标准：

- **privileged**：不限制（系统组件用）；
- **baseline**：基线——挡掉明显高危项（特权容器、hostNetwork、高危 hostPath）；
- **restricted**：严格——强制本章上面的全套清单（非 root、丢能力、只读根文件系统等）。

给 namespace 打标签即可生效：

```
kubectl label ns prod pod-security.kubernetes.io/enforce=restricted
```

从此，任何"不合规"的 Pod 在 `prod` 里直接被拒之门外——**用制度代替自觉**。建议：业务 namespace 统一 `enforce=restricted`（或先 `audit` 观察再收紧），系统组件 namespace 保持 privileged。

17.4 第三道防线：网络隔离（NetworkPolicy）

第 12 章讲过 NetworkPolicy 的机制，这里从安全视角再强调它的战略地位：**默认全通的 Pod 网络，意味着任何一个被攻破的 Pod 都可以横向扫描、访问集群里任何服务（包括数据库、kubelet API、云元数据接口）**。

云元数据接口（169.254.169.254）是近年云上攻击的经典跳板——容器拿到节点凭证后借此窃取云账号权限。防守要点：

1. 业务 namespace 之间用 NetworkPolicy 收紧，默认拒绝入站；
2. 明确禁止 Pod 访问元数据地址（Calico/Cilium 均可针对该地址写策略，或云侧禁用元数据直通）；
3. 数据库类服务只接受特定标签来源的流量（如 `app=orders`）。

"零信任网络"在 K8s 里的起步动作，就是一张全集群的默认拒绝 Ingress 策略 + 白名单逐个放行。

17.5 第四道防线：运行时检测与密钥管理

运行时检测（Runtime Security）：策略挡得住"已知的坏"，挡不住"未知的坏"。Falco 这类工具基于系统调用实时检测异常行为——"nginx 容器里 spawn 了 shell""读取了 /etc/shadow""容器内 mknod"——并即时告警。它是安全体系里的"监控摄像头"，与"门禁"（PSA）互为补充。

密钥管理：第 15 章已经讲过 Secret 的安全实践，这里从架构上补充一句：企业级方案是把密钥的"存放与轮换"交给 Vault/云 KMS，应用通过 CSI 驱动或 External Secrets Operator 动态获取，Secret 只承担"最后一级注入"。密钥的**定期轮换**与**最小暴露面**，比"存储方式多花哨"更重要。

准入策略引擎（OPA/Gatekeeper、Kyverno）：当 PSA 不够用时——比如"生产镜像必须来自公司仓库""必须有资源限制""标签必须齐全"——策略引擎可以自定义任意规则，并给出违反报告。Kyverno 用 YAML 写策略，门槛较低，推荐起步选择。

17.6 把防线拼成体系

把本章内容叠起来，就是一套容器安全的纵深防御：

```
供应链：    镜像精简 → 扫描 → 签名校验
运行前：    Pod Security（restricted）→ 策略引擎自定义规则
运行中：    SecurityContext 收紧 → NetworkPolicy 隔离 → 运行时检测
身份层：    RBAC 最小权限（第 16 章）→ SA token 不自动挂载
数据层：    Secret 加密与轮换（第 15 章）
基础设施：  节点内核补丁 → etcd 加密与备份 → API Server 审计日志
```

> **提示：** 安全不是一次性的项目，而是持续的过程。起步建议按"性价比排序"：① 收紧 RBAC（尤其 secrets 权限）② 全 namespace 启用 PSA baseline ③ CI 集成镜像扫描 ④ 默认拒绝的 NetworkPolicy ⑤ etcd 加密与审计日志。这五步能挡掉绝大多数常见攻击路径。

17.7 本章小结

- 容器共享内核，隔离弱于虚拟机：**内核补丁、非 root 运行、慎用特权能力**是三块地基。
- 供应链三抓手：**精简镜像、流水线扫描、签名验证**。
- SecurityContext 是容器的"纪律清单"（非 root、只读根目录、丢能力、seccomp）；**Pod Security Admission** 按 namespace 强制三档标准，用制度代替自觉。
- 默认全通的 Pod 网络是横向移动的温床：**NetworkPolicy 默认拒绝 + 白名单放行**是零信任的起步动作，还要防云元数据接口。
- 运行时检测（Falco）、策略引擎（Kyverno/OPA）、密钥管理（Vault/KMS）补齐"未知威胁"与"密钥治理"。
- 纵深防御：每一层都假设上一层可能失守，层层设防，逐层收窄。

安全篇章到此收官。接下来进入第七部分——调度与弹性：调度器是怎么给 Pod"选房"的？集群如何自动伸缩？下一章见。

第18章 调度器：Kubernetes 的选房系统

前面的章节里，我们不止一次提到"调度器为 Pod 选择节点"。这一章，我们把这个"选房系统"拆开，看它如何在一秒内为成百上千个 Pod 找到合适的家，以及如何用亲和性、污点、拓扑约束等手段"指挥"它。

18.1 调度的本质：一个两轮筛选的决策过程

当一个新的 Pod 出现且未指定节点时（`nodeName` 为空），它进入调度队列。调度器（kube-scheduler）对每个 Pod 执行两轮决策：

第一轮：过滤（Filter/Predicate）——先淘汰不及格的。 逐个节点检查硬性条件，不满足者出局：

- 资源够吗？（节点的可分配资源 ≥ Pod 的 requests 之和）
- 端口冲突吗？
- 节点的污点（taint），Pod 能容忍（toleration）吗？
- 节点选择器/亲和性能满足吗？
- Pod 要挂的卷，能在这个可用区挂上吗？（拓扑约束）
- 有没有 PV/PVC 或 Pod 亲和性等硬约束冲突？

第二轮：打分（Score/Priority）——再给及格者排座次。 对幸存节点逐项打分（0～100），加权求和：

- **资源均衡分**：优先选择"分配后资源分布更均匀"的节点（避免把所有 Pod 堆到一台机器）；
- **亲和性得分**：满足 Pod 亲和性的加分（软性 preferred 权重）；
- **镜像本地性**：镜像已在节点的加分（省去拉取时间）；
- **Pod 拓扑分散**：同一服务的 Pod 分散部署加分；
- 以及各类自定义插件的打分。

得分最高者胜出，调度器把结果（Pod 的 `spec.nodeName`）写回 apiserver，后续 kubelet 接手拉起容器。

> **比喻：** 先按"硬性门槛"筛掉不合格的房源（预算外的、位置太远的），再按"宜居程度"打分排序（通勤、采光、物业），选分最高的。硬约束是"一票否决"，软偏好是"加分项"——这个区分贯穿本章所有机制。

调度是快照决策：调度器基于当前集群状态做决定，决策之后状态还会变（比如新 Pod 占了资源）。极端情况下会出"过度承诺"，所以还有**抢占（Preemption）**机制兜底：高优先级 Pod 调度不进去时，可以"挤掉"低优先级 Pod 腾出资源。

18.2 requests 与调度：你已经决定了调度

一个重要认知：**你在 Pod 里写的 requests，就是调度器眼里的"体积"**。调度器不看 Pod 实际用了多少 CPU，只看 requests 声明了多少。

这带来几个实用推论：

- **requests 设得虚高** → 节点看似"满了"（分配率高），实际利用率很低 → 集群资源浪费、扩容频繁；
- **requests 设得过低** → 节点超卖（分配的资源总量超过实际容量）→ 负载高峰时节点 CPU 争抢、内存耗尽触发 OOM → 生产事故；
- **不设 requests** → Pod 属于 BestEffort，资源紧张时第一批被驱逐（见下文 QoS）。

第 4 章埋的伏笔在这里收线：**QoS（服务质量）等级**由 requests/limits 配置自动决定：

QoS 等级	条件	资源紧张时
Guaranteed	每项资源 requests == limits	最后被驱逐
Burstable	设了 requests，但 < limits	中间层
BestEffort	全都没设	第一批被驱逐

kubelet 在节点内存紧张时按 QoS 从低到高驱逐 Pod（BestEffort → Burstable → Guaranteed）。**关键服务要给 Guaranteed 或高 Burstable 配置**，让它们在"节点风雨飘摇"时活到最后。

18.3 nodeSelector 与亲和性：指挥调度器

默认调度已经足够智能，但有些需求必须显式表达——"这个服务必须跑在 SSD 节点上""这个 Pod 最好和那个服务部署在同一可用区"。工具按约束强度排序如下。

18.3.1 nodeSelector：最简单的节点筛选

```
spec:
  nodeSelector:
    disktype: ssd     # 只调度到带这个标签的节点
```

节点先打标签：`kubectl label node node-3 disktype=ssd`。简单直接，但只能表达"必须"，且只有等于匹配。

18.3.2 nodeAffinity：更强大的节点亲和

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:   # 硬约束
        nodeSelectorTerms:
        - matchExpressions:
          - key: kubernetes.io/arch
            operator: In
            values: ["amd64"]
      preferredDuringSchedulingIgnoredDuringExecution:  # 软偏好
      - weight: 80
        preference:
          matchExpressions:
          - key: disktype
            operator: In
            values: ["ssd"]
```

名字很长，但结构化拆解后非常好记：`required`（必须满足，否则不调度）vs `preferred`（尽量满足，weight 是加分权重）；`DuringScheduling`（调度时检查）vs `IgnoredDuringExecution`（运行后不管——节点标签变了也不会驱逐已运行的 Pod）。支持 `In/NotIn/Exists/DoesNotExist/Gt/Lt` 操作符。

18.3.3 podAffinity / podAntiAffinity：Pod 之间的"合租"与"分居"

```
spec:
  affinity:
    podAntiAffinity:      # 反亲和：尽量分开部署
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchLabels:
              app: web
          topologyKey: kubernetes.io/hostname   # 分散的"粒度"：按节点
```

这个配置是高可用部署的标配：**同一服务的多个副本尽量分散到不同节点**（把 topologyKey 换成 `topology.kubernetes.io/zone` 则是分散到可用区——跨 AZ 高可用的标准写法）。`required` 版本是硬性"分居"（一个节点最多一个副本），节点不够时 Pod 会 Pending。

对应的 podAffinity 是"合租"——把通信密集的服务放同一可用区（降低跨区延迟）。

18.3.4 topologySpreadConstraints：更精细的分散控制

Pod 反亲和只能做到"尽量分散"，想精确控制"每个域最多偏差 1 个副本"，用拓扑分布约束：

```
spec:
  topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: kubernetes.io/hostname
    whenUnsatisfiable: ScheduleAnyway    # 或 DoNotSchedule（硬约束）
    labelSelector:
      matchLabels:
        app: web
```

它保证副本在各节点的分布"不均匀度"不超过 maxSkew——Deployment 有 6 个副本、3 台节点时，分布一定是 2/2/2，而不会是 4/1/1。

18.4 污点与容忍：节点的"谢客牌"

前面的机制都是"Pod 挑节点"，**污点（Taint）机制反过来了——节点挑 Pod**。节点打上污点后，默认拒绝所有不能"容忍"它的 Pod。

```
kubectl taint node node-3 dedicated=gpu:NoSchedule
```

污点三要素：`key=value:effect`。effect 有三档：

- **NoSchedule**：不调度新 Pod（已在跑的不动）；
- **PreferNoSchedule**：尽量不调度（软性谢客）；
- **NoExecute**：不调度新 Pod，且**驱逐**已运行的不能容忍的 Pod（最严厉）。

tolerations（容忍） 写在 Pod 侧，表示"我有通行证"：

```
spec:
  tolerations:
  - key: "dedicated"
    operator: "Equal"
    value: "gpu"
    effect: "NoSchedule"    # 只容忍这一个污点
```

污点的经典用途：

- **专用节点**：GPU 节点打污点，只有声明 GPU 需求且能容忍的 Pod 进来，防止普通任务挤占昂贵资源；
- **节点维护**：`kubectl cordon`（封锁调度）+ `kubectl drain`（驱逐 Pod，内部会打 NoExecute 污点）——第 22 章节点升级的标准流程就是它；
- **系统保留**：控制平面节点默认带 `node-role.kubernetes.io/control-plane:NoSchedule`，防止业务 Pod 挤进"舰桥"。

注意污点与亲和性的方向差异：**亲和性是"Pod 说我想去哪"，污点是"节点说谁别来"**——一个正向吸引，一个反向排斥，组合使用可以实现精细的集群资源管控。

18.5 优先级与抢占

当集群资源紧张时，谁该活下来？**PriorityClass** 给出了答案：

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
value: 1000000
globalDefault: false
preemptionPolicy: PreemptLowerPriority
```

Pod 引用后获得高优先级。当它调度不出去时，调度器会寻找"驱逐哪些低优先级 Pod 能腾出资源"的方案并执行抢占——低优先级 Pod 被驱逐（进入正常终止流程），高优先级 Pod 顶上。

实践建议：核心系统组件（如日志、监控的 DaemonSet）、关键业务给高优先级；批处理任务给低优先级（可被抢占）。**不要滥用抢占**——它可能造成低优先级服务的反复重启，优先考虑扩容集群。

18.6 多调度器与调度框架

Kubernetes 支持并行多个调度器（Pod 用 `schedulerName` 指定），也支持通过**调度框架（Scheduling Framework）**扩展默认调度器——在过滤、打分的各个环节插入自定义插件（比如"优先调度到温度低的节点""优先复用已绑定 IP 的节点"）。大数据调度（Volcano）、批量任务调度（Kueue）等都是基于调度框架的定制调度器。

> **深入：** 这又是"扩展而非修改"哲学的体现——与控制器之于 apiserver、CNI 之于网络、CSI 之于存储一样，调度器的自定义能力也通过标准化接口开放。Kubernetes 的每个核心环节，都可以被生态"接管"，而核心保持稳定。

18.7 本章小结

- 调度 = **过滤（硬性淘汰）+ 打分（软性偏好）**；调度基于 requests 的"声明体积"而非实际用量。
- **requests/limits 决定 QoS**：Guaranteed/Burstable/BestEffort，资源紧张时驱逐顺序从低到高——关键服务别裸奔。
- 指挥调度器的工具箱：**nodeSelector（简单硬筛）→ nodeAffinity（强弱结合）→ pod(anti)affinity（合租/分居）→ topologySpreadConstraints（精确分散）**。

- **污点是节点侧的谢客牌**（NoSchedule/PreferNoSchedule/NoExecute），容忍是 Pod 侧的通行证；drain 维护流程的本质就是 NoExecute 驱逐。
- **优先级与抢占**兜底资源紧张场景；调度框架允许生态定制调度器。

调度器决定"Pod 放在哪"。但当业务流量波动时，"放几个 Pod"才是更动态的问题——下一章讲自动扩缩容三件套：HPA、VPA 与 Cluster Autoscaler。

第19章 HPA 与自动扩缩容：让集群呼吸起来

凌晨三点流量低谷，十几个副本空转烧钱；中午流量高峰，副本数不够请求排队。**"放几个 Pod"不该是一个静态答案**——这一章讲 Kubernetes 的自动扩缩容三件套：**HPA**（水平自动扩容）、**VPA**（垂直自动调整）与 **Cluster Autoscaler**（集群节点自动伸缩），以及它们如何协作与冲突。

19.1 HPA：副本数的自动驾驶仪

HPA（Horizontal Pod Autoscaler，水平 Pod 自动扩缩器） 是使用最广泛的自动伸缩机制：根据指标（CPU、内存或自定义指标）**自动增减 Deployment 的副本数**。

一个最基本的 CPU 型 HPA：

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web
  minReplicas: 2          # 缩不下去的下限（保底可用性）
  maxReplicas: 20         # 扩不上来的上限（保护集群资源）
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60    # 目标：平均 CPU 使用率 60%
  behavior:                    # 伸缩行为微调（v2 支持）
    scaleDown:
      stabilizationWindowSeconds: 300   # 缩容前观察 5 分钟
      policies:
      - type: Percent
        value: 25                # 每次最多缩 25%
        periodSeconds: 60
```

目标语义：让所有副本的 CPU 平均使用率维持在 60%——流量涨 → 使用率超 60% → 扩容摊薄 → 回到 60%；流量落 → 缩容。这是典型的**负反馈控制**。

计算公式（理解它，就理解了 HPA 的行为）：

```
期望副本数 = ceil(当前副本数 × 当前指标 / 目标指标)
```

例如：10 个副本、平均 CPU 90%、目标 60% → 期望 = ceil(10 × 90/60) = 15 个副本。

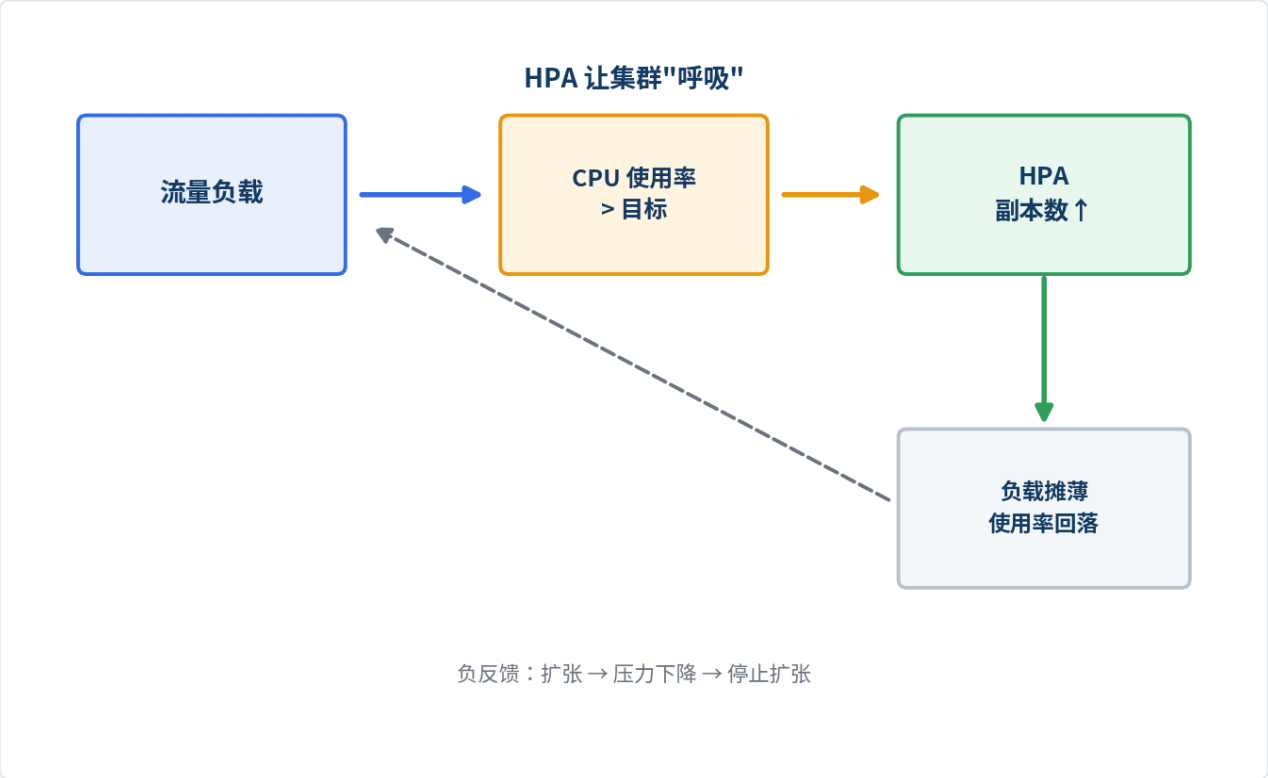


图19-1 HPA 的负反馈回路：负载升高→扩容→压力摊薄→停止扩张

19.1.1 指标从哪来：metrics-server

Resource 类型的指标来自 **metrics-server**——一个集群级组件，它从各节点 kubelet 抓取容器指标，聚合后通过 Metrics API 供 HPA 与 `kubectl top` 使用。**没有 metrics-server，HPA 直接罢工**（`kubectl get hpa` 的 TARGETS 列显示 `<unknown>`）。托管集群通常已预装，自建集群需要单独部署。

19.1.2 自定义指标与缩放行为

CPU 有个致命缺陷：**它不是业务负载的好指标**——Node.js 单进程吃满一核不代表压力大，Java 堆内存高不代表吞吐高，而 QPS、消息积压量才是真正的业务指标。HPA 支持自定义指标（通过 Prometheus Adapter 或 KEDA 等组件对接）：

- **Prometheus Adapter**：把 Prometheus 里的指标（http_requests_per_second、kafka_consumer_lag）暴露成 HPA 可用的 Metrics API；
- **KEDA**：微软开源的事件驱动伸缩器，原生支持几十种数据源（Kafka 队列深度、Redis 长度、云服务队列），甚至支持**缩容到 0**（无事件时零副本，事件来了拉起）——Serverless 化批处理的利器。

`behavior` 字段是生产调优的重点：**扩容要快，缩容要慢**。突发流量扩容晚了就是事故，而流量波动导致频繁缩扩（抖动）同样伤稳定性——`stabilizationWindowSeconds` 与缩容百分比策略就是为此设计的。

> **提示：** HPA 只会在 **1 和 maxReplicas 之间**做决策。真正决定"冷启动速度"的还有 Pod 自身的启动时间（拉镜像 + 启动 + 就绪探针通过）——镜像瘦身与 readinessProbe 快速通过，是 HPA 生效快慢的隐藏变量。

19.2 VPA：垂直方向的"量体裁衣"

HPA 解决"数量"，**VPA（Vertical Pod Autoscaler）** 解决"大小"：自动分析 Pod 的实际资源用量，**调整 requests/limits**。它的价值场景：

- **推荐模式（推荐起步）**：只输出建议值（`kubectl describe vpa` 显示推荐的 requests），人工确认后更新清单——安全，先观察；
- **自动模式**：VPA 直接改写 Pod 的资源请求。**注意：这需要重建 Pod 才能生效**（资源请求不能在线改），所以 Auto 模式的 VPA 会杀掉 Pod 重建——对无状态服务可用，但有中断；配合 on-demand 节点可平滑；
- **.Initial / Off 模式**：只对新建 Pod 生效或不做任何动作。

HPA 与 VPA 不能同时对同一资源维度工作：HPA 按 CPU 使用率伸缩 + VPA 改 CPU 请求，会形成死循环（VPA 调高 CPU 请求 → HPA 看到使用率下降 → 缩容 → 单 Pod 压力升高……）。共存守则：**HPA 用 CPU 指标时，VPA 只管内存；或 HPA 改用自定义指标，VPA 管资源**。

VPA 的最佳定位是"**成本的精算师**"：帮你发现"这个服务实际只需要 200m CPU，却声明了 1000m"——把第 18 章提到的 requests 虚高问题量化出来。实践上，很多团队用 VPA 的推荐模式定期校准 requests，比 Auto 模式更稳妥。

19.3 Cluster Autoscaler：节点层的伸缩

Pod 伸缩到极限后，集群本身也要伸缩。**Cluster Autoscaler（CA）** 负责节点层：

扩容：当出现"**因资源不足而 Pending 的 Pod**"时——注意触发条件是 Pending 的 Pod，不是 CPU 使用率——CA 向云厂商申请新节点，节点加入后 Pod 被调度上去。

缩容：节点连续一段时间（默认 10 分钟）**利用率低于阈值（默认 50%）**，且上面的 Pod 都能被迁移到其他节点，该节点被回收（Pod 被驱逐重建）。

与云厂商的节点组/节点池集成后，CA 的部署通常就是一条 Helm 命令 + 节点组标签配置。新一代的 **Karpenter**（AWS 开源，已支持多云）直接 Watch Pending Pod，按需直接申请**最合适规格**的节点（而不是从固定节点组里选），扩容速度更快、规格更贴合，正在成为云上首选。

19.3.1 别让节点"缩不走"：PDB 与注解

缩容驱逐可能破坏服务的可用性底线——**PodDisruptionBudget（PDB）** 是防线：

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: web-pdb
spec:
  minAvailable: 2        # 或 maxUnavailable: 1
  selector:
    matchLabels:
      app: web
```

语义：任何"自愿驱逐"（CA 缩容、drain 维护）都必须保证 `app=web` 始终至少 2 个可用副本，否则驱逐等待/失败。PDB 与 CA 配合：节点上所有 Pod 都满足各自的 PDB 才允许缩容。

两个常见的"节点缩不走"原因：**没有 PDB 但有本地存储/单副本关键服务**（驱逐有风险被 CA 保守跳过）、**Pod 带了 `cluster-autoscaler.kubernetes.io/safe-to-evict: "false"` 注解**（显式拒绝驱逐）。

19.4 三件套的协作全景

把三层伸缩拼在一起，看一次流量高峰的完整响应链条：

```
流量高峰来临
  → 单 Pod 压力上升，CPU/自定义指标超阈值
  → HPA 扩容副本（分钟级）
  → 集群资源不够，新 Pod Pending
  → CA 扩容节点（数分钟级，Karpenter 更快）
  → 新 Pod 调度上节点，流量摊薄
  → 高峰回落
  → HPA 观察期后缩容副本
  → 节点利用率持续低 → CA 缩容节点
```

三个经验法则收尾：

1. **先 Pod 层，后节点层**：绝大多数弹性问题 HPA 就能解决，CA 是"兜底的兜底"；
2. **requests 要准**：HPA 看的是相对 requests 的比例、调度看的是 requests 占用——requests 不准，两层伸缩全部失真；
3. **演练缩容**：扩容的失败是"慢"，缩容的失败是"事故"（PDB 配置错误导致节点永远缩不掉，或过度缩容导致抖动）。把"低峰缩容"当作与"高峰扩容"同等级的测试场景。

19.5 本章小结

- **HPA 是副本数的自动驾驶仪**：目标指标（CPU/自定义）+ min/max 边界 + behavior 快扩慢缩；依赖 metrics-server；CPU 不是万能指标，业务指标 + KEDA 更贴切。
- **VPA 是成本的精算师**：推荐模式先行，Auto 模式会重建 Pod；与 HPA 的资源维度必须错开，避免反馈死循环。
- **CA/Karpenter 是节点层的弹性**：触发条件是 Pending Pod；缩容看利用率且受 **PDB 约束**；PDB 是"自愿驱逐"的安全底线，同时服务于缩容与节点维护。
- 三层协作的根基是**准确的 requests**；先 Pod 层后节点层，扩容要快、缩容要慢、缩容要演练。

到这里，一个"会呼吸"的集群已经成形。但会呼吸还不够——你得能**看见**它在呼吸：监控、日志、告警，是运维的眼睛。下一章进入可观测性部分。

第20章 日志与监控：给集群装上仪表盘

一个集群跑着几百个 Pod，你能回答这几个问题吗：**现在系统健康吗？用户请求的 99 分位延迟是多少？凌晨两点的报错是谁发出的？内存用量趋势如何、什么时候需要扩容？**

回答不了，就等于开着一辆没有仪表盘的车。这一章讲 Kubernetes 的"仪表盘工程"——**可观测性（Observability）** 的三大支柱：指标（Metrics）、日志（Logs）与追踪（Traces），以及 Kubernetes 上的标准实现。

20.1 可观测性三支柱

先厘清三个概念，它们回答不同的问题：

- **指标（Metrics）**：数值化的聚合数据（QPS、延迟分位数、内存用量）。回答"**总体健康吗？趋势如何？**"——查询快、存储省、适合告警与仪表盘；
- **日志（Logs）**：离散的事件文本。回答"**具体发生了什么？**"——细节最丰富，但量大、检索成本高；
- **追踪（Traces）**：一次请求穿越多个服务的完整路径。回答"**这次慢/失败发生在哪一跳？**"——微服务排障的瑞士军刀。

三者互补：告警靠指标定位异常范围，日志还原异常细节，追踪定位跨服务问题。成熟系统三支柱齐备，起步阶段指标 + 日志即可覆盖 80% 的需求。

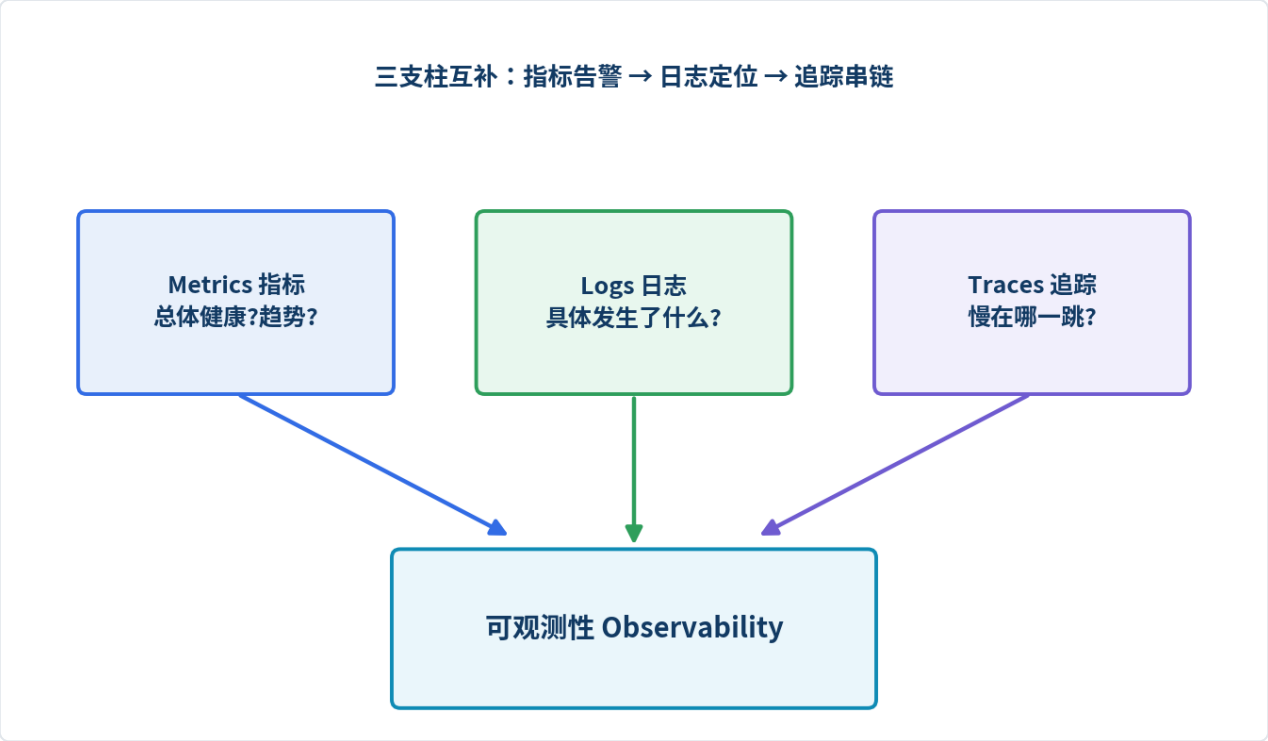


图20-1 可观测性三支柱：指标、日志、追踪各司其职

20.2 指标体系：Prometheus 与其模型

Prometheus 是云原生监控的事实标准。它的核心模型：

- **拉取（Pull）**：Prometheus 定期抓取各目标的 `/metrics` HTTP 端点。应用只需暴露一个文本格式的指标端点；
- **时序数据**：每条指标是一个时间序列，由**指标名** + **标签集**唯一标识（如 `http_requests_total{service="web", status="500"}`）；
- **PromQL**：强大的查询语言，支持聚合、速率、分位数计算。

Kubernetes 的指标分三层，逐层看：

第一层：节点与容器指标。 node-exporter（DaemonSet）暴露节点硬件指标；kubelet/cAdvisor 暴露容器 CPU/内存/网络指标——这就是 `kubectl top` 和 HPA 的数据源。

第二层：Kubernetes 自身指标。 apiserver、etcd、调度器、CoreDNS 都暴露 `/metrics`——它们能回答"API 延迟多高？etcd 同步慢吗？调度排队多长？"

第三层：应用指标。 你的业务暴露 QPS、错误率、延迟（RED 方法论：Rate、Errors、Duration），以及业务指标（订单数、支付成功率）。主流语言框架（Spring Boot Actuator、Go prometheus client 等）开箱即用。

20.2.1 部署与四条黄金指标

部署 Prometheus 生态的事实标准是 **kube-prometheus-stack** Helm Chart，一键包含 Prometheus、Alertmanager、Grafana 及核心看板。装好后，每层建议盯住"四条黄金指标"：

指标	问题	典型告警
流量（Rate）	服务在承受多少请求?	QPS 突降
错误率（Errors）	失败比例多高?	5xx > 1% 持续 5 分钟
延迟（Duration）	响应多快? 99 分位是否恶化?	P99 > 1s
饱和度（Saturation）	资源还剩多少余量?	CPU/内存/连接池 > 80%

Grafana 负责可视化：官方提供大量现成 Dashboard（节点监控、容器监控等导入即用），应用层建议每个服务一张 RED 看板。

20.2.2 告警：让人该睡就睡

告警系统最大的敌人是"告警疲劳"——几百条没人看的告警等于没有告警。几条铁律：

1. **告警必须"可行动"**：收到告警的人知道该做什么，否则降级为看板指标；
2. **按症状告警，不按原因告警**："用户请求错误率高"值得半夜叫人，"某 Pod 重启过"不值得；
3. **分级路由**：P0 电话叫醒、P1 IM 通知、P2 邮件周报（Alertmanager 的路由分组能力）；
4. **持续修剪**：每月复盘告警，删掉从未触发行动的规则。

20.3 日志体系：从 stdout 到检索平台

Kubernetes 对日志有一个重要的架构约定（十二要素第 11 条）：**应用把日志写到 stdout/stderr，不管理文件与轮转**。容器运行时负责收集，Kubernetes 统一提供 `kubectl logs`：

```
kubectl logs web-7d9c8b6f5-xk2p9          # 单个 Pod
kubectl logs -f -l app=web --tail=100     # 按标签聚合多个 Pod（调试利器）
kubectl logs web-pod -c log-collector     # 多容器指定 -c
kubectl logs web-pod --previous           # 崩溃前的日志（CrashLoop 排障必用）
```

但 `kubectl logs` 只能看"现在"，且 Pod 重建后旧日志就没了。**持久化日志体系**的标准架构（EFK/ELK）：

```
应用(stdout) → 容器运行时落盘(/var/log/containers)
  → DaemonSet 采集器(fluent-bit) 解析、加标签(k8s 元数据)
  → 传输/缓冲(kafka 可选)
  → 存储+检索(Elasticsearch / Loki)
  → 可视化(Kibana / Grafana)
```

两个主流后端的取舍：

- **Elasticsearch（EFK 栈）**：全文检索能力强、生态成熟；资源消耗大（JVM、分片管理），运维成本高；
- **Loki（Grafana 系）**：只索引标签不索引全文，存储成本极低；查询用 LogQL，适合"按服务/时间窗过滤"为主的场景；全文模糊搜索能力弱。

云上还有云原生日志服务（如 SLS、CloudWatch Logs），免运维、与告警集成好。**中小团队的务实之选通常是 Loki 或云日志服务**。

20.3.1 日志规范的工程要求

日志体系的上游是日志质量，几条团队规范值得写入开发手册：

1. **结构化日志**（JSON 格式）：包含 level、time、trace_id、service 等字段——结构化是"可检索"的前提，纯文本日志在采集端很难解析；
2. **携带 trace_id**：一次请求在所有服务的日志中共享同一个 trace_id（追踪系统注入），跨服务排障时用它串起完整链路；
3. **正确的级别使用**：ERROR 是"需要人看的故障"，WARN 是"异常但已自愈"——把 INFO 级别的日志打成 ERROR，告警系统会被淹死；
4. **不要打印敏感信息**：密码、token、身份证号进日志是合规事故（第 17 章安全篇的呼应）。

20.4 追踪：分布式系统的"GPS"

微服务架构下，一次下单请求可能经过网关 → 订单 → 库存 → 支付 → 消息队列。"用户说慢"，慢在哪一跳？日志只能逐个服务翻，**分布式追踪**给出完整答案：

Tracing 的模型：一次请求生成一个全局 **trace**（含唯一 trace_id），每经过一个服务产生一个 **span**（记录该跳的开始/结束时间、标签）。所有 span 按父子关系组成一棵"调用树"，在 UI 里一屏看清全链路耗时分布。

实现标准：**OpenTelemetry（OTel）** 已成为可观测性的统一标准（指标、日志、追踪三合一），SDK 埋点 → Collector 收集 → 后端存储展示（Jaeger、Tempo、商业 APM）。Java/Go 等语言支持零代码/低代码接入（Java Agent 自动埋点）。

采样策略值得了解：全量追踪成本高，生产常用"头部采样"（如 10%）+ "尾部采样"（错误与慢请求全保留）的组合，兼顾成本与排障需求。

20.5 排障实战：一次 5 分钟定位

把工具串起来，演示一次标准排障流（假设"下单接口 502"）：

```
# 1. 指标确认范围：Grafana 看 order-service 的 RED
#    → 错误率飙升，时间点 14:32

# 2. 集群侧快速检查
kubectl get pods -n shop -l app=orders          # 有没有 CrashLoop/Pending?
kubectl get events -n shop --sort-by=.lastTimestamp | tail   # 最近事件
kubectl top pods -n shop                        # 是否有 OOM 迹象

# 3. 锁定异常 Pod，看日志
kubectl logs orders-66f8c94d21-wq9rt --previous
# → 数据库连接池耗尽

# 4. 下钻：是 DB 慢还是流量突增?
#    Grafana 数据库看板 → 连接数打满；追踪系统看慢 SQL
```

5 分钟内从"用户报障"到"根因假设"，靠的正是三支柱的闭环：**指标发现异常 → 事件/日志定位范围 → 日志与追踪锁定根因**。

20.6 本章小结

- 可观测性三支柱各司其职：**指标管"健康与趋势"（告警）、日志管"具体细节"、追踪管"跨服务路径"**。
- **Prometheus（拉取 + 时序 + PromQL）是指标标准**；指标三层：节点/容器 → 集群组件 → 应用 RED；kube-prometheus-stack 一键部署。
- 告警铁律：**可行动、按症状、分级路由、持续修剪**——对抗告警疲劳。
- 日志架构：**应用只写 stdout，DaemonSet 采集 → Loki（省钱）或 ES（强检索）**；结构化日志 + trace_id 是可检索的前提。
- **OpenTelemetry 统一三大信号**；追踪模型 trace/span 一屏看穿跨服务耗时；采样平衡成本。
- 排障标准流：**指标发现 → 事件确认 → 日志下钻 → 追踪定位**，熟练后 5 分钟锁定根因不是玄学。

看得见了，还得多练"手感"——下一章是实战性最强的一章：当真正的故障来临时，如何有条不紊地排查：Pod 起不来、节点挂了、网络不通……每一类问题都有固定的排查路径。

第21章 故障排查：当船漏水时

前面的章节分散地讲过不少排障技巧，这一章把它们系统地串起来——建立一套**排查方法论**，并对 Kubernetes 最高频的故障场景给出标准处置路径。故障排查能力是区分"用过 K8s"和"精通 K8s"的分水岭。

21.1 方法论：五个原则

原则一：先止血，后查因。 生产故障的第一目标不是找根因，是恢复服务。重启 Pod、回滚版本、摘除故障节点、扩容缓冲——先让业务活下来，再从容复盘。记住：**用户不会因为你找到了根因而少受一分钟影响**。

原则二：从观察开始，别急着动手。 排障大忌是"猜一个原因就乱改"，改完不但没好，还污染了现场。先看：Pod 状态、事件、日志、指标——用证据收敛假设。

原则三：分层定位。 故障发生在哪一层？从上往下过一遍：应用层（代码 bug、配置错误）→ 工作负载层（Pod 异常、探针失败）→ 网络层（Service/DNS/NetworkPolicy）→ 存储层（PVC 挂载失败）→ 节点层（资源不足、内核问题）→ 控制平面（apiserver/etcd）。**每一层用该层的证据确认或排除**。



图21-1 故障排查的分层模型：自上而下逐层确认或排除

原则四：只改一个变量。 同时改镜像和配置，好了不知道谁治的病，坏了不知道谁下的毒。

原则五：记录与复盘。 每次故障记录：现象、排查路径、根因、修复、预防措施。三个月后你会感谢这份记录——同样的问题往往还会来。

21.2 排障工具箱总览

先把"武器库"清点一遍，后面按场景取用：

```
# 状态总览
kubectl get pods,services,endpoints -n <ns>
kubectl get events -n <ns> --sort-by=.lastTimestamp

# 深入详情（第一入口）
kubectl describe pod <pod>        # Events 是金矿
kubectl get pod <pod> -o yaml     # 看完整 spec 与 status

# 日志
kubectl logs <pod> --previous     # 崩溃前日志
kubectl logs -l app=web -f --tail=100

# 交互调试
kubectl exec -it <pod> -- sh      # 钻进容器
kubectl port-forward pod/<pod> 8080:80   # 本地直连
kubectl debug -it <pod> --image=busybox --target=app   # 临时调试容器（目标容器无 shell 时）

# 节点与资源
kubectl top nodes / kubectl top pods
kubectl describe node <node>      # 资源分配、conditions、污点

# 集群组件
kubectl get componentstatuses     # 老版本
kubectl -n kube-system get pods   # CoreDNS、kube-proxy 等健康度
```

21.3 高频故障场景手册

21.3.1 Pod 起不来：Pending

```
kubectl describe pod <pod>   # 看 Events
```

Events 里的 `FailedScheduling` 会写明原因，常见三种：

- **Insufficient cpu/memory**：节点资源不够。处理：降低 requests、或扩容集群、或清理资源大户。检查 `kubectl describe node` 的 Allocated resources；
- **node(s) had untolerated taint**：污点不容忍（专用节点、维护中节点）。处理：调整容忍或节点选择器；
- **node(s) didn't match pod affinity**：亲和性约束太紧（如 required 反亲和，节点数 < 副本数）。处理：放宽为 preferred 或加节点。

还有一类隐蔽的：**PVC 未绑定**（有状态应用）—— `kubectl get pvc` 检查。

21.3.2 镜像拉取失败：ImagePullBackOff

```
kubectl describe pod <pod> | grep -A5 "Failed to pull"
```

- **镜像名/tag 错误**（最常见，尤其是 tag 写成不存在的小版本）；
- **私有仓库未认证**：配置 imagePullSecrets，确认 Secret 的 dockerconfigjson 正确；
- **网络问题**：节点出网受限、仓库限流（如 Docker Hub 的 pull rate limit）——生产建议自建/代理仓库。

21.3.3 崩溃循环：CrashLoopBackOff

Pod 起来又崩、起来又崩，重启计数狂飙。排查路径：

```
kubectl logs <pod> --previous   # 上一次崩溃的日志——80% 的答案在这
kubectl describe pod <pod>      # 看 Last State: Terminated, Reason: OOMKilled / Error
```

- **OOMKilled**：内存超 limits 被杀。处理：调大 limits 或查内存泄漏。确认方法：describe 里 Last State 显示 OOMKilled，Exit Code 137；
- **应用启动即退出**：配置错误（连不上数据库、配置格式错）、端口占用、依赖缺失。看日志定位；
- **探针误杀**：liveness 探针太激进（超时太短、失败阈值太低），应用启动慢被反复杀。处理：加 startupProbe 或放宽参数（第 4 章的配方）；
- **退出码速查**：137 = SIGKILL（OOM 或手动强杀）、143 = SIGTERM（正常终止信号）、1 = 应用自身错误。

21.3.4 Running 但不就绪：0/1 Running

容器活着，readinessProbe 一直失败：

- describe 看探针失败详情（Unhealthy: HTTP probe failed with statuscode: 500）；
- 进容器手动验证：`kubectl exec` 后 curl 探针路径，看真实返回；
- 常见原因：应用依赖的数据库/缓存没起来（就绪探针检查了依赖）、探针路径写错、应用还没初始化完（缺 startupProbe）。

21.3.5 网络不通：层层剥洋葱

"Pod A 访问不了 Pod B"，按这个顺序剥：

```
# 1. DNS 解析正常吗?
kubectl exec -it <pod-a> -- nslookup service-b

# 2. Service 有就绪后端吗?
kubectl get endpoints service-b        # 空的? → 后端 Pod 没就绪或 selector 不匹配

# 3. 直连 Pod IP 通吗? (绕过 Service 与 DNS)
kubectl exec -it <pod-a> -- curl <pod-b-ip>:8080
# 通 → 问题在 Service/DNS 层; 不通 → 问题在网络路径层

# 4. NetworkPolicy 误伤?
kubectl get networkpolicy -n <ns>
# Cilium 环境: cilium connectivity test / hubble observe

# 5. 同节点通吗? 跨节点不通? → 走 CNI 层排查 (encap/路由)
```

第 12 章的网络原理在这一步发挥作用——理解 DNAT、conntrack、CNI 流派，才知道每一层该看什么。

21.3.6 节点故障：NotReady

```
kubectl get nodes                      # 哪个 NotReady
kubectl describe node <node>           # Conditions: MemoryPressure/DiskPressure/Ready
```

- **kubelet 失联**：ssh 上节点查 kubelet 服务状态（`systemctl status kubelet`）、日志（磁盘满？证书过期？）；
- **NotReady 后的 Pod**：默认 5 分钟（tolerationSeconds）后被驱逐重建到其他节点。**有状态 Pod 的重建可能卡在卷 detach**（云盘还挂在"僵尸"节点）——需要强制删除 Pod 触发卷强制分离；
- **磁盘压力（DiskPressure）**：镜像垃圾、日志膨胀、emptyDir 暴涨——清理镜像（crictl rmi --prune）、查日志轮转；
- **恢复后**：`kubectl uncordon <node>` 解除封锁。

21.3.7 证书过期：集群"集体失联"

apiserver 证书过期后，所有 kubectl 命令报 `x509: certificate has expired`。用 `kubeadm certs check-expiration` 检查、`kubeadm certs renew all` 续期（kubeadm 集群）。**在日历上提前标记证书到期日**——这是一个完全可预防的"经典事故"。

21.4 进阶武器：kubectl debug 与 ephemeral 容器

生产镜像往往是 distroless（没有 shell），`kubectl exec` 无从下手。**临时容器（Ephemeral Containers）** 解决这个问题——往运行中的 Pod 里"注入"一个调试容器，共享目标容器的进程命名空间：

```
kubectl debug -it orders-66f8c94d21-wq9rt \
  --image=nicolaka/netshoot \
  --target=orders      # 附加到目标容器，可查看其进程与网络
```

这既保持了生产镜像的精简与安全，又不牺牲可调试性——distroless 与可观测不再矛盾。

21.5 事故复盘：从救火到防火

排障的终点不是"修好了"，是"不再发生"。复盘四问：

1. **为什么会发生？**（根因，不止表层）
2. **为什么没有更早发现？**（监控盲区 → 补告警）
3. **为什么影响这么大？**（缺少熔断/降级/冗余 → 架构改进）
4. **如何避免复发？**（自动化检查、流程改进、文档沉淀）

> **提示：** 高水平团队的标志不是"不出故障"，而是**故障响应快、复发率低、同类问题不再犯**。把每次事故的处置脚本沉淀成 Runbook（运维手册），下次同样的告警响起，值班同学按图索骥即可。

21.6 本章小结

- 排查五原则：**先止血、先观察、分层定位、单变量修改、复盘沉淀**。
- 工具箱五件套：**get/events 总览 → describe 深入 → logs 取证 → exec/debug 交互 → top/describe node 查资源**。
- 高频故障的标准路径：**Pending 看 FailedScheduling、ImagePullBackOff 查镜像与凭证、CrashLoopBackOff 看 previous 日志与退出码（137=OOM）、网络不通按 DNS → Endpoints → 直连 IP → NetworkPolicy 逐层剥**。
- 节点 NotReady 关注 kubelet 与磁盘；有状态 Pod 的卷 detach 可能需强制干预；证书过期可预防，要日历化。
- distroless 时代用 **kubectl debug 临时容器**调试；复盘四问把救火转化为防火。

危机应对能力就绪。但最好的运维是"让危机少发生"——下一章讲主动出击的运维：集群升级、备份恢复与高可用设计。

第22章 升级、备份与高可用：永不沉没的船

故障应对是"被动挨打时的反应"，这一章讲"主动出击"：**集群升级**怎么做才不翻车、**备份恢复**怎么设计才真的能救火、**高可用架构**怎么搭才能真正抗住故障。这三件事决定了你的 Kubernetes 是"生产级"还是"玩具级"。

22.1 集群升级：最不该翻车的常规操作

Kubernetes 大约每 4 个月发布一个小版本，每个版本有约 14 个月的维护期。**不升级的后果是双重的**：安全补丁停发（安全风险），且一旦落后多个版本，升级将变得异常艰难（需要逐版本跳）。

升级的核心纪律：**逐个小版本升级，不跳级**——1.28 → 1.29 → 1.30，绝不能 1.28 → 1.30。跨版本之间组件的兼容矩阵（API 弃用、etcd 版本、kubelet 版本偏移）都会坑你。

22.1.1 升级前的准备清单

1. **确认 API 弃用**：每个版本都会移除一批旧 API（如 1.25 移除 `batch/v1beta1` 的 CronJob）。用 `kubectl-convert` 或 Pluto 工具扫描存量清单，提前改写；
2. **检查兼容性**：CNI、CSI、ingress controller 等插件的版本支持矩阵；
3. **备份 etcd**（下文详述）——这是后悔药；
4. **测试环境先走一遍**完整升级流程，记录耗时与坑点；
5. **安排维护窗口并公告**：升级过程中 Pod 会被驱逐重建（滚动），要有容量冗余。

22.1.2 kubeadm 集群的标准升级流程

```
# 1. 升级控制平面（第一台 master）
kubeadm upgrade plan                        # 预检与计划
apt install kubeadm=1.29.4-00
kubeadm upgrade apply v1.29.4               # 升级本机控制平面
apt install kubelet kubectl=1.29.4-00

# 2. 其余控制平面节点（每台 drain → upgrade node → uncordon）
kubectl drain k8s-master-2 --ignore-daemonsets
kubeadm upgrade node
apt install kubelet kubectl=1.29.4-00
kubectl uncordon k8s-master-2

# 3. 工作节点逐台（一台一台来！）
kubectl drain k8s-worker-1 --ignore-daemonsets --delete-emptydir-data
apt install kubeadm kubelet kubectl=1.29.4-00
kubeadm upgrade node
kubectl uncordon k8s-worker-1
```

`drain` 的本质（第 18 章讲过）：给节点打 NoExecute 污点驱逐 Pod，Pod 在其他节点重建。`--ignore-daemonsets` 跳过 DaemonSet（它们跟着节点走），`--delete-emptydir-data` 允许清空临时卷。

托管集群（EKS/ACK/TKE/GKE）的升级被大幅简化：控制平面由云厂商一键升级，你只需处理节点池滚动升级与清单兼容性。这也是"生产环境优先托管"的又一个理由。

22.1.3 升级期间业务会中断吗

控制平面升级：apiserver 短暂不可用（多主架构下逐台滚动，整体可用），**业务 Pod 不受影响**；节点升级：被 drain 的节点上的 Pod 滚动重建——**只要你的服务有多副本 + PDB + 反亲和分散**，业务无感。这三个条件缺一个，升级就是"生产事故模拟器"。可以说：**升级翻车是架构缺陷的照妖镜**。

22.2 备份：唯一能救你命的东西

第 3 章说过："etcd 里的数据就是集群本身。" 现在展开讲备份体系——它分两个层面，缺一不可。

22.2.1 集群状态备份（etcd 与资源清单）

etcd 备份覆盖：所有资源对象定义（Deployment、Service、ConfigMap、RBAC……）、集群自身状态（节点、证书、配置）。丢了这个，集群要重建一切。

```
# etcd 快照（在控制平面节点执行）
ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-snap.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

# 验证与恢复
etcdctl snapshot status /backup/etcd-snap.db
etcdctl snapshot restore /backup/etcd-snap.db --data-dir=/var/lib/etcd-restored
```

Velero 是应用最广的 K8s 备份工具，能力更完整：定时备份（schedule）、按 namespace/label 选择性备份、**连同 PV 数据一起备份**（CSI 快照集成）、跨集群恢复（迁移场景）。建议策略：

- etcd 快照：每小时一次，保留 7 天以上，**异机存放**（备份放在同一台机器上等于没备份）；
- Velero：每日全量 + 资源变更实时（可选）；
- **至少每季度演练一次完整恢复**——没有验证过的备份等于没有备份。演练还能暴露"配置靠记忆"的问题。

22.2.2 应用数据备份（PV 内容）

etcd 备份不含 PV 里的数据（那是存储侧的东西）。数据库类数据要用**专业备份方案**：数据库原生工具（pg_dump/xtrabackup）+ 定时 CronJob（第 9 章的示例）+ 异地副本（对象存储）。卷快照（第 14 章）是辅助手段，不能作为唯一备份。

22.3 高可用架构：假设一切都会坏

高可用设计的出发点是一个残酷假设：**任何单点都会坏，任何操作都可能出错**。逐层检查你的集群：

控制平面高可用： - apiserver ≥ 3 副本，前面挂负载均衡； - etcd ≥ 3 节点（奇数！Raft 协议需要多数派），跨故障域分布； - 所有组件通过 LB 访问 apiserver，而不是直连某一台。

工作负载高可用： - 副本数 ≥ 2（关键服务 ≥ 3），且**反亲和分散到不同节点/可用区**（第 18 章）； - **PDB** 保住驱逐底线（第 19 章）； - readiness/liveness 探针齐全（第 4 章）； - 多可用区部署：节点池跨 AZ，topologySpreadConstraints 按 zone 分散。

依赖高可用： - 镜像仓库挂了，集群拉不到镜像——本地镜像缓存/多仓库容灾； - 外部 DNS、证书签发（cert-manager 的 ACME）、云 API（LB 创建）这些"集群外的腿"，也要有冗余方案。

区域级容灾：单集群多 AZ 能扛节点与 AZ 故障；扛"区域故障"需要**多集群**（多区域各一集群，流量层做主备或双活），配合 GitOps（第 24 章）让两个集群配置保持一致。这是"持续交付能力"的终极考验：**能用 Git 重建整个集群，才算真正拥有它**。

22.4 混沌工程：主动破坏的勇气

高可用配置是否真的有效？纸上谈兵不算数。**混沌工程（Chaos Engineering）** 的思路：在生产（或等比预发）环境**主动注入故障**，验证系统的韧性。Kubernetes 生态的注入工具：Chaos Mesh（CNCF 项目，Pod 杀死、网络延迟/分区、磁盘 IO 注入等）、LitmusChaos。

入门实验清单：

1. **随机杀 Pod**：业务是否自动恢复？用户是否无感？（验证自愈与探针）
2. **drain 一个节点**：服务容量与可用性如何？（验证反亲和与 PDB）
3. **断开数据库网络 30 秒**：应用是雪崩还是优雅降级？（验证超时、重试、熔断）
4. **APZ 整体失联演练**（预发环境）：流量能否切到存活 AZ？

每次实验记录"预期 vs 实际"，发现的每一个意外都是修复"隐形故障"的机会。**故障演练做得越多，真故障来临时越从容**。

22.5 本章小结

- 升级纪律：**逐版本、先测试、备份先行、节点逐台 drain/uncordon**；升级无感的前提是多副本 + 反亲和 + PDB——升级是架构的照妖镜。
- 备份两层：**etcd/资源清单（集群本体）+ 应用数据（PV/数据库）**；工具首选 Velero；**没演练过恢复的备份等于没有备份**。
- 高可用逐层设防：控制平面（3+ etcd 奇数节点）、工作负载（多副本 + 分散 + PDB + 探针）、依赖（镜像仓库等集群外的腿）、区域级（多集群 + GitOps 重建能力）。
- **混沌工程主动验证韧性**：随机杀 Pod、节点驱逐、依赖断连，从预发开始练。

集群的"生存能力"建设完毕。最后一个部分，我们把视野投向生态：当 Kubernetes 满足不了业务需求时，如何教它学新本领？下一章从最流行的"应用包管理器"Helm 讲起。

第23章 Helm：Kubernetes 的包管理器

当你的应用有十几个 YAML 文件、要部署到三个环境、参数各不相同时，"复制粘贴改配置"的管理方式开始崩塌。这一章的主角 **Helm**——Kubernetes 世界的"apt/yarn/brew"，专门解决"**应用的打包、分发与参数化部署**"问题。

23.1 为什么需要包管理器

回望第 5 章的困境：一个服务 = Deployment + Service + ConfigMap + Ingress + HPA + PDB……六七个文件。如果三个环境要不同的副本数、镜像 tag、域名，你需要 18 个文件；改一处公共配置要同步改 6 处。更别说团队间"复用一套部署方案"的需求——别人开发的中间件（Redis、Kafka），难道要照着它的 GitHub 手工抄一遍 YAML?

包管理器要解决的就是三件事：

1. **打包**：把一组相关的资源清单打成一个可分发的单元；
2. **参数化**：同一份模板，不同环境传不同参数；
3. **分发复用**：别人"一键安装"你发布的应用。

Helm 的答案是：**Chart（模板包）+ Values（参数文件）+ Release（安装实例）**。

> **比喻：** Chart 是"装修图纸模板"，Values 是"业主的需求单"（三室改两室、墙刷米色），Release 是"按图纸 + 需求单装修出来的那一套房"。同一张图纸，可以给一千个业主装出一千套不同的房子，每套房子登记在册、可单独升级或拆除。

23.2 五分钟上手 Helm

```
# 安装 Redis（从官方仓库）
helm repo add bitnami https://charts.bitnami.com/bitnami
helm install my-redis bitnami/redis --set auth.password=secret123

# 查看已安装的 Release
helm list
# NAME       NAMESPACE  REVISION  STATUS    CHART
# my-redis   default    1         deployed  redis-19.6.1

# 升级（改参数后重新下发）
helm upgrade my-redis bitnami/redis \
  --set auth.password=secret123 --set replica.replicaCount=3

# 回滚到上一个版本
helm rollback my-redis 1

# 卸载
helm uninstall my-redis
```

注意 `REVISION` 列——**Helm 自带版本管理**，每次 install/upgrade 生成一个版本号，`helm rollback` 一键回到任意历史版本。这是它比裸 `kubectl apply` 强的地方。

23.3 Chart 的解剖：模板 + 参数

自己写一个 Chart，看看内部结构：

```
myapp/
├── Chart.yaml          # 元数据：名称、版本、描述
├── values.yaml         # 默认参数
├── templates/          # 模板目录
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── configmap.yaml
│   ├── ingress.yaml
│   └── _helpers.tpl    # 可复用的模板片段
└── .helmignore
```

Chart.yaml：

```
apiVersion: v2
name: myapp
description: A demo web application chart
version: 0.1.0        # Chart 自身版本
appVersion: "2.1.0"   # 所打包应用的版本（语义独立于 chart 版本）
```

values.yaml（参数默认值）：

```
replicaCount: 2
image:
  repository: myrepo/myapp
  tag: "2.1.0"
resources:
  requests: {cpu: 100m, memory: 128Mi}
ingress:
  enabled: false
  host: ""
```

templates/deployment.yaml（模板——Go template 语法）：

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "myapp.fullname" . }}
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
      - name: {{ .Chart.Name }}
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
        resources:
          {{- toYaml .Values.resources | nindent 10 }}
```

`.Values.xxx` 引用参数，`{{ include }}` / `{{- }}` 是模板控制语法。使用时覆盖参数的优先级：**命令行 --set > -f 自定义 values 文件 > Chart 内置 values.yaml**。

调试模板的三板斧：

```
helm template myapp ./myapp -f prod-values.yaml   # 渲染出最终 YAML 看结果
helm lint ./myapp                                  # 静态检查
helm install myapp ./myapp --dry-run --debug      # 模拟安装不落盘
```

> **提示：** `helm template` 应该进入 CI 流水线——在真正 apply 之前，先渲染 + diff，确认变更符合预期。这能拦住绝大多数模板错误。

23.4 多环境管理：values 的分层覆盖

生产级用法是**每个环境一个 values 文件**：

```
helm install myapp ./myapp -f values-dev.yaml
helm install myapp ./myapp -f values-prod.yaml
```

`values-prod.yaml` 只写与默认值的差异：

```
replicaCount: 6
image:
  tag: "2.1.0"
ingress:
  enabled: true
  host: shop.example.com
```

这套"模板 + 分层参数"模式，正是后面第 24 章 GitOps 的标准输入——CI/CD 只需在发布时选择不同的 values 文件。

23.5 Chart 仓库与分发

Chart 打包成 `.tgz`（`helm package`），发布到 **Chart 仓库**（类似镜像仓库之于镜像）：可以用 ChartMuseum 自建，可以托管在 OCI 兼容仓库（Harbor 已支持 `oci://` 协议，推荐），也可以走 GitHub Pages。

分发的意义在内部复用上：公司的基础设施团队发布标准化的"中间件 Chart"（带好监控、日志、安全默认配置），业务团队一条命令部署合规的 Redis/Kafka——**把最佳实践打包分发，而不是写文档让人抄**。

23.6 Helm 的边界与替代视角

Helm 不是万能的，认识它的局限同样重要：

- **模板是图灵完备的**：复杂的模板里嵌套着 if/else、range、函数调用，可读性堪忧——"模板地狱"与"YAML 地狱"只是换了一种痛；
- **Helm 是"命令式使用"的声明式工具**：upgrade 靠人执行，谁最后执行谁说了算，缺乏"仓库即真相"的持续调和；
- **状态存于 Helm 自身**（Release 记录），集群被人手动改过之后，Helm 的升级可能与实际状态冲突。

替代方案 **Kustomize**（kubectl 内置）：不用模板，用"基础清单 + 补丁覆盖"，纯 YAML 无逻辑，可读性好，但没有包分发能力，参数能力也弱于 Helm。社区的现实共识：**消费第三方应用用 Helm Chart，管理自己的应用配置用 Kustomize 或 GitOps 工具**，两者混用很常见（用 Kustomize 包一层调整官方 Chart 的参数）。

23.7 本章小结

- Helm 解决三件事：**打包（Chart）、参数化（Values）、复用分发（仓库）**；Release 自带版本管理与回滚。
- 模板三宝：`.Values` 引参数、`_helpers.tpl` 复用片段、`helm template --dry-run` 调试——**渲染结果进 CI**。
- 多环境 = 一张模板 + 分层 values 文件；优先级 --set > -f > 内置默认。
- Chart 仓库（推荐 OCI 协议）让最佳实践以"包"的形式在组织内流通。
- 边界认知：复杂模板伤可读性；Helm 缺持续调和；Kustomize 是纯声明式补充。两者常配合使用。

包管理解决了"装得快"。但"配置的最终真相"应该在哪里？当 Git 成为部署的唯一入口，会发生什么？下一章：GitOps 与 Kustomize——声明式的终极形态。

第24章 GitOps 与 Kustomize：声明式的终极形态

Helm 解决了"装得快、参数化"，但把视角拉远：整个部署体系的"真相"到底存在哪里？谁有权触碰生产？配置漂移如何杜绝？这一章的 **GitOps** 给出当代云原生的标准答案：**Git 仓库是期望状态的唯一真相源，控制器自动让集群与 Git 保持一致**。

24.1 GitOps 的四原则

GitOps 不是"用 Git 存配置"那么简单，它是一套完整的运营模型，四条原则：

1. **声明式**：系统的全部期望状态（应用配置、基础设施）以声明式描述，可存储、可版本化（Kubernetes 天然满足）；
2. **版本化与不可变**：期望状态存储在 Git，完整历史，任何变更都有提交记录与审阅痕迹；
3. **自动拉取（Pull 模型）**：代理（控制器）自动从 Git 拉取期望状态并应用到集群——**没有人在 CI 里直接对集群执行 kubectl**；
4. **持续调和（Reconcile）**：代理持续对比实际状态与 Git 状态，发现漂移就纠正（或告警）。

与"传统 CI/CD 推模型"的本质区别在于**方向**：传统方式是 CI 流水线"推"到集群（流水线持有集群凭证，人触发部署）；GitOps 是集群内的代理"拉" Git（集群持有只读的仓库凭证，Git 状态触发部署）。

> **比喻：** 推模型像"外卖员每次送菜到你家"——外卖员（CI）需要你家钥匙（集群凭证），换锁要通知所有外卖员；拉模型像"你家有自动进货机器人，每天去仓库（Git）按清单核对并补货"——机器人永远知道清单，仓库永远是最新的，谁想改菜单必须改仓库里的清单。

Pull 模型的安全收益是颠覆性的：CI 系统不再需要生产集群的管理员凭证（CI 被攻破不再等于生产沦陷）；生产变更的唯一入口是"合并一个 PR"，审计与回滚都变成了 Git 操作（`git revert`）。

24.2 Argo CD：GitOps 控制器

两大主流工具 **Argo CD**（CNCF 毕业项目）与 Flux CD 能力相当，以 Argo CD 为例看工作方式：

```
# 安装
kubectl create namespace argocd
kubectl apply -n argocd -f https://.../install.yaml

# 声明一个"应用"：从哪个仓库、哪个路径、部署到哪
argocd app create myapp \
  --repo https://git.example.com/platform/prod-manifests.git \
  --path apps/myapp/overlays/prod \
  --dest-server https://kubernetes.default.svc \
  --dest-namespace shop
```

从此 Argo CD 接管这个应用：每 3 分钟对比一次 Git 与集群（Sync Status），界面上一目了然地显示 **Synced / OutOfSync / Degraded**；你甚至可以开启**自动同步 + 自动修剪**（prune，Git 里删了资源集群里也删）。

"手动改线上"的下场：有人 `kubectl edit` 改了线上副本数？Argo CD 检测到 OutOfSync，自动同步把它改回 Git 里的值。第 15 章说的"配置漂移"问题，在这里被机制性消灭。

部署新版本 = 合并 PR：CI 构建镜像成功后，通过工具（Argo CD Image Updater、CI 里的 argocd CLI）向配置仓库提交一个"更新镜像 tag"的 commit → PR 审核 → 合并 → Argo CD 自动部署。**发布流程的全部环节（构建、审核、部署、回滚）都是 Git 操作**。

进阶玩法 **App of Apps** 模式：一个"根应用"管理几百个子应用，整个平台的应用拓扑本身就是 Git 里的一棵目录树——新集群上线 = Argo CD 指向仓库 = 全部应用自动铺设（第 22 章的"用 Git 重建集群"由此实现）。

24.3 Kustomize：无模板的配置叠加

GitOps 仓库里，配置的组织方式首推 **Kustomize**——kubectl 内置（`kubectl apply -k`），不引入模板语言，靠"**基础 + 覆盖**"的叠加模型：

```
prod-manifests/
└── apps/myapp/
    ├── base/                    # 基础清单（三个环境的共性）
    │   ├── deployment.yaml
    │   ├── service.yaml
    │   └── kustomization.yaml
    └── overlays/
        ├── dev/
        │   ├── kustomization.yaml      # 引用 base，声明覆盖
        │   └── patch-replicas.yaml
        └── prod/
            ├── kustomization.yaml
            ├── patch-replicas.yaml     # 副本数补丁
            └── patch-image.yaml        # 镜像 tag 覆盖
```

`overlays/prod/kustomization.yaml`：

```
resources:
- ../../base
patches:
- path: patch-replicas.yaml
images:
- name: myrepo/myapp
  newTag: "2.1.0"
namespace: shop-prod
```

产出验证：`kubectl kustomize overlays/prod` 渲染最终 YAML（GitOps 仓库的标准验证步骤）。

Kustomize 与 Helm 的分工（第 23 章预告）：**Helm 适合"从零生成"复杂模板与分发第三方包；Kustomize 适合"微调既有清单"**——比如给官方 Chart 的输出再打补丁。GitOps 仓库里两者混用很常见：Argo CD 原生支持 Helm Chart + Kustomize 补丁的组合。

24.4 GitOps 的目录组织与秘密管理

生产级 GitOps 仓库的典型布局：

```
infra-repo/
├── platform/            # 平台层：Argo CD、监控、日志、ingress 等
│   ├── argocd/
│   └── monitoring/
├── apps/                # 业务应用层（按团队分目录）
│   ├── team-a/
│   │   ├── orders/overlays/{dev,staging,prod}
│   │   └── users/...
│   └── team-b/
└── clusters/            # 每个集群的差异化配置
    ├── prod-cn/
    └── prod-eu/
```

两个关键决策：

单仓库 vs 多仓库：小团队单仓库（infra + 应用配置）省心；多团队大仓库按"平台层 / 各团队应用层"拆分，配 CODEOWNERS 实现目录级审阅权限（"改 team-a 的配置必须 team-a 的人批准"）。

秘密的存放（老话题的 GitOps 版本）：Git 里**永远不存明文密码**。方案：**Sealed Secrets**（加密后的 Secret 只有集群内私钥能解开，可安全入库）、**SOPS + 云 KMS**（提交前加密）、**External Secrets Operator**（Git 里只存外部密钥管理系统的引用）。三者选一即可，原则相同：Git 可见的一切都可以公开。

24.5 渐进式交付：GitOps 的下一层

GitOps 保证了"Git 说的就是集群跑的"，但没有回答"**发布的新版本怎么安全地放量**"。**Argo Rollouts**（或 Flagger）补上这块：用自定义的 Rollout 资源替代 Deployment，声明金丝雀策略：

```
spec:
  strategy:
    canary:
      steps:
      - setWeight: 10          # 放 10% 流量
      - pause: {duration: 10m} # 观察 10 分钟
      - analysis:              # 跑指标分析模板（错误率、P99）
          templates: [success-rate]
      - setWeight: 30
      - pause: {duration: 10m}
      - setWeight: 60
      - setWeight: 100
```

指标达标自动推进，异常自动回滚——**金丝雀发布从"人肉盯盘"变成"声明式 + 自动化"**。这是 GitOps 体系的自然延伸，也是第 7 章发布策略的工程化落地。

24.6 本章小结

- GitOps 四原则：**声明式、版本化、Pull 模型、持续调和**；核心转变是"CI 推"变"控制器拉"，CI 不再持有生产凭证。
- **Argo CD**：App = 仓库 + 路径 + 目标集群；自动同步 + 修剪机制性消灭配置漂移；部署 = 合并 PR，回滚 = git revert。
- **Kustomize**：base + overlays 的无模板叠加，`kubectl apply -k` 原生内置；与 Helm 分工——分发用 Chart、微调用 Kustomize。
- 仓库组织：平台层/应用层/集群层分层，CODEOWNERS 管审阅权限；秘密用 Sealed Secrets / SOPS / External Secrets，Git 永不存明文。
- **Argo Rollouts** 把金丝雀发布声明式化、自动化，指标驱动放量与回滚。

至此，应用的交付与配置管理闭环完成。但有些需求，Kubernetes 的内置资源真的满足不了——比如"一键部署一套完整的数据库集群并自动运维主从切换"。怎么办？教 Kubernetes 学新本领。下一章：CRD 与 Operator。

第25章 CRD 与 Operator：教 Kubernetes 新本领

全书至此，你接触的每一种资源——Pod、Deployment、Service、PV——都是 Kubernetes "出厂自带"的。但真实业务总有内置资源覆盖不了的需求：**"给我部署一套三节点的 PostgreSQL，自动配置主从，故障时自动切换主库"**——没有任何内置资源能表达这种"领域知识"。

这一章讲 Kubernetes 最强大的扩展机制：**CRD（自定义资源定义）与 Operator（自定义控制器）**。理解了它们，你将真正理解 Kubernetes 的设计野心——它不是一组固定的功能，而是**一个可以无限生长的平台内核**。

25.1 CRD：给 API 添加新词汇

CRD（Custom Resource Definition） 允许你向集群注册一种全新的资源类型。定义一个 `Database` 类型：

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: databases.demo.example.com    # 名字格式：复数.组名
spec:
  group: demo.example.com             # API 组（自定义域名）
  names:
    kind: Database
    plural: databases
    shortNames: ["db"]
  scope: Namespaced
  versions:
  - name: v1
    served: true
    storage: true
    schema:                           # 用 OpenAPI 校验字段！
      openAPIV3Schema:
        type: object
        properties:
          spec:
            type: object
            required: ["size", "version"]
            properties:
              size:
                type: integer
                minimum: 1
              version:
                type: string
              storage:
                type: string
```

`kubectl apply` 之后，集群立刻"学会"了一种新资源——你可以像操作内置资源一样操作它：

```
$ kubectl get databases          # 或 kubectl get db
$ kubectl explain database.spec  # schema 让 explain 生效
$ kubectl apply -f my-db.yaml
```

```
apiVersion: demo.example.com/v1
kind: Database
metadata:
  name: orders-db
spec:
  size: 3
  version: "16.2"
  storage: 100Gi
```

关键认知一：CRD 注册后，整个生态立刻兼容——kubectl 能操作、RBAC 能授权、Argo CD 能同步、Prometheus 能发现……**新资源不是"二等公民"，而是一等公民**。这就是 Kubernetes API 机器的强大之处。

关键认知二：CRD 只定义了"词汇表"，没有任何行为——创建 `Database` 对象后，集群里什么都不会发生（没有 Pod 被拉起、没有盘被创建）。**让新资源"活起来"的，是下一节的主角**。

25.2 Operator：带着领域知识的控制器

回忆第 6 章的控制循环：控制器 Watch 某种资源，对比期望与实际，执行动作消除差异。**Operator = 自定义控制器 + CRD**：它 Watch 你定义的新资源，把某个软件的**领域运维知识**编码进控制循环。

以 `Database` Operator 为例，它的 Reconcile 逻辑：

```
Watch 到一个 Database 对象（spec: 3 副本，PG 16.2，100Gi）
  → 实际没有对应 StatefulSet？→ 创建（模板内含主从配置、探针、反亲和）
  → PVC 数量不够？→ 等待/创建存储卷
  → 版本与 spec 不符？→ 执行滚动升级（数据库升级的特殊顺序）
  → 实际副本数 < 3？→ 执行扩容流程（新从库加入集群的注册步骤）
  → 主库 Pod 故障？→ 执行主从切换（选新主、更新 Service 指向）
  → 每天备份、监控指标暴露、告警规则生成……
```

注意这些动作——**主从切换、注册新节点、数据库专属的升级顺序**——StatefulSet 一概不懂，只有懂 PostgreSQL 的人才知道。Operator 把"DBA 的大脑"封装成了程序。

> **比喻：** StatefulSet 像一位"通用的宿管阿姨"，只会发钥匙、管秩序；Operator 像"专属管家"，懂这一家人的生活习惯——谁的药几点吃、停电了先关哪个闸。CRD 是管家的工作手册（需求清单），控制器是管家本人。

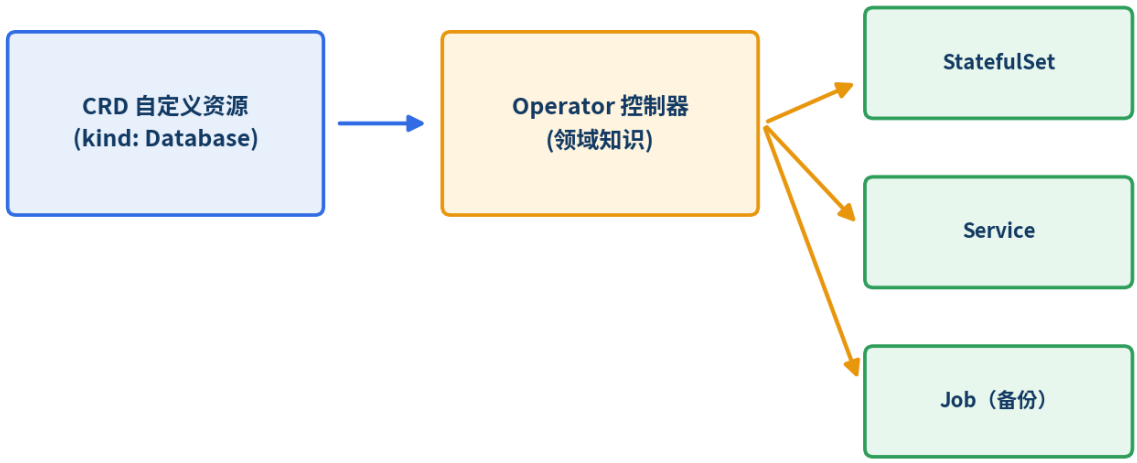


图25-1 Operator 将领域知识编码进控制循环，编排底层资源

25.3 生态中的 Operator：站在巨人肩上

理解 Operator 的最快方式是看生态：**几乎所有主流有状态软件都有成熟的 Operator**：

软件	Operator 能力示例
PostgreSQL（CloudNativePG、Zalando）	一键集群、自动故障切换、时间点恢复（PITR）
MySQL（Oracle/Vitess）	主从/组复制、备份、滚动升级
Kafka（Strimzi）	集群伸缩、分区再均衡、滚动升级
Elasticsearch（ECK）	节点角色编排、快照生命周期管理
Prometheus（Prometheus Operator）	声明式定义抓取规则、告警规则（ServiceMonitor CRD）
cert-manager	证书的自动签发与轮换（Certificate CRD）

使用视角：运维一套 Kafka 从"背文档、写脚本、值守故障"变成"apply 一个 YAML + 盯 Operator 日志"。Operator 之于有状态应用，就是 Deployment 之于无状态应用。

开发视角：写 Operator 有成熟的脚手架——**Kubebuilder**（Go，社区标准）、**Operator SDK**（含 Ansible/Helm 两种低代码模式）。核心就是实现 Reconcile 函数：读对象 → 算差异 → 调 apiserver 创建/更新真实资源（StatefulSet、ConfigMap、Job……）。Helm 模式甚至允许把"已有 Helm Chart + 补充逻辑"包装成 Operator。

25.4 设计 Operator 之前的三问

Operator 是强大工具，也是复杂度放大器。动手前先过三问：

一问：真的需要自定义资源吗？ 如果逻辑只是"部署一组固定的资源"，Kustomize/Helm 就够了。Operator 的价值在于**持续的、有状态的管理逻辑**（故障切换、生命周期钩子）——没有这些，别上 Operator。

二问：领域知识真的能编码吗？ 有些运维流程依赖人的判断（"这次要切主吗？数据可能不一致"）。把半自动的流程强行自动化，Operator 可能比人错得更快。稳妥路线：先做成"半自动 Operator"（自动执行、关键决策人来批注），再逐步提高自动化程度。

三问：谁来长期维护它？ Operator 是"活代码"——它依赖的软件升级、K8s 版本演进，都要跟着维护。没有owner 的 Operator 是负资产。

> **提示：** 有一个务实的起步模式：**"Operator 化"你的 Runbook**。把运维手册里"出现 X 就执行 Y"的步骤写成控制器逻辑——这是最低风险、最快见效的 Operator 起步方式，也让团队的运维知识从文档升级为代码。

25.5 Reconcile 的工程纪律

自己写 Operator 时，有几条铁律来自社区的集体血泪：

1. **幂等**：Reconcile 可能被重复触发（事件、重试、重启），同样的输入必须产生同样的结果——每一步先"检查再执行"（或直接用 createOrUpdate）；
2. **乐观并发与冲突重试**：多副本/多控制器同时 Reconcile 时用 resourceVersion 防冲突，冲突就重试；
3. **status 回写与 conditions**：把内部状态写回 `status.conditions`（如 Ready/Progressing/Error），让 kubectl 和上层系统可观测——"不写 status 的 Operator 是黑盒"；
4. **Finalizer 处理删除**：需要清理外部资源（云盘、外部集群注册）时用 finalizer 拦截删除，清理完成再放行；
5. **事件与日志**：关键动作发 Kubernetes Event（`kubectl describe` 能看到），内部细节打结构化日志。

25.6 本章小结

· **CRD 扩展词汇表**：注册新资源类型，全生态（kubectl/RBAC/GitOps/监控）自动兼容；但 CRD 本身没有行为。

- **Operator = CRD + 领域控制器**：把软件的运维知识（部署拓扑、故障切换、升级顺序、备份）编码进 Reconcile 循环——StatefulSet 管通用有状态语义，Operator 管专属领域知识。
- 生态已成熟：PG/MySQL/Kafka/ES/Prometheus 都有生产级 Operator，**优先用现成的，其次"Operator 化"自己的 Runbook，最后才考虑从零自研**。
- 自研铁律：幂等、冲突重试、status 可观测、finalizer 清理、事件与日志。
- 更大的图景：CRD/Operator 是"平台工程"的基石——把平台的复杂度封装成声明式 API，让业务团队面对的永远是简单的"申请单"。

Kubernetes 的可扩展性至此圆满。但还有一个维度的问题：**服务一多，服务之间的流量治理（超时、重试、熔断、加密、灰度）要去哪里管？** 下一章，也是生态部分的最后一章：服务网格 Istio。

第26章 服务网格 Istio：网络里的情报系统

微服务规模一上来，一组新问题随之而来：服务之间的调用要加超时和重试（哪个团队去改代码？）；服务间流量要 mTLS 加密（每个服务都集成证书逻辑？）；要按比例灰度两个版本（改 Ingress？要按 header 呢？）；调用链路上谁依赖谁（加埋点？）……

这些需求的共同点：**它们都是"流量治理"，却散落在每个服务的业务代码里**。服务网格（Service Mesh）的答案是：**把流量治理从业务代码里抽出来，下沉到基础设施层**。

26.1 服务网格的核心思想：Sidecar 接管流量

Istio 是最流行的服务网格实现。它的核心架构只有一招——**Sidecar 代理**（Envoy）：

给命名空间打上注入标签后，每个 Pod 创建时自动多出一个容器——Envoy 代理（通过第 5 章讲过的 MutatingWebhook 注入）。此后，**该 Pod 收发的所有流量都被强制经过 Envoy**（通过 iptables 流量劫持）：

```
业务容器 ⇄ (localhost) ⇄ Envoy sidecar ⇄ 网络 ⇄ 对端 Envoy sidecar ⇄ 对端业务容器
```

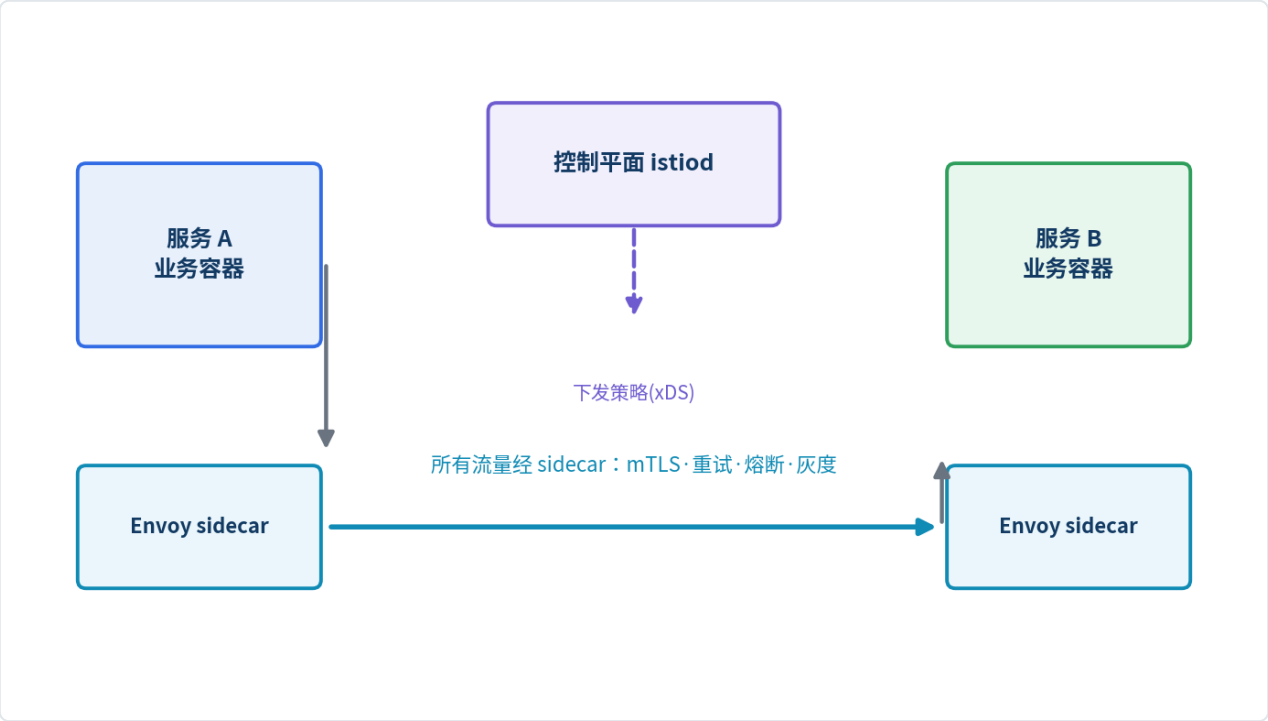


图26-1 服务网格：Sidecar 接管流量，控制平面统一下发策略

于是：重试、超时、熔断、加密、灰度分流、指标采集——全部由 Envoy 执行，**配置由网格的控制平面统一下发，业务代码零改动**。

> **比喻：** 每个服务像一个外交官，原本要自己学各国语言、自己安排安保、自己记录行程；服务网格给每人配了一位**全能随从**（Envoy）：翻译、安保、行程记录全包，外交官只管谈正事。所有随从还听命于同一个"礼宾司"（控制平面），全局政策一夜生效。

控制平面 **istiod** 负责三件事：把用户声明（VirtualService 等 CRD）翻译成 Envoy 配置并下发（xDS 协议）、维护服务发现信息、签发 mTLS 证书。数据平面就是那成百上千个 Envoy。

26.2 三个核心 CRD：VirtualService、DestinationRule、Gateway

Istio 的流量管理通过一组 CRD 声明（第 25 章的知识立刻派上用场）。

VirtualService——"路由规则"：定义流量怎么走：

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts: [reviews]
  http:
  - match:
    - headers:
        x-beta-user: {exact: "true"}     # 请求头匹配
    route:
    - destination: {host: reviews, subset: v2}
  - route:
    - destination: {host: reviews, subset: v1}
      weight: 90                          # 其余流量 9:1 分流
    - destination: {host: reviews, subset: v2}
      weight: 10
    timeout: 2s                           # 每跳的超时
    retries:
      attempts: 2
      perTryTimeout: 500ms
```

按 header 精确分流（内部用户先体验新版）+ 按权重灰度 + 超时重试——**第 7 章费尽心思的灰度方案，在网格里是几行声明**。

DestinationRule——"目标策略"：定义访问目标的方式（版本划分 subset、负载均衡算法、连接池、熔断）：

```
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: reviews
spec:
  host: reviews
  trafficPolicy:
    loadBalancer:
      simple: LEAST_REQUEST      # 最少连接
    outlierDetection:            # 熔断：连续 5 次错误摘除 30 秒
      consecutive5xxErrors: 5
      interval: 10s
      baseEjectionTime: 30s
  subsets:
  - name: v1
    labels: {version: v1}
  - name: v2
    labels: {version: v2}
```

Gateway——"网格的大门"：管理南北向（外部进出）流量，角色类似 Ingress，但由 Envoy 执行、配置方式与 VirtualService 一体化。

东西向（服务间）用 VirtualService/DestinationRule，南北向用 Gateway——一套语法管全部流量。

26.3 mTLS：服务间加密的"免费午餐"

网格最迷人的能力之一：**服务间通信自动 mTLS 加密 + 双向身份认证**，业务代码零改动。

原理：Envoy 之间用 istiod 签发的证书建立 TLS，证书里带着服务的身份标识（namespace/serviceaccount）。由此获得三重收益：

1. **传输加密**：内部流量不再明文裸奔（合规刚需）；
2. **身份认证**：请求方身份可验证——"这个请求确实来自 orders 服务"；
3. **授权基础**：可以声明"只有 app=orders 的服务能调 payments"（AuthorizationPolicy CRD），零信任网络（第 17 章的愿景）落地。

迁移的平滑性也设计好了：`PeerAuthentication` 支持 `PERMISSIVE` 模式（同时接受明文与 mTLS），全量切换到 `STRICT` 之前可以先观测。

26.4 可观测性的"免费升级"

流量经过 Envoy 的副作用是：**每个服务自动获得统一的黄金指标**——请求量、错误率、延迟分位数（P50/P90/P99），按 source/destination/version 等维度打标，直接进 Prometheus。

叠加 **Kiali**（网格可视化）可以看到实时服务拓扑图：谁调用谁、每条边的 QPS 与健康度、配置生效情况。配合 OpenTelemetry 追踪（第 20 章），"跨服务排障"的两大痛点（看不见拓扑、日志串不起来）同时解决。

> **提示：** 很多团队上 Istio 的第一动机就是"不改造代码就能获得全套指标和追踪上下文"——这是完全正当的收益，不必等到需要高级流量治理才考虑。

26.5 Istio 的代价：清醒地看复杂度

服务网格不是免费的，三笔账要算清：

性能账：每个请求多两跳 sidecar（进出各一次），增加毫秒级延迟与内存/CPU 开销；规模越大，sidecar 舰队本身就是一个需要运维的"分布式系统"。

复杂度账：流量"不走了直线"，排障时要多看一层（Envoy 日志、xDS 配置同步状态）；VirtualService/DestinationRule 的组合语义有学习曲线；Istio 升级本身也是运维工作。

收益门槛：服务少（<10）时，K8s 原生能力 + Ingress + 应用内重试库已经够用，网格的收益体现不出来。

社区的应对是**淡化 Sidecar**：**Istio Ambient 模式**（1.24+ GA）用节点级的共享代理（ztunnel，处理 mTLS/L4；waypoint，按需处理 L7）替代每 Pod 一个 sidecar——去掉每 Pod 的资源开销与注入复杂度，保留网格能力。eBPF 系（Cilium Service Mesh）则更激进地把 L4 处理压进内核。方向一致：**网格的"体感"要像基础设施一样隐形**。

引入建议：微服务 > 20 个、或团队 > 5 个、或有强合规加密需求时，认真评估；服务少时优先打磨 K8s 原生能力。也可以只开 Ambient 的 L4 能力（mTLS + 遥测），按需再加 L7。

26.6 本章小结

- 服务网格的本质：**流量治理从业务代码下沉到基础设施**——每 Pod 注入 Envoy sidecar（或 Ambient 的节点级代理），控制平面统一下发策略。
- 三个核心 CRD：**VirtualService（路由/超时/重试）、DestinationRule（版本子集/负载均衡/熔断）、Gateway（南北向入口）**；一套语法管东西向与南北向。
- **mTLS 自动加密 + 服务身份**是零信任的落地路径，PERMISSIVE → STRICT 平滑迁移。
- 可观测性"免费升级"：黄金指标 + Kiali 拓扑 + 追踪上下文注入。
- 代价要清醒：延迟与资源开销、排障多一层、维护成本；**服务规模是引入网格的核心决策变量**；Ambient/eBPF 正在降低这笔代价。

生态篇至此完成。前面 26 章的所有积木——概念、工作负载、网络、存储、安全、调度、可观测、生态工具——现在到了检验的时刻。下一章（终章）：从零开始，把它们组装成一个完整的微服务应用。

第27章 从零到一：部署一个微服务应用

全书最后一步：把 26 章学到的一切组装起来，完整走一遍"**一个微服务应用从代码到生产**"的旅程。这一章是一个大型综合实验——每个环节都会标注"用到了第几章的知识"，你可以把它当作全书的实战索引，也可以在本地集群跟着做一遍。

27.1 场景设定

我们要部署一个简化的电商系统，包含四个服务：

- **frontend**（React 静态页 + Nginx）：用户界面，无状态；
- **orders**（Go/Java API）：订单服务，无状态，依赖 PostgreSQL 与 Kafka；
- **users**（Python API）：用户服务，无状态，依赖 PostgreSQL；
- **postgres + kafka**：数据与消息中间件（用现成 Operator 部署）。

目标架构：

```
用户 → Ingress(Gateway) → frontend → orders / users
                                        |        |
                                  PostgreSQL  PostgreSQL
                                        |
                                   Kafka（订单事件）
```

目标能力清单：HTTPS 入口、健康检查、资源限制、多副本高可用、HPA 自动伸缩、配置与密码分离、金丝雀发布、监控日志齐备——**每一项对应前面的一章**。

27.2 第一步：环境准备（第 3、22 章）

本地实验环境推荐 Kind（Kubernetes in Docker）：

```
# 安装 kind 与 kubectl 后
kind create cluster --name demo --config kind-config.yaml
kubectl cluster-info

# 部署基础设施组件
helm install ingress-nginx ingress-nginx/ingress-nginx \
  -n ingress-nginx --create-namespace        # 第 11 章：入口
helm install metrics-server metrics-server/metrics-server \
  -n kube-system                              # 第 19 章：HPA 依赖
helm install prometheus kube-prometheus-stack ...   # 第 20 章：监控
```

生产环境则从"托管集群 + 多可用区节点池"开始，跳过自建控制平面的复杂度（第 22 章的取舍）。

27.3 第二步：设计清单（第 5、15、4 章）

先定目录结构（第 24 章的 GitOps 仓库雏形）：

```
shop/
├── base/                    # 公共清单
│   ├── frontend/
│   ├── orders/
│   └── users/
└── overlays/
    ├── dev/
    └── prod/
```

orders 的 Deployment——注意每一处细节对应的知识点：

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: orders
  labels: {app: orders}
spec:
  replicas: 3
  strategy:
    rollingUpdate: {maxSurge: 1, maxUnavailable: 0}   # 第 7 章：无损滚动
  selector:
    matchLabels: {app: orders}
  template:
    metadata:
      labels: {app: orders, version: v1}    # version 标签：为灰度做准备
    spec:
      serviceAccountName: orders-sa          # 第 16 章：专用 SA
      automountServiceAccountToken: false
      securityContext:                        # 第 17 章：安全基线
        runAsNonRoot: true
      containers:
      - name: orders
        image: registry.example.com/shop/orders:v1.4.2   # 明确 tag
        ports: [{containerPort: 8080}]
        env:
        - name: DB_HOST
          value: orders-db-pg-rw              # Operator 提供的 Service 名
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef: {name: orders-db-cred, key: password}  # 第 15 章
        resources:
          requests: {cpu: 250m, memory: 512Mi}
          limits:   {cpu: "1",  memory: 1Gi}   # 第 18 章：QoS/Burstable
        startupProbe:                           # 第 4 章：慢启动保护
          httpGet: {path: /healthz, port: 8080}
          failureThreshold: 30
          periodSeconds: 2
        livenessProbe:
          httpGet: {path: /healthz, port: 8080}
        readinessProbe:
          httpGet: {path: /ready, port: 8080}  # /ready 检查 DB 连接
          periodSeconds: 5
        lifecycle:
          preStop:
            exec: {command: ["sh","-c","sleep 5"]}   # 第 4 章：终止竞态
        securityContext:                        # 第 17 章
          allowPrivilegeEscalation: false
          readOnlyRootFilesystem: true
          capabilities: {drop: ["ALL"]}
      terminationGracePeriodSeconds: 45
```

Service、HPA、PDB、Ingress:

```
# service.yaml（第 10 章）
apiVersion: v1
kind: Service
metadata: {name: orders}
spec:
  selector: {app: orders}
  ports: [{port: 80, targetPort: 8080}]
---
# hpa.yaml（第 19 章）
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: {name: orders}
spec:
  scaleTargetRef: {apiVersion: apps/v1, kind: Deployment, name: orders}
  minReplicas: 3
  maxReplicas: 12
  metrics:
  - type: Resource
    resource: {name: cpu, target: {type: Utilization, averageUtilization: 65}}
---
# pdb.yaml（第 19、22 章）
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: {name: orders}
spec:
  minAvailable: 2
  selector: {matchLabels: {app: orders}}
---
# ingress.yaml（第 11 章）
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt    # 第 17 章生态：自动证书
spec:
  ingressClassName: nginx
  tls: [{hosts: [shop.example.com], secretName: shop-tls}]
  rules:
  - host: shop.example.com
    http:
      paths:
      - path: /api/orders
        pathType: Prefix
        backend: {service: {name: orders, port: {number: 80}}}
      - path: /
        pathType: Prefix
        backend: {service: {name: frontend, port: {number: 80}}}
```

数据库用 Operator（第 25 章），一条 YAML 获得完整运维能力：

```
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata: {name: orders-db}
spec:
  instances: 3                # 自动主从 + 故障切换
  storage: {size: 50Gi}
  monitoring:
    enablePodMonitor: true    # 自动接入 Prometheus
```

27.4 第三步：发布流水线（第 23、24、7 章）

CI/CD 采用 GitOps 模式：

```
代码提交 → CI 构建镜像 + 跑测试 + Trivy 扫描
       → CI 向配置仓库提交"更新 image tag"的 PR
       → 团队审阅合并 → Argo CD 自动同步到集群
       → 金丝雀：Argo Rollouts 按 10%→30%→100% 放量
```

Argo Rollouts 把第 6 章的 Deployment 换成 Rollout 资源（其余清单不变），发布策略声明式化：

```
spec:
  strategy:
    canary:
      steps:
      - setWeight: 10
      - pause: {duration: 5m}
      - analysis: {templates: [{templateName: orders-success-rate}]}
      - setWeight: 50
      - pause: {duration: 5m}
```

指标不达标（第 20 章的 PromQL 分析模板判定错误率超标）→ 自动回滚。**发布这件事，从"人肉操作"变成了"声明 + 自动化"**。

27.5 第四步：上线后的守护（第 20、21、19 章）

上线不是终点。守护清单：

- **看板**：Grafana 上每个服务的 RED 面板；Kiali 看服务拓扑；
- **告警**：错误率、P99 延迟、HPA 顶到 maxReplicas、Pod 重启频次——全部"按症状"配置；
- **日志**：Loki/ES 里能按 `service=orders` + 时间窗过滤；trace_id 贯穿四层调用；
- **弹性验证**：压测触发 HPA 扩容；观察 metrics-server 与行为参数是否符合预期；

- **故障演练**：随机杀 orders Pod（自愈验证）；drain 一个节点（PDB 生效验证）；断 DB 网络 30 秒（熔断与降级验证）。

27.6 复盘：这本书教给你的思维

走完全程，回看这一章的每个 YAML——没有一个字段是凭空出现的，每一个都对应前面某一章的"为什么"。这就是全书的最终目的：让你掌握的不是一堆 YAML 片段，而是一套**云原生的思维方式**：

1. **声明式思维**：描述期望状态，让系统收敛（spec/status、控制循环、GitOps）；
2. **分层解耦思维**：每个抽象层解决一类问题，通过接口（CRI/CNI/CSI/API 组）组合——所以遇到问题先问"这在哪一层"；
3. **易逝与自愈思维**：一切皆可重建，状态外置，自愈交给控制器——不再"心疼任何一台机器"；
4. **最小权限与纵深防御思维**：每一层都假设上一层会失守；
5. **可观测与自动化思维**：看不见的东西管不了，重复的事情交给机器。

27.7 继续前行的路标

技术地图永远比一本书大。合上书之后，建议的进阶方向：

- **动手**：CKA（管理员）/ CKAD（开发者）认证是检验与系统化的好途径；
- **深入底层**：容器运行时（containerd 源码级）、eBPF（Cilium）、内核网络——本书第 12 章是入口；
- **平台工程**：Backstage（开发者门户）、Crossplane（基础设施即代码）、多集群治理；
- **读源文档**：kubernetes.io 的 Concepts 与 Tasks 是最好的"进阶读物"，此时你已具备读懂它们的全部前置知识。

云计算原生的浪潮还在继续，但万变不离其宗——**容器、声明式 API、控制循环、声明的基础设施**。这些内核你已经掌握。剩下的，是在真实世界的风浪里，把它们打磨成肌肉记忆。

祝你航行顺利，船长。

（全书完）

第28章 多集群与混合云：舰队协同作战

单个集群再高可用，也扛不住"整个机房失联"级别的故障，也解决不了"多地合规部署"的问题。当业务规模继续增长，多集群管理成为绕不开的课题。这一章讲多集群的动机、架构模式与协同技术。

28.1 为什么要多集群

一个集群搞不定的场景，才是引入多集群的正经理由：

故障域隔离：单集群的"爆炸半径"是整个集群——一次误操作（删 namespace）、一次失控的升级、一次 etcd 故障，都会波及所有业务。多集群把爆炸半径切小：一个集群出事，业务整体仍可服务。

地域与延迟：用户分布在多地时，就近接入（北集群服务北方、南集群服务南方）能显著降低延迟。跨地域的单集群不可行——etcd 的 Raft 协议对跨数据中心延迟极其敏感。

合规与数据主权：数据不能出境、不同行业的数据隔离要求，物理上要求不同区域/不同集群。

环境隔离：dev/staging/prod 各自独立集群，彻底避免"测试打挂生产"。

升级节奏错开：先升级小集群（金丝雀集群），验证一周再升级生产集群——把升级风险从"一次性赌博"变成"渐进验证"。

对应的反模式也要点破：**用多集群解决"单集群资源不够"通常是伪命题**——单集群万级节点是完全可行的，缺的往往是容量规划与资源治理，不是集群数量。

28.2 多集群的四种架构模式

模式一：独立舰队（每集群自治）。最简单的形态：N 个互不相干的集群。管理成本随集群数线性增长（每个都要升级、备份、监控），一致性靠纪律，容易漂移。适合集群数量少（2～3 个）的阶段。

模式二：GitOps 统管（推荐基线）。所有集群由同一个 Git 仓库（Argo CD/Flux 多实例或多集群接入）管理，集群差异化通过 overlays 表达。配置一致性有机制保证，"新集群 10 分钟铺设完毕"（App of Apps）。这是多集群管理的事实基线，第 24 章的仓库结构（platform/apps/clusters 三层）就是为此设计的。

模式三：服务互访（多集群网络）。集群间的服务可以互相调用，实现跨集群容灾与调度。实现路径：Submariner、Cilium ClusterMesh（打通集群间 Pod 网络）、或 Istio 多集群（跨集群服务发现与流量治理）。代价：网络方案的复杂度显著上升，通常只在"需要跨集群容灾"的核心业务上启用。

模式四：流量级多活。全局流量入口（GSLB/云全局负载均衡）按健康度与地域把用户流量分发到多个独立完整的多活集群。每个集群都能独立承载 100% 流量——任何单集群故障，流量秒级切换。这是最高形态，成本也最高（全量冗余）。

28.3 混合云与云间迁移

混合云（部分在自建 IDC、部分在公有云）的常见形态：IDC 跑敏感数据与稳态业务，公有云承载弹性与突发。技术要点：

- **统一控制面**：Rancher、阿里云 ACK One、Anthos 等多集群管理平台，提供统一的 API、认证与可见性；
- **网络打通**：专线/VPN 连接 IDC 与云上 VPC，集群网络方案需支持跨网段路由；
- **镜像与依赖的可达性**：跨环境的镜像仓库同步、配置中心与 DNS 的互通——"集群外的腿"（第 22 章）在混合云下是双份的；
- **避免云锁定**：清单层使用标准 API（少用云专属注解），存储抽象走 CSI，基础设施定义走 Terraform/Crossplane。

云间迁移的务实路径：新集群（GitOps 铺设）→ 数据同步（数据库主从跨云）→ 流量按比例切（DNS 权重/网关分流）→ 观察期 → 旧集群退役。**用 GitOps 重建的集群与原集群行为一致**，是整个迁移可行性的前提。

28.4 多集群的核心挑战

一致性漂移：N 个集群 N 份配置，手动管理的漂移几乎必然发生。解法只有一个：GitOps 强约束（集群配置必须来自 Git，手动变更加锁）。

可观测性割裂：每个集群一套 Prometheus/Grafana，全局视角缺失。解法：指标联邦或 Thanos/Mimir 全局查询层，日志与追踪接入统一平台，告警聚合到统一值班体系。

安全边界扩大：集群间的信任关系（跨集群 token、证书）是新攻击面。原则：最小信任、短时凭证、mTLS 全覆盖。

成本与冗余的平衡：全量多活成本翻倍。务实做法：核心业务多活、普通业务主备、边缘业务单集群——**按业务等级分配冗余度**。

28.5 本章小结

- 多集群的正当理由：**故障域隔离、地域延迟、合规、环境隔离、升级错峰**；"资源不够"通常不是理由。
- 架构演进路径：独立自治 → **GitOps 统管（基线）** → 服务互访 → 流量级多活；按需逐级上难度。
- 混合云的关键：统一控制面、网络打通、跨环境依赖可达、用标准 API 降锁定。
- 多集群三大挑战——**漂移、观测割裂、信任面扩大**——分别用 GitOps 强约束、全局观测层、最小信任原则应对。

第29章 成本优化：把钱花在刀刃上

云原生的弹性是双刃剑：用得好省钱，用不好烧钱。FinOps（云财务运营）在 K8s 语境下的核心问题只有一个：**每一分钱对应哪个业务？怎么让它更少？** 这一章给出一条完整的成本优化路线。

29.1 看见成本：优化的前提

第一步：成本可见。 节点账单是"总账"，必须分摊到 namespace/团队/服务：

- **OpenCost**（CNCF 项目）：基于 requests 与实际用量，把节点成本分摊到 Pod → Deployment → Namespace，输出分摊报表；
- **云厂商的成本分析**：按标签（集群名、namespace 注入到节点标签）拆账单；
- 建立"**每团队每月成本看板**"——看不见的东西管不住，这是 FinOps 第一定律。

第二步：识别浪费的三大典型：

1. **request 虚高**：声明 2 核实际用 0.3 核，节点按声明分配——账单为虚高的声明买单（第 18 章 VPA 推荐模式的用武之地）；
2. **僵尸资源**：没人用的测试集群、忘删的 LoadBalancer（每个都是持续计费的）、绑定了没人用的 PV、闲置的未挂载云盘；
3. **非弹性调度**：固定为峰值容量买单（24 小时按高峰规模运行），而流量有明显波谷。

29.2 六个高性价比优化手段

手段一：校准 requests/limits。 基于 Prometheus 的历史 P95/P99 用量重设 requests（`quantile_over_time` 查询或 VPA 推荐值）。这一步通常能"免费"释放 20%～40% 的集群容量。

手段二：启用 HPA + CA。 让副本数与节点数跟随负载伸缩（第 19 章），把"峰值固定成本"变成"均值动态成本"。配合缩容行为调优（快扩慢缩），低谷时段的节省相当可观。

手段三：抢占式/竞价实例跑可中断负载。 批处理（Job/CronJob）、CI runner、无状态 worker 用 Spot 实例，价格通常低 60%～90%。要点：Pod 可容忍驱逐（无本地强状态）、配合多可用区与重新供给策略。**核心业务的关键服务不要用 Spot。**

手段四：按使用时段开关环境。 dev/staging 环境晚上与周末缩容到 0（Kube-downscaler 类工具按时间表自动缩放）——开发环境利用率往往只有每周 40 小时，其余时间全是浪费。

手段五：存储与流量治理。 未使用的 PV 定期回收；大盘改小卷（只增不减，所以**初始就要规划好**）；日志保留期与索引量治理（日志是可观测成本里的大头）；跨可用区/跨区域流量费 awareness（大流量场景把通信密集的服务放同区）。

手段六：平台层的规模化红利。 多团队共享集群（配额隔离，第 5 章）vs 每团队一个集群——共享显著提升利用率；中间件自建 vs 托管的成本对比要把**运维人力成本**算进去，不只是机器钱。

29.3 建立成本治理机制

技术手段见效快，机制才可持续：

1. **成本归因到人**：每个 namespace 有明确的 owner（标签强制，策略引擎校验），成本看板按 owner 聚合；
2. **预算与告警**：分摊成本超阈值自动通知 owner（OpenCost + 告警规则）；
3. **效率指标**：集群分配率（requests 占容量比）与利用率（实际用量占容量比）双指标监控——分配率高但利用率低 = request 虚高；两者都低 = 集群过大；
4. **季度成本评审**：像复盘事故一样复盘成本，删僵尸、调配置、验证优化项效果。

> **提示：** 成本优化有一条铁律：**先测量，再动手；每次只动一个变量**。盲目下调 requests 可能引发调度失败与 OOM，省下的钱抵不上一次事故。

29.4 本章小结

- FinOps 起点：**成本分摊可见**（OpenCost/账单标签）+ 识别三大浪费（request 虚高、僵尸资源、非弹性）。
- 六手段按性价比排序：**校准 requests → HPA/CA → Spot 实例 → 环境分时开关 → 存储流量治理 → 平台规模化**。
- 可持续的优化靠机制：成本归因到 owner、预算告警、双指标（分配率/利用率）、季度评审。
- 记住隐性成本：运维人力、事故损失、机会成本——最便宜的方案未必是总成本最低的。

第30章 平台工程：从集群到"自助平台"

全书的最后，我们把视野拉到最高层。当组织里有十个团队都在用 Kubernetes 时，一个新的问题出现了：**每个团队都要成为 K8s 专家吗？** 平台工程（Platform Engineering）的回答是：**不——由平台团队把 Kubernetes 封装成"自助服务平台"，让业务团队用简单的接口获得生产级能力**。

30.1 平台工程的定位

平台工程不是运维 2.0，也不是 SRE 的别名。它回答的是产品问题：**平台是一个内部产品，业务团队是它的用户**。

平台团队的产品通常包括：

- **应用脚手架**：新服务从模板一键创建（含 CI/CD、监控、告警、安全基线全配好）；
- **部署界面**：业务团队面对的是"改一个镜像 tag + 点发布"，而不是手写 Deployment YAML；
- **中间件目录**：数据库、缓存、消息队列按目录申请，自动开通（Operator/云 API 驱动）；
- **统一可观测**：接入即有 RED 看板、日志检索、追踪；
- **护栏（Guardrails）**：安全基线、配额、网络策略默认生效——**用默认值代替培训**。

> **比喻：** Kubernetes 是一整套专业的厨房设备——功能强大但需要专业厨师。平台工程开的是"连锁餐厅"：中央厨房把复杂度封装好，门店只需要按菜单出餐。菜单越简单，门店越多越稳。

30.2 关键技术支柱

Backstage（CNCF）：开发者门户的事实标准。软件目录（所有服务的一屏视图：负责人、仓库、部署状态、依赖）、技术文档站、Scaffolder 模板（"创建新服务"向导）。它把"平台"从一堆工具变成一个统一的门户入口。

Crossplane：基础设施的声明式管理——把云资源（数据库实例、Bucket、消息队列）做成 K8s 自定义资源（第 25 章的 CRD 思想延伸到基础设施层）。业务团队 `apply` 一个 `PostgreSQL` 对象，平台层自动开通云数据库并注入连接信息——**内部平台的"目录申请"由此实现**。

策略即代码：Kyverno/OPA 把"必须贴 owner 标签""必须有资源限制""镜像必须来自公司仓库"写成自动校验的策略——平台的护栏不依赖人的自觉。

Golden Path（黄金路径）：平台工程的核心理念——为 90% 的常见场景提供预制好的最佳实践路径（脚手架 + 模板 + 流水线），路径外的东西允许但不支持。**默认安全、默认合规、默认可观测**，偏离路径才需要审批。

30.3 平台工程的避坑指南

坑一：平台没人用。 平台团队闭门造车，做出来的门户难用、模板僵化，业务团队绕道直连 K8s。解法：把业务团队当客户——需求调研、体验打磨、采纳率（adoption）作为平台团队的核心指标。

坑二：过度封装。 把所有东西都藏起来，业务团队失去逃生舱（escape hatch），特殊场景寸步难行。解法：分层封装——默认走黄金路径，能力阶梯暴露（简单 → 高级 → 原始 YAML），底层永远可达。

坑三：平台团队变成运维瓶颈。 所有变更都找平台团队，平台成了新的单点。解法：自助化优先——能自动开通的不走工单，能声明式管理的不走人工。

坑四：为平台而平台。 团队只有 5 个人、服务只有 10 个，先建门户是浪费。平台工程的启动信号：**同质的重复劳动明显增多**（每个团队都在配监控、写流水线、踩相同的坑）——这个信号出现之前，用模板仓库 + 文档就够了。

30.4 全书终章：你的云原生地图

三十年前，部署一个应用需要机房、服务器、系统管理员；十年前，需要虚拟机、负载均衡、自动化脚本；今天，需要的是一张 YAML 声明，和背后整套自动化的基础设施。而明天，随着 AI 驱动的运维（AIOps）、声明式基础设施的成熟，部署这件事会越来越像"许愿"——但支撑"许愿"的，永远是本书讲过的那些原理：

- **容器与镜像**：一次构建，处处运行（第 1～2 章）；
- **控制循环**：声明期望，系统收敛（第 5、6、25 章）；
- **分层解耦**：CRI/CNI/CSI 与 API 组的接口哲学（第 3、12、14 章）；
- **易逝与自愈**：一切皆可重建（第 4、6、22 章）；
- **最小权限与纵深防御**（第 16、17 章）；
- **可观测先行**（第 20、21 章）。

工具会更替，内核长存。愿你带着这张地图，在云原生的海洋里，航行得更远。

（全书完）

第31章 动手实验一：从零搭建本地集群

这是动手实验篇的第一章。全篇的实验都可以在一台普通电脑上完成，无需任何云资源。本章先搭好"地基"：一个本地 Kubernetes 集群 + 一套趁手的工具链。后面的每一章实验都建立在这个地基之上。

31.1 实验目标

完成本章后，你将拥有：

1. 一个可用的本地 Kubernetes 集群（用 Kind）；
2. 正确配置的 kubectl 与命令补全；
3. 两个极其实用的排障工具：`kubectl-ctx` / `kubectl-ns`（可选）与一个网络调试镜像；
4. 对集群组件的一手认识（亲手看看控制平面与工作节点长什么样）。

31.2 环境要求

- Docker 或 Podman（Kind 用它来模拟节点）；
- 至少 4GB 可用内存、2 核 CPU；
- 稳定的网络（需要拉取镜像）。

31.3 安装 Kind 与 kubectl

macOS（Homebrew）：

```
brew install kind kubectl
```

Linux：

```
# kubectl
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

# kind
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.24.0/kind-linux-amd64
chmod +x ./kind
sudo mv ./kind /usr/local/bin/kind
```

Windows：使用 `winget install Kubernetes.kubectl` 与 `winget install Kubernetes.kind`，或下载二进制放到 PATH。

验证安装：

```
$ kubectl version --client
Client Version: v1.29.4
$ kind version
kind v0.24.0 go1.22.2 linux/amd64
```

> **提示：** 所有实验基于 kubectl 1.29 与 Kind 0.24 编写，相近版本均可。云上托管集群（ACK/EKS/TKE/GKE）的命令完全一致，只是创建集群部分换成控制台或云 CLI。

31.4 创建你的第一个集群

Kind 用 YAML 描述集群。先写一个配置：

```
# kind-config.yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane           # 控制平面节点
- role: worker                  # 两个工作节点
- role: worker
```

创建并验证：

```
$ kind create cluster --name k8s-lab --config kind-config.yaml
Creating cluster "k8s-lab" ...
 ✓ Ensuring node image (kindest/node:v1.29.4) 🖼
 ✓ Preparing nodes 📦 📦 📦
 ✓ Writing configuration 📜
 ✓ Starting control-plane 🕹️
 ✓ Installing CNI 🔌
 ✓ Installing StorageClass 💾
Set kubectl context to "kind-k8s-lab"

$ kubectl get nodes -o wide
NAME                    STATUS   ROLES           AGE   VERSION
k8s-lab-control-plane   Ready    control-plane   1m    v1.29.4
k8s-lab-worker          Ready    <none>          1m    v1.29.4
k8s-lab-worker2         Ready    <none>          1m    v1.29.4
```

三个节点——一个控制平面、两个工作节点，和本书第 3 章的"航母"结构完全对应。

31.5 看看集群内部组件

控制平面组件都跑在 `kube-system` 命名空间：

```
$ kubectl get pods -n kube-system
NAME                                          READY   STATUS    RESTARTS   AGE
coredns-76f75df574-4z9m2                      1/1     Running   0          2m
coredns-76f75df574-bxq7h                      1/1     Running   0          2m
etcd-k8s-lab-control-plane                    1/1     Running   0          2m
kube-apiserver-k8s-lab-control-plane          1/1     Running   0          2m
kube-controller-manager-k8s-lab-control-plane 1/1     Running   0          2m
kube-proxy-...                                1/1     Running   0          2m
kube-scheduler-k8s-lab-control-plane          1/1     Running   0          2m
```

对照第 3 章，逐个认出它们：**etcd、apiserver、controller-manager、scheduler** 是控制平面四大件，**kube-proxy** 每个节点都有，**coredns** 是集群 DNS。Kind 的 CNI 用的是内置的 kindnet。

> **深入：** 这些"控制平面 Pod"其实是以静态 Pod（Static Pod）方式运行的——kubelet 直接读取 `/etc/kubernetes/manifests` 目录下的 YAML 拉起它们，而非经过调度器。可以用 `docker exec k8s-lab-control-plane ls /etc/kubernetes/manifests` 亲眼看看（Kind 环境下）。

31.6 配置命令补全（强烈推荐）

kubectl 的子命令和资源名极多，命令补全能救命：

```
# bash
echo 'source <(kubectl completion bash)' >> ~/.bashrc
source ~/.bashrc

# zsh
echo 'source <(kubectl completion zsh)' >> ~/.zshrc
source ~/.zshrc

# fish
kubectl completion fish > ~/.config/fish/completions/kubectl.fish
```

再加两个高频别名：

```
alias k='kubectl'
alias kg='kubectl get'
alias kd='kubectl describe'
alias kl='kubectl logs'
```

从现在起，书中的 `kubectl` 都可以用 `k` 代替。

31.7 准备两个"百宝箱"工具

工具一：网络调试镜像。第 12、21 章反复出现的 `netshoot`，含 tcpdump、dig、curl、iperf、mtr 全套网络工具。先拉下来备着：

```
docker pull nicolaka/netshoot
```

需要时一键起一个调试 Pod：

```
kubectl run net-debug --rm -it --image=nicolaka/netshoot -- bash
```

工具二：切换上下文与命名空间。多集群/多命名空间时，原生的 `kubectl config use-context` 与 `-n` 会显得啰嗦。装 `kubectx` 与 `kubens`：

```
brew install kubectx          # macOS
sudo apt install kubectx      # Ubuntu
```

```
$ kubectx                     # 列出并切换集群上下文
kind-k8s-lab
$ kubens kube-system          # 切换默认命名空间
Context "kind-k8s-lab" modified.
Active namespace is "kube-system".
```

31.8 部署一个"看板"：验证集群健康

最后部署一个最小应用，验证整条链路通：

```
kubectl create deployment hello --image=nginx:1.25 --replicas=2
kubectl expose deployment hello --port=80 --target-port=80
kubectl run curl-test --rm -it --image=curlimages/curl -- \
  sh -c 'curl -s http://hello | head -c 100'
```

看到 nginx 的欢迎页 HTML，说明"调度 → 网络 → Service → DNS"整条链路都是通的。

清理这个临时应用（保持实验环境干净）：

```
kubectl delete deployment hello
kubectl delete service hello
```

31.9 实验总结与思考

- 你的集群有几个节点？`kubectl get nodes` 的 ROLES 列分别是什么？
- 控制平面组件为什么以 Pod 形式出现？它们和普通 Pod 的运行方式有何不同？（提示：Static Pod）
- 尝试 `kubectl get pods -A`，集群启动后自带了多少系统 Pod？它们分别属于哪些命名空间？

> **提示：** 后面所有实验都假设集群名为 `k8s-lab`、默认命名空间为 `default`。若集群被休眠或重启，用 `kind get clusters` 查看、`docker start` 相关容器即可恢复。玩坏了想重来？`kind delete cluster --name k8s-lab` 然后重新 create，30 秒满血复活——这正是 Kind 作为"实验沙盒"的最大价值。

第32章 动手实验二：从代码到 Pod 的完整旅程

这一章走完"应用容器化"的完整链路：写一个真实的小应用 → 写 Dockerfile 构建镜像 → 写 Pod 清单 → 部署到集群 → 通过探针与调试容器验证。这是理解"容器与镜像"（第 1～2 章）与"Pod"（第 4 章）最扎实的方式。

32.1 实验目标

1. 亲手构建一个多阶段镜像，理解"小镜像"的价值；
2. 编写并运行单容器、多容器两种 Pod；
3. 亲手验证探针（liveness/readiness/startup）的行为差异；
4. 学会用 `kubectl exec`、`port-forward`、`logs` 三个调试命令。

32.2 写一个最小 HTTP 应用

用 Go 写一个带健康检查端点的 HTTP 服务（如果你不熟 Go，用任意语言的等价实现即可，逻辑都一样）：

```
// main.go
package main

import (
    "fmt"
    "log"
    "net/http"
    "os"
    "time"
)

var startTime = time.Now()

func main() {
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintf(w, "Hello from %s! uptime=%s\n",
            os.Getenv("POD_NAME"), time.Since(startTime).Round(time.Second))
    })
    // /healthz 回答"我还活着吗"——永远健康（liveness 用）
    http.HandleFunc("/healthz", func(w http.ResponseWriter, r *http.Request) {
        w.WriteHeader(200)
        w.Write([]byte("ok"))
    })
    // /ready 回答"我现在能服务吗"——启动后 10 秒内返回 503（readiness 用）
    http.HandleFunc("/ready", func(w http.ResponseWriter, r *http.Request) {
        if time.Since(startTime) < 10*time.Second {
            w.WriteHeader(503)
            w.Write([]byte("warming up"))
            return
        }
        w.WriteHeader(200)
        w.Write([]byte("ready"))
    })
    log.Println("listening on :8080")
    log.Fatal(http.ListenAndServe(":8080", nil))
}
```

这个应用故意让 `/ready` 在启动后 10 秒内返回 503，用于演示就绪探针的作用——**新 Pod 在"热身"期间不会被分配流量**。

32.3 写一个多阶段 Dockerfile

```
# Dockerfile
# 阶段一：编译
FROM golang:1.22 AS builder
WORKDIR /src
COPY main.go .
RUN CGO_ENABLED=0 go build -o /app main.go

# 阶段二：运行（用极小的基础镜像）
FROM gcr.io/distroless/static-debian12
COPY --from=builder /app /app
USER 65532:65532                    # 非 root 运行（第 17 章）
EXPOSE 8080
ENTRYPOINT ["/app"]
```

多阶段构建的精髓：编译环境（几百 MB 的 golang 镜像）和运行环境（几十 MB 的 distroless）分离，最终镜像又小又安全。

构建并确认体积：

```
docker build -t myapp:v1 .
docker images | grep myapp
# myapp  v1   xxxxx   约 10MB（对比：若直接用 golang 镜像会超过 800MB）
```

32.4 把镜像交给 Kind 集群

Kind 的节点是 Docker 容器，本地镜像默认不会被它们"看到"。三种办法：

```
# 办法一（最简单）：直接加载到集群
kind load docker-image myapp:v1 --name k8s-lab

# 办法二：构建时直接用 kind 的节点作为目标（构建到集群里）
# 办法三：推送到一个集群可访问的镜像仓库（生产场景的标准做法）
```

生产环境里，镜像是推送到中央仓库（Harbor 等）再被各节点拉取的——本地实验用 `kind load` 省去这个环节。

32.5 运行单容器 Pod

```
# pod-basic.yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
spec:
  containers:
  - name: myapp
    image: myapp:v1
    env:
    - name: POD_NAME
      valueFrom:
        fieldRef:
          fieldPath: metadata.name     # 注入 Pod 名（downwardAPI）
    ports:
    - containerPort: 8080
    resources:
      requests: {cpu: 50m, memory: 64Mi}
      limits:   {cpu: 200m, memory: 128Mi}
    readinessProbe:
      httpGet: {path: /ready, port: 8080}
      periodSeconds: 2
    livenessProbe:
      httpGet: {path: /healthz, port: 8080}
      periodSeconds: 5
```

创建并观察（注意 READY 列在启动后 10 秒才变 1/1）：

```
$ kubectl apply -f pod-basic.yaml
pod/myapp-pod created

$ kubectl get pod myapp-pod -w
NAME        READY   STATUS    RESTARTS   AGE
myapp-pod   0/1     Running   0          2s      # 容器已运行，但就绪探针还没通过
myapp-pod   1/1     Running   0          11s     # 10 秒热身后就绪
```

验证应用：

```
# 端口转发到本地
kubectl port-forward pod/myapp-pod 8080:8080

# 另开终端
curl localhost:8080/
# Hello from myapp-pod! uptime=45s
curl localhost:8080/ready
# ready
```

32.6 亲手验证探针行为

实验 A：让就绪探针失败。 临时把 `/ready` 改成永远 503？不必改代码——用 `kubectl exec` 看探针日志，或做一个更直观的实验：故意让 liveness 失败。编辑 Pod 把 liveness 的路径改成 `/notexist`：

```
kubectl patch pod myapp-pod --type=json \
  -p='[{"op":"replace","path":"/spec/containers/0/livenessProbe/httpGet/path","value":"/notexist"}]'
```

> **注意：** 直接 patch Pod 会因"与模板不一致"被拒绝——Pod 的 spec 大多字段不可变。这个实验的正确做法是用 `kubectl edit` 观察会失败，改用 Deployment（下一章）来演示探针重启。这里我们先记住结论：**探针配置要改在控制器模板里，而不是改运行中的 Pod。**

实验 B：观察就绪探针的摘除效果（配合 Service）。 创建 Service 指向这个 Pod：

```
kubectl expose pod myapp-pod --port=80 --target-port=8080 --name=myapp-svc
kubectl get endpoints myapp-svc
# NAME       ENDPOINTS
# myapp-svc  10.244.x.x:8080      ← 只有一个就绪后端
```

现在再起一个永远不就绪的"坏 Pod"（镜像用 `myapp:v1`，但 readiness 路径写错）：

```
# pod-bad.yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-bad
  labels: {app: myapp}
spec:
  containers:
  - name: myapp
    image: myapp:v1
    readinessProbe:
      httpGet: {path: /this-path-does-not-exist, port: 8080}
      periodSeconds: 2
```

```
kubectl apply -f pod-bad.yaml
kubectl get endpoints myapp-svc      # 端点没有变化！坏 Pod 未被加入
kubectl get pod myapp-bad            # 0/1 Running，一直不就绪
```

结论一目了然：Service 只把流量给"就绪"的后端——第 10 章的理论在此得到验证。坏 Pod 虽然 Running，但永远接不到流量。

32.7 多容器 Pod：Sidecar 实战

把第 4 章的 sidecar 模式跑起来：主容器写日志到共享卷，sidecar 读取并"上传"（这里简化为打印）。

```
# pod-sidecar.yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-with-logger
spec:
  containers:
  - name: app                # 主容器：每分钟写一条日志
    image: busybox:1.36
    command: ["sh", "-c",
      "while true; do echo \"[app] $(date) hello world\" >> /var/log/app.log; sleep 5; done"]
    volumeMounts:
    - name: logdir
      mountPath: /var/log
  - name: log-forwarder      # 边车：读共享卷并"转发"（打印）
    image: busybox:1.36
    command: ["sh", "-c", "tail -f /var/log/app.log"]
    volumeMounts:
    - name: logdir
      mountPath: /var/log
      readOnly: true
  volumes:
  - name: logdir
    emptyDir: {}
```

```
kubectl apply -f pod-sidecar.yaml

# 分别看两个容器的日志
kubectl logs app-with-logger -c app
kubectl logs app-with-logger -c log-forwarder -f
```

你会发现 sidecar 实时打印主容器写入的日志——两个容器通过 emptyDir 共享文件，这正是 Pod 存在的意义（第 4、13 章）。

32.8 实验总结与思考

- 你的最终镜像有多大？多阶段构建省掉了什么？（提示：编译工具链、源代码、包管理器缓存）
- 坏 Pod 为什么 Running 却接不到流量？它和好 Pod 在 `kubectl get endpoints` 里有什么区别？
- sidecar 和主容器是"同时"启动的吗？如果要求 sidecar 先启动（比如先下载配置），应该用什么？（提示：initContainer）
- 试着 `kubectl delete pod myapp-pod` 再 `get`，它还会自动回来吗？为什么？（提示：裸 Pod 没有控制器）

第33章 动手实验三：Deployment 发布全生命周期

这是最核心的一个实验——把第 6、7 章的理论（Deployment、滚动更新、回滚、金丝雀）在真实集群里完整走一遍。你会发现，许多"纸上谈兵"的概念，亲手操作一次就再也忘不掉。

33.1 实验目标

1. 创建 Deployment 并理解它的三层结构（Deployment → ReplicaSet → Pod）；
2. 亲手验证"自愈"——删 Pod 自动补；
3. 完整走一遍滚动更新，观察新旧 ReplicaSet 的此消彼长；
4. 制造一次"发布事故"，再练习回滚；
5. 用金丝雀 Deployment 做一次灰度发布。

33.2 准备工作

沿用第 32 章构建的 `myapp:v1`。我们再构建一个 v2 版本（改一下返回内容，便于肉眼分辨版本）：

```
// 把 main.go 里输出的 "Hello" 改为 "Hello v2"
fmt.Fprintf(w, "Hello v2 from %s! ...")
```

```
docker build -t myapp:v2 .
kind load docker-image myapp:v2 --name k8s-lab
kind load docker-image myapp:v1 --name k8s-lab   # 两个版本都要在集群里
```

33.3 创建 Deployment 并理解三层结构

```
# deploy.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  selector:
    matchLabels: {app: web}
  strategy:
    type: RollingUpdate
    rollingUpdate: {maxSurge: 1, maxUnavailable: 0}
  template:
    metadata:
      labels: {app: web}
    spec:
      containers:
      - name: web
        image: myapp:v1
        ports: [{containerPort: 8080}]
        readinessProbe:
          httpGet: {path: /ready, port: 8080}
          periodSeconds: 2
```

```
kubectl apply -f deploy.yaml

# 一次看全三层
kubectl get deploy,rs,pod -l app=web
NAME                    READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/web     3/3     3            3           20s

NAME                               DESIRED   CURRENT   READY   AGE
replicaset.apps/web-5d78b9c6f4     3         3         3       20s

NAME                           READY   STATUS    AGE
pod/web-5d78b9c6f4-7xk2p       1/1     Running   20s
pod/web-5d78b9c6f4-9mq4r       1/1     Running   20s
pod/web-5d78b9c6f4-z8t1v       1/1     Running   20s
```

三个 Pod 的名字中间都带 `5d78b9c6f4`——这是当前模板的哈希。记住它，待会儿它会被新哈希取代。

33.4 验证自愈

```
$ kubectl delete pod web-5d78b9c6f4-7xk2p
pod "web-5d78b9c6f4-7xk2p" deleted

$ kubectl get pods -l app=web
NAME                      READY   STATUS    AGE
web-5d78b9c6f4-9mq4r      1/1     Running   3m
web-5d78b9c6f4-z8t1v      1/1     Running   3m
web-5d78b9c6f4-fk9wd      1/1     Running   2s   # 秒级补位
```

旧 Pod 被删除后，ReplicaSet 立刻新建了一个补位 Pod（新名字、新后缀）。**注意：没有任何人"重启"旧 Pod，是一个全新的 Pod 被创建了**——这就是第 6 章讲的"易逝 + 自愈"。

33.5 滚动更新：一帧一帧看

现在把镜像升级到 v2。**开两个终端**，一个执行更新，一个持续观察：

```
# 终端一：触发更新
kubectl set image deployment/web web=myapp:v2
# 或修改 YAML 后 apply（推荐，保持文件即真相）

# 终端二：实时观察
kubectl get pods -l app=web -w
```

你会看到精确的节奏（因为配置了 maxSurge=1, maxUnavailable=0）：

```
web-5d78b9c6f4-9mq4r   1/1   Running       # 3 个旧 Pod
web-5d78b9c6f4-z8t1v   1/1   Running
web-5d78b9c6f4-fk9wd   1/1   Running
web-8f4c9d2b1e-abcde   0/1   Pending       # 第 1 个新 Pod 创建（surge）
web-8f4c9d2b1e-abcde   0/1   Running       # 起来但未就绪（热身 10 秒）
web-8f4c9d2b1e-abcde   1/1   Running       # 就绪了！
web-5d78b9c6f4-9mq4r   1/1   Terminating   # 才杀掉 1 个旧 Pod
web-8f4c9d2b1e-defgh   0/1   Pending       # 继续第 2 个
...（循环）
```

观察两个 ReplicaSet 的世代更替：

```
$ kubectl get rs -l app=web
NAME              DESIRED   CURRENT   READY   AGE
web-8f4c9d2b1e    3         3         3       1m    # 新版
web-5d78b9c6f4    0         0         0       15m   # 旧版，缩到 0 但对象保留
```

要点提炼：滚动更新的本质是"新 Pod 就绪一个，旧 Pod 才下线一个"；旧 ReplicaSet 缩到 0 但**保留对象**，这正是回滚能秒级完成的原因。

33.6 制造事故 + 回滚

发布 v2 后，假设我们发现 v2 有严重 bug。先看发布历史：

```
$ kubectl rollout history deployment/web
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
```

CHANGE-CAUSE 是空的——因为没配注解。先补上（第 7 章的教训），顺便回滚：

```
# 回滚到上一版（revision 1）
kubectl rollout undo deployment/web

# 观察：v1 重新滚动铺开（同样平滑，不是瞬间全变）
kubectl get pods -l app=web -w

# 确认回到 v1
kubectl get rs -l app=web
# web-5d78b9c6f4 又变成 3 个，web-8f4c9d2b1e 缩到 0
```

验证回滚结果（端口转发确认内容回到 v1）：

```
kubectl port-forward deployment/web 8080:8080
curl localhost:8080/     # 输出 "Hello from ..."（没有 v2）
```

33.6.1 让历史可读：change-cause 注解

给模板加注解（它本身也会触发一次滚动，因为模板变了）：

```
spec:
  template:
    metadata:
      annotations:
        kubernetes.io/change-cause: "upgrade to v2"
```

```
kubectl annotate deployment/web kubernetes.io/change-cause="initial v1" --overwrite
kubectl rollout history deployment/web
# REVISION  CHANGE-CAUSE
# 1         <none>
# 3         initial v1
```

> **提示：** 生产里 change-cause 由 CI 自动写入（每次发布带上版本号/提交号），回滚时一眼定位目标版本。

33.7 金丝雀发布：灰度 10%

现在用"独立 canary Deployment + 共享 Service"实现灰度（第 7 章的穷人版金丝雀）：

第一步：创建 Service，selector 只匹配 `app: web`（两个 Deployment 的 Pod 都带这个标签）：

```
# svc.yaml
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  selector: {app: web}
  ports: [{port: 80, targetPort: 8080}]
```

第二步：主版本（v1）跑 9 个副本，金丝雀（v2）跑 1 个：

```
# canary.yaml（独立 Deployment，镜像 v2，仅 1 副本）
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-canary
spec:
  replicas: 1
  selector:
    matchLabels: {app: web, version: canary}
  template:
    metadata:
      labels: {app: web, version: canary}   # 同时带 app:web 和 version 标签
    spec:
      containers:
      - name: web
        image: myapp:v2
        readinessProbe:
          httpGet: {path: /ready, port: 8080}
```

注意主 Deployment 的 Pod 模板也要加 `version: stable` 标签，这样两个 Deployment 的 selector 才能各认各的 Pod（主 Deployment 的 selector 改为 `app: web, version: stable`）。

```
kubectl apply -f svc.yaml -f canary.yaml
kubectl scale deployment/web --replicas=9

# 此时 Service 的后端 = 9 个 v1 + 1 个 v2 ≈ 10% 流量走新版
kubectl get endpoints web
```

第三步：反复请求，统计版本比例（验证约 10% 命中 v2）：

```
kubectl run curl-test --rm -it --image=curlimages/curl -- \
  sh -c 'for i in $(seq 1 50); do curl -s http://web | grep -o "v2" || echo v1; done' | sort | uniq -c
# 约 45 次 v1，5 次 v2（比例接近 9:1，因负载均衡有波动）
```

第四步：灰度验证通过后"全量"——把主 Deployment 升级到 v2，删掉 canary：

```
kubectl set image deployment/web web=myapp:v2
kubectl delete deployment web-canary
```

灰度发布完成。这就是第 7 章金丝雀策略的最小可用实现，零额外工具。

33.8 实验总结与思考

- Deployment 三层结构中，删除 Pod 后补位的是"新 Pod"，你能从名字看出新旧吗?
- 滚动更新时为什么是"先多一个（surge）再少一个"? 如果 maxUnavailable=1 会是什么节奏?
- 回滚为什么这么快? 它回滚的到底是什么? （提示：旧 ReplicaSet 的模板）
- 金丝雀的比例是怎么控制出来的? 它精确吗? 要精确 5% 该怎么做? （提示：副本数、Ingress canary 注解、Argo Rollouts）
- 如果金丝雀的 Service selector 写错（比如漏了 version 标签），会发生什么?

第34章 动手实验四：Service 与网络排障实战

这一章把第 10、12 章的网络知识变成"手上功夫"。我们会部署一个多副本服务，用 Service 暴露它，然后用一套固定的排查命令，把"服务发现 → 负载均衡 → 流量路径"看得清清楚楚。最后故意制造两个经典网络故障，练习定位。

34.1 实验目标

1. 亲手创建 ClusterIP / NodePort / headless 三种 Service；
2. 从 Pod 内部验证 DNS 解析与负载均衡；
3. 观察 Service 的端点（EndpointSlice）如何随 Pod 就绪状态变化；
4. 练习用固定套路定位两类网络故障。

34.2 部署后端并暴露 Service

用一个能"自报身份"的镜像（返回 Pod 名字，方便观察负载均衡）。用第 32 章的 `myapp` 即可——它返回 `POD_NAME`：

```
kubectl create deployment backend --image=myapp:v1 --replicas=3
kubectl expose deployment backend --port=80 --target-port=8080
```

```
$ kubectl get svc backend
NAME      TYPE        CLUSTER-IP      PORT(S)   AGE
backend   ClusterIP   10.96.142.7    80/TCP    10s
```

34.3 从 Pod 内部验证服务发现与负载均衡

起一个调试 Pod：

```
kubectl run net-debug --rm -it --image=nicolaka/netshoot -- bash
```

在 Pod 内依次执行：

```
# 1. DNS 解析：短名自动补全
nslookup backend
# Server:   10.96.0.10 (CoreDNS)
# Name:     backend.default.svc.cluster.local
# Address:  10.96.142.7

# 2. 完整域名也可解析
nslookup backend.default.svc.cluster.local

# 3. 反复请求，观察负载均衡到不同后端
for i in $(seq 1 9); do curl -s http://backend | grep -o 'backend-[a-z0-9-]*'; done
# backend-5d78b9c6f4-7xk2p
# backend-5d78b9c6f4-9mq4r
# backend-5d78b9c6f4-z8t1v
# ...（三个 Pod 轮流出现）
```

三个 Pod 轮流命中——Service 在三个后端之间做了负载均衡（默认 iptables 概率随机）。

34.4 观察端点随就绪状态变化

```
# 在调试 Pod 外开新终端
kubectl get endpointslices -l kubernetes.io/service-name=backend
# NAME            ADDRESSES      PORTS
# backend-7xk2p   10.244.1.2,10.244.1.3,10.244.2.2   8080
```

现在把其中一个 Pod 的 readinessProbe 干掉（或直接删 Pod 看端点短暂收缩），观察端点变化：

```
kubectl delete pod backend-5d78b9c6f4-7xk2p
# 立刻看端点：旧 IP 消失，新 Pod 就绪后新 IP 加入
kubectl get endpointslices -l kubernetes.io/service-name=backend -w
```

34.5 三种 Service 类型实测

ClusterIP（已有）：只能在集群内访问。

NodePort：

```
kubectl expose deployment backend --name=backend-np --type=NodePort \
  --port=80 --target-port=8080
kubectl get svc backend-np
# TYPE       PORT(S)         →  80:30080/TCP（自动分配的 nodePort）
```

在**集群外**（你的宿主机）访问任意节点的 30080 端口。Kind 环境需要先做端口映射，或者用 `kubectl port-forward` 模拟；真实集群直接 `curl http://<任意节点IP>:30080` 即可。

headless Service：

```
kubectl expose deployment backend --name=backend-headless --cluster-ip=None
kubectl get svc backend-headless
# TYPE      CLUSTER-IP   →  None
```

回到调试 Pod：

```
nslookup backend-headless
# 直接返回所有 Pod IP（而不是一个虚拟 IP）：
# Name: backend-headless.default.svc.cluster.local
# Address: 10.244.1.3
# Address: 10.244.2.2
# Address: 10.244.1.4
```

对比 `nslookup backend` 返回单个 ClusterIP——**headless 与普通 Service 的核心区别一目了然**（第 8、10 章）。

34.6 故障演练一：Service 没有后端

这是新手最高频的坑——"Service 创建了，但访问报 503/connection refused"。

制造故障：把 Service 的 selector 改成一个不存在的标签：

```
kubectl patch svc backend -p '{"spec":{"selector":{"app":"no-such-app"}}}'
```

排查：

```
kubectl get endpoints backend
# NAME      ENDPOINTS   AGE
# backend   <none>      10m     ← 空端点！问题定位

kubectl describe svc backend
# Selector:  app=no-such-app（与实际 Pod 标签不符）
```

修复：

```
kubectl patch svc backend -p '{"spec":{"selector":{"app":"backend"}}}'
kubectl get endpoints backend      # 恢复
```

> **提示：** "访问不通"的第一检查永远是 `kubectl get endpoints`——端点为空说明 selector 没匹配到任何就绪 Pod，问题在标签不在网络。

34.7 故障演练二：Pod 起了但没就绪

制造故障：部署一个 readiness 探针永远失败的服务：

```
kubectl create deployment broken --image=myapp:v1
kubectl patch deployment broken --type=json -p='[{
  "op":"add","path":"/spec/template/spec/containers/0/readinessProbe",
  "value":{"httpGet":{"path":"/does-not-exist","port":8080},"periodSeconds":2}}]'
kubectl expose deployment broken --port=80 --target-port=8080
```

观察与排查：

```
kubectl get pods -l app=broken     # 0/1 Running（一直不就绪）
kubectl get endpoints broken       # <none>（没有就绪后端）
kubectl describe pod broken-xxx    # Events 里：Readiness probe failed: HTTP probe failed with
statuscode: 404
```

修复：把探针路径改回 `/ready`：

```
kubectl patch deployment broken --type=json -p='[{
  "op":"replace","path":"/spec/template/spec/containers/0/readinessProbe/httpGet/path",
  "value":"/ready"}]'
kubectl rollout status deployment broken     # 等就绪
kubectl get endpoints broken                 # 恢复
```

34.8 一套标准的网络排障口诀

把本章的排查步骤固化成口诀，贴在工位上：

```
访问不通 → 查 DNS（nslookup）
DNS 通 → 查端点（get endpoints，空则查 selector 与就绪探针）
端点有 → 查连通（curl ClusterIP → curl Pod IP 直连）
直连也不通 → 查网络策略与 CNI（NetworkPolicy / pod 所在节点）
```

层层递进，每一步都在缩小问题范围——第 21 章的方法论在本章落地为具体命令。

34.9 实验总结与思考

- ClusterIP 和 headless Service 的 `nslookup` 结果有什么本质区别？各自适合什么场景？
- 负载均衡是"轮询"还是"随机"？从你的连续请求中能判断吗？（提示：iptables 模式是概率随机）
- 端点为空的两种最常见原因分别是什么？如何一句话区分？
- NodePort 默认端口范围是多少？为什么生产不推荐用 NodePort 做入口？
- 试着删掉一个 Pod，在 `-w` 模式下观察端点从"3 个 → 2 个 → 3 个"的变化过程，这说明了什么？

第35章 动手实验五：Ingress 路由与 HTTPS 实战

上一章解决了集群内部与基础的对外暴露，这一章走通真正的"生产级入口"：部署 Ingress Controller，用域名和路径把流量分发到多个服务，最后配置 HTTPS 证书。对应第 11 章。

35.1 实验目标

1. 部署 Ingress NGINX Controller；
2. 配置基于域名和路径的路由规则；
3. 理解 Ingress 资源与 Controller 的分工（亲手验证"没有 Controller，Ingress 是废纸"）；
4. 配置自签名证书，走通 HTTPS 链路。

35.2 部署 Ingress Controller

Kind 环境一条命令搞定（生产环境用 Helm 安装或云厂商托管入口）：

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/
provider/kind/deploy.yaml
```

等待就绪：

```
kubectl get pods -n ingress-nginx -w
# ingress-nginx-controller-xxx   1/1   Running
kubectl get ingressclass
# NAME    CONTROLLER
# nginx   k8s.io/ingress-nginx   ← 默认 IngressClass 已就绪
```

35.3 准备两个后端服务

模拟 `api` 和 `web` 两个服务：

```
kubectl create deployment api --image=myapp:v1
kubectl expose deployment api --port=80 --target-port=8080 --name=api-svc

kubectl create deployment web --image=myapp:v1
kubectl expose deployment web --port=80 --target-port=8080 --name=web-svc
```

35.4 基于路径的路由

```
# ingress-path.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: path-ingress
spec:
  ingressClassName: nginx
  rules:
  - http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service: {name: api-svc, port: {number: 80}}
      - path: /
        pathType: Prefix
        backend:
          service: {name: web-svc, port: {number: 80}}
```

```
kubectl apply -f ingress-path.yaml
kubectl get ingress
# NAME          CLASS   HOSTS   ADDRESS   PORTS
# path-ingress  nginx   *       localhost 80
```

验证（Kind 的 ingress-nginx 监听 localhost:80）：

```
curl http://localhost/api   # 命中 api 服务
curl http://localhost/      # 命中 web 服务
```

> **提示：** 如果 `curl http://localhost` 没反应，多半是 Ingress Controller 还没就绪。`kubectl get pods -n ingress-nginx` 确认 Running，再试。

35.5 基于域名的路由

多个域名指向同一入口，按 Host 分发：

```
# ingress-host.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: host-ingress
spec:
  ingressClassName: nginx
  rules:
  - host: api.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend: {service: {name: api-svc, port: {number: 80}}}
  - host: web.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend: {service: {name: web-svc, port: {number: 80}}}
```

用 Host 头模拟不同域名访问（本地无需真实 DNS）：

```
curl -H "Host: api.example.com" http://localhost/   # 命中 api
curl -H "Host: web.example.com" http://localhost/   # 命中 web
```

35.6 亲手验证"没有 Controller，Ingress 是废纸"

这是第 11 章最重要的认知，值得亲眼看一次。先删除 Controller 的 Pod（或直接看一个指向不存在 Class 的 Ingress）：

```
# 创建一个指向不存在 Controller 的 Ingress
cat <<EOF | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: orphan-ingress
spec:
  ingressClassName: no-such-controller
  rules:
  - http:
      paths:
      - path: /
        pathType: Prefix
        backend: {service: {name: web-svc, port: {number: 80}}}
EOF

kubectl get ingress orphan-ingress
# 创建成功！但访问完全无效——因为没有任何 Controller 读取它
curl http://localhost/    # 依然走 path-ingress（无变化）
```

结论：Ingress 对象只是"规则"，没有 Controller 执行，它就是一条无人理会的声明。删掉它：

```
kubectl delete ingress orphan-ingress
```

35.7 配置 HTTPS（自签名证书）

生产环境用 cert-manager 自动签发，这里用自签名理解原理。

第一步：生成自签名证书：

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout tls.key -out tls.crt \
  -subj "/CN=web.example.com"
```

第二步：创建 Secret（类型必须是 kubernetes.io/tls）：

```
kubectl create secret tls web-tls --key tls.key --cert tls.crt
kubectl get secret web-tls
# NAME      TYPE                DATA   AGE
# web-tls   kubernetes.io/tls   2      5s
```

第三步：Ingress 引用：

```
# ingress-tls.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: tls-ingress
spec:
  ingressClassName: nginx
  tls:
  - hosts: [web.example.com]
    secretName: web-tls
  rules:
  - host: web.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend: {service: {name: web-svc, port: {number: 80}}}
```

```
kubectl apply -f ingress-tls.yaml

# HTTPS 访问（-k 跳过自签名校验）
curl -k https://web.example.com --resolve web.example.com:443:127.0.0.1
# 返回 web 服务的内容，TLS 生效
```

深入： 生产环境的正确姿势是 cert-manager——它自动向 Let's Encrypt 申请证书、存为 Secret、到期自动续期。Ingress 只需加一条注解 `cert-manager.io/cluster-issuer: letsencrypt`，其余全自动（第 17 章生态、第 27 章实战已提及）。

35.8 用注解做限流（可选进阶）

Ingress NGINX 的注解能力（第 11 章）：

```
metadata:
  annotations:
    nginx.ingress.kubernetes.io/limit-rps: "2"    # 每秒限 2 个请求
```

```
kubectl apply -f <加了限流的 ingress>

# 快速连续请求，观察第 3 个起返回 503
for i in 1 2 3 4; do curl -s -o /dev/null -w "%{http_code}\n" http://localhost/api; done
# 200 200 503 503
```

限流生效——注解是 Ingress Controller 的"方言"，换 Controller 注解就不认了（第 11 章的教训）。

35.9 实验总结与思考

- Ingress 资源与 Ingress Controller 的分工是什么？如何一句话验证"没有 Controller 就没有路由"？
- `pathType: Prefix` 的 `/api` 会匹配 `/api/orders` 吗？会匹配 `/apiv2` 吗？动手试试。
- 自签名证书的 Secret 类型是什么？真实证书和自签名证书在生产上的差别只是"信任"吗？
- 试着用 `nginx.ingress.kubernetes.io/rewrite-target: /` 注解，让 `/api` 前缀在转发前被去掉，观察后端收到的路径变化。
- 多个 Ingress 同时匹配同一 Host 和路径时，谁的优先级更高？（提示：最长路径匹配优先）

第36章 动手实验六：存储持久化实战

这一章把第 13、14 章讲清楚——用真实的实验回答三个问题：容器重启数据还在吗？Pod 重建数据还在吗？节点重启数据还在吗？然后走通 StatefulSet 的"稳定身份 + 专属存储"。这是理解"有状态 vs 无状态"的关键一课。

36.1 实验目标

1. 亲手证明"容器可写层不持久"；
2. 理解 emptyDir、hostPath、PVC 三者的生命周期差异；
3. 走通 PVC 动态供应的完整链路（本地环境用 local-path-provisioner）；
4. 用 StatefulSet 验证"稳定身份 + 专属存储"。

36.2 预备：安装本地存储供应器

Kind 集群默认有一个 `standard` StorageClass（用 kind 内置的 local-path）。确认：

```
kubectl get storageclass
# NAME                 PROVISIONER             RECLAIMPOLICY
# standard (default)   rancher.io/local-path   Delete
```

看到 `(default)` 标记即可。如果是裸集群（Minikube/自建），用 local-path-provisioner 一行安装：

```
kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-provisioner/master/deploy/local-path-storage.yaml
```

36.3 实验 A：容器可写层不持久

```
# pod-volatile.yaml
apiVersion: v1
kind: Pod
metadata: {name: volatile}
spec:
  containers:
  - name: writer
    image: busybox:1.36
    command: ["sh", "-c", "echo precious-data > /data.txt && sleep 3600"]
```

```
kubectl apply -f pod-volatile.yaml

# 1. 容器内写文件后，重启容器（不删 Pod，仅重启容器）
kubectl exec volatile -- cat /data.txt        # precious-data
kubectl exec volatile -- kill 1               # 杀掉主进程 → 容器重启
kubectl exec volatile -- cat /data.txt        # 数据还在吗?
```

结论：**容器重启，可写层还在**（只要容器没被删除重建）。真正丢数据的是"Pod 重建"——试试：

```
kubectl delete pod volatile                    # 删除 Pod（相当于重建）
kubectl run volatile2 --image=busybox:1.36 --restart=Never -- \
  sh -c 'cat /data.txt 2>&1 || echo "data lost!"'
# data lost!
```

结论：容器可写层随 Pod 删除而销毁。持久化必须用 Volume。

36.4 实验 B：emptyDir 的生命周期

```
# pod-emptydir.yaml
apiVersion: v1
kind: Pod
metadata: {name: with-emptydir}
spec:
  containers:
  - name: writer
    image: busybox:1.36
    command: ["sh", "-c", "echo hello > /cache/note.txt && sleep 3600"]
    volumeMounts:
    - name: cache
      mountPath: /cache
  volumes:
  - name: cache
    emptyDir: {}
```

```
kubectl apply -f pod-emptydir.yaml
kubectl exec with-emptydir -- cat /cache/note.txt   # hello

kubectl delete pod with-emptydir                    # Pod 没了
kubectl apply -f pod-emptydir.yaml                  # 同名重建
kubectl exec with-emptydir -- cat /cache/note.txt   # cat: ... No such file
```

结论：emptyDir 随 Pod 生死——适合"多容器共享的临时文件"（第 4 章 sidecar），不适合持久化。

36.5 实验 C：PVC 动态供应全链路

这才是正经持久化。创建一个 PVC：

```
# pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata: {name: data-claim}
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests: {storage: 1Gi}
  storageClassName: standard      # 默认 SC
```

```
kubectl apply -f pvc.yaml

# 观察供应链路：PVC → 自动创建 PV → Bound
kubectl get pvc,pv
# NAME          STATUS  VOLUME          CAPACITY  ACCESS MODES
# data-claim    Bound   pvc-abc123...   1Gi       RWO

# 说明：SC 的 provisioner 自动"开了一块盘"，生成了 PV 并绑定
```

把 PVC 挂给一个写数据的 Pod：

```
# pod-with-pvc.yaml
apiVersion: v1
kind: Pod
metadata: {name: db-pod}
spec:
  containers:
  - name: db
    image: busybox:1.36
    command: ["sh", "-c", "echo 'important data' > /data/db.txt && sleep 3600"]
    volumeMounts:
    - name: data
      mountPath: /data
  volumes:
  - name: data
    persistentVolumeClaim:
      claimName: data-claim
```

```
kubectl apply -f pod-with-pvc.yaml

# 写数据 → 删 Pod → 重建 Pod → 数据还在!
kubectl exec db-pod -- cat /data/db.txt          # important data
kubectl delete pod db-pod
kubectl apply -f pod-with-pvc.yaml
kubectl exec db-pod -- cat /data/db.txt          # important data（还在！）
```

结论：PVC 挂载的数据跨 Pod 生死持久。 对比 36.4 的 emptyDir，差异一目了然。

> **注意：** `ReadWriteOnce` 的卷在同一时刻只能被一个节点挂载。如果两个 Pod 恰好调度到不同节点同时挂它，后一个会卡住。多副本共享存储要用 RWX（第 14 章的提醒）。

36.6 实验 D：StatefulSet 的稳定身份与专属存储

把第 8 章跑起来。先建 headless Service：

```
# sts.yaml
apiVersion: v1
kind: Service
metadata: {name: sts-svc}
spec:
  clusterIP: None
  selector: {app: sts-demo}
  ports: [{port: 80}]
---
apiVersion: apps/v1
kind: StatefulSet
metadata: {name: sts-demo}
spec:
  serviceName: sts-svc
  replicas: 3
  selector:
    matchLabels: {app: sts-demo}
  volumeClaimTemplates:
  - metadata: {name: data}
    spec:
      accessModes: [ReadWriteOnce]
      resources: {requests: {storage: 100Mi}}
  template:
    metadata: {labels: {app: sts-demo}}
    spec:
      containers:
      - name: app
        image: busybox:1.36
        command: ["sh", "-c", "echo \"I am $HOSTNAME\" > /data/whoami.txt && sleep 3600"]
        env:
        - name: HOSTNAME
          valueFrom: {fieldRef: {fieldPath: metadata.name}}
        volumeMounts:
        - name: data
          mountPath: /data
```

```
kubectl apply -f sts.yaml

# 1. 观察有序创建：0 就绪才建 1，1 就绪才建 2
kubectl get pods -l app=sts-demo -w

# 2. 每个 Pod 都有专属 PVC
kubectl get pvc
# NAME            STATUS  CAPACITY
# data-sts-demo-0 Bound   100Mi
# data-sts-demo-1 Bound   100Mi
# data-sts-demo-2 Bound   100Mi

# 3. 验证稳定身份：删掉 sts-demo-1，重建后名字不变（对比 Deployment 的随机名）
kubectl delete pod sts-demo-1
kubectl get pods -l app=sts-demo     # sts-demo-1 回来了，名字一模一样

# 4. 验证专属存储：sts-demo-1 重建后，它的数据还在
kubectl exec sts-demo-1 -- cat /data/whoami.txt   # I am sts-demo-1
```

结论：StatefulSet 的 Pod 删了，回来的是"同名同盘"的同一个它——与 Deployment 的"随机名 + 新盘"形成鲜明对比（第 8 章）。

36.7 实验总结与思考

- 依次排出四者的持久性：容器可写层 < emptyDir < hostPath < PVC，各自"活多久"？
- PVC 从 Pending 到 Bound 之间发生了什么？谁创建了 PV？（提示：StorageClass 的 provisioner）
- StatefulSet 删 Pod 后重建，为什么名字不变？这带来什么好处？
- 删除 StatefulSet 后，PVC 还在吗？删掉 PVC 会发生什么（先想想再试）？回收策略 Delete 和 Retain 的区别是什么？
- 如果想模拟"节点故障"，`kind` 里可以 `docker stop k8s-lab-worker` 停掉一个节点，观察 StatefulSet 的 Pod 迁移是否卡在卷 detach（第 21 章场景）。

第37章 动手实验七：ConfigMap、Secret 与热更新

这一章把第 15 章的配置管理变成手上功夫：环境变量注入 vs 文件挂载、热更新 vs 冷更新、Secret 的 base64 真相、以及 subPath 那个著名的大坑。每一个都是生产踩坑率极高的知识点。

37.1 实验目标

1. 亲手验证"环境变量注入不热更新、文件挂载可热更新";
2. 验证 subPath 挂载不热更新的大坑;
3. 看穿 Secret 的 base64 编码本质;
4. 用 ConfigMap 挂载 Nginx 配置并热加载。

37.2 创建 ConfigMap

```
# cm.yaml
apiVersion: v1
kind: ConfigMap
metadata: {name: app-config}
data:
  LOG_LEVEL: "info"
  APP_MODE: "dev"
  nginx.conf: |
    server {
      listen 8080;
      location / { return 200 "version-1\n"; }
    }
```

```
kubectl apply -f cm.yaml
kubectl get configmap app-config -o yaml
```

37.3 实验 A：环境变量注入不热更新

```
# pod-env.yaml
apiVersion: v1
kind: Pod
metadata: {name: env-pod}
spec:
  containers:
  - name: app
    image: busybox:1.36
    command: ["sh", "-c", "echo LOG_LEVEL=$LOG_LEVEL && sleep 3600"]
    envFrom:
    - configMapRef: {name: app-config}     # 整个 CM 注入环境变量
```

```
kubectl apply -f pod-env.yaml
kubectl logs env-pod          # LOG_LEVEL=info

# 修改 ConfigMap
kubectl patch configmap app-config -p '{"data":{"LOG_LEVEL":"debug"}}'

# 等 1 分钟，再看环境变量——纹丝不动
kubectl exec env-pod -- printenv LOG_LEVEL     # info（没变！）

# 只有重建 Pod 才生效
kubectl delete pod env-pod
kubectl apply -f pod-env.yaml
kubectl logs env-pod          # LOG_LEVEL=debug（变了）
```

结论：环境变量是进程启动时读取的，改 CM 不会改写已运行进程的环境——必须重启 Pod。 这是"环境变量不适合热更新配置"的根本原因。

37.4 实验 B：文件挂载可以热更新

```
# pod-file.yaml
apiVersion: v1
kind: Pod
metadata: {name: file-pod}
spec:
  containers:
  - name: app
    image: busybox:1.36
    command: ["sh", "-c", "while true; do cat /etc/config/LOG_LEVEL; echo; sleep 3; done"]
    volumeMounts:
    - name: conf
      mountPath: /etc/config
      readOnly: true
  volumes:
  - name: conf
    configMap:
      name: app-config
```

```
kubectl apply -f pod-file.yaml
kubectl logs file-pod -f      # 持续输出 info
```

另开终端修改 CM：

```
kubectl patch configmap app-config -p '{"data":{"LOG_LEVEL":"warn"}}'
```

回到日志终端，约 1 分钟内你会看到输出从 `info` 变成 `warn` ——**文件挂载的配置热更新了**（kubelet 周期性同步 CM 到挂载目录）。

> **提示：** 热更新有分钟级延迟，且依赖应用"重新读文件"。我们的脚本每 3 秒 `cat` 一次所以能立刻看到；真实应用若只在启动时读一次，文件变了它也不知道（除非支持信号重载）。

37.5 实验 C：subPath 不热更新（著名大坑）

用 `subPath` 只挂载单个文件：

```
# pod-subpath.yaml
apiVersion: v1
kind: Pod
metadata: {name: subpath-pod}
spec:
  containers:
  - name: app
    image: busybox:1.36
    command: ["sh", "-c", "while true; do cat /etc/config/LOG_LEVEL; echo; sleep 3; done"]
    volumeMounts:
    - name: conf
      mountPath: /etc/config/LOG_LEVEL
      subPath: LOG_LEVEL        # 只挂载这一个文件
  volumes:
  - name: conf
    configMap: {name: app-config}
```

```
kubectl apply -f pod-subpath.yaml
kubectl logs subpath-pod -f    # 持续输出 info

# 修改 CM
kubectl patch configmap app-config -p '{"data":{"LOG_LEVEL":"error"}}'
# 等 2 分钟……日志依然是 info，永不更新!
```

结论：subPath 挂载的单个文件不随 ConfigMap 更新——这是官方文档明确的行为，也是生产事故高发区。要热更新，就整目录挂载，不要用 subPath。

37.6 实验 D：看穿 Secret 的 base64

```
# 创建 Secret
kubectl create secret generic db-cred --from-literal=password='S3cure!Pass'
kubectl get secret db-cred -o yaml
# data:
#   password: UzNjdXJlIVBhc3M=      ← 这不是加密，是 base64 编码
```

一秒钟"解密"：

```
echo 'UzNjdXJlIVBhc3M=' | base64 -d
# S3cure!Pass
```

任何能 `get secret` 的人都能瞬间还原明文——**base64 只是"编码"不是"加密"**（第 15 章的警示）。真实的安全来自 RBAC 收紧、etcd 加密、以及密钥不进 Git。

37.7 实验 E：ConfigMap 挂 Nginx 配置 + 热加载

这是配置热更新最有生产价值的应用。部署一个 Nginx，配置来自 ConfigMap：

```
# nginx-cm.yaml
apiVersion: v1
kind: ConfigMap
metadata: {name: nginx-conf}
data:
  default.conf: |
    server {
      listen 8080;
      location / { return 200 "I am version-A\n"; }
    }
---
apiVersion: apps/v1
kind: Deployment
metadata: {name: nginx}
spec:
  replicas: 1
  selector:
    matchLabels: {app: nginx}
  template:
    metadata: {labels: {app: nginx}}
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        volumeMounts:
        - name: conf
          mountPath: /etc/nginx/conf.d
          readOnly: true
      volumes:
      - name: conf
        configMap: {name: nginx-conf}
```

```
kubectl apply -f nginx-cm.yaml
kubectl port-forward deployment/nginx 8080:8080 &
curl localhost:8080     # I am version-A
```

修改 CM 中的配置，并手动触发 Nginx reload（ConfigMap 更新到文件后，Nginx 不会自动重载）：

```
kubectl patch configmap nginx-conf --type=json \
  -p='[{"op":"replace","path":"/data/default.conf","value":"server {\n  listen 8080;\n  location /
{ return 200 \"I am version-B\\n\"; }\n}"}]'

# 等 CM 同步到 Pod（约 1 分钟），然后 reload Nginx
kubectl exec deployment/nginx -- nginx -s reload
curl localhost:8080     # I am version-B
```

完整链路打通：改 ConfigMap → kubelet 同步文件 → `nginx -s reload` → 新配置生效。生产中用 `reloader` 这类工具自动监听 CM 变化并触发 reload，省去手动步骤。

37.8 实验总结与思考

- 环境变量注入和文件挂载，谁支持热更新？各自的适用场景是什么？
- subPath 挂载为什么不热更新？这个坑在你的项目里会不会踩到？
- Secret 的 base64 为什么不是安全机制？真正的安全依赖哪几层？
- ConfigMap 热更新后，Nginx 为什么还需要手动 reload？还有哪些应用需要"信号重载"？
- 一个 ConfigMap 最多多大？超大配置应该放哪里？（提示：约 1MB 上限，静态资源走镜像或对象存储）

第38章 动手实验八：RBAC 与 Pod 安全实战

安全是"看不见摸不着"的，所以更要亲手做实验——亲眼看到"权限拒绝""非 root 强制""镜像策略拦截"，才会真正重视第 16、17 章。这一章用两个角色（平台管理员、应用开发者）的视角走完整个 RBAC 流程，再体验 Pod Security 的强制标准。

38.1 实验目标

1. 创建 ServiceAccount，授予最小权限，用 `can-i` 验证权限边界；
2. 体验"程序用 SA token 访问 API"与"禁用 token 自动挂载"；
3. 亲手被 Pod Security Admission 拦截一次，理解 restricted 标准；
4. 用非 root、只读文件系统加固一个 Pod。

38.2 实验 A：RBAC 最小权限全流程

场景：CI 机器人 `ci-bot` 只允许在 `team-a` 命名空间部署应用（操作 Deployment、Service、ConfigMap），其他一律禁止。

第一步：建命名空间与 ServiceAccount：

```
kubectl create namespace team-a
kubectl create serviceaccount ci-bot -n team-a
```

第二步：写 Role（最小权限）：

```
# role.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: deployer
  namespace: team-a
rules:
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["get", "list", "create", "update", "patch", "delete"]
- apiGroups: [""]
  resources: ["services", "configmaps"]
  verbs: ["get", "list", "create", "update", "delete"]
```

第三步：RoleBinding 绑定：

```
# binding.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: ci-bot-deployer
  namespace: team-a
subjects:
- kind: ServiceAccount
  name: ci-bot
  namespace: team-a
roleRef:
  kind: Role
  name: deployer
  apiGroup: rbac.authorization.k8s.io
```

第四步：验证权限边界（`can-i --as` 是神器，无需真的切换身份）：

```
# 允许的
kubectl auth can-i create deployments -n team-a --as=system:serviceaccount:team-a:ci-bot
# yes

# 越界的：跨命名空间？
kubectl auth can-i create deployments -n default --as=system:serviceaccount:team-a:ci-bot
# no

# 越界的：删除 Secret？
kubectl auth can-i delete secrets -n team-a --as=system:serviceaccount:team-a:ci-bot
# no

# 越界的：读全集群 Pod？
kubectl auth can-i get pods --all-namespaces --as=system:serviceaccount:team-a:ci-bot
# no
```

权限边界清晰可见：ci-bot 只能动 team-a 的三个资源类型，其余全是 no。这就是最小权限原则的落地。

> **深入：** 真实 CI 系统里，流水线就以这个 SA 的身份运行（通过 token 或云厂商的 workload identity 注入）。`kubectl auth can-i --as` 让你在授权之前"预演"任何主体的权限，是安全审查的第一工具。

38.3 实验 B：程序如何用 token 访问 API

一个 Pod 默认挂载着所属 SA 的 token，任何进程都能拿它调 API：

```
kubectl run token-test --rm -it --image=nicolaka/netshoot -n team-a -- bash
```

在 Pod 内：

```
# 找到自动挂载的 token
ls /var/run/secrets/kubernetes.io/serviceaccount/
# ca.crt  namespace  token

# 用它访问 API（读当前命名空间的 Service）
CA=/var/run/secrets/kubernetes.io/serviceaccount/ca.crt
TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
APISERVER=https://${KUBERNETES_SERVICE_HOST}:${KUBERNETES_SERVICE_PORT}

curl -s --cacert $CA -H "Authorization: Bearer $TOKEN" \
  $APISERVER/api/v1/namespaces/team-a/services
# 返回 Service 列表（default SA 有 get 权限）
```

这就是"程序访问 K8s API"的完整机制（第 16 章）。现在理解为什么**不需要 API 访问的 Pod 应该禁掉 token**——一旦 Pod 被攻破，这个 token 就是攻击者扩大战果的钥匙。

禁用自动挂载：

```
spec:
  automountServiceAccountToken: false
  containers: [...]
```

```
kubectl run no-token --image=busybox:1.36 --restart=Never \
  --overrides='{"spec":{"automountServiceAccountToken":false}}' \
  --command -- sleep 3600
kubectl exec no-token -- ls /var/run/secrets/kubernetes.io/serviceaccount/
# ls: cannot access ... No such file or directory   ← 没有 token 了
```

38.4 实验 C：被 Pod Security Admission 拦截

先给命名空间上 `restricted`（最严格档）：

```
kubectl label ns team-a pod-security.kubernetes.io/enforce=restricted
kubectl label ns team-a pod-security.kubernetes.io/warn=restricted
```

现在试着部署一个"不合规"的 Pod（特权容器）：

```
# privileged.yaml
apiVersion: v1
kind: Pod
metadata: {name: privileged-pod, namespace: team-a}
spec:
  containers:
  - name: app
    image: busybox:1.36
    securityContext:
      privileged: true          # 高危：特权容器
```

```
kubectl apply -f privileged.yaml
# Error from server (Forbidden): error when creating "privileged.yaml":
# pods "privileged-pod" is forbidden: violates PodSecurity "restricted:latest":
# privileged (container "app" must not set securityContext.privileged=true)
```

被拦下来了！ 这就是"用制度代替自觉"——不用写一行校验代码，集群自动拒绝高危 Pod。再试一个"以 root 运行"的：

```
# root.yaml
apiVersion: v1
kind: Pod
metadata: {name: root-pod, namespace: team-a}
spec:
  containers:
  - name: app
    image: busybox:1.36        # busybox 默认以 root 运行
    command: ["sleep", "3600"]
```

```
kubectl apply -f root.yaml
# forbidden: violates PodSecurity "restricted": runAsNonRoot (container must run as non-root)
```

修好它，让它通过 restricted：

```
# compliant.yaml
apiVersion: v1
kind: Pod
metadata: {name: compliant-pod, namespace: team-a}
spec:
  containers:
  - name: app
    image: busybox:1.36
    command: ["sleep", "3600"]
    securityContext:
      runAsNonRoot: true
      runAsUser: 1000
      allowPrivilegeEscalation: false
      capabilities: {drop: ["ALL"]}
      readOnlyRootFilesystem: true      # 注意：busybox sleep 不需要写根目录，可通过
```

```
kubectl apply -f compliant.yaml    # 通过!
kubectl get pod compliant-pod -n team-a
```

> **提示：** `readOnlyRootFilesystem: true` 会让很多应用失败（它们默认写 /tmp、/var/run 等），需要把这些路径单独挂 emptyDir。改造有成本，但这是 restricted 标准的核心价值所在——逼着你审视应用到底需要写哪里（第 17 章）。

38.5 实验 D：一条完整的安全基线对比

把"不安全"和"加固后"的 Pod 并排看（节选关键字段）：

项目	不安全	加固后
运行用户	root	runAsNonRoot: true, runAsUser: 1000
内核能力	默认全部	drop: ALL
特权提升	允许	allowPrivilegeEscalation: false
根文件系统	可写	readOnlyRootFilesystem: true
SA token	自动挂载	automountServiceAccountToken: false
镜像来源	任意	由准入策略限制为可信仓库

这份对照表就是第 17 章"SecurityContext 纪律清单"的浓缩版，可打印张贴。

38.6 实验总结与思考

- `kubectl auth can-i --as` 的作用是什么？如何在授权前预演权限？
- 一个被攻破的 Pod，如果挂着 SA token，攻击者能做什么？（提示：`kubectl auth can-i --list` 看 default SA 的权限）
- Pod Security 的 restricted 标准拦下了哪些行为？它在哪一层、什么时机生效？（提示：准入控制）
- 如果某系统组件确实需要特权容器，应该怎么放行？（提示：给它专用的 privileged namespace + 标签）
- 试着用 `kubectl describe ns team-a` 查看 Pod Security 的标签，理解 enforce/audit/warn 三者的区别。

第39章 动手实验九：HPA 弹性伸缩实战

这一章把第 18、19 章跑起来——亲手让集群"呼吸"：装上 metrics-server，给服务配上 HPA，用压测把 CPU 拉高，眼睁睁看着副本数自动涨上去、再自动缩回来。这是 Kubernetes 最让人上瘾的能力之一。

39.1 实验目标

1. 安装 metrics-server，理解 HPA 的指标来源；
2. 配置 CPU 型 HPA；
3. 压测触发扩容，观察副本数的自动变化；
4. 观察缩容的"延迟"（stabilizationWindow）；
5. 理解 requests 对 HPA 的直接影响。

39.2 安装 metrics-server

Kind 集群默认没有 metrics-server，装一个：

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/
components.yaml
```

> **注意：** Kind 环境通常需要给 metrics-server 加 `--kubelet-insecure-tls` 参数（因为本地 kubelet 用自签证书）。patch 一下：

```
kubectl patch deployment metrics-server -n kube-system --type=json -p='[
  {"op":"add","path":"/spec/template/spec/containers/0/args/-",
   "value":"--kubelet-insecure-tls"}]'
```

等待就绪后验证：

```
kubectl top nodes        # 能看到节点指标即成功
kubectl top pods
```

> **深入：** metrics-server 从各节点 kubelet 拉取 cAdvisor 采集的容器指标，聚合后通过 Metrics API 暴露。**没有它，HPA 和 `kubectl top` 都用不了**——这就是第 19 章说的"HPA 的指标来源"。

39.3 部署一个"能吃 CPU"的应用

用一个计算密集的应用（或直接用 busybox 跑死循环），配好 requests（HPA 的基准）：

```
# cpu-app.yaml
apiVersion: apps/v1
kind: Deployment
metadata: {name: cpu-app}
spec:
  replicas: 1
  selector:
    matchLabels: {app: cpu-app}
  template:
    metadata: {labels: {app: cpu-app}}
    spec:
      containers:
      - name: worker
        image: busybox:1.36
        command: ["sh", "-c", "while true; do :; done"]   # 空转吃 CPU
        resources:
          requests: {cpu: 100m, memory: 32Mi}
          limits:   {cpu: 200m, memory: 64Mi}
```

> 注意：这里 requests=100m、limits=200m。**HPA 的目标利用率是相对 requests 计算的**——牢记这一点，后面观察时用得上。

39.4 创建 HPA

```
# hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: {name: cpu-app}
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: cpu-app
  minReplicas: 1
  maxReplicas: 8
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50     # 平均 CPU 使用率目标 50%
```

```
kubectl apply -f hpa.yaml

# 观察 HPA 状态（初始 TARGETS 可能是 <unknown>，等 metrics 就绪）
kubectl get hpa -w
# NAME      REFERENCE            TARGETS   MINPODS   MAXPODS   REPLICAS
# cpu-app   Deployment/cpu-app   0%/50%    1         8         1
```

39.5 压测触发扩容

在另一个终端制造负载——起一个压测 Pod 持续请求（这里直接用 busybox 发高负载，或用一个 load 工具）：

```
# 让一个副本把 CPU 打满：在 Pod 里跑多个死循环
kubectl exec deployment/cpu-app -- sh -c \
  'for i in $(seq 1 4); do while true; do :; done & done; wait'
```

> 更贴近真实的方式是用压测工具（如 hey、vegeta、wrk）对一个 HTTP 服务打流量。这里为了简单直接吃 CPU，效果相同。

观察 HPA 的变化（几分钟内）：

```
kubectl get hpa -w
# cpu-app   Deployment/cpu-app   198%/50%   1   8   1
# cpu-app   Deployment/cpu-app   198%/50%   1   8   2    ← 扩容了！
# cpu-app   Deployment/cpu-app   99%/50%    1   8   4    ← 继续扩
# cpu-app   Deployment/cpu-app   49%/50%    1   8   8    ← 直到达标
```

同时看副本数：

```
kubectl get pods -l app=cpu-app -w
```

发生了什么：单副本 CPU 飙到接近 limits（200m，即 requests 的 200%），远超 50% 目标 → HPA 套用公式 `ceil(1 × 198/50) = 4` 扩容 → 新副本摊薄负载 → 使用率回落 → 继续调整直到稳定在 50% 附近。

> **提示：** 冷启动注意——HPA 有个默认的扩容冷却期（约 15 秒采样、稳定窗口），新副本还要等调度 + 拉镜像 + 就绪，所以从"指标飙升"到"副本就绪接流量"需要 1～3 分钟。这就是第 19 章说的"扩容要快、但快不过冷启动"。

39.6 停止压测，观察缩容

停掉负载（杀掉压测进程 / 删掉压测 Pod）：

```
kubectl delete pod <压测pod>     # 或 exec 里 kill
```

观察缩容的"克制"：

```
kubectl get hpa -w
# ...  0%/50%  1  8  8
# ...  0%/50%  1  8  5     ← 不是瞬间回到 1，而是逐步、缓慢缩
# ...  0%/50%  1  8  3
# ...  0%/50%  1  8  1     ← 几分钟后才回到 1
```

为什么缩得慢？ 默认 `stabilizationWindowSeconds=300`（观察 5 分钟）——HPA 要确认负载确实"持续很低"才缩容，避免流量波动导致反复伸缩（抖动）。这是第 19 章"快扩慢缩"的直观体现。

39.7 实验进阶：验证 requests 的影响

把 requests 从 100m 改成 200m（= limits），再跑一次压测：

```
kubectl patch deployment cpu-app --type=json -p='[
  {"op":"replace","path":"/spec/template/spec/containers/0/resources/requests/cpu","value":"200m"}]'
```

现在同样的负载下，CPU 使用率基准变了（相对 200m 计算），HPA 的扩容决策也随之变化。观察 `kubectl get hpa` 的 TARGETS 百分比差异——**同样 200m 的实际用量，requests=100m 时是 200%，requests=200m 时是 100%**。

结论：requests 是 HPA 的"标尺"，标尺不准，弹性决策全失真。这正是第 19 章反复强调"requests 要准"的原因。

39.8 实验总结与思考

- HPA 的扩容公式是什么？`ceil(当前副本 × 当前指标 / 目标指标)` 在你的实验数据里成立吗？
- 为什么扩容有"延迟"，缩容更"慢"？各由什么参数控制？
- requests 从 100m 改到 200m 后，同样的负载为什么触发的扩容更少？
- `kubectl get hpa` 里 TARGETS 显示 `<unknown>` 通常是什么原因？
- 如果想把指标从 CPU 换成 QPS 或消息积压量，需要什么？（提示：Prometheus Adapter / KEDA）

第40章 动手实验十：Helm 从零到发布

这一章把第 23 章跑起来——亲手写一个 Helm Chart，把它参数化、部署到两个环境、升级、回滚。走完这一遍，你会彻底理解"模板 + 参数 + 实例"三者是怎么协作的。

40.1 实验目标

1. 从零创建一个 Helm Chart；
2. 理解 values.yaml、模板、_helpers.tpl 的关系；
3. 用不同 values 部署 dev / prod 两个环境；
4. 完成一次升级与回滚；
5. 体验 `helm template` 渲染预览。

40.2 创建一个 Chart

```
helm create myweb
# 生成标准脚手架：Chart.yaml、values.yaml、templates/、charts/
tree myweb
# myweb/
# ├── Chart.yaml
# ├── values.yaml
# ├── templates/
# │   ├── deployment.yaml
# │   ├── service.yaml
# │   ├── _helpers.tpl
# │   └── tests/
# └── .helmignore
```

先看 `Chart.yaml` 与 `values.yaml`，把它们精简成我们需要的：

```
# Chart.yaml
apiVersion: v2
name: myweb
description: 演示用的 web 应用
version: 0.1.0
appVersion: "1.0"
```

```
# values.yaml
replicaCount: 2
image:
  repository: myapp
  tag: v1
service:
  port: 80
```

40.3 写模板

templates/deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-web
  labels:
    app: {{ .Release.Name }}-web
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ .Release.Name }}-web
  template:
    metadata:
      labels:
        app: {{ .Release.Name }}-web
    spec:
      containers:
      - name: web
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
        ports:
        - containerPort: 8080
```

templates/service.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-web
spec:
  selector:
    app: {{ .Release.Name }}-web
  ports:
  - port: {{ .Values.service.port }}
    targetPort: 8080
```

几个关键点（对应第 23 章）：

- `{{ .Release.Name }}`：Release 实例名——同一张模板装出多个实例，靠它区分；
- `{{ .Values.xxx }}`：读取 values.yaml 的参数；
- 模板渲染是纯文本替换，先 `helm template` 看结果再部署。

40.4 渲染预览（重要习惯）

```
helm template dev ./myweb
# 渲染出完整 YAML，检查无误再装
```

这一步应该进 CI（第 23 章的忠告）——**渲染结果 = 即将部署的东西，先看清楚再动手**。

40.5 部署两个环境

```
# dev 环境（2 副本，镜像 v1）
helm install myweb-dev ./myweb -n dev --create-namespace

# prod 环境（5 副本，覆盖参数）
helm install myweb-prod ./myweb \
  --set replicaCount=5 --set image.tag=v1 \
  -n prod --create-namespace
```

查看两个 Release：

```
helm list -A
# NAME        NAMESPACE  REVISION  STATUS    CHART
# myweb-dev   dev        1         deployed  myweb-0.1.0
# myweb-prod  prod       1         deployed  myweb-0.1.0

kubectl get pods -n dev
kubectl get pods -n prod    # 5 个副本
```

同一张模板，两个参数组合，两个独立实例——Helm 的"打包 + 参数化"价值在此体现。

40.6 升级与回滚

升级 prod（改副本数 + 镜像）：

```
helm upgrade myweb-prod ./myweb \
  --set replicaCount=8 --set image.tag=v2 -n prod

kubectl get pods -n prod    # 8 个，滚动更新到 v2

helm list -n prod
# myweb-prod  prod  2  deployed    ← REVISION 变成 2
```

回滚：

```
helm rollback myweb-prod 1 -n prod
# Rollback was a success!

helm list -n prod          # REVISION 3（回滚本身也是新版本）
kubectl get pods -n prod   # 又回到 v1、5 副本
```

要点：`helm rollback` 秒级完成，因为底层就是"用旧 values 重新渲染 + 滚动更新"——Helm 把每次 install/upgrade 的 values 快照都存着。

40.7 用 values 文件（替代一堆 --set）

参数多了以后，`--set` 会失控。改用文件：

```
# prod-values.yaml
replicaCount: 6
image:
  repository: myapp
  tag: v2
```

```
helm upgrade myweb-prod ./myweb -f prod-values.yaml -n prod
```

第 24 章 GitOps 的标准输入，就是这样的 values 文件。

40.8 调试三板斧

```
helm lint ./myweb                      # 静态检查模板语法
helm template dev ./myweb --debug      # 渲染 + 调试信息
helm install myweb-test ./myweb --dry-run   # 模拟安装不落盘
```

40.9 实验总结与思考

- Chart 里 `{{ .Release.Name }}`、`{{ .Values.xxx }}` 分别是什么？它们由谁在何时填充？

- 两个环境共用一个 Chart，靠什么区分？镜像 tag、副本数这些"环境差异"应该写在哪里？
- `helm upgrade` 和 `helm rollback` 底层发生了什么？为什么回滚也走滚动更新？
- 如果模板渲染出错了，最快怎么发现？（提示：helm template / lint）
- 一个 Release 的"版本"（REVISION）和 Chart 的"版本"（version）是一回事吗？（提示：第 23 章 Chart.yaml 的 version vs appVersion）

第41章 动手实验十一：GitOps 全流程实战

这是全篇最"现代"的一个实验——把第 24 章跑通：装 Argo CD，建一个 Git 仓库当"真相源"，让集群自动与 Git 保持一致，然后亲手制造一次"配置漂移"看它被自动纠正。走完这一遍，你会理解为什么大厂都把 GitOps 当发布标准。

41.1 实验目标

1. 部署 Argo CD；
2. 把一个应用接入 GitOps（Git 仓库 → 自动同步到集群）；
3. 体验"改 Git 即改集群"（Pull 模型）；
4. 亲手制造配置漂移，看自动纠正；
5. 理解"回滚 = git revert"。

41.2 部署 Argo CD

```
kubectl create namespace argocd
kubectl apply -n argocd -f \
  https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

kubectl get pods -n argocd -w     # 等到全部 Running
```

获取初始 admin 密码并登录（本地用端口转发访问 UI）：

```
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath="{.data.password}" | base64 -d; echo
kubectl port-forward svc/argocd-server -n argocd 8080:443
# 浏览器打开 https://localhost:8080（忽略证书告警），用户名 admin
```

41.3 准备"真相源"仓库

GitOps 的核心是：**Git 仓库里存放应用的期望状态**。本地建一个仓库（真实环境推送到 GitLab/GitHub）：

```
mkdir gitops-repo && cd gitops-repo
git init

# 应用的 Kustomize 或纯 YAML 清单
mkdir -p apps/demo
cat > apps/demo/deployment.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: demo-app
spec:
  replicas: 3
  selector:
    matchLabels: {app: demo-app}
  template:
    metadata:
      labels: {app: demo-app}
    spec:
      containers:
      - name: app
        image: myapp:v1
        ports: [{containerPort: 8080}]
---
apiVersion: v1
kind: Service
metadata: {name: demo-app}
spec:
  selector: {app: demo-app}
  ports: [{port: 80, targetPort: 8080}]
EOF

git add . && git commit -m "init demo app"
```

> **提示：** 本实验用本地 git 仓库 + Argo CD 的"本地仓库"能力即可跑通（生产是远程 Git 仓库 + 自动同步）。若想用 GitHub，把仓库推到远程，Argo CD 里填远程 URL 与凭证即可。

41.4 把应用接入 Argo CD

用命令行创建一个 Application（也可以 UI 操作）：

```
argocd login localhost:8080 --username admin --password <上面的密码> --insecure

argocd app create demo \
  --repo file:///root/gitops-repo \        # 本地仓库（生产填远程 URL）
  --path apps/demo \
  --dest-server https://kubernetes.default.svc \
  --dest-namespace default \
  --sync-policy none                       # 先手动同步，看清过程
```

> 若 Argo CD 无法访问宿主机本地路径，可改用真实 Git 仓库，或先 `git daemon` / `python3 -m http.server` 暴露仓库。云上托管 Argo CD + 远程 Git 是生产标准。

查看应用状态：

```
argocd app list
# NAME  CLUSTER  NAMESPACE  PROJECT  STATUS     HEALTH
# demo  in-cluster  default  default  OutOfSync  Missing
```

`OutOfSync` ——Argo CD 发现集群里没有这个应用，与 Git 不一致。同步：

```
argocd app sync demo
# 同步完成后：
argocd app list
# demo  ...  Synced  Healthy
kubectl get pods -l app=demo-app    # 3 个副本，由 Argo CD 创建
```

41.5 体验"改 Git 即改集群"

改仓库里的副本数，提交：

```
sed -i 's/replicas: 3/replicas: 5/' apps/demo/deployment.yaml
git add . && git commit -m "scale to 5"
```

再次同步（或开启自动同步）：

```
argocd app sync demo
kubectl get pods -l app=demo-app    # 5 个副本
```

整个流程没有一句 `kubectl apply` ——改 Git 提交，Argo CD 把集群"拉"到与 Git 一致。这就是 Pull 模型的精髓（第 24 章）。

41.6 制造配置漂移，看自动纠正

现在有人（或某个意外）直接改线上：

```
kubectl scale deployment demo-app --replicas=10
kubectl get pods -l app=demo-app    # 10 个（线上被人改了！）
```

观察 Argo CD 的反应（3 分钟后，或手动触发检测）：

```
argocd app get demo
# STATUS: OutOfSync（检测到漂移）

# 开启自动同步后，它会被自动纠正回 5
argocd app set demo --sync-policy automated --self-heal
# 片刻后：
kubectl get pods -l app=demo-app    # 回到 5（漂移被自动纠正）
```

`--self-heal`（**自愈**）会持续对比 Git 与集群，任何漂移都被拉回。第 15 章说的"配置漂移"问题，在这里被机制性消灭。

41.7 回滚 = git revert

假设这次"扩到 5"是个错误，要回滚：

```
git revert HEAD            # 生成一个"撤销"提交
argocd app sync demo
kubectl get pods -l app=demo-app    # 回到 3
```

回滚就是一次普通的 Git 操作——有 commit 历史可审计、可 diff、可 blame。这就是 GitOps 相对"人在终端敲命令"的根本优势。

41.8 实验总结与思考

- Argo CD 的 Application 由哪几部分组成？（repo + path + dest cluster/namespace）
- `OutOfSync`、`Synced`、`Healthy` 分别是什么状态？三者关系如何？
- "改 Git 即改集群"和传统"CI 推送"的本质区别是什么？安全上有什么收益？
- `--self-heal` 关闭时，漂移只会被"检测"而不会被"纠正"——为什么有时反而要关掉它？
- 如果要管理 100 个应用，怎么组织仓库和 Application？（提示：App of Apps 模式）

第42章 动手实验十二：故障排查实战演练

这是动手实验篇的收官——把第 21 章的排查方法论变成肌肉记忆。我们会**故意制造几类真实故障**，然后沿着标准路径逐一定位。练得多了，真故障来临时你才不慌。所有演练都在本地集群进行，放心折腾。

42.1 实验目标

1. 亲手制造并定位：镜像拉取失败、崩溃循环、就绪失败、Pending 调度失败；
2. 掌握 `describe` 的 Events 是"第一手线索"；
3. 走一遍网络排障的标准顺序；
4. 建立"止损优先、先观察后动手"的肌肉记忆。

42.2 故障一：ImagePullBackOff

制造：部署一个镜像名错误的 Deployment：

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: bad-image}
spec:
  replicas: 1
  selector:
    matchLabels: {app: bad-image}
  template:
    metadata: {labels: {app: bad-image}}
    spec:
      containers:
      - name: app
        image: myapp:v9.9.9-not-exist    # 这个镜像不存在
```

```
kubectl apply -f bad-image.yaml
kubectl get pods
# bad-image-xxx   0/1   ImagePullBackOff   0   1m
```

定位：

```
kubectl describe pod bad-image-xxx | grep -A6 Events
# Events:
#   Failed   Failed to pull image "myapp:v9.9.9-not-exist":
#   ... not found
```

根因一目了然：镜像 tag 不存在。修复 = 改对镜像名。私有仓库还要查 `imagePullSecrets`（凭证）、网络（拉不到）。

42.3 故障二：CrashLoopBackOff

制造：容器启动即退出：

```
apiVersion: v1
kind: Pod
metadata: {name: crasher}
spec:
  containers:
  - name: app
    image: busybox:1.36
    command: ["sh", "-c", "echo 'boom' && exit 1"]   # 启动就退出，非零码
```

```
kubectl apply -f crasher.yaml
kubectl get pods
# crasher   0/1   CrashLoopBackOff   3   2m     ← RESTARTS 不断增长
```

定位三板斧：

```
# 1. 看崩溃前的日志（--previous 是关键！）
kubectl logs crasher --previous        # boom

# 2. 看退出原因
kubectl describe pod crasher | grep -A3 "Last State"
#   Last State:   Terminated
#     Reason:     Error        ← 应用自身错误退出
#     Exit Code:  1

# 3. 退出码速查（第 21 章）：137=OOM、143=SIGTERM、1=应用错误
```

根因：命令 `exit 1`，应用启动即失败。真实场景里换成"配置错误""依赖连不上"等，看日志都能定位。

进阶：再制造一个 OOMKilled——把 limits 设到极小，跑一个吃内存的容器：

```
kubectl run oom-demo --image=busybox:1.36 --restart=Never \
  --limits=memory=32Mi --command -- sh -c \
  'dd if=/dev/zero of=/dev/null bs=1M count=1000'
kubectl describe pod oom-demo | grep -A3 "Last State"
#   Reason:     OOMKilled
#   Exit Code:  137
```

Exit Code 137 = 内存超限被内核强杀——第 4 章 OOM 理论在此验证。

42.4 故障三：Pending（调度失败）

制造：申请一个集群不可能满足的资源：

```
apiVersion: v1
kind: Pod
metadata: {name: greedy}
spec:
  containers:
  - name: app
    image: myapp:v1
    resources:
      requests: {cpu: "100", memory: 1000Gi}    # 100 核、1TB 内存，不可能满足
```

```
kubectl apply -f greedy.yaml
kubectl get pods
# greedy   0/1   Pending   0   1m
```

定位：

```
kubectl describe pod greedy | grep -A5 Events
# Events:
#   Warning  FailedScheduling  ... 0/3 nodes are available:
#   3 Insufficient cpu, 3 Insufficient memory.
```

根因：资源不足（requests 远超节点容量）。处理：降低 requests、扩容集群、或清理资源。还有两类 Pending 要会认：污点不容忍（`untolerated taint`）、亲和性不满足（`didn't match pod affinity`）——第 21 章的速查表。

42.5 故障四：网络不通（逐层剥洋葱）

制造：Service 的 selector 指向不存在的标签（第 34 章也做过，这里固化流程）：

```
kubectl create deployment svc-backend --image=myapp:v1
kubectl expose deployment svc-backend --port=80 --target-port=8080 --name=svc-demo
# 故意改错 selector
kubectl patch svc svc-demo -p '{"spec":{"selector":{"app":"wrong"}}}'
```

标准排障顺序（第 12、21、34 章的结晶）：

```
# 1. DNS 通吗?
kubectl run d --rm -it --image=nicolaka/netshoot -- nslookup svc-demo
# 解析成功（DNS 没问题）→ 继续

# 2. Service 有后端吗?
kubectl get endpoints svc-demo
# <none>     ← 问题在这里!

# 3. 为什么没后端? 看 selector 和 Pod 标签
kubectl get svc svc-demo -o jsonpath='{.spec.selector}'
# {"app":"wrong"}     ← 与 Pod 标签 app=svc-backend 不符
kubectl get pods --show-labels -l app=svc-backend
```

修复：

```
kubectl patch svc svc-demo -p '{"spec":{"selector":{"app":"svc-backend"}}}'
kubectl get endpoints svc-demo    # 恢复
```

口诀再背一遍（第 34 章）：`访问不通 → 查 DNS → 查端点 → 查连通 → 查策略`。

42.6 一套"秒级止损"清单

排障的黄金法则永远不变：**先止血，后查因**。生产故障的第一反应不是找根因，是恢复服务：

故障现象	秒级止损动作
新版本异常	`kubectl rollout undo` 回滚
单 Pod 崩溃	`kubectl delete pod` 触发重建（临时）
节点异常	`kubectl cordon` 摘除流量，Pod 迁移
流量暴增	`kubectl scale --replicas=N` 快速扩容
有状态服务主库故障	交给 Operator / 按 Runbook 切换

止血之后，再从容地用本章的路径定位根因。

42.7 实验总结与思考

- 四个故障的"第一手线索"分别在 `kubectl get` 和 `kubectl describe` 的哪个字段?
- `kubectl logs --previous` 为什么是 CrashLoop 排查的必杀技?
- Exit Code 137、143、1 分别代表什么? OOM 的 137 和手动 kill 的 137 如何区分? (提示: describe 的 Reason 字段)
- Pending 的三种常见原因如何从 Events 一句话区分?
- 把这套流程写进团队 Runbook, 下次同样的告警响起, 值班同学按图索骥需要几分钟?

附录F 动手实验篇索引与完成清单

动手实验篇(第 31～42 章)共十二个实验, 覆盖了全书从入门到实战的完整路径。把它们做成一张可勾选的清单, 作为你的"通关记录":

#	实验	对应正文	核心技能	完成
1	本地集群搭建	第 3 章	Kind、kubectl、集群组件	☐
2	从代码到 Pod	第 1、4 章	多阶段构建、探针、sidecar	☐
3	Deployment 全生命周期	第 6、7 章	滚动更新、回滚、金丝雀	☐
4	Service 与网络	第 10、12 章	服务发现、负载均衡、排障	☐
5	Ingress 与 HTTPS	第 11 章	路由、TLS、注解	☐
6	存储持久化	第 13、14、8 章	PVC、StatefulSet、生命周期	☐
7	配置与 Secret	第 15 章	热更新、subPath 坑、base64	☐
8	RBAC 与安全	第 16、17 章	最小权限、PSA、加固	☐
9	HPA 弹性伸缩	第 18、19 章	metrics-server、压测、扩缩容	☐
10	Helm 从零到发布	第 23 章	Chart、模板、升级回滚	☐
11	GitOps 全流程	第 24 章	Argo CD、Pull 模型、自愈	☐
12	故障排查演练	第 21 章	四类故障定位、止损	☐

> **提示：** 全部勾完，你就拥有了一线工程师的"手感"。建议把实验环境（Kind 集群）长期保留，学习新工具时随手在上面验证——这比读十篇博客都管用。

附录A 常用命令速查表

本附录按使用场景汇总全书出现的高频命令，可作为日常速查卡片。

A.1 集群与节点

```
kubectl cluster-info                          # 集群信息
kubectl get nodes -o wide                     # 节点列表（含 IP/版本）
kubectl describe node <node>                  # 节点详情（资源分配/事件/污点）
kubectl top nodes                             # 节点资源用量
kubectl cordon <node>                         # 封锁节点（不再调度新 Pod）
kubectl drain <node> --ignore-daemonsets      # 驱逐 Pod（维护前）
kubectl uncordon <node>                       # 解除封锁
kubectl taint node <node> key=value:NoSchedule    # 打污点
kubectl label node <node> disktype=ssd        # 打标签
```

A.2 工作负载（第 4、6～9 章）

```
kubectl apply -f app.yaml                     # 声明式创建/更新
kubectl get pods -l app=web -o wide           # 按标签查询
kubectl describe pod <pod>                    # 详情与事件（排障第一入口）
kubectl logs <pod> --previous -c <container>  # 崩溃前日志
kubectl exec -it <pod> -- sh                  # 进入容器
kubectl port-forward pod/<pod> 8080:80        # 本地端口转发
kubectl scale deploy web --replicas=5         # 手动扩缩容（记得同步 YAML）
kubectl set image deploy/web nginx=nginx:1.26 # 更新镜像
kubectl rollout status deploy/web             # 观察发布进度
kubectl rollout history deploy/web            # 发布历史
kubectl rollout undo deploy/web --to-revision=2   # 回滚
kubectl autoscale deploy web --min=2 --max=10 --cpu-percent=65   # HPA
```

A.3 网络与服务（第 10～12 章）

```
kubectl get svc,endpointslices -A             # 服务与后端端点
kubectl run debug --rm -it --image=nicolaka/netshoot -- bash  # 临时网络调试容器
kubectl exec -it debug -- nslookup user-service        # DNS 验证
kubectl exec -it debug -- curl -sv http://svc:80       # 连通性验证
kubectl get networkpolicy -n <ns>             # 网络策略
```

A.4 存储与配置（第 13～15 章）

```
kubectl get pvc,pv,storageclass              # 存储三件套
kubectl patch pvc data -p '{"spec":{"resources":{"requests":{"storage":"100Gi"}}}}'   # 在线扩容
kubectl create configmap app-config --from-file=nginx.conf        # 从文件建 CM
kubectl create secret generic db-cred --from-literal=password=xxx  # 建 Secret
kubectl create secret docker-registry regcred \
  --docker-server=... --docker-username=... --docker-password=...  # 镜像凭证
```

A.5 RBAC 与安全（第 16～17 章）

```
kubectl auth can-i create deployments -n prod # 我能不能做
kubectl auth can-i delete pods --as=system:serviceaccount:team-a:ci-bot  # 模拟他人
kubectl get rolebinding,clusterrolebinding -A # 全部绑定关系
kubectl who-can delete pods -n prod          # 谁能做（插件 kubectl-who-can）
kubectl label ns prod pod-security.kubernetes.io/enforce=restricted  # PSA 强制
```

A.6 排障与调试（第 21 章）

```
kubectl get events -A --sort-by=.lastTimestamp     # 全集群最新事件
kubectl get pods -A --field-selector=status.phase!=Running   # 异常 Pod
kubectl debug -it <pod> --image=busybox --target=app   # 注入临时调试容器
kubectl get pod <pod> -o jsonpath='{.status.containerStatuses[0].lastState}'   # 崩溃详情
```

A.7 Helm 与 GitOps（第 23～24 章）

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm install myapp ./chart -f values-prod.yaml     # 安装
helm upgrade myapp ./chart -f values-prod.yaml     # 升级
helm rollback myapp 1                              # 回滚
helm template myapp ./chart -f values-prod.yaml    # 渲染预览（进 CI）
kubectl apply -k overlays/prod                     # Kustomize 部署
argocd app sync myapp                              # 手动触发同步
```

A.8 etcd 备份（第 22 章）

```
ETCDCTL_API=3 etcdctl snapshot save /backup/etcd.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
```

附录B 关键术语对照表

术语	通俗解释	本书章节
Cluster（集群）	由控制平面 + 一组节点组成的整体	3
Control Plane（控制平面）	集群的"大脑"：apiserver、etcd、调度器、控制器	3
Node（节点）	集群中的一台机器（工作节点跑 Pod）	3
Pod	调度最小单元，一组共享网络的容器	4
Sidecar	同 Pod 内的辅助容器	4、26
Init 容器	主容器启动前按序执行的前置容器	4
探针（Probe）	liveness（管生死）/ readiness（管接客）/ startup（管慢启动）	4
Namespace（命名空间）	资源的逻辑分区（不是安全边界）	5
Label / Selector	标签与选择器，资源的关联粘合剂	4～6
声明式 API	描述期望状态，系统自动收敛	5
Reconcile（调谐）	控制循环：对比期望与实际并纠偏	5、6、25
ReplicaSet	保证 N 个副本存在的控制器	6
Deployment	管理 ReplicaSet 世代更替，支持滚动更新/回滚	6、7
StatefulSet	有序、有稳定身份与专属存储的控制器	8
DaemonSet	每节点一个副本	9
Job / CronJob	跑完即止 / 定时任务	9
Service	稳定的虚拟 IP + 负载均衡（总机）	10
ClusterIP / NodePort / LoadBalancer	Service 的三种暴露类型	10
Headless Service	clusterIP: None，解析出全部 Pod IP	8、10
Ingress	七层入口规则（需控制器执行）	11
Gateway API	Ingress 的下一代标准	11
CNI	容器网络接口（Calico/Cilium/Flannel 等）	12

术语	通俗解释	本书章节
NetworkPolicy	Pod 网络的默认拒绝式门禁	12、17
Volume / emptyDir / hostPath	存储卷 / 临时卷 / 宿主机卷	13
PV / PVC / StorageClass	库房 / 申请单 / 自动供应商	14
CSI	容器存储接口	14
ConfigMap / Secret	明文配置 / 敏感信息	15
RBAC	基于角色的访问控制	16
ServiceAccount	程序的身份（机器身份证）	16
SecurityContext / PSA	容器安全上下文 / Pod 安全准入	17
QoS（Guaranteed/Burstable/BestEffort）	由 requests/limits 决定的服务等级	18
Affinity / Anti-affinity	亲和性（合租）/ 反亲和（分居）	18
Taint / Toleration	污点（节点谢客）/ 容忍（通行证）	18
HPA / VPA / Cluster Autoscaler	副本伸缩 / 资源规格调整 / 节点伸缩	19
PDB	驱逐时的可用性底线	19、22
Prometheus / Grafana	指标采集与可视化	20
EFK / Loki	日志采集检索方案	20
OpenTelemetry	可观测性统一标准（指标/日志/追踪）	20
CrashLoopBackOff / OOMKilled	崩溃循环 / 内存超限被杀	21
滚动更新 / 金丝雀 / 蓝绿	三大发布策略	7
Helm Chart / Values / Release	包 / 参数 / 安装实例	23
Kustomize	base + overlay 的无模板配置叠加	24
GitOps / Argo CD	Git 为真相源的持续部署	24
CRD / Operator	自定义资源定义 / 领域知识控制器	25
Service Mesh / Istio / Envoy	服务间流量治理层	26

术语	通俗解释	本书章节
mTLS	服务间双向加密与身份认证	26

附录C 学习资源推荐

C.1 官方文档（第一手资料）

- **Kubernetes 官方文档**（kubernetes.io/zh-cn）：有官方中文版。Concepts 部分系统读一遍，Tasks 部分按需查阅；
- **Kubernetes API 参考**：所有资源的完整字段定义，写 YAML 时的权威依据；
- **CNCF 官网**（cncf.io）：云原生全景图（Landscape），了解生态全景。

C.2 动手环境

- **Kind**（Kubernetes in Docker）：最轻量的本地集群，CI 中也常用；
- **Minikube**：功能全、带 Dashboard 的本地集群；
- **k3s / k3d**：边缘与轻量场景；
- **Killercoda / Play with Kubernetes**：浏览器内免费实验环境，零安装。

C.3 认证体系（CNCF）

- **CKA**（Certified Kubernetes Administrator）：集群运维向，实操考试；
- **CKAD**（Developer）：应用开发部署向；
- **CKS**（Security）：安全向，难度最高，需要先有 CKA。

三者均为线上实操考试（无选择题），备考过程本身就是极好的实战训练。

C.4 进阶主题路线

- **底层原理**：《Kubernetes in Action》（进阶经典）、Kubernetes 源码导读系列；
- **网络深潜**：Calico/Cilium 官方文档、《Kubernetes Networking》相关 talks、eBPF 基础；
- **平台工程**：Backstage、Crossplane、多集群管理与 Fleet；
- **云原生安全**：《Kubernetes Security》、CIS Benchmark for Kubernetes。

C.5 保持更新

Kubernetes 每年三个版本，跟进渠道：

- K8s 官方 Blog 与 Release Notes（每次版本的新特性与弃用列表）；
- KubeCon + CloudNativeCon 大会演讲（YouTube 官方频道，免费）；
- CNCF 的 Newsletter 与中文社区（K8S 技术社区、各大云厂商技术公众号）。

最后一条建议：**技术会过时，原理不会**。把控制循环、声明式、分层解耦这些"不变的东西"真正内化，无论工具如何演进，你都能快速上手。

附录D 生产就绪检查清单

上线前的"体检表"。每一项都可以回答"是"才算就绪。建议打印出来，每次上线逐项核对。

D.1 应用层（单个服务）

资源与调度

- [ ] 设置了合理的 requests/limits（基于压测数据，不是拍脑袋）
- [ ] 副本数 ≥ 2（关键服务 ≥ 3）
- [ ] 配置了 Pod 反亲和，副本分散在不同节点/可用区
- [ ] 配置了 PodDisruptionBudget（minAvailable 或 maxUnavailable）
- [ ] 关键服务设置了合适的 QoS 等级（Guaranteed 或高 Burstable）
- [ ] 资源画像经过校准（无长期 request 虚高或 OOM）

健康与生命周期

- [ ] readinessProbe 配置且检查了关键依赖（决定接客）
- [ ] livenessProbe 配置且只检查自身（避免依赖抖动放大事故）
- [ ] 启动超过 30 秒的应用配置了 startupProbe
- [ ] 应用正确处理 SIGTERM（优雅退出，处理完存量请求）
- [ ] 配置了 preStop 睡眠 3～5 秒，规避端点移除竞态
- [ ] terminationGracePeriodSeconds 覆盖了最慢的优雅退出耗时

配置与安全

- [ ] 配置外置于 ConfigMap，敏感信息用 Secret（且不在 Git 明文存放）
- [ ] 镜像 tag 明确（禁止 latest），镜像来自可信仓库
- [ ] CI 中完成镜像漏洞扫描，高危漏洞不进生产
- [ ] 容器以非 root 运行，丢弃全部 capabilities
- [ ] ServiceAccount 最小权限，或禁用 token 自动挂载
- [ ] 应用日志输出到 stdout，格式结构化（JSON），携带 trace_id
- [ ] 应用暴露 /metrics 端点（RED 指标 + 业务指标）

D.2 流量与网络

- [ ] Service 的 selector 与 Pod 标签匹配（端点非空已验证）

- [ ] Ingress/Gateway 域名、TLS 证书配置完成且自动续期
- [ ] HTTPS 全覆盖（HTTP 重定向到 HTTPS）
- [ ] 关键路由配置了超时与重试（避免无限等待）
- [ ] 内部服务间调用走 Service DNS 名（不写死 Pod IP）
- [ ] NetworkPolicy 部署并实测过（默认拒绝 + 白名单放行）
- [ ] gRPC 长连接场景评估过负载均衡方案

D.3 数据层

- [ ] 有状态服务使用 StatefulSet 或 Operator（不是 Deployment）
- [ ] PVC 使用生产级 StorageClass（回收策略 Retain 或有快照兜底）
- [ ] 备份策略存在且**演练过恢复**（数据库每日备份 + 异地副本）
- [ ] 缩容操作有 quorum/多数派保护意识
- [ ] 主从切换经过演练（或由 Operator 自动处理）

D.4 发布与回滚

- [ ] 发布走 CI/CD 流水线（GitOps 优先）
- [ ] rollout status --timeout 进 CI，失败自动阻断
- [ ] 回滚命令写在 Runbook 里，人人会用
- [ ] 数据库 Schema 变更向后兼容（新代码跑旧库不炸）
- [ ] 有金丝雀或按比例放量机制
- [ ] minReadySeconds ≥ 15 秒（观察期）

D.5 可观测性

- [ ] 指标看板就绪（RED + 资源 + 业务指标）
- [ ] 告警规则"按症状"配置且经过评审（可行动）
- [ ] 告警路由分级（P0 电话 / P1 IM / P2 邮件）
- [ ] 日志可按服务/时间窗检索，保留期符合合规要求
- [ ] 分布式追踪接入（或至少 trace_id 贯穿日志）

D.6 集群与平台层

- [ ] etcd 每小时快照，异机存放，恢复流程演练过
- [ ] 控制平面高可用（apiserver ≥ 3，etcd ≥ 3 奇数节点）

- [ ] 集群版本不在支持窗口外（落后 ≤ 2 个小版本）
- [ ] 证书到期日在日历上（提前续期）
- [ ] 节点自动维修/替换机制就绪（云厂商或自建）
- [ ] HPA 边界合理（min 保可用，max 保护集群）
- [ ] 命名空间配额（ResourceQuota）防止资源抢占
- [ ] 审计日志开启并接入检索

D.7 流程与人

- [ ] Runbook 覆盖已知故障模式（对照第 21 章场景手册）
- [ ] 值班安排与升级路径明确
- [ ] 每次事故有复盘记录，行动项有人跟进
- [ ] 故障演练定期执行（混沌工程实验清单）
- [ ] 变更全部走审阅（包括"只是改个配置"）

附录E 高频面试题精选与解析

Kubernetes 岗位面试的高频问题精选，每题给出"面试官想听到的要点"。这也是检验本书掌握程度的自测题。

E.1 基础概念类

Q1：Pod 为什么存在？为什么 Kubernetes 不直接调度容器？

要点：三个理由——① 有些容器必须紧密协作（共享网络/存储），需要绑定为一个调度单元；② 同 Pod 容器共享网络命名空间，localhost 互通，契合"同机多进程"的传统模式；③ sidecar 等模式让业务镜像与通用能力（日志、代理）解耦。加分：能说出 pause 容器持有网络命名空间的机制。

Q2：Deployment、ReplicaSet、Pod 三者的关系？

要点：Deployment 管理 ReplicaSet（每次模板变更生成新 RS，旧 RS 保留用于回滚）；ReplicaSet 通过 label selector 维持 Pod 数量；Pod 名字中段的哈希是模板指纹。加分：能完整讲出滚动更新中"新 Pod 就绪 → 旧 Pod 下线"的编排逻辑与 maxSurge/maxUnavailable 的作用。

Q3：Pod 的 IP 为什么不能直接对外服务？

要点：Pod 生命周期短（重建即换 IP）；Pod IP 只在集群内可达；缺少负载均衡与健康检查。所以需要 Service 提供稳定的 VIP + DNS + 端点管理。

E.2 原理机制类

Q4：描述 kubectl apply 一个 Deployment 后集群内发生的事。

要点（顺序）：apiserver 认证授权准入 → 写 etcd（此时无 Pod）→ Deployment 控制器创建 ReplicaSet → RS 控制器创建 Pod（Pending）→ 调度器过滤打分选定节点 → kubelet Watch 到分配，调 CRI 拉镜像建容器 → CNI 分配 IP → 探针通过 → Running/Ready。加分：强调"无组件直接命令他人，全部基于 Watch 的事件驱动"。

Q5：Service 的 ClusterIP 是怎么工作的？ping 它为什么不通？

要点：ClusterIP 是虚拟 IP，无进程监听；kube-proxy 在每个节点写 iptables/IPVS 规则做 DNAT，把流量改写为真实 Pod IP；conntrack 保证回程。ping 是 ICMP，不匹配 TCP/UDP 的 NAT 规则，所以不通——这是正常现象。

Q6：什么是控制循环（Reconcile）？为什么要幂等？

要点：读期望状态 → 读实际状态 → 计算差异 → 执行动作消除差异，循环往复。幂等是必须的：Reconcile 会被事件、重试、周期 resync 重复触发，同一输入必须产生同样结果，否则重复执行会破坏状态。加分：能联系 Operator 开发的五条纪律。

Q7：etcd 在集群中的角色？如何备份？

要点：唯一的存储组件，保存集群全部状态；apiserver 是唯一读写方；基于 Raft 多数派。备份：etcdctl snapshot save 定期快照 + 异机存放 + 恢复演练；应用层工具 Velero 可一并处理资源与 PV 数据。

E.3 网络与存储类

Q8：跨节点的两个 Pod 如何通信？

要点：K8s 网络模型要求 Pod 间免 NAT 直连，实现交给 CNI。三大流派：Overlay 封装（VXLAN，兼容性好）、BGP 路由（性能好、要求网络可控）、eBPF 原生（性能与可观测最优）。能比较优劣即高分。

Q9：Pod 的优雅终止流程？如何避免滚动更新时的请求报错？

要点：Pod 标记 Terminating → 从 Endpoints 摘除 → SIGTERM → grace period（默认 30s）→ SIGKILL。报错根因：端点移除传播有延迟，旧 Pod 仍可能收到请求。解法：preStop sleep 3～5 秒 + 应用处理 SIGTERM + readinessProbe 快速摘除。

Q10：PV、PVC、StorageClass 的关系？PVC 一直 Pending 怎么排查？

要点：PVC 是申请单、PV 是实际卷、SC 是动态供应商目录。Pending 排查：SC 不存在或无默认 → provisioner（CSI 驱动）未装 → 可用区不匹配（用 WaitForFirstConsumer）→ 配额/容量问题。看 `kubectl describe pvc` 的 Events。

E.4 实战排障类

Q11：Pod 一直 CrashLoopBackOff，怎么排查？

要点：`kubectl logs --previous` 看崩溃前日志；describe 看 Last State（OOMKilled→Exit 137→调大 limits；应用错误→改配置/修 bug）；检查探针是否误杀（启动慢缺 startupProbe）。强调"止损优先"：生产环境先回滚。

Q12：线上 Pod 频繁重启但日志没有明显错误，可能是什么？

要点：OOMKilled（查看 RESTARTS 与 Last State 的 reason）；liveness 探针超时误杀（应用 GC 停顿、探针 timeout 太短）；节点资源压力驱逐（describe node 看 events）。逐项用证据排除。

Q13：节点 NotReady 后集群发生了什么？

要点：Node 控制器在 grace period 后给节点打 NoExecute 污点驱逐 Pod → Pod 迁移重建（新 IP）→ 有状态 Pod 可能卡在卷 detach（需强制驱逐触发强制分离）→ 排查 kubelet 服务、磁盘、证书。云环境节点自愈可自动替换。

E.5 设计与架构类

Q14：如何设计一个高可用的微服务部署？

要点：多副本 + 反亲和（节点/AZ 两级分散）+ PDB + 探针 + HPA + 无状态设计 + 依赖高可用（DB 主从/Operator）+ GitOps 保证可重建。能说出"升级是照妖镜"（drain 不中断的前提是以上全配齐）是加分项。

Q15：什么是有状态应用？StatefulSet 提供了什么？

要点：稳定身份（有序名 + 专属 DNS）、稳定存储（volumeClaimTemplates 一 Pod 一 PVC）、有序管理（部署/更新/缩容按序号）。更新默认倒序，partition 支持金丝雀。加分：能讨论"数据库上 K8s"的收益与风险权衡。

Q16：Helm 和 Kustomize 的区别？如何选型？

要点：Helm 是模板引擎 + 包管理（分发第三方应用、参数化强、模板复杂时难读）；Kustomize 是 base + overlay 纯声明式叠加（无模板语言、可读性好、无分发能力）。常见组合：第三方用 Helm Chart，自有配置用 Kustomize/GitOps 管理。

Q17：什么是 Operator？什么场景需要？

要点：CRD（自定义资源）+ 自定义控制器，把领域运维知识（故障切换、备份、升级顺序）编码进 Reconcile 循环。需要"持续的有状态管理逻辑"时才用；先找现成 Operator，其次 Operator 化自己的 Runbook，最后才自研。加分：幂等、status 可观测、finalizer 等开发纪律。

Q18：说说你对 GitOps 的理解，它比传统 CI/CD 好在哪？

要点：四原则（声明式、版本化、Pull 模型、持续调和）。核心优势：CI 不持有生产凭证（安全）、配置漂移被机制性消灭、回滚 = git revert、审计 = commit 历史。加分：能讲 Argo CD 的 App 模型与 OutOfSync 检测。

Q19：requests 和 limits 分别影响什么？设置不当会怎样？

要点：requests 影响调度（节点按"声明体积"分配）与最低保障；limits 限制上限（CPU 超限限流、内存超限 OOMKilled）。requests 虚高→资源浪费与调度失败；过低→超卖与 OOM；不设→BestEffort，驱逐第一批。QoS 三档与驱逐顺序。

Q20：如何让应用在 K8s 中支持配置热更新？

要点：文件挂载 ConfigMap（kubelet 周期同步，约 1 分钟内）；注意 subPath 挂载不热更新、直接持有文件句柄的应用需重新打开或支持信号重载；环境变量方式不热更新。最终"热"到什么程度取决于应用自身是否支持感知变化。

附录G 常用 YAML 模板库

本附录提供一套**可直接复制使用**的生产级 YAML 模板。每个模板都遵循本书的最佳实践（资源限制、探针、安全上下文、滚动策略），可作为新项目的起点。使用时把 `<...>` 占位符替换为实际值。

G.1 无状态服务（Deployment 全套）

```
# 01-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: <service-name>
  labels: {app: <service-name>}
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate: {maxSurge: 1, maxUnavailable: 0}
  selector:
    matchLabels: {app: <service-name>}
  template:
    metadata:
      labels: {app: <service-name>}
    spec:
      terminationGracePeriodSeconds: 45
      containers:
      - name: app
        image: <registry>/<image>:<tag>
        ports: [{containerPort: 8080}]
        resources:
          requests: {cpu: 250m, memory: 512Mi}
          limits:   {cpu: "1", memory: 1Gi}
        startupProbe:
          httpGet: {path: /healthz, port: 8080}
          failureThreshold: 30
          periodSeconds: 3
        livenessProbe:
          httpGet: {path: /healthz, port: 8080}
          periodSeconds: 10
          timeoutSeconds: 2
          failureThreshold: 3
        readinessProbe:
          httpGet: {path: /ready, port: 8080}
          periodSeconds: 5
        lifecycle:
          preStop:
            exec: {command: ["sh", "-c", "sleep 5"]}
        securityContext:
          runAsNonRoot: true
          allowPrivilegeEscalation: false
          readOnlyRootFilesystem: true
          capabilities: {drop: ["ALL"]}
      affinity:
        podAntiAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
          - weight: 100
```

```
            podAffinityTerm:
              labelSelector:
                matchLabels: {app: <service-name>}
              topologyKey: kubernetes.io/hostname
---
# 02-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: <service-name>
spec:
  selector: {app: <service-name>}
  ports:
  - port: 80
    targetPort: 8080
---
# 03-hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: <service-name>
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: <service-name>
  minReplicas: 3
  maxReplicas: 12
  metrics:
  - type: Resource
    resource:
      name: cpu
      target: {type: Utilization, averageUtilization: 65}
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
      - {type: Percent, value: 25, periodSeconds: 60}
---
# 04-pdb.yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: <service-name>
spec:
  minAvailable: 2
  selector:
    matchLabels: {app: <service-name>}
```

G.2 有状态服务（StatefulSet）

```
apiVersion: v1
kind: Service
metadata: {name: <app>-headless}
spec:
  clusterIP: None
  selector: {app: <app>}
  ports: [{port: 6379}]
---
apiVersion: apps/v1
kind: StatefulSet
metadata: {name: <app>}
spec:
  serviceName: <app>-headless
  replicas: 3
  selector:
    matchLabels: {app: <app>}
  volumeClaimTemplates:
  - metadata: {name: data}
    spec:
      accessModes: [ReadWriteOnce]
      storageClassName: <storage-class>
      resources: {requests: {storage: 20Gi}}
  template:
    metadata: {labels: {app: <app>}}
    spec:
      containers:
      - name: app
        image: <registry>/<image>:<tag>
        ports: [{containerPort: 6379}]
        volumeMounts:
        - name: data
          mountPath: /data
        resources:
          requests: {cpu: 500m, memory: 1Gi}
          limits:   {cpu: "2", memory: 4Gi}
```

G.3 定时任务（CronJob）

```yaml
apiVersion: batch/v1
kind: CronJob
metadata: {name: daily-backup}
spec:
  schedule: "30 2 * * *"
  timeZone: "Asia/Shanghai"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  startingDeadlineSeconds: 300
  jobTemplate:
    spec:
      ttlSecondsAfterFinished: 86400
      template:
        spec:
          restartPolicy: OnFailure
          containers:
          - name: backup
            image: <backup-image>
            command: ["/backup.sh"]
            resources:
              requests: {cpu: 100m, memory: 128Mi}
              limits:   {cpu: 500m, memory: 512Mi}
```

G.4 守护进程（DaemonSet）

```
apiVersion: apps/v1
kind: DaemonSet
metadata: {name: log-collector, namespace: logging}
spec:
  selector:
    matchLabels: {app: log-collector}
  updateStrategy:
    type: RollingUpdate
    rollingUpdate: {maxUnavailable: 1}
  template:
    metadata: {labels: {app: log-collector}}
    spec:
      hostNetwork: true
      tolerations:
      - {key: node-role.kubernetes.io/control-plane, operator: Exists, effect: NoSchedule}
      volumes:
      - name: varlog
        hostPath: {path: /var/log}
      containers:
      - name: collector
        image: <collector-image>
        volumeMounts:
        - {name: varlog, mountPath: /var/log, readOnly: true}
        resources:
          requests: {cpu: 50m, memory: 64Mi}
          limits:   {cpu: 200m, memory: 256Mi}
```

G.5 配置与密钥

```
# ConfigMap
apiVersion: v1
kind: ConfigMap
metadata: {name: app-config}
data:
  APP_ENV: "prod"
  LOG_LEVEL: "info"
  app.yaml: |
    server:
      port: 8080
---
# Secret（用 kubectl create secret 或外部工具创建，勿直接提交明文到 Git）
# kubectl create secret generic app-secret \
#   --from-literal=DB_PASSWORD='<password>' \
#   --from-literal=API_KEY='<key>'
```

G.6 网络入口（Ingress）

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: main-ingress
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt
spec:
  ingressClassName: nginx
  tls:
  - hosts: [<domain>]
    secretName: <domain>-tls
  rules:
  - host: <domain>
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service: {name: api-svc, port: {number: 80}}
      - path: /
        pathType: Prefix
        backend:
          service: {name: web-svc, port: {number: 80}}
```

G.7 存储（PVC）

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata: {name: data-claim}
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: <storage-class>
  resources:
    requests: {storage: 50Gi}
```

G.8 RBAC（最小权限）

```
apiVersion: v1
kind: ServiceAccount
metadata: {name: app-sa, namespace: <ns>}
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: {name: app-role, namespace: <ns>}
rules:
- apiGroups: [""]
  resources: ["configmaps", "secrets"]
  resourceNames: ["app-config", "app-secret"]
  verbs: ["get", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: {name: app-binding, namespace: <ns>}
subjects:
- kind: ServiceAccount
  name: app-sa
  namespace: <ns>
roleRef:
  kind: Role
  name: app-role
  apiGroup: rbac.authorization.k8s.io
```

G.9 NetworkPolicy（默认拒绝 + 白名单）

```
# 默认拒绝入站
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: deny-all, namespace: <ns>}
spec:
  podSelector: {}
  policyTypes: [Ingress]
---
# 白名单：只允许 app=frontend 访问
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: allow-frontend, namespace: <ns>}
spec:
  podSelector:
    matchLabels: {app: backend}
  policyTypes: [Ingress]
  ingress:
  - from:
    - podSelector:
        matchLabels: {app: frontend}
    ports:
    - port: 8080
```

> **提示：** 这套模板可以直接作为新项目的"黄金路径"起点（第 30 章平台工程）。建议把它们连同生产检查清单（附录 D）一起纳入团队的脚手架，让每个新服务默认就带上安全、弹性、可观测的"出厂配置"。

附录H 参考文献与延伸阅读

本书在写作过程中参考了以下官方文档、经典著作与开源项目资料。它们也是你继续深入的最佳入口——链接均以原文形式列出，便于直接查阅。

H.1 官方文档与规范

- **Kubernetes 官方文档**（含中文版）：https://kubernetes.io/docs/ ｜ https://kubernetes.io/zh-cn/
- **Kubernetes API 参考**（全部资源的字段定义）：https://kubernetes.io/docs/reference/
- **Kubernetes 官方教程**（Tutorials）：https://kubernetes.io/docs/tutorials/
- **CNCF（云原生计算基金会）**：https://www.cncf.io/
- **CNCF Cloud Native Landscape**（云原生全景图）：https://landscape.cncf.io/
- **The Twelve-Factor App**（十二要素应用方法论）：https://12factor.net/
- **OCI（Open Container Initiative）规范**：https://opencontainers.org/
- **CNI（Container Network Interface）规范**：https://github.com/containernetworking/cni
- **CSI（Container Storage Interface）规范**：https://github.com/container-storage-interface/spec
- **CRI（Container Runtime Interface）规范**：https://github.com/kubernetes/cri-api

H.2 经典书籍

- Marko Lukša：《Kubernetes in Action》（第一版 / 第二版），Manning Publications。
- Brendan Burns、Joe Beda、Kelsey Hightower：《Kubernetes: Up and Running》，O'Reilly（中文版《Kubernetes 权威指南》）。
- Brendan Burns：《Designing Distributed Systems》，O'Reilly（中文版《分布式系统设计》）。
- Liz Rice：《Container Security》，O'Reilly（容器安全专题）。
- 张磊：《深入剖析 Kubernetes》，人民邮电出版社（兼有极客时间专栏）。
- Kelsey Hightower 等：《Kubernetes: The Hard Way》（自建集群实践教程）：https://github.com/kelseyhightower/kubernetes-the-hard-way

H.3 核心组件与开源项目文档

运行时与存储

- etcd：https://etcd.io/docs/
- containerd：https://containerd.io/docs/
- Ceph（Rook 存储编排）：https://rook.io/docs/

网络

- Calico：https://docs.tigera.io/calico/
- Cilium：https://docs.cilium.io/
- Flannel：https://github.com/flannel-io/flannel
- CoreDNS：https://coredns.io/manual/
- Submariner（跨集群网络）：https://submariner.io/

交付与配置

- Helm：https://helm.sh/docs/
- Kustomize：https://kubectl.docs.kubernetes.io/
- Argo CD：https://argo-cd.readthedocs.io/
- Argo Rollouts：https://argoproj.github.io/argo-rollouts/
- Flux CD：https://fluxcd.io/docs/

可观测性

- Prometheus：https://prometheus.io/docs/
- Grafana：https://grafana.com/docs/
- Grafana Loki：https://grafana.com/docs/loki/
- OpenTelemetry：https://opentelemetry.io/docs/
- Jaeger：https://www.jaegertracing.io/docs/

安全

- cert-manager：https://cert-manager.io/docs/
- Trivy（镜像扫描）：https://aquasecurity.github.io/trivy/
- Falco（运行时检测）：https://falco.org/docs/
- Kyverno：https://kyverno.io/docs/
- OPA Gatekeeper：https://open-policy-agent.github.io/gatekeeper/
- 密封密钥与外部密钥管理：Sealed Secrets（https://github.com/bitnami-labs/sealed-secrets）、External Secrets Operator（https://external-secrets.io/）

服务网格与弹性

- Istio：https://istio.io/latest/docs/
- Linkerd：https://linkerd.io/2.15/overview/
- KEDA（事件驱动伸缩）：https://keda.sh/docs/
- Karpenter（节点弹性）：https://karpenter.sh/docs/
- Chaos Mesh：https://chaos-mesh.org/docs/

平台与扩展

- Kubebuilder（Operator 开发）：https://book.kubebuilder.io/
- Operator SDK：https://sdk.operatorframework.io/
- Velero（备份恢复）：https://velero.io/docs/
- OpenCost（成本分摊）：https://www.opencost.io/docs/
- Backstage（开发者门户）：https://backstage.io/docs/
- Crossplane（基础设施声明式管理）：https://docs.crossplane.io/

主流 Operator

- CloudNativePG（PostgreSQL）：https://cloudnative-pg.io/documentation/
- Strimzi（Kafka）：https://strimzi.io/documentation/
- Prometheus Operator：https://prometheus-operator.dev/
- ECK（Elasticsearch）：https://www.elastic.co/guide/en/cloud-on-k8s/

H.4 认证与社区资源

- **CNCF 认证**（CKA / CKAD / CKS）：https://www.cncf.io/training/certification/
- **KubeCon + CloudNativeCon** 大会演讲（官方 YouTube 频道，免费）：https://www.youtube.com/c/CNCF
- **Kubernetes Release Notes**（每次版本的新特性与弃用清单）：https://kubernetes.io/releases/

> **提示：** 参考资料的阅读顺序建议与本书同步——先精读官方 Concepts 部分建立概念，再按需查阅各项目文档；遇到版本差异时，一律以你实际使用的 Kubernetes 版本所对应的官方文档为准。